System Design Documents

Table of Contents · 36 documents · 295 pages

1. Designing a Multi-Tenant CI/CD Platform (Cloud-Native) 3
2. Designing a Sandboxed Cloud IDE / Notebook (Colab-like) 17
3. Designing a Cloud DevBox Platform (Remote Dev Environments) 32
4. Designing an Instagram-like Photo/Video Sharing App (Feed-Focused) 49
5. Designing an In-Memory Key–Value Database (Redis-like) 65
6. Designing a Distributed Rate Limiter 79
7. Distributed Rate Limiter — One-Page Cheat Sheet 90
8. Designing a Slack-like Team Messaging System (Real-Time Focus) 93
9. Designing an Online Chess Platform (Real-Time, Timed Games) 113
10. Designing a URL Shortener (Bit.ly) 134
11. Designing Dropbox (File Storage and Sync) 141
12. Designing Ticketmaster (Ticket Booking) 149
13. Designing a News Feed (Facebook-style) 155
14. Designing Uber (Ride Sharing) 161
15. Designing WhatsApp (1:1 and Group Messaging) 167
16. Designing YouTube (Video Upload and Streaming) 174
17. Designing a Web Crawler 180
18. Designing an Ad Click Aggregator 186
19. Designing Top-K (YouTube Top Videos / Trending) 192
20. Designing a Distributed Cache 197
21. Designing a Job Scheduler 202
22. Designing a Payment System (Stripe-style) 208

System Design Documents

Table of Contents · 36 documents · 295 pages

23. Designing Google Docs (Real-Time Collaborative Editing) 214

24. Designing LeetCode (Online Coding Judge) 219

25. Designing Yelp (Business Search and Reviews) 225

26. Designing Strava (Fitness Activity Tracking) 231

27. Designing Robinhood (Stock Brokerage) 237

28. Designing Tinder (Swipe-Based Matching) 243

29. Designing Google Calendar 248

30. Designing an Online Auction (eBay-style) 255

31. Designing Facebook Live Comments 261

32. Designing Facebook Post Search 267

33. Designing a Price Tracking Service (CamelCamelCamel) 273

34. Designing Gopuff (Local Delivery Service) 279

35. Designing a Metrics Monitoring System 285

36. Designing a Notification Service 291

Click any entry to jump to that document.

Designing a Multi-Tenant CI/CD Platform (Cloud-Native)

> A complete, interview-ready walkthrough: requirements → estimates → architecture/components → **multi-tenancy & security** → **execution semantics** → **scaling** → **billing** → **observability/compliance** → trade-offs → rollout plan. Use the headings as your whiteboard agenda. The hard parts are **strong tenant isolation while running untrusted build code, fair scheduling, and correct DAG execution semantics** — spend your time there.

> Think GitHub Actions / GitLab CI / CircleCI / Buildkite. Tenants connect their repos; pushes/PRs trigger pipelines that run as DAGs of jobs on ephemeral runners; artifacts, caches, and logs flow back — all isolated per tenant, metered per tenant, and scaled elastically.

---

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements and assumptions.
2. **Estimate** scale (tenants, builds/day, concurrent jobs, runner fleet, storage).
3. **Split control plane vs data plane (runners)** — the key architectural cut.
4. **Walk the trigger→schedule→execute→report flow.**
5. **Pick the isolation boundary** for untrusted build code (the security core).
6. **Define execution semantics**: DAG, conditionals, matrices, retries, timeouts, cancel, idempotency.
7. **Scale**: autoscale runners, shard the scheduler/control plane, multi-region.
8. **Meter & bill** per tenant; **observe & comply**; **trade-offs**; **rollout plan**.

Keep saying *"here's the trade-off…"* — that's what's being graded.

---

1. Problem & motivation

A managed platform where many independent tenants run **continuous integration and delivery** pipelines triggered by source-control events, on a shared elastic fleet, without seeing or harming each other.

What makes it hard:

- **Untrusted code execution** — a build runs arbitrary tenant code (`make`, `npm`, Docker builds). Multi-tenant on shared infra → must isolate like a hostile workload.
- **Bursty, spiky load** — a monorepo push or a "merge train" can spawn thousands of jobs in seconds; nights/weekends idle. Elastic from zero.
- **Heterogeneous jobs** — seconds-long lint to hour-long integration suites; Linux default, plus Windows/macOS pools.
- **Stateful-ish caching** — dependency/build caches make or break build speed but must stay tenant-scoped.
- **Correctness under failure** — webhooks retry, runners die mid-job, networks flake → need idempotency, retries, and exactly-one logical run per trigger.
- **Fairness** — one tenant's 5,000-job burst must not starve everyone else (noisy neighbor).

The central cut: a **durable, sharded control plane** (orchestration, state, APIs) vs an **ephemeral, elastic data plane** (runners that execute untrusted jobs in disposable sandboxes).

---

2. Requirements

Functional

- **Integrate with source control** (GitHub/GitLab/Bitbucket): OAuth/Apps, **webhooks**, repo read/clone, commit-status/checks write-back.
- **Trigger pipelines** on push/PR/tag/schedule/manual/API.
- **Execute pipelines as DAGs** of jobs/steps with conditionals, matrices, dependencies.
- **Runners/agents** execute jobs in isolated environments (Linux default; Windows/macOS pools).
- **Artifacts & images** — store/retrieve build outputs and container images.
- **Caching** — source, dependency, and build caches to speed builds.
- **Secrets** — securely inject tenant secrets into jobs, scoped.
- **Logs** — stream live job logs to the UI; persist them.
- **Retries, timeouts, cancellation, re-runs.**

Non-functional

- **Strong tenant isolation** (security is primary — untrusted code).
- **Low scheduling latency** — a job starts in seconds (warm), not minutes.
- **High availability** — control plane HA; a runner/host loss doesn't lose pipeline state.
- **Scalable** — 100k+ tenants, millions of jobs/day, tens of thousands of concurrent jobs.
- **Fair** — no noisy-neighbor starvation; per-tenant quotas.
- **Observable & compliant** — logs/metrics/traces, **audit**, retention, DR.
- **Cost-attributable** — per-tenant/project metering and billing.

Assumptions (state them)

- Managed cloud + Kubernetes available. Linux runners default; Windows/macOS via dedicated pools (often macOS = bare-metal/VM, can't containerize).
- At-least-once webhooks (must dedup). Eventual consistency fine for logs/metrics; **strong** for pipeline state transitions.
- Server-side encryption (not customer-managed E2E unless asked).

Clarifying questions

- **Tenant trust** — fully untrusted public (OSS forks running on our infra) or enterprise tenants? (Public PRs = the hardest threat model.)
- **Self-hosted runners** — allow tenants to bring their own compute? (Yes — big enterprise need.)
- **Max concurrency / build minutes** per plan tier?
- **Caching guarantees** — best-effort (fine) vs correctness-critical?
- **Multi-region / data residency** requirements?
- **Deployment (CD) scope** — do we also deploy to tenant environments, or stop at build/test/artifact?

---

3. Back-of-the-envelope estimation

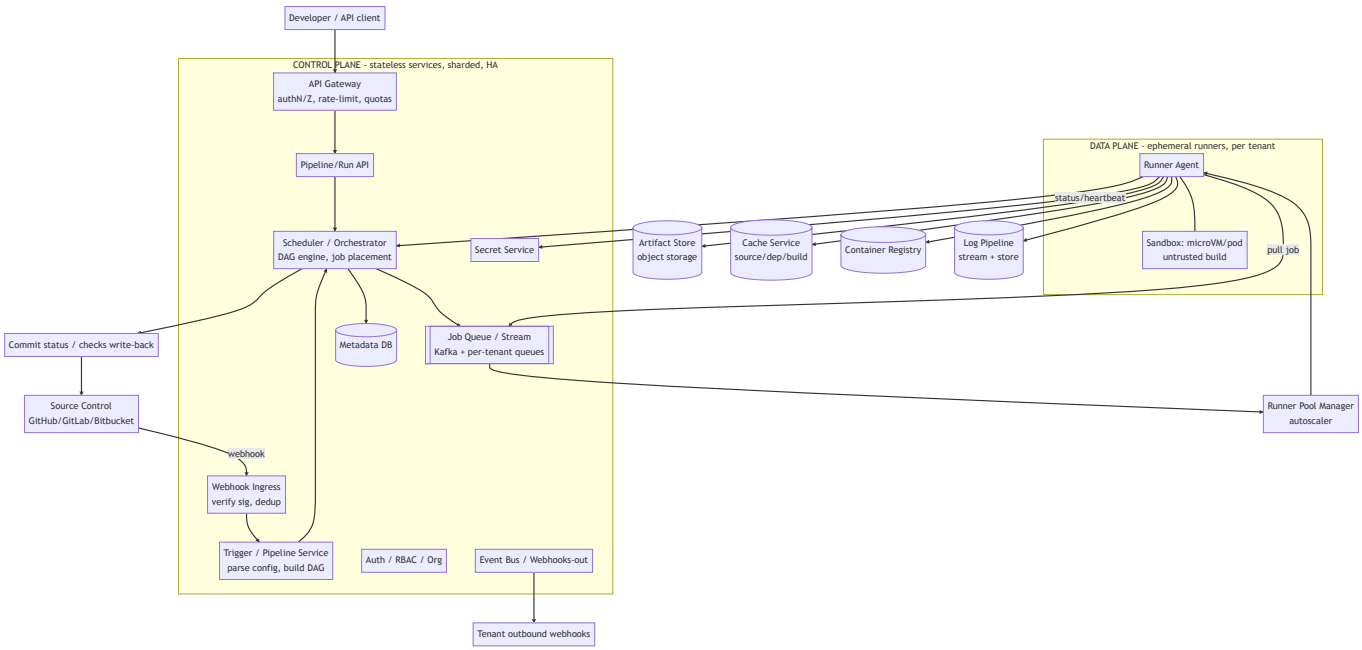
Quantity	Assumption	Result
Tenants / repos	—	100k tenants, ~1M repos
Builds/day	~10 pushes/active repo subset	~~10M pipeline runs/day ≈ **~~120/sec avg, ~1–2K/sec peak**
Jobs per pipeline	DAG avg ~8 jobs (matrices inflate)	~~80M jobs/day ≈ **~~1K/sec avg, ~10K+/sec peak**
Concurrent jobs	peak bursts	**~20k–50k concurrent runners**
Avg job	2 vCPU, 4–8 GB, ~5 min	30k × 2 = **~60k vCPU** live at peak
Runner host packing	32 vCPU hosts, ~8 jobs each	**~few thousand hosts** at peak (scale to ~0 off-peak)
Log volume	~1 MB/job	~80 TB/day logs (hot→cold tiered)
Artifacts/cache	~100 MB/build avg	tens of PB; cache hit-rate is the speed lever
Webhook ingress	every SCM event	**~thousands/sec peak**, must dedup

Takeaways that drive the design:

1. **10k+ jobs/sec peak, untrusted** → ephemeral sandboxed runners + warm pools; isolation is non-negotiable.
2. **Spiky 0→huge** → autoscale-from-zero; queue + fair scheduler absorb bursts.
3. **Tens of PB artifacts/cache** → object storage + CDN; per-tenant scoping + GC.
4. **At-least-once webhooks** → idempotency/dedup at ingress.

---

4. Architecture & key components



Component responsibilities & choices

Component	Responsibility	Choice / notes
Source-control integration	OAuth/**App** install, webhook receipt, clone/mirror, status write-back	Use a **GitHub/GitLab App** (fine-grained, per-install tokens) not a personal token; short-lived installation tokens to clone
Webhook ingress	Verify HMAC signature, **dedup** (delivery id), enqueue	At-least-once → idempotency key = delivery id / `(repo, sha, event)`
Trigger/Pipeline service	Fetch pipeline config (`.ci.yml`), parse → **DAG**, expand matrices	Config-as-code in the repo; validate + version
Scheduler/Orchestrator	Walk the DAG, decide ready jobs, place on runners, track state	The brain; **sharded by tenant/run**; durable state machine
Job queue/stream	Decouple schedule↔execute, absorb bursts, fair dispatch	**Kafka** (durable log) + per-tenant logical queues for fairness
Runner Pool Manager	Autoscale runners, manage warm pools & OS pools	K8s + cluster-autoscaler / Karpenter; pools per OS/size/tenant-class
Runner agent	Pull job, set up sandbox, run steps, stream logs, upload artifacts	Stateless; **dials out**; least-privileged
Artifact & image storage	Build outputs + container images	**Object store** (S3) + CDN; **OCI registry** for images; per-tenant prefixes
Secret management	Store + inject tenant secrets, scoped	Vault/KMS; **short-lived**, scoped per repo/env; never in logs/images
Cache service	Source mirror, dependency cache, build cache	Object store + CDN; tenant-scoped keys; restore/save steps
Metadata DB	Tenants, repos, pipelines, runs, jobs, RBAC, quotas	Postgres (HA, sharded); strong consistency for state transitions
APIs / eventing	External REST + webhooks, internal gRPC, outbound events	REST/GraphQL external; **gRPC** internal; outbound webhooks + event bus

Control plane vs data plane (the key cut)

- **Control plane** — durable, HA, **sharded** services holding *pipeline state* (runs, jobs, DAG progress) in Postgres; schedules and reconciles. Survives runner churn.
- **Data plane** — **ephemeral runners** that execute untrusted jobs in disposable sandboxes, **pull** work from the queue, **dial out** (no inbound), and report status. Cattle: a runner dies → reschedule the job.
- **Logs/artifacts bypass the control plane** — they flow runner → log pipeline / object store directly, so the orchestrator isn't a data bottleneck.

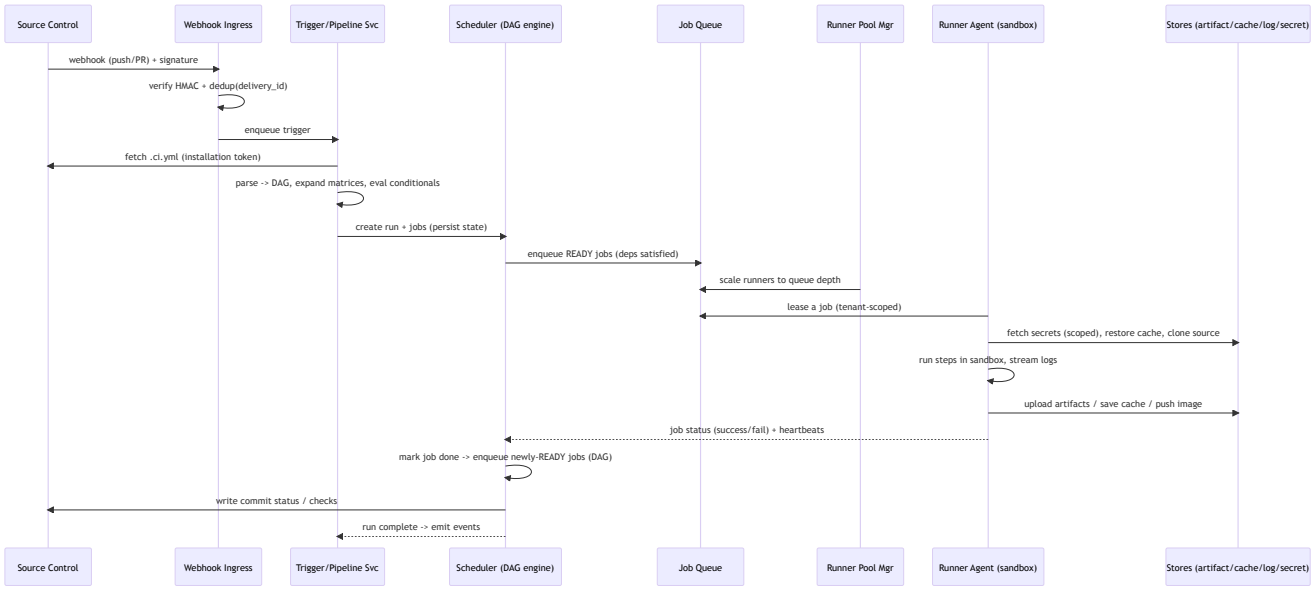
Internal & external APIs (REST / gRPC / webhooks / eventing)

```
# External REST (tenant-facing)
POST /v1/repos/{id}/pipelines:run        {ref, inputs}        → {run_id}
GET  /v1/runs/{run_id}                    → {status, jobs[], dag}
POST /v1/runs/{run_id}:cancel
GET  /v1/runs/{run_id}/jobs/{jid}/logs?follow=true            # live log stream (SSE/WS)
POST /v1/webhooks (inbound from SCM)      # signature-verified
```

- **Internal: gRPC** between control-plane services and **agent↔scheduler** (lease, heartbeat, status) — typed, low-latency, streaming for logs.
- **Eventing**: every state change (`run.started`, `job.failed`, `run.succeeded`) publishes to an **event bus**; tenants subscribe via **outbound webhooks**; internal consumers drive notifications, metering, and the UI.

---

5. Data flow: trigger → schedule → execute → report



Key points:

1. **Verify + dedup at ingress** — HMAC signature (authenticity) + delivery-id dedup (at-least-once → exactly-one run).
2. **Config-as-code** — pipeline defined in the repo; parsed into a DAG; **matrices expanded**, **conditionals evaluated** at plan time (and per-job at runtime for dynamic ones).
3. **Persist run/job state before dispatch** — durable state machine; a control-plane restart resumes the DAG.
4. **Lease-based dispatch** — runners *pull* and **lease** jobs (visibility timeout); if a runner dies, the lease expires and the job is redelivered (with idempotency to avoid double-effect).
5. **DAG progression** — finishing a job unlocks dependents; the scheduler enqueues newly-ready jobs until the DAG completes.
6. **Status write-back** — commit statuses/checks update the PR; events fire to tenant webhooks.

---

6. Multi-tenancy & security (focus area — the core)

We run **untrusted tenant code** (including public-PR forks) on shared infra. Isolation is the product's foundation.

6.1 Tenant isolation

Compute isolation — pick the boundary by threat model:

Boundary	Isolation	Startup	Density	Use
Container/K8s pod (namespaces/cgroups)	Shared kernel — **weak** for untrusted	~ms	High	Trusted/enterprise tenants
gVisor (userspace kernel)	Medium-strong	~tens ms	High	Hardened pods
microVM (Firecracker/Kata) ✅	**Own kernel — strong**	~125 ms	High	**Untrusted / public PRs**
Dedicated VM/host pool	Strongest	~slow	Low	High-security / Windows/macOS

Recommendation: one job = one fresh, ephemeral sandbox, destroyed after the job (no reuse → no state leakage between tenants). Use **microVMs** for untrusted/public workloads; pods (optionally gVisor-hardened) for trusted enterprise tenants; dedicated VM pools for Windows/macOS and high-security tiers.

Network & data isolation:

- **Per-tenant network policy** — default-deny egress + allowlist; **block the cloud metadata endpoint** (169.254.169.254) so jobs can't steal node IAM creds (SSRF); per-tenant subnets/namespaces; jobs can't reach each other.
- **Per-tenant data scoping** — artifacts/caches/logs keyed by `tenant_id` with **per-tenant encryption keys** (KMS); object-store IAM scoped so a job's credentials can only touch its tenant's prefix.
- **Per-tenant runner groups** — dedicated pools for tenants who require it (and for self-hosted runners they own).
- **K8s namespaces per tenant** (or vCluster) for control-plane-side resources, with NetworkPolicies + ResourceQuotas.

6.2 RBAC, role scopes & secret scoping

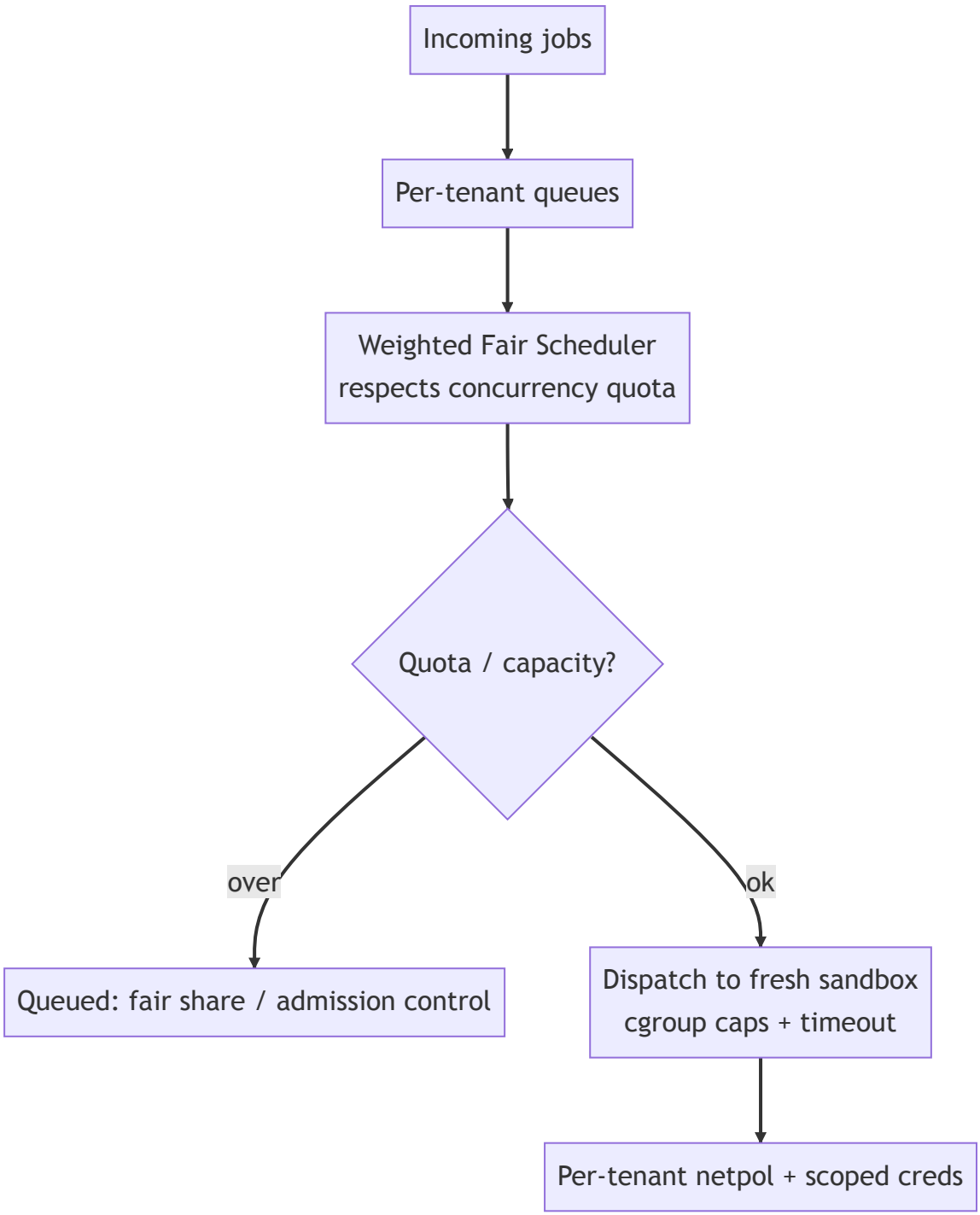
- **Hierarchy**: Org → Project/Repo → Pipeline; **roles** (Owner/Admin/Maintainer/Developer/Viewer) with scoped permissions (run pipeline, manage secrets, approve deploy, view logs).
- Enforced at the API gateway on **every** call and at **job dispatch** (can this pipeline use this runner group / this secret?).
- **Secret scoping** — secrets bound to a scope (org / repo / environment); a job only gets secrets for **its** scope, injected as **short-lived** env/files in the sandbox, **masked in logs**, never persisted to artifacts/images. **Protected/production secrets** gated by environment approvals and **not exposed to fork PRs** (the classic supply-chain attack — untrusted PR exfiltrating prod secrets).

6.3 Rate limiting & quotas

- **API rate limits** per tenant/token (gateway).
- **Concurrency quotas** — max concurrent jobs per tenant/plan (the main fairness lever).
- **Storage quotas** — artifact/cache GB per tenant; **build-minutes** budget per tier.
- **Enforced transactionally** (atomic reserve) at dispatch so two bursts can't both exceed the cap.

6.4 Noisy-neighbor protection (fair scheduling)

- **Per-tenant queues + weighted fair queuing** — round-robin/WFQ across tenants so one tenant's 5,000-job burst can't monopolize runners; each tenant gets a fair share, bursts drain as capacity frees.
- **Resource limits per job** — cgroup CPU/mem/PID/IO caps; hard timeouts; OOM-kills the job, not the host.
- **Admission control** — queue beyond quota; surface "queued (concurrency limit)".
- **Isolation of the build cache** so a poisoned/huge cache from one tenant can't affect others.



One-liner: *"One job runs in one **fresh microVM** with default-deny egress (metadata endpoint blocked), tenant-scoped encrypted storage, and short-lived scoped secrets that never reach fork PRs — and a **weighted-fair, per-tenant-quota scheduler** keeps any single tenant's burst from starving the fleet."*

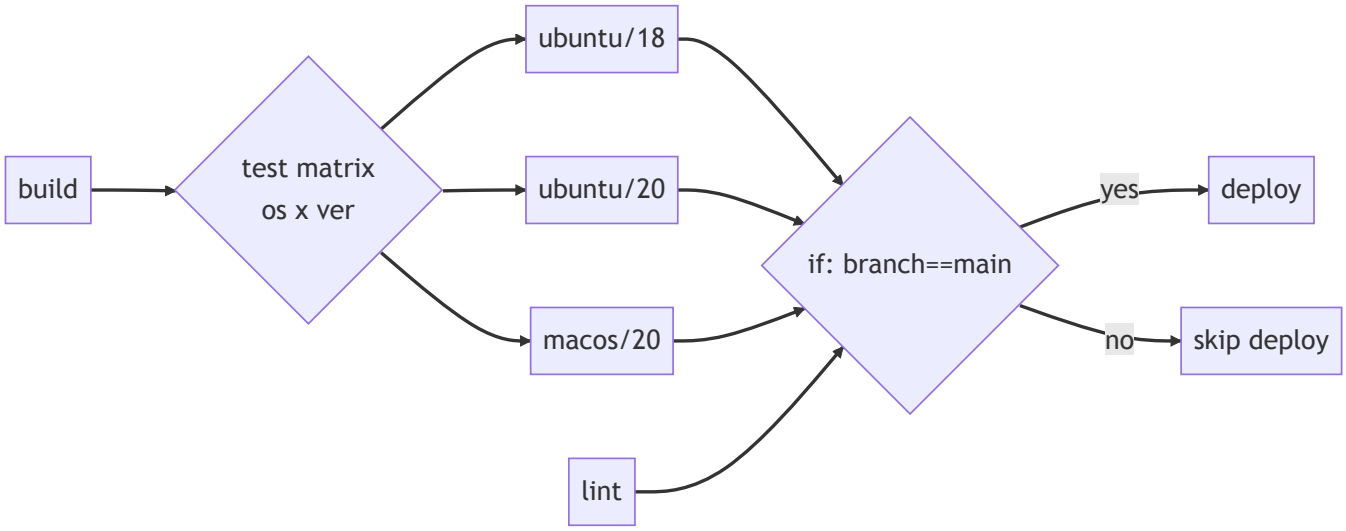
---

7. Pipeline execution semantics (focus area)

7.1 DAG model, conditionals, matrices

- **DAG of jobs**, each with `needs:`/`depends_on:` edges → topological execution; independent branches run in parallel; a job starts only when **all** dependencies succeed (or per policy).
- **Steps** within a job run sequentially in the same sandbox (share workspace).

- **Conditionals** — `if:` expressions on branch, event, file paths, prior results, or inputs; evaluated at plan time for static gates and at runtime for dynamic ones (skip → mark `skipped`, propagate to dependents).
- **Matrices** — expand a job template over a parameter grid (`os × version`) into N parallel jobs; `fail-fast` cancels siblings on first failure (configurable); `include`/`exclude` to tune the grid.
- **Fan-in/fan-out** — fan-out a matrix, fan-in a "deploy" job that `needs:` all of them.



7.2 Retries/backoff, timeouts, cancellations

- **Retries** — per-step/job retry on failure with **exponential backoff + jitter**; distinguish **infra failures** (runner died, spot reclaimed → auto-retry, free) from **user failures** (test failed → don't auto-retry). Cap attempts.
- **Timeouts** — per-step, per-job, per-run wall-clock limits; on expiry, **cancel** (SIGTERM→SIGKILL), mark `timed_out`, free the runner.
- **Cancellations** — user/API cancel, or **superseded** (new push to the same PR auto-cancels the in-flight run — "concurrency group: cancel-in-progress"). Propagate cancel to running sandboxes (signal agent) and dependents (mark `cancelled`).

7.3 Idempotency & deduplication

- **Trigger dedup** — webhook delivery id / `(repo, sha, event)` idempotency key → one logical run per trigger despite at-least-once webhooks/retries.
- **Job lease idempotency** — at-least-once dispatch (a redelivered job after a dead runner) must not double-apply side effects: jobs get a **unique attempt id**; artifact/cache/registry writes are **idempotent** (content-addressed or attempt-scoped paths); "publish"/deploy steps guarded by idempotency keys.
- **Exactly-once *effect*, at-least-once *execution*** — the realistic guarantee: a job may *run* more than once (retry), but its observable result is reconciled to one (last-writer/attempt-scoped).
- **Deterministic run id** so UI/API/events refer to one canonical run.

One-liner: *"Pipelines are DAGs with matrices and conditionals; infra failures auto-retry with backoff while user failures don't; new pushes supersede in-flight runs; and at-least-once dispatch is made safe with attempt-scoped, content-addressed, idempotent writes so a re-run never double-publishes."*

---

8. Scaling (focus area)

8.1 Autoscaling runners/agents

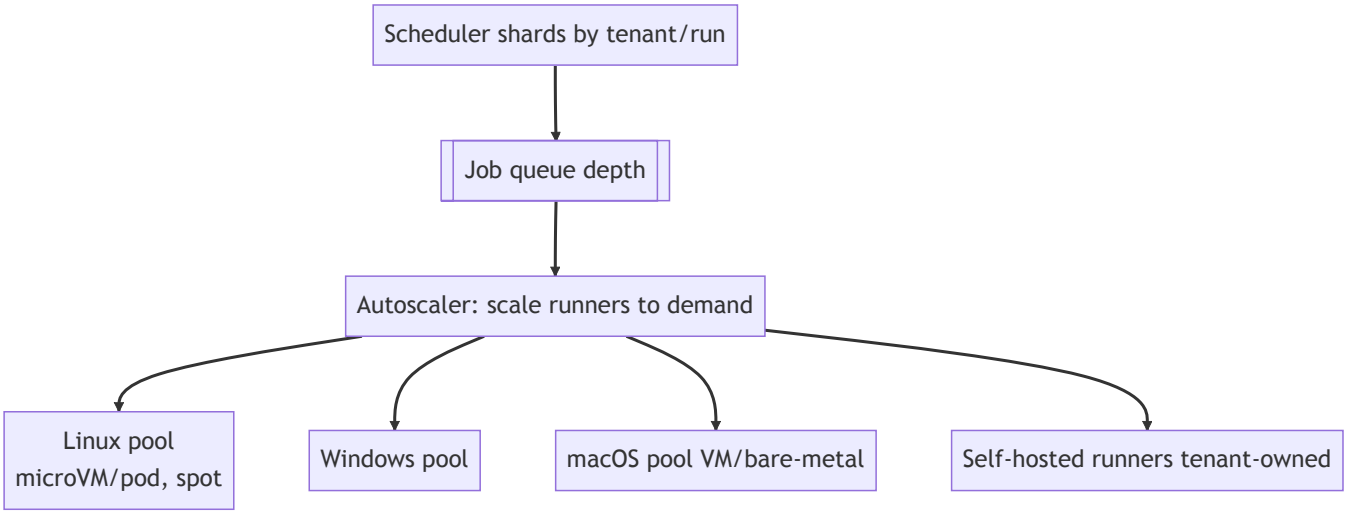
- **Scale-to-zero + warm pools** — keep a small buffer of pre-booted runners per OS/size for fast starts; autoscale the rest on **queue depth** (jobs waiting) and per-pool utilization.
- **K8s cluster-autoscaler / Karpenter** (or cloud ASGs) provision nodes; bin-pack jobs; **spot/preemptible** for cheap, retry-safe jobs (infra-retry covers reclaim), on-demand for critical/long jobs.
- **OS pools** — separate autoscaled pools for Linux (containers/microVMs), Windows, macOS (VM/bare-metal, slower to provision → larger warm buffer).
- **Self-hosted runners** — tenants register their own agents (dial-out, labeled); the scheduler routes labeled jobs to them; tenant owns that capacity/cost.

8.2 Horizontal scaling & sharding of control plane + scheduler

- **Stateless services** scale horizontally behind LBs.
- **Shard the scheduler by tenant/run** (consistent hashing) so each shard owns a subset of runs → no global lock; the DAG engine for a run lives on one shard (ownership + failover).
- **Queue partitioned by tenant** for fairness + parallelism.
- **Metadata DB** sharded by `tenant_id`; read replicas for UI/API reads; strong consistency on the write path for state transitions.
- **Idempotent, lease-based** job handling so any scheduler replica can recover an orphaned run.

8.3 Multi-region

- **Regional control planes + runner fleets**; route a tenant/repo to a **home region** (data residency, latency).
- **Webhook ingress is global/anycast**, forwards to the repo's home region.
- **Artifacts/caches regional** with cross-region replication for DR; registry geo-replicated.
- **Self-hosted runners** can pin builds to a tenant's own region/VPC.
- Keep a run **within one region** (DAG locality) to avoid cross-region chatter; replicate only state needed for failover.



---

9. Cost tracking & billing per tenant (focus area)

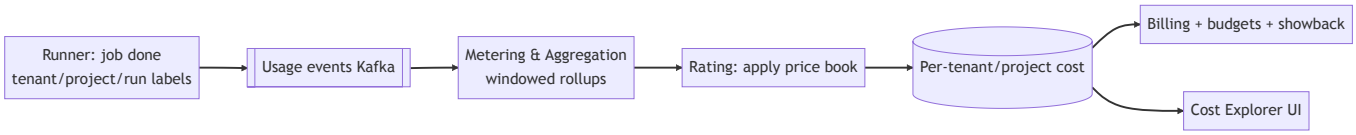
Meter everything a job consumes, attribute to tenant/project, aggregate, bill.

- **Meters (emitted as usage events):**
  - **Compute** — runner **vCPU·seconds, GB·seconds, GPU·seconds**, by OS/instance type (macOS/Windows priced higher).
  - **Build minutes** — wall-clock per job × a multiplier per runner class.
  - **Storage** — artifact/cache **GB·month**, registry storage.
  - **Network egress** — artifact/log/image download bytes.

    - **API/requests** — if metered.
- **Attribution** — every job carries `tenant_id/project_id/pipeline_id/run_id/attempt_id` labels; the agent emits a usage event on job completion (and periodically for long jobs) tagged with these → cost attributes precisely down to project/pipeline.
- **Pipeline:**

```
Runner agent → usage events → Kafka → Metering/Aggregation service
            → rated usage (apply price book) → per-tenant rollups (hourly/daily)
            → Billing (invoices, budgets, showback/chargeback) + Cost Explorer UI
```

- **Aggregation** — stream-aggregate usage events into per-tenant/per-project counters (windowed); reconcile against cloud bills (the platform's own cost) for margin.
- **Budgets & controls** — per-tenant budgets, alerts, and **hard caps** (stop scheduling when build-minutes/budget exhausted) — also a cost-abuse guard (crypto-mining in CI).
- **Idempotent metering** — usage events keyed by `attempt_id` so retries/redeliveries don't double-bill.



One-liner: *"Each job emits idempotent, attempt-keyed usage events tagged with tenant/project/run; a streaming metering pipeline rates them against a price book and rolls them up per tenant for billing, budgets, and hard caps — and we reconcile against our own cloud bill for margin."*

---

10. Observability & compliance (focus area)

Logs, metrics, traces, correlation IDs

- **Logs** — live job logs **streamed** runner → log pipeline → UI (SSE/WebSocket) and persisted (hot store for recent, object store + tiering for old). **Secret-masked**. Separate **system logs** (scheduler/agent) from **user build logs**.
- **Metrics** — queue depth, scheduling latency, runner utilization, job success/failure/retry rates, p50/p99 **time-to-first-byte** (queued→running) and build duration, autoscaler lag → TSDB + SLOs/alerts.
- **Traces** — distributed trace across webhook → trigger → schedule → dispatch → run; find the slow stage.
- **Correlation IDs** — a `run_id` (+ `job_id`, `attempt_id`, `trace_id`, webhook `delivery_id`) propagated through every service, log line, metric, and event → one-click "everything about this run."

Audit logging & retention

- **Append-only, tamper-evident audit** of security-relevant actions: secret create/access, RBAC/role changes, runner-group changes, pipeline config changes, manual approvals, run cancel/override, login. Per-tenant, queryable, exported to SIEM, with **configurable retention** (compliance: SOC2/ISO/GDPR). PII handling + data-deletion workflows.

Disaster recovery

- **Backups** — Postgres PITR (point-in-time recovery), cross-region replicas; object store + registry cross-region replication; periodic restore drills.
- **RPO/RTO targets** — e.g., RPO minutes (replicated state), RTO < 1h (stand up control plane in DR region, runners autoscale from zero).
- **Graceful degradation** — if a region fails, fail over the home region; in-flight runs may need re-run (idempotent triggers make that safe); queued work survives in replicated Kafka.

Zero-downtime deploys & backward-compatible migrations

- **Rolling / blue-green / canary** deploys of control-plane services behind LBs; **drain** runners (finish in-flight jobs, stop leasing new) before node replacement.
- **Expand-contract (backward-compatible) DB migrations** — add columns/tables first (expand), deploy code that writes both/reads new, backfill, then drop old (contract) — never a breaking schema change in one step. **Versioned APIs** + **versioned agent protocol** (new control plane supports old agents during rollout). Feature-flag risky changes.

---

11. Trade-offs (focus area — the money section)

Shared vs dedicated runner pools

	Shared pool	Dedicated pool (per tenant/group)
Cost / utilization	✅ High (bin-packed, scale-to-zero)	❌ Lower (idle reserved capacity)
Isolation	◑ Relies on sandbox boundary	✅ Physical/pool separation
Noisy neighbor	◑ Needs fair scheduling	✅ None (tenant owns the pool)
Compliance	◑ Harder to argue	✅ Easy (dedicated, even single-tenant nodes)
Best for	Most tenants, OSS, cost-sensitive	Enterprise, regulated, high-security

Guidance: shared by default (microVM isolation + fair scheduling) for cost/utilization; offer **dedicated pools** as a premium/enterprise tier for compliance and predictable performance.

Managed vs self-hosted agents

	Managed (our fleet)	Self-hosted (tenant's compute)
Ops burden	✅ We run it	❌ Tenant runs it
Cost model	We meter & bill compute	Tenant pays their own cloud
Network access	◑ Public/our VPC	✅ Inside tenant VPC (private resources, on-prem)
Security/trust	We isolate untrusted code	✅ Tenant trusts their own; ◑ they own patching
Best for	Fast onboarding, most tenants	Private networks, custom hardware (GPU), data residency

Guidance: managed by default; **self-hosted runners** for tenants needing private-network access, special hardware, or strict data residency — they register dial-out agents we route labeled jobs to.

One-liner to say out loud: *"I'd split a **sharded, HA control plane** (trigger, DAG scheduler, metadata in Postgres, per-tenant queues on Kafka) from an **ephemeral runner data plane** where **one job runs in one fresh microVM** with default-deny egress and short-lived scoped secrets. **Weighted-fair per-tenant scheduling** kills noisy neighbors, **autoscale-from-zero + warm pools** handle spiky load, **idempotent attempt-scoped writes** make at-least-once dispatch safe, and **per-job usage events** drive tenant billing. Shared pools by default for cost, dedicated/self-hosted for enterprise isolation and private networks."*

---

12. Deployment plan (phased rollout, validation & guardrails)

Phase	Scope	Validation & guardrails
0. Foundations	Multi-tenant data model, authN/Z, metadata DB, API skeleton, IaC	Threat model review; tenancy isolation tests; everything behind feature flags
1. Core CI (private beta)	SCM App + webhooks, DAG scheduler, **Linux microVM runners**, logs, artifacts	Internal dogfooding; load-test webhook ingress + scheduler; **isolation pentest** (try to escape sandbox / read another tenant) before any external code
2. Caching + secrets + matrices	Dependency/build cache, secret service, matrices/conditionals, retries/timeouts/cancel	Correctness suite (DAG edge cases, idempotency, dedup); secret-leak scans; chaos test (kill runners → jobs reschedule, no double-effect)
3. Scale & fairness (GA)	Autoscaling, per-tenant quotas, **weighted fair scheduling**, rate limits	Soak/burst tests (one tenant 5k jobs ≠ starvation); quota-enforcement tests; SLO dashboards + alerts; gradual tenant onboarding (% rollout)
4. Billing & observability	Metering pipeline, cost explorer, audit logs, tracing	**Reconcile metered cost vs cloud bill** (±% accuracy gate); audit completeness review; retention policies
5. Multi-OS + dedicated/self-hosted	Windows/macOS pools, dedicated pools, self-hosted runners	Per-OS isolation validation; self-hosted security guidance; pin-to-region tests
6. Multi-region + DR	Regional fleets, replication, DR runbooks	**DR game-day** (fail a region, hit RTO/RPO); restore drills; data-residency verification

Cross-cutting guardrails:

- **Feature flags + canary + % rollout** for every phase; instant rollback.
- **Progressive delivery** — new scheduler/agent versions canaried on a small tenant cohort first.
- **Backward-compatible (expand-contract) migrations** and **versioned agent protocol** so rollouts never break in-flight runs.
- **Guardrail metrics** — error rate, scheduling latency, isolation-violation alerts, cost-anomaly detection; auto-halt rollout on breach.
- **Security gates** — isolation pentest before untrusted code; secret-handling review; fork-PR secret policy enforced from day one.

---

13. Networking, security & performance best practices

Networking

- **Webhook ingress**: verify **HMAC signature** + dedup delivery id; **anycast/global** ingress forwarding to the repo's home region.
- **Runners dial out** (no inbound); **self-hosted runners** reach the tenant's private VPC/on-prem.
- **Per-tenant/per-job network policy** — default-deny egress + allowlist; **block the metadata endpoint** (fork-PR credential theft).
- **gRPC** internal + agent↔scheduler; **artifact/log/image uploads go direct** to object store / registry / CDN (not through the control plane).

Security

- **One job = one fresh microVM** (untrusted), destroyed after — no reuse, no leak.
- **SCM App + short-lived installation tokens** (least privilege); never personal/long-lived tokens.

- **Secret scoping** — only the job's scope, **masked in logs, never to fork PRs**; protected/prod secrets gated by **environment approvals**.
- **Supply-chain security** — **signed images (cosign/Sigstore)**, SBOM, **SLSA provenance**, pinned action versions; **OIDC keyless** auth to cloud (no static secrets).
- **Per-tenant KMS** for artifacts/cache; **tamper-evident audit**.

Performance

- **Autoscale-from-zero + warm pools**; **spot/preemptible** for retry-safe jobs; bin-packing.
- **Source/dependency/build cache** (CDN-backed) — the single biggest build-speed lever.
- **DAG parallelism** + **weighted-fair per-tenant scheduling**; **layer cache + lazy pull** for images.

---

14. Staying current — modern & emerging approaches

- **Reference systems:** GitHub **Actions**, GitLab CI, CircleCI, Buildkite; **Argo Workflows**, **Tekton**, Drone for K8s-native pipelines.
- **Runners: Actions Runner Controller (ARC)** on K8s, **Firecracker/Kata** isolation, **Karpenter** autoscaling.
- **Build/cache: BuildKit**, **Bazel** remote cache/RBE, sccache, **Nix** for reproducible builds.
- **Supply chain: SLSA** framework, **Sigstore/cosign**, in-toto attestations, SBOM (Syft/Trivy), **OIDC** federation (kill static cloud secrets).
- **Artifacts:** OCI images + **OCI artifacts**, Harbor registry.
- **How I stay current:** GitHub/GitLab changelogs, SLSA & Sigstore projects, KubeCon CD talks, the CNCF landscape.

---

15. Likely follow-up questions (rehearse these)

- Shared vs dedicated runner pools — the trade-off? *(cost/utilization vs isolation/compliance)*
- How do you stop a fork PR from exfiltrating prod secrets? *(scope secrets, none to forks, env approvals)*
- Runner dies mid-job — double side-effects? *(lease + idempotent, attempt-scoped, content-addressed writes)*
- New push while a build runs — what happens? *(concurrency group: cancel-in-progress / supersede)*
- One tenant fires 5,000 jobs — noisy neighbor? *(per-tenant queue + weighted fair scheduling + quota)*
- Untrusted build stealing node IAM creds? *(microVM + block metadata endpoint + no host creds)*
- At-least-once webhooks — idempotency? *(delivery-id / `(repo, sha, event)` dedup → one run)*
- Accurate per-tenant billing? *(idempotent, attempt-keyed per-job usage events → rating → rollups)*

---

16. Summary checklist (whiteboard recap)

- **Control plane vs data plane** — sharded HA orchestrator + ephemeral runners; control plane owns durable DAG state, runners are cattle.
- **SCM integration** — App-based OAuth, signature-verified + deduped webhooks, installation-token clone, status/checks write-back.
- **Isolation** — **one job = one fresh microVM**, default-deny egress (block metadata endpoint), tenant-scoped encrypted storage, short-lived scoped secrets **never exposed to fork PRs**.
- **Fairness** — per-tenant queues + **weighted fair scheduling** + concurrency/storage/API quotas + cgroup caps.
- **Execution semantics** — DAG + matrices + conditionals; infra-retry vs user-fail; timeouts; supersede-on-new-push cancel; **idempotent attempt-scoped** writes for exactly-once *effect*.
- **Scaling** — autoscale-from-zero + warm pools, spot for retry-safe jobs, **scheduler sharded by tenant/run**, DB sharded by tenant, multi-region home-region.
- **Billing** — idempotent per-job **usage events** (vCPU·s, GB·s, minutes, storage, egress) → metering → rating → per-tenant rollups + budgets/hard caps.

- **Observability/compliance** — streamed secret-masked logs, metrics, traces, **`run_id` correlation**, append-only audit + retention, PITR/cross-region **DR**, **expand-contract** migrations + versioned agent protocol for zero-downtime.
- **Trade-offs** — shared pools (cost) vs dedicated (compliance); managed (easy) vs self-hosted (private network/residency).
- **Rollout** — phased: foundations → core CI → cache/secrets → scale/fairness → billing/obs → multi-OS/dedicated → multi-region/DR, gated by isolation pentests, chaos tests, billing reconciliation, and DR game-days, all behind flags + canaries.

Designing a Sandboxed Cloud IDE / Notebook (Colab-like)

> A complete, interview-ready walkthrough: assumptions → requirements → estimates → APIs → architecture → **compute substrate, isolation, output streaming, lifecycle, persistence** → failure handling → trade-offs. Use the headings as your whiteboard agenda. **Security (isolation) is primary**, and the signature feature is **near-real-time output/log streaming** of running cells — spend your time there.

> Think Google Colab / Jupyter / Deepnote / Replit / Hex. A browser-based, multi-tenant IDE where a user edits code and **runs cells/commands in an isolated sandbox**, watching stdout/stderr and rich output stream back live.

---

0. How to drive the interview (talk track)

1. **State assumptions** + clarify functional/non-functional requirements.
2. **Estimate** scale (users, concurrent sessions, start rate, output throughput).
3. **Split control plane vs data plane**; identify the **kernel/sandbox** as the core unit.
4. **Pick the compute substrate** (VM vs container vs microVM) — security-first.
5. **Define the isolation model**: filesystem, network, process, credentials.
6. **Design the output/log streaming path** (the signature feature).
7. **Define the session lifecycle**: create → run → idle → suspend/resume → terminate.
8. **Persist** workspace files + checkpoints.
9. **Failure + trade-offs.**

Keep saying *"here's the trade-off…"* — that's what's being graded.

---

1. Assumptions (state these up front)

- **Workspace = project/notebook**; a **session = one running sandbox** bound to a workspace.
- Code is **untrusted and arbitrary** (`pip install`, network calls, fork bombs, crypto-miners) — adversarial multi-tenant is the threat model.
- **Notebook/REPL model**: a long-lived **kernel** holds in-memory state; cells execute against it and return stdout/stderr + **rich output** (HTML, images, plots, tables).
- **Eventual consistency** is fine for files; **ordering** of streamed output within a cell must be preserved.
- A session is **more ephemeral** than a full dev box — minutes to hours, often idle, frequently recycled. Workspace **files** persist; **kernel memory** generally does not (unless we add suspend/resume).
- Single cloud, multi-region later. Free tier (small, capped) + paid tiers (bigger, GPU).
- Collaboration is **optional** — called out separately in §13.

---

2. Requirements

Functional

- **Provision an isolated compute environment** per workspace/session.
- **Execute arbitrary user code safely** (sandboxed), cell-by-cell or as commands.
- **Stream execution output/logs** (stdout, stderr, rich MIME output) to the browser in **near real time**.

- **File operations**: upload/download, browse, and **persisted workspace state**.
- **Interrupt / restart** the kernel; **install packages**.
- (Optional) **Real-time collaboration** — co-editing + shared kernel.

Non-functional (the stated priorities)

- **Strong tenant isolation** — *security is primary*. VM-grade boundary.
- **Reasonable startup latency** — new session ready in **a few seconds** (warm), not minutes.
- **Autoscaling + fair resource sharing** — many concurrent sandboxes; no tenant starves others.
- **Observability** — metrics, tracing, **audit logs**.
- **Reliability** — a crashed kernel/host shouldn't lose the user's *files*.

Clarifying questions to ask the interviewer

- **Isolation bar** — adversarial multi-tenant on shared hosts (→ microVMs) or trusted/dedicated?
- **Session persistence** — must kernel **memory** survive idle/suspend (true resume), or only files?
- **GPU** needed (ML notebooks)? Affects scheduling/pools/cost.
- **Egress policy** — can user code reach the internet (`pip install`, API calls)? Usually yes-but-policed.
- **Max runtime / idle limits**, resource caps per tier?
- **Languages** — Python-first (Jupyter kernels) or polyglot?
- **Collaboration** in scope now or later?

---

3. Back-of-the-envelope estimation

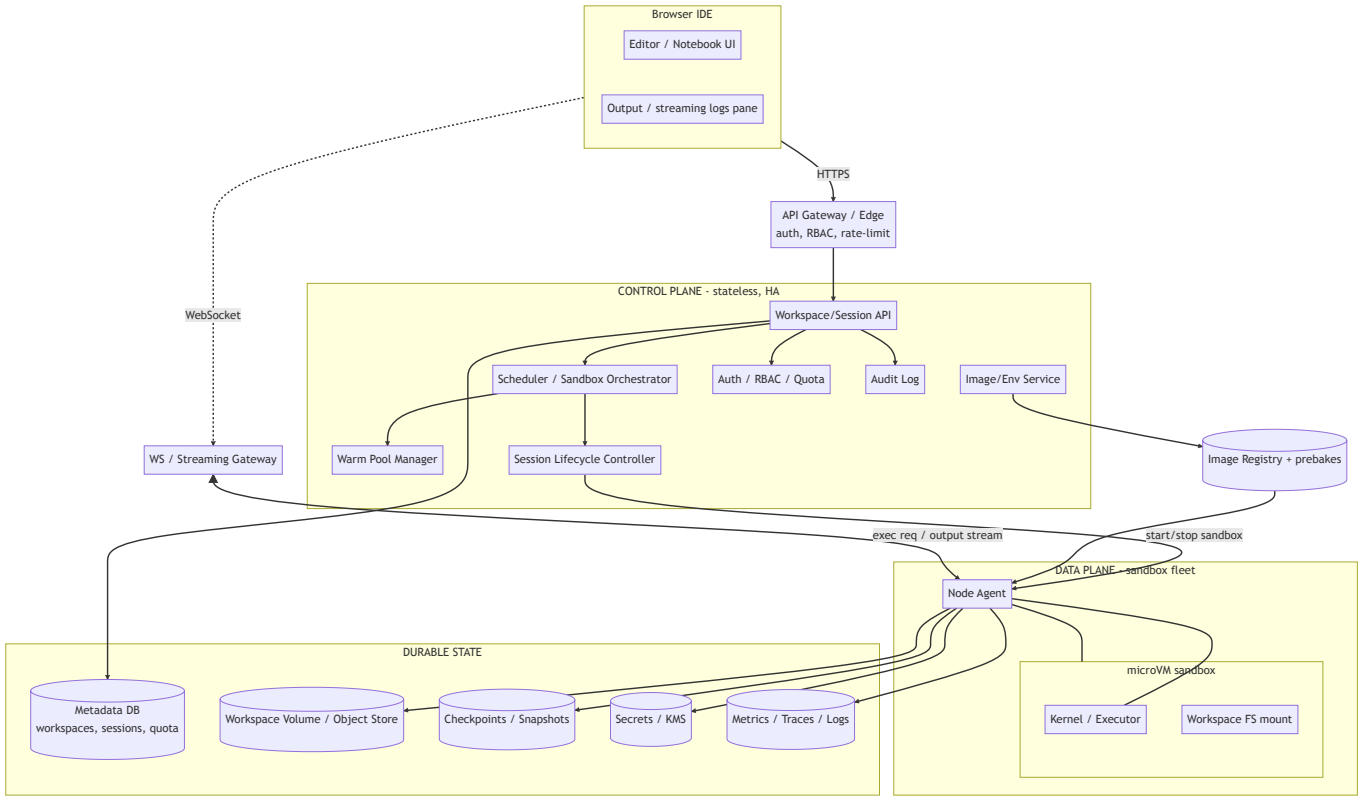
Quantity	Assumption	Result
MAU	—	5,000,000
Concurrent sessions	~2% online at peak	**~100,000 sandboxes live**
Session starts/day	~3 per active user	~~10–15M/day ≈ **~~150/sec avg, ~1–2K/sec peak**
Avg session	2 vCPU, 4–8 GB	100k × 2 = **~200k vCPU**, ~600 TB RAM live
Host packing	64 vCPU / 256 GB, ~16 small sandboxes/host	**~6,000+ hosts** at peak
Output streaming	active cells emit logs/plots	bursty KB/s–MB/s per session; **fan to 1+ viewers**
Workspace storage	~1–5 GB/workspace, 10M workspaces	**tens of PB** (tiered, mostly cold)
Idle ratio	most sessions idle between runs	**idle-cull + suspend = top cost lever**

Takeaways that drive the design:

1. **1–2K session starts/sec peak** → cold container/VM boot is too slow → **warm pools + prebaked images**.
2. **100K live sandboxes, adversarial code** → **microVM isolation** + strict resource caps for fair sharing.
3. **Output is bursty and ordered** → a **streaming pipeline** (kernel → broker → WebSocket) with backpressure, not request/response.
4. **Files must persist; memory usually not** → decouple durable **workspace volume/object store** from ephemeral kernels.

---

4. Architecture overview — control plane vs data plane



Why this split (control vs data plane)?

- **Control plane** (stateless, HA): holds *desired state* in a metadata DB, schedules sandboxes, manages warm pools and lifecycle — reconciler/operator pattern. Restart/scale freely without killing running sessions.
- **Data plane**: worker hosts running each sandbox as a **microVM** with a **node agent** that launches kernels, mounts the workspace, and proxies the **exec** ↔ **output** stream.
- **Streaming bypasses the control plane**: cell-run requests and output flow client ↔ **WS/streaming gateway** ↔ agent ↔ kernel, so live output latency never depends on the orchestrator.

Core components

- **Workspace/Session API** — CRUD workspaces, start/stop sessions, validate quota/RBAC.
- **Scheduler/Orchestrator** — place sandboxes (bin-pack, GPU/zone affinity), manage **warm pools**, autoscale nodes, enforce **fair-share**.
- **Session Lifecycle Controller** — drives the session **state machine**; reconciles desired vs actual; idle-culls.
- **Node Agent** — per-host daemon: launch/stop microVMs + kernels, mount FS, stream output, enforce cgroup caps, heartbeat.
- **Kernel/Executor** — runs user code (Jupyter kernel for Python, etc.), emits stdout/stderr + rich MIME output.
- **WS/Streaming Gateway** — terminates browser WebSockets, routes messages to the right sandbox, fans out output (and to collaborators).
- **Image/Env Service** — prebaked base images + per-workspace dependency layers.
- **Auth/RBAC/Quota, Audit, Observability** — multi-tenant control surfaces.

---

5. API sketch

REST/JSON for control; **WebSocket** for the live exec + output stream (the important one).

```
# --- Workspaces & files (control) ---
POST   /v1/workspaces                          {name, template}      → {workspace_id}
GET    /v1/workspaces/{id}/files?path=/        → file tree
PUT    /v1/workspaces/{id}/files?path=/a.py    (body = bytes)        # upload/save
GET    /v1/workspaces/{id}/files?path=/a.py    → bytes               # download

# --- Sessions (sandbox lifecycle) ---
POST   /v1/workspaces/{id}/sessions  {resources:{vcpu,mem,gpu}}      → {session_id, ws_url}
GET    /v1/sessions/{sid}                       → {state, host, started_at}
POST   /v1/sessions/{sid}/suspend               # snapshot + free compute
POST   /v1/sessions/{sid}/resume                # restore
DELETE /v1/sessions/{sid}                       # terminate
POST   /v1/sessions/{sid}/interrupt             # SIGINT the kernel
POST   /v1/sessions/{sid}/restart               # fresh kernel, keep files
```

WebSocket protocol (Jupyter-style message bus)

Client opens `wss://stream/v1/sessions/{sid}` and exchanges typed messages:

```
// client → server: run a cell
{ "type":"execute_request", "msg_id":"c1", "cell_id":"42", "code":"print(df.head())" }

// server → client: ordered stream for msg_id "c1"
{ "type":"status", "msg_id":"c1", "state":"busy" }
{ "type":"stream", "msg_id":"c1", "name":"stdout", "text":"  a  b\n0 1 2\n", "seq":1 }
{ "type":"display_data", "msg_id":"c1", "mime":"image/png", "data":"<base64>", "seq":2 }
{ "type":"error", "msg_id":"c1", "ename":"ValueError", "evalue":"...", "traceback":[...] }
{ "type":"execute_reply", "msg_id":"c1", "status":"ok", "exec_count":7 }
{ "type":"status", "msg_id":"c1", "state":"idle" }
```

Design notes:

- Every output carries `(msg_id, seq)` → the UI **orders** and **associates** output with the right cell, and can **resume** after a reconnect by requesting `seq > last_seen`.
- **Rich output** via MIME bundles (`text/plain`, `text/html`, `image/png`, `application/vnd.*`) → plots, tables, widgets.
- **Interrupt/restart** are control messages, not in-band, so they work even when stdout is flooding.

---

6. Compute substrate — VMs vs containers vs microVMs (focus area)

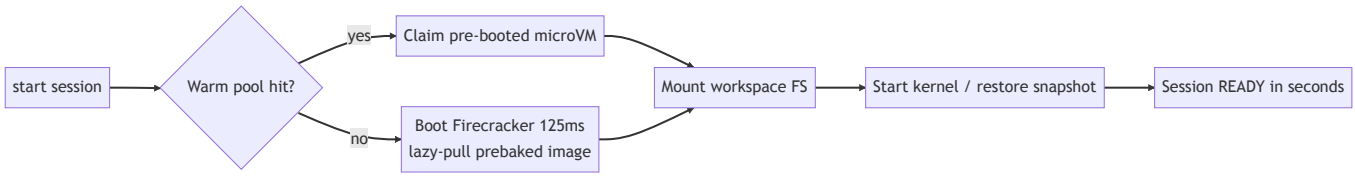
We execute **untrusted, adversarial** code from many tenants on shared hosts. The substrate choice **is** the security decision.

Substrate	Boundary	Cold start	Density	Security (untrusted)	Verdict
Plain container (ns/cgroups)	**Shared host kernel**	~ms	Highest	**Weak** — one kernel 0-day = full escape	❌ not for adversarial multi-tenant
gVisor (userspace kernel)	Syscall interception	~tens ms	High	Medium-strong (smaller attack surface, some perf cost)	◑ good middle ground
Kata / microVM (Firecracker) ✅	**Own guest kernel + hypervisor**	~125 ms	High	**Strong (VM-grade)**	✅ **recommended**
Full VM	Hypervisor	~10s+	Low	Strong	for GPU/heavy/dedicated

Recommendation: microVMs (Firecracker/Kata). Each sandbox runs in its **own minimal VM with its own kernel** — VM-grade isolation at near-container speed and density. This is precisely why Colab-like products and serverless runtimes (AWS Lambda, Fly.io) use Firecracker. gVisor is an acceptable fallback where microVMs aren't available; **plain containers are not acceptable** for arbitrary untrusted code.

Managing the substrate:

- **Warm pools** of pre-booted microVMs per common size/image → claim-and-personalize instead of cold boot (kills startup latency).
- **Prebaked images** (base OS + interpreter + popular libs) cached node-local + regional pull-through; **lazy-pull** layers so the kernel starts before the whole image lands.
- **Snapshot/restore** of a pre-initialized VM (kernel imported, libs warm) → resume in well under a second.
- **Bin-packing + autoscaler** scales nodes with demand; **fair-share scheduler** caps per-tenant footprint.



---

7. Isolation model — filesystem, network, process, credentials (focus area)

Defense in depth *around* the microVM (security is primary):

Filesystem

- **Read-only base image**; user writes land on a **per-session overlay** + the mounted **workspace volume** (only the user's own files).
- No host paths exposed; the guest sees only its rootfs + `/workspace`. Per-tenant **encryption at rest** (tenant-scoped KMS keys).
- **Disk quota** per session (stop a notebook from filling the host); scratch in tmpfs with caps.

Network

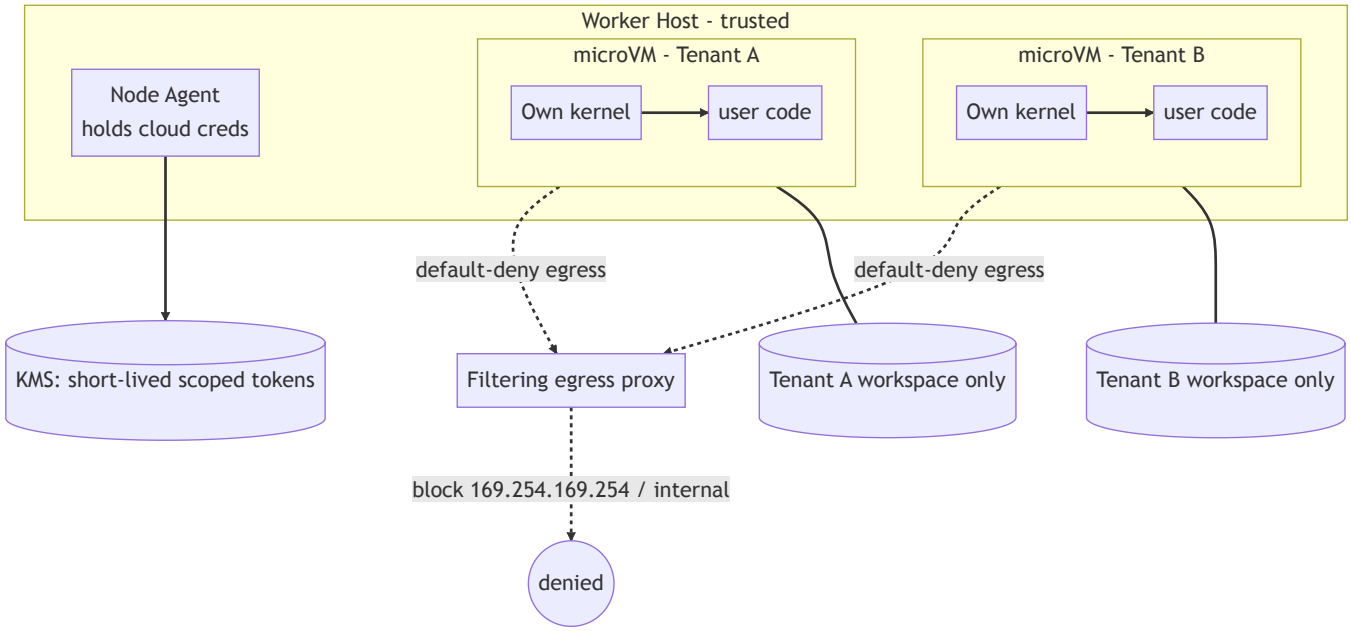
- **Default-deny egress**; allowlist what's needed (e.g., the package mirror). Block the **cloud metadata endpoint** (169.254.169.254) and internal ranges → prevents SSRF to IAM/credential theft.
- Each sandbox on its **own virtual NIC / namespace**; **sandboxes cannot see each other** (no lateral movement).
- Egress through a **filtering proxy** for `pip`/`apt` (policy + audit); per-tenant bandwidth caps.

Process / runtime

- **microVM = own kernel**; inside it, drop privileges, **seccomp**/syscall filtering, no `CAP_SYS_ADMIN`.
- **cgroup caps**: CPU shares, memory limit (OOM-kill the sandbox, not the host), **PID limit** (fork-bomb guard), CPU-time/wall-clock limits, IO throttling → **fair sharing** + DoS protection.
- **Hard timeouts** on cells/sessions; kill runaway kernels.

Credentials

- **No host/cloud credentials inside the guest.** The **node agent (host-trusted)** performs any privileged cloud calls; the sandbox receives only **short-lived, narrowly-scoped** tokens (e.g., a signed URL to its own workspace bucket).
- **Secrets** (user-provided API keys) injected at runtime into **tmpfs/env**, never baked into images or persisted in clear; wiped on terminate.
- Every privileged action and connection is **audited**.



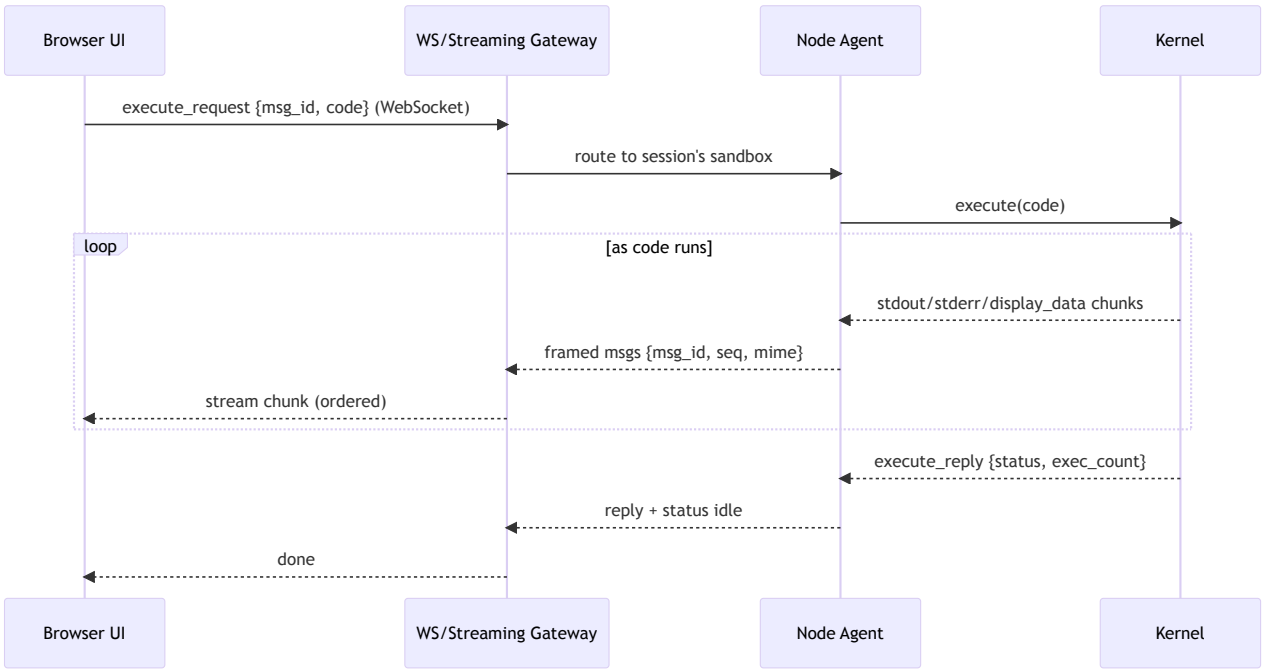
One-liner: *"Each sandbox is a Firecracker microVM with its **own kernel**, a read-only base + per-tenant workspace mount, **default-deny egress** that blocks the metadata endpoint, cgroup CPU/mem/PID caps, and **zero host credentials** inside — privileged cloud calls happen host-side via the agent with short-lived scoped tokens."*

---

8. Output / log streaming architecture (the signature feature)

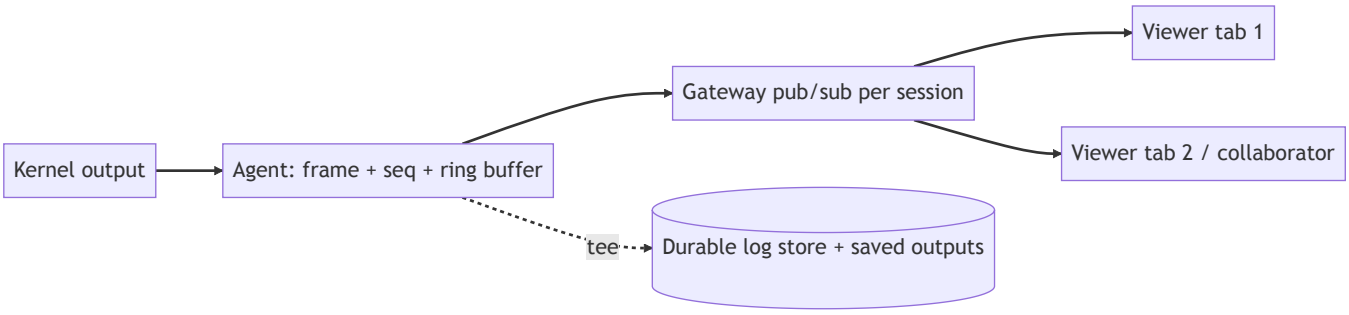
The product *feels* like Colab because output appears **live, ordered, and resumable**. This is a streaming problem, not request/response.

Path



Key design decisions

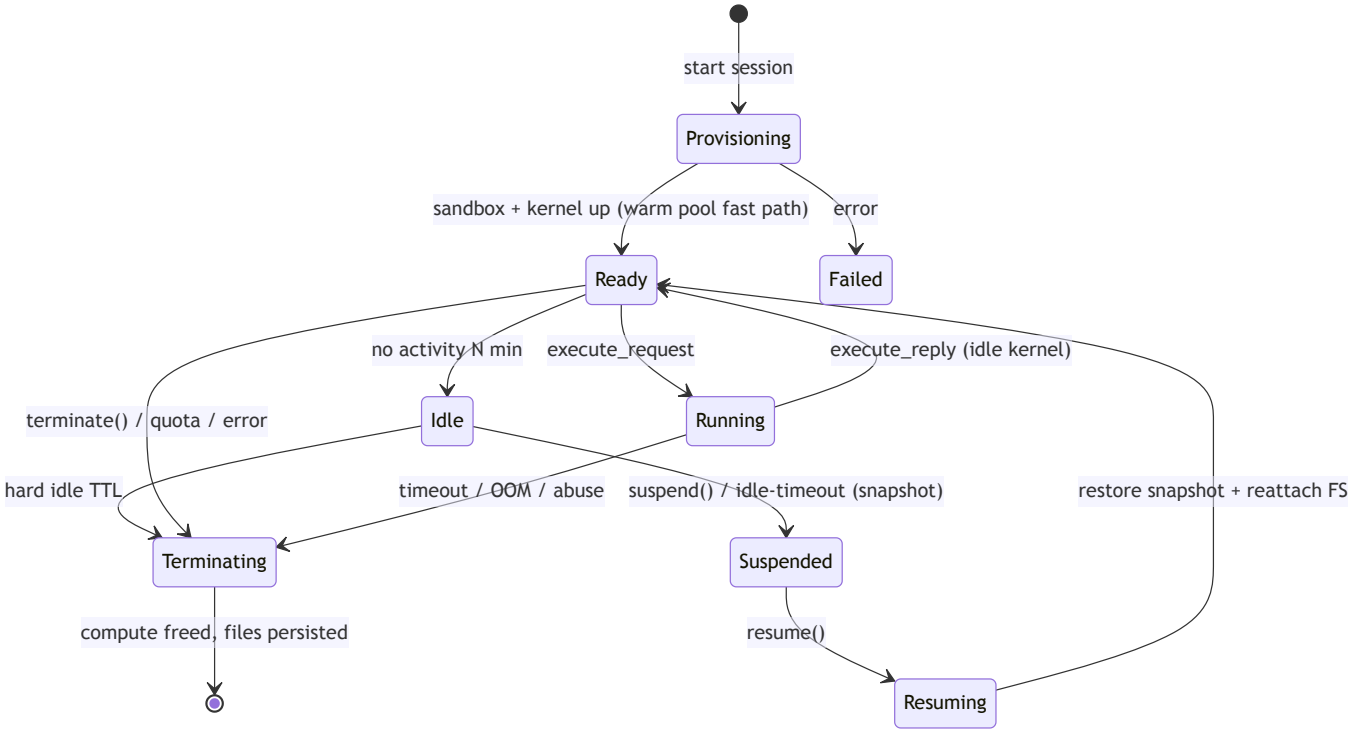
- **WebSocket (bidirectional, low-latency)** for the live channel — not long-polling. SSE is a fallback for output-only.
- **Ordering & resume** — each chunk has `(msg_id, seq)`. The agent keeps a small **ring buffer** of recent output per cell; on reconnect the UI sends `last_seq` and replays the gap → **no lost or duplicated** output across flaky networks.
- **Backpressure** — a `while True: print(x)` can emit MB/s. Apply flow control: **rate-limit/coalesce** chunks (batch small writes every ~50 ms), **cap total output per cell** (truncate with "output limit reached"), and respect WS buffer high-water marks so a slow client can't OOM the gateway or kernel.
- **Rich output** — MIME bundles streamed the same way; large blobs (images, files) can be **offloaded to object storage** and streamed as a URL reference instead of inline base64.
- **Fan-out** (collaboration / multiple tabs) — the gateway **publishes** a session's output to all subscribers via an in-memory pub/sub (or Redis pub/sub across gateway nodes), so N viewers see the same live stream.
- **Durable logs** — tee the stream to a **log store** (and the notebook's saved outputs) for audit, post-hoc viewing, and the "reopen and see last run's output" experience. Separate **system logs** (kernel lifecycle, errors) from **user output**.
- **Stateful routing** — output is tied to a specific sandbox, so the WS gateway uses **sticky routing** (session_id → sandbox/agent) rather than round-robin.



One-liner: *"Output streams over a WebSocket as ordered `(msg_id, seq)` chunks with an agent-side ring buffer for gap-free reconnect, backpressure + per-cell caps so a runaway `print` loop can't melt the gateway, rich output offloaded to object storage, and a per-session pub/sub fan-out so multiple viewers (or collaborators) share one live stream — teed to a durable log store for replay and audit."*

---

9. Lifecycle management — create, run, idle, suspend/resume, terminate (focus area)



Phase	Compute	Kernel memory	Workspace files	Cost	Resume
Running	Active	In RAM	Mounted, live	Full	—
Idle	Active (briefly)	In RAM	Mounted	Full	instant
Suspended	**Freed**	**Snapshotted** (optional)	Persisted	Storage only	sub-sec–seconds
Terminated	Freed	**Gone**	**Persisted**	Storage only	cold start

- **Create** — fast path via warm pool (§6); mount workspace; start kernel; return WS URL.
- **Run** — exec messages over WS; kernel holds REPL state between cells.
- **Idle** — kernel goes quiet; after N minutes start reclaiming. **Idle culling is the #1 cost + fair-share lever** (most sessions idle).
- **Suspend/resume** — two options:
  - *Cheap:* **terminate the kernel, keep files**; resume = cold kernel (memory lost). Default for free tier.
  - *Premium:* **memory snapshot** (Firecracker snapshot to object store) → resume restores variables/imports exactly. Costs storage; great UX.
- **Terminate** — graceful kernel shutdown, **flush files + save outputs**, detach volume, free compute, GC overlay/scratch. Always persist the workspace.
- **Self-healing** — agent heartbeats; lifecycle controller reschedules sandboxes off dead hosts and reaps orphans.

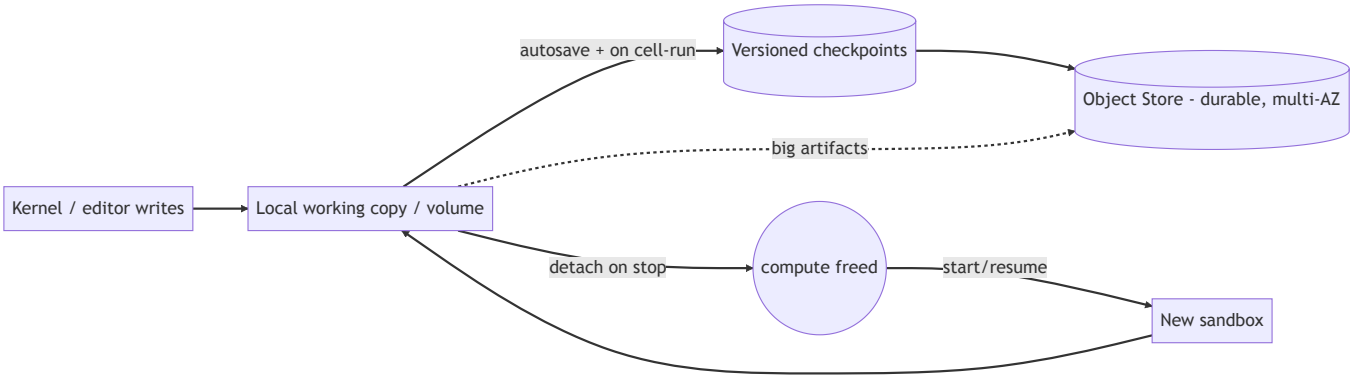
---

10. Data persistence — workspace files & checkpoints (focus area)

Principle: separate durable state from ephemeral compute. Kernels are cattle; the workspace is the pet.

Data	Store	Notes
Workspace files (notebooks, code, data)	**Network volume** (per workspace) *or* **object store** + local cache	Durable, detach/reattach across hosts; survives kernel/host loss
Saved notebook outputs	With the notebook (+ object store for big blobs)	"Reopen and see last run"
Checkpoints / autosave	Versioned object store	Periodic + on-edit; crash recovery, undo
Memory snapshots (suspend)	Object store	Optional true resume
Large artifacts/datasets	Object store, streamed	Don't bloat the volume; signed-URL access
Metadata (workspaces, sessions, quota)	Postgres (HA)	Source of truth

- **Two models:**
  - **Network block volume per workspace** (EBS-like) — POSIX FS, mount on start, detach on stop, **snapshot** for checkpoints. Simple mental model; great for "it's just a filesystem."
  - **Object store + local working copy** — files live in S3; sandbox syncs a working copy on start and **writes back** (autosave/on-close). Cheaper, infinitely scalable, naturally versioned; sync adds complexity.
  - *Common hybrid:* object store as source of truth + a fast local/volume cache for live editing.
- **Autosave + checkpoints** — periodically and on cell-run, snapshot the notebook (content + outputs) to versioned storage → crash never loses more than a few seconds; users can restore.
- **Durability** — replicated (multi-AZ) volume/object store; checkpoints give point-in-time recovery.
- **Crash consistency** — `fsync`/journaling; flush on graceful terminate; filesystem-consistent snapshots.
- **GC** — reap scratch/overlay on terminate; expire orphaned volumes and old anonymous workspaces after retention.



One-liner: *"Files live on a replicated per-workspace volume (or object store) decoupled from the kernel, with periodic + on-run **checkpoints** to versioned storage — so a kernel crash, OOM, or host failure costs you a restart, never your notebook."*

---

11. Autoscaling & fair resource sharing

- **Warm-pool autoscaling** — keep a buffer of pre-booted microVMs per size; scale predictively (weekday/class-hour surges) so starts stay fast under load.
- **Node autoscaler + bin-packing** — add/remove hosts with demand; pack small sandboxes densely; **scale to zero** off-hours.
- **Fair-share scheduler** — per-tenant/per-org caps on concurrent sandboxes and total vCPU/RAM/GPU; **weighted fair queuing** so a heavy user can't starve others; **admission control** + queue under saturation (graceful "waiting for a runtime…").

- **Resource caps per session** — cgroup CPU/mem/PID/IO limits (also a security control) guarantee one notebook can't hog a host.
- **Cost levers** — **idle culling**, session TTLs, suspend-instead-of-run-idle, **spot/preemptible** hosts for free-tier (with checkpoint so preemption only costs a restart), GPU time-slicing/pools.

---

12. Observability (a stated requirement)

- **Metrics** — start latency p50/p99, **warm-pool hit rate**, sandboxes live, scheduler queue depth, kernel exec latency, output throughput, OOM/timeout rates, per-tenant usage → TSDB + SLOs/alerts.
- **Tracing** — trace the **start path** (API → schedule → boot → kernel → ready) and the **exec path** (request → kernel → first-output → done) to find slow stages. Propagate `session_id`/`msg_id`.
- **Logs** — separate **user output** (streamed/saved) from **system logs** (kernel/agent/control plane); centralized, per-tenant scoped.
- **Audit logs** — append-only, tamper-evident: session create/terminate, **code execution events**, file up/download, package installs, secret access, egress decisions, role/quota changes → SIEM + retention. Critical for a system running untrusted code.
- **Golden signal**: **time-to-ready** (start → first cell runnable) and **time-to-first-output** (run → first stream chunk).

---

13. Collaboration (optional — called out)

If included: **co-editing** via **CRDT/OT** (e.g., Yjs) over the same WebSocket layer for conflict-free shared text, plus a **shared kernel** so collaborators see the **same live output** (the §8 pub/sub already fans output to multiple subscribers). Add presence/cursors and per-user RBAC on the workspace. **Trade-off:** a shared kernel means shared mutable state and a shared blast radius (one user's `os.system` affects all) — fine within a trusted team, gated by workspace membership. Call it out as **v2** to keep the core scope focused.

---

14. Failure scenarios — *"what if X fails?"*

Failure	Impact	Mitigation
Kernel crashes / OOM	Cell fails, memory lost	OOM-kill sandbox not host; surface error; **files + last checkpoint safe**; one-click restart
Worker host dies	Running sandboxes lost	Workspace is network-attached/object-store → **reschedule** elsewhere; memory lost unless snapshotted
Control plane down	Can't start/stop sessions	CP stateless + HA; **running sandboxes keep streaming** (data-plane independent)
WS/streaming gateway dies	Output stream drops	Multiple gateways; client **reconnects**, replays from `last_seq` via agent ring buffer
Output flood (`while True: print`)	Gateway/client overload	Backpressure, coalescing, **per-cell output cap**, truncate
Image registry slow	Cold starts stall	Node-local + regional cache, lazy-pull, warm pool absorbs
Runaway / abusive code (miner, fork bomb)	Host degraded / abuse	cgroup CPU/mem/PID/IO caps, timeouts, egress policy, anomaly detection → auto-suspend
Tenant escape attempt	Cross-tenant breach	microVM (own kernel) + default-deny egress + no host creds + seccomp
Storage/AZ failure	Risk to files	Multi-AZ replication + versioned checkpoints (DR restore)
Start storm (class begins)	Latency spike	Predictive warm-pool scaling + admission queue + fair-share

Guiding principle: the **data plane outlives the control plane**, and **durable files outlive the kernel**. A crash should cost a *restart*, never the user's work.

---

15. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Isolation vs speed/density	Full VM (strong, slow)	Container (fast, weak)	**microVM** — VM-grade isolation at near-container speed ✅ (gVisor as fallback; never bare containers for untrusted code)
Startup vs cost	Big warm pool (instant)	No pool (cheap, slow)	Pool sized to predicted demand + snapshots; scale-to-zero off-hours
Suspend: files-only vs memory	Terminate kernel (cheap, lose RAM)	Memory snapshot (instant resume, storage cost)	Files-only default; memory snapshot premium
Streaming: WS vs polling	WebSocket (live, bidirectional)	SSE/long-poll (simpler)	WS for the live exec channel; SSE fallback for output-only
Output: inline vs offload	Inline base64 (simple)	Object-store + URL (scalable)	Inline small, **offload large** blobs
Persistence: volume vs object store	Block volume (POSIX, simple)	Object store (cheap, versioned)	Hybrid: object store source-of-truth + local/volume cache
Consistency vs availability	Strong everywhere	Eventual reconcile	Strong for quota/RBAC/billing; eventual for fleet; available data plane under CP partition

CAP/PACELC framing: quota/RBAC/metadata is **CP** (no double-spend, no privilege bypass). **Fleet state** is eventually consistent (controllers reconcile desired→actual). The **streaming data plane** is designed to **stay available** even when the control plane is partitioned — a running notebook keeps streaming.

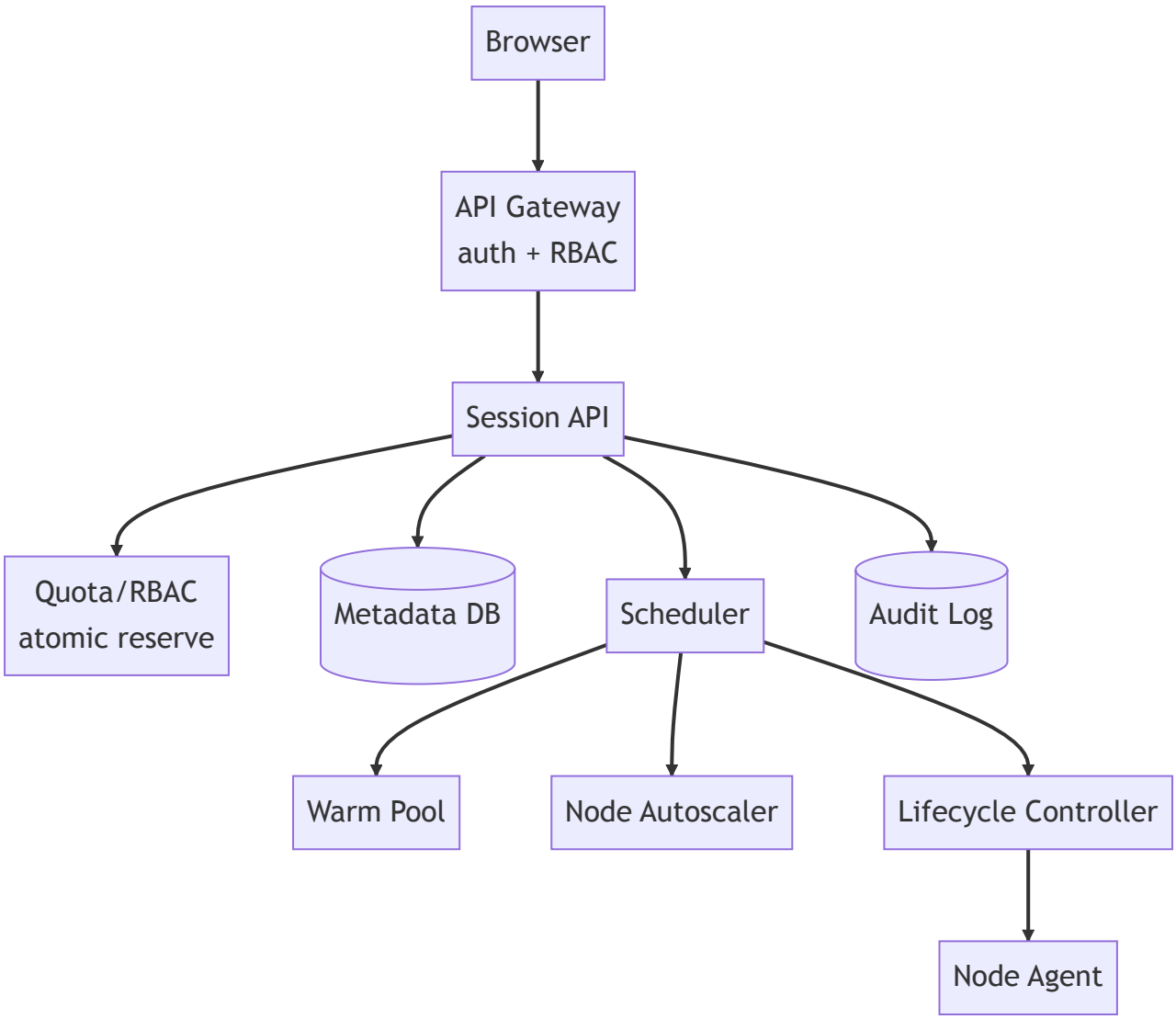
One-liner to say out loud: *"I'd run each session as a **Firecracker microVM** (own kernel, default-deny egress, cgroup caps, no host creds) for security-first multi-tenant isolation, fronted by a **stateless HA control plane** that schedules from **warm pools + prebaked images** so starts take seconds. Cell execution and output ride a **WebSocket** as ordered `(msg_id, seq)` chunks with an **agent-side ring buffer** for gap-free reconnect, backpressure + per-cell caps to survive output floods, and per-session **pub/sub fan-out** for collaborators — teed to durable logs. **Files live on a replicated volume/object store with checkpoints**, decoupled from the kernel, so crashes cost a restart, not the user's work. Idle-culling, fair-share scheduling, and spot hosts keep it economical."*

---

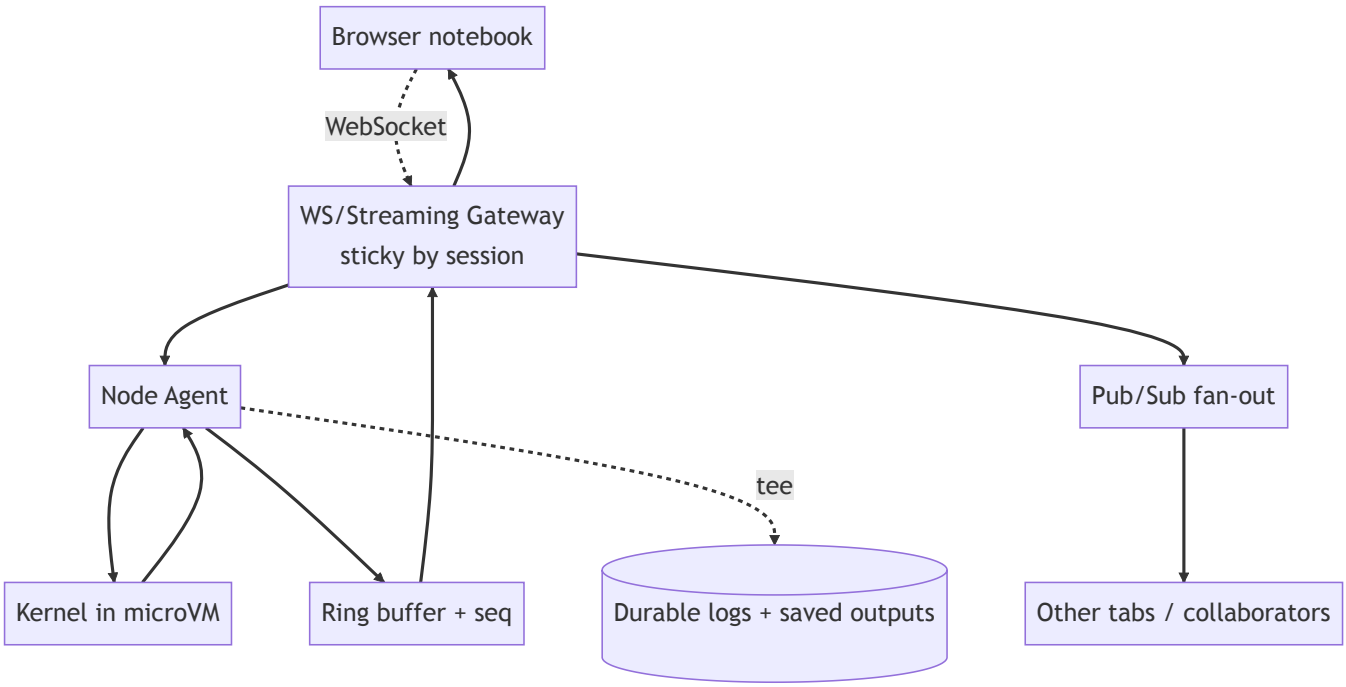
16. Full system design (detailed)

End-to-end, split into **(A)** the control/start path, **(B)** the exec + output streaming data path, and **(C)** persistence & lifecycle.

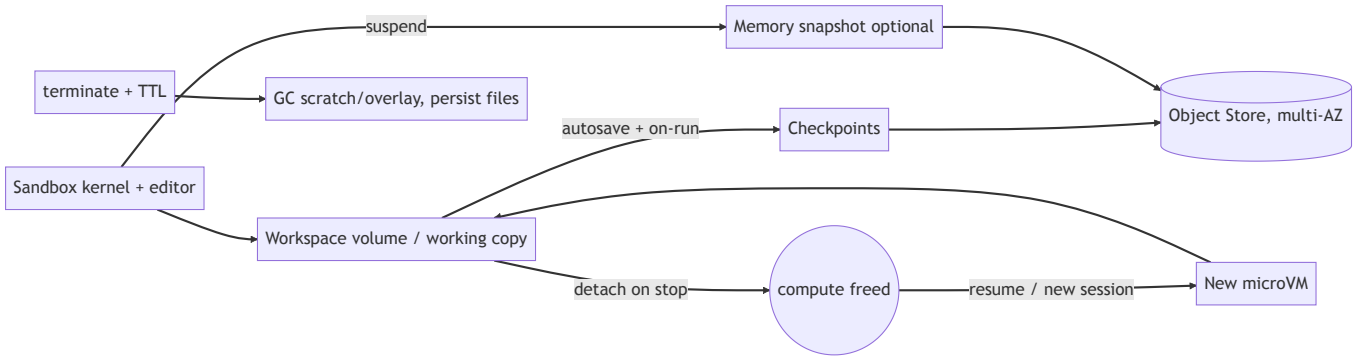
16A. Control path — start a session



16B. Data path — execute & stream output



16C. Persistence & lifecycle



---

17. Networking, security & performance best practices

Networking

- **WebSocket** for the live exec/output channel (bidirectional, low-latency); **SSE fallback** for output-only clients.
- **Sticky routing** `session_id → sandbox` at the streaming gateway; **TLS at edge**, HTTP/2/3 for the UI, **gRPC** agent↔control.
- **Per-sandbox network namespace**; **default-deny egress** through a filtering proxy; **block the metadata endpoint** (IAM/SSRF).
- **Per-gateway fan-out** of a session's output to multiple viewers via pub/sub.

Security

- **microVM per session** (own kernel); **one session = one fresh sandbox, destroyed after** → no cross-tenant state leak.
- **No host creds in the guest**; short-lived scoped tokens (signed URL to the session's own bucket).
- **Read-only base + per-tenant overlay/workspace**; per-tenant **KMS encryption**.
- **seccomp, dropped caps, cgroup CPU/mem/PID/IO caps** (fork-bomb/miner guard); hard cell/session timeouts.
- **Secrets in tmpfs, masked in logs**; **audit code-execution events** (running untrusted code makes this essential).

Performance

- **Warm pools + prebaked images + snapshot/restore** → seconds to first cell.
- **Output backpressure + per-cell caps + coalescing**; **offload large blobs** to object storage (stream a URL, not base64).
- **Agent-side ring buffer** for gap-free reconnect; **idle-cull + scale-to-zero**.

---

18. Staying current — modern & emerging approaches

- **Isolation: Firecracker** / **Kata** / **gVisor**; **WebAssembly (WASI, Pyodide)** for ultra-light or in-browser sandboxing.
- **Reference systems:** Google **Colab**, **Jupyter/JupyterHub + BinderHub**, **Deepnote**, **Hex**, **Replit** (Nix + Firecracker).
- **Protocols: Jupyter messaging protocol** (kernels), **Language Server Protocol**, **CRDTs** (Yjs/Automerge) for collaboration.
- **Streaming: WebTransport/HTTP3**, gRPC streaming for output.
- **GPU:** time-slicing / **MIG** / fractional GPUs for ML notebooks.
- **How I stay current:** Jupyter & Firecracker releases, Replit/Colab engineering blogs, KubeCon/PyCon talks.

---

19. Likely follow-up questions (rehearse these)

- Why a microVM over a container for arbitrary code? *(own kernel; bare containers are unsafe for untrusted code)*
- Output floods (`while True: print`) — how do you survive? *(backpressure + per-cell caps + coalescing)*
- Client reconnects mid-cell — no lost/duplicated output? *(`(msg_id, seq)` + agent ring buffer replay)*
- Does kernel memory survive idle? *(terminate-and-keep-files by default; memory snapshot for premium resume)*
- How do you start a session in seconds? *(warm pool + prebaked image + snapshot/restore)*
- Stop a notebook stealing node IAM creds? *(block the metadata endpoint; no host creds in guest)*
- Fair sharing across tenants? *(weighted fair queuing + cgroup caps + admission control)*
- Collaboration consistency? *(CRDT co-editing + shared kernel — call out the shared-blast-radius trade-off)*

---

20. Summary checklist (whiteboard recap)

- **Control plane vs data plane** — stateless HA orchestrator + sandbox fleet; data plane keeps streaming even if CP is down.
- **microVM substrate** (Firecracker) — own kernel per session; security-first for adversarial code (gVisor fallback, never bare containers).
- **Isolation** — read-only base + per-tenant FS, default-deny egress (block metadata endpoint), cgroup CPU/mem/PID caps, **no host creds in guest**, audited.
- **Output streaming** — WebSocket, ordered `(msg_id, seq)`, agent ring buffer for gap-free reconnect, backpressure + per-cell caps, rich-output offload, pub/sub fan-out, teed to durable logs.
- **Fast start** — warm pools + prebaked images + snapshot/restore = seconds ("restore, not build").
- **Lifecycle** — create → run → idle → suspend/resume (optional memory snapshot) → terminate; **idle-culling** is the top cost/fair-share lever.
- **Persistence** — files on replicated volume/object store + **checkpoints**, decoupled from the kernel; crash = restart, not data loss.
- **Autoscaling + fair-share** — predictive pools, bin-packing, weighted fair queuing, admission control, spot for free tier.
- **Observability** — time-to-ready / time-to-first-output SLOs, exec tracing, append-only audit of code execution.
- **CAP** — CP for quota/RBAC/billing; eventual fleet reconcile; available streaming data plane under partition.

Designing a Cloud DevBox Platform (Remote Dev Environments)

> A complete, interview-ready walkthrough: requirements → estimates → APIs → data model → lifecycle → **provisioning, isolation, persistence, connectivity** → multi-tenancy (quota/RBAC/audit/cost) → failure handling → trade-offs. Use the headings as your whiteboard agenda. The hard parts are **fast provisioning, strong isolation, and reliable persistence** — spend your time there.

> Think GitHub Codespaces / Gitpod / Coder / AWS Cloud9 / Replit. A managed service that hands a developer a ready-to-code remote machine in seconds, reachable from a **browser, SSH, or an IDE plugin** (VS Code Remote, JetBrains Gateway).

---

0. How to drive the interview (talk track)

1. **Clarify** product goals + functional/non-functional requirements.
2. **Estimate** scale (devs, concurrent boxes, provisioning rate, storage) to justify choices.
3. **Split control plane vs data plane** — the single most important architectural cut.
4. **Define the lifecycle** (create/start/stop/pause/resume/delete) as a state machine.
5. **Pick the isolation boundary** (VM vs container vs microVM) and justify.
6. **Make provisioning fast** (prebuilds, warm pools, snapshots, image caching).
7. **Persist reliably** (home dir volumes, snapshots, what survives stop/pause).
8. **Connect** the browser/SSH/IDE through a gateway/tunnel.
9. **Multi-tenancy**: quota, RBAC, audit, cost controls.
10. **Failure + trade-offs.**

Keep saying *"here's the trade-off…"* — that's what's being graded.

---

1. Problem & motivation

Give developers an **instant, reproducible, powerful** development machine that lives in the cloud instead of on their laptop.

Why teams want this:

- **Zero onboarding friction** — "works on my machine" dies; a new hire is coding in minutes from a template, not days of setup.
- **Reproducibility** — environment is defined as code (image + toolchain + dotfiles), identical for everyone.
- **Power & elasticity** — attach 32 vCPUs or a GPU for a build/ML task, then give it back.
- **Security** — source code and secrets stay in the cloud, never on an unmanaged laptop; centrally revocable.
- **Cost efficiency** — pause idle boxes; pay for what's used.

What makes it hard:

- **Fast provisioning** — a cold "create from template + clone repo + install deps" can take minutes; users expect **seconds**.
- **Strong isolation** — untrusted code (arbitrary `npm install`, Docker-in-Docker) from many tenants on shared hardware → must not let one box read/attack another or the host.
- **Reliable persistence** — a developer's uncommitted work and home dir **must survive** stop/pause/restart and host failures.

- **Stateful, long-lived, bursty** — unlike stateless web services, each box is a pet-ish stateful workload that can be paused for days and resumed.

---

2. Requirements

Functional

- **Lifecycle**: create, **start, stop, pause, resume**, delete a dev box.
- **Templates**: predefined images (OS + toolchain) users instantiate from; org-custom templates.
- A dev box bundles: **OS image, language/toolchain deps, repo checkout, user home dir, secrets, allocated CPU/mem/storage, optional GPU**.
- **Access** via browser (web IDE), **SSH**, and **IDE plugins** (VS Code Remote, JetBrains Gateway).
- **Orgs & teams**, **quota enforcement**, **RBAC**, **audit logs**, **cost controls**.
- Resize / reconfigure resources; snapshot/restore.

Non-functional (the stated optimization targets)

- **Fast provisioning** — warm start in **seconds**, cold create in well under a minute.
- **High availability** — control plane HA; a host failure must not lose a developer's data.
- **Scalability** — tens of thousands of **concurrent** boxes, thousands of provisions/min at peak.
- **Strong isolation** — VM-grade boundary between tenants.
- **Reliable persistence** — durable home dir / workspace; survives pause/stop/host loss.
- **Operational observability** — metrics, logs, traces, audit, cost attribution per box/team.

Out of scope (name then park)

Source-control hosting itself, CI/CD pipelines, the IDE's editing features. We integrate with Git providers and IDEs; we don't rebuild them.

Clarifying questions to ask the interviewer

- **Disposable vs persistent** — both? (Yes — ephemeral PR-review boxes *and* long-lived personal boxes.)
- **Isolation bar** — multi-tenant on shared hosts (need microVMs) or dedicated per-org hosts?
- **Pause semantics** — preserve **RAM state** (true hibernate/resume) or just stop the compute and keep disk?
- **GPU** — required? (Affects scheduling, pools, cost.)
- **Cloud** — single cloud, multi-cloud, or on-prem (Coder-style)? Affects the infra abstraction.
- **Max idle / cost policy** — auto-stop after N minutes idle? Hard org budgets?
- **Self-hosted runners vs our fleet** — do we run the compute or does the customer (BYO-cloud)?

---

3. Back-of-the-envelope estimation

Assume a large enterprise/SaaS scale:

Quantity	Assumption	Result
Developers (MAU)	—	1,000,000
Concurrent active boxes	~5% online at peak	**~50,000 concurrent**
Provisions/day	each dev starts ~2×/day	~~2M/day ≈ **~~25/sec avg, ~200–500/sec peak**
Avg box size	4 vCPU, 8–16 GB RAM	50k × 4 = **~200k vCPU**, ~600 TB RAM live
Host packing	64 vCPU / 256 GB hosts, ~8 boxes/host	**~6,000–8,000 hosts** at peak
Workspace storage	~20 GB/box, 2M total boxes	**~40 PB** persistent volumes (tiered)
Image/prebuild cache	hundreds of templates × layers	TBs of cached images per region/node
Idle ratio	most boxes idle most of the time	**auto-pause is the #1 cost lever**

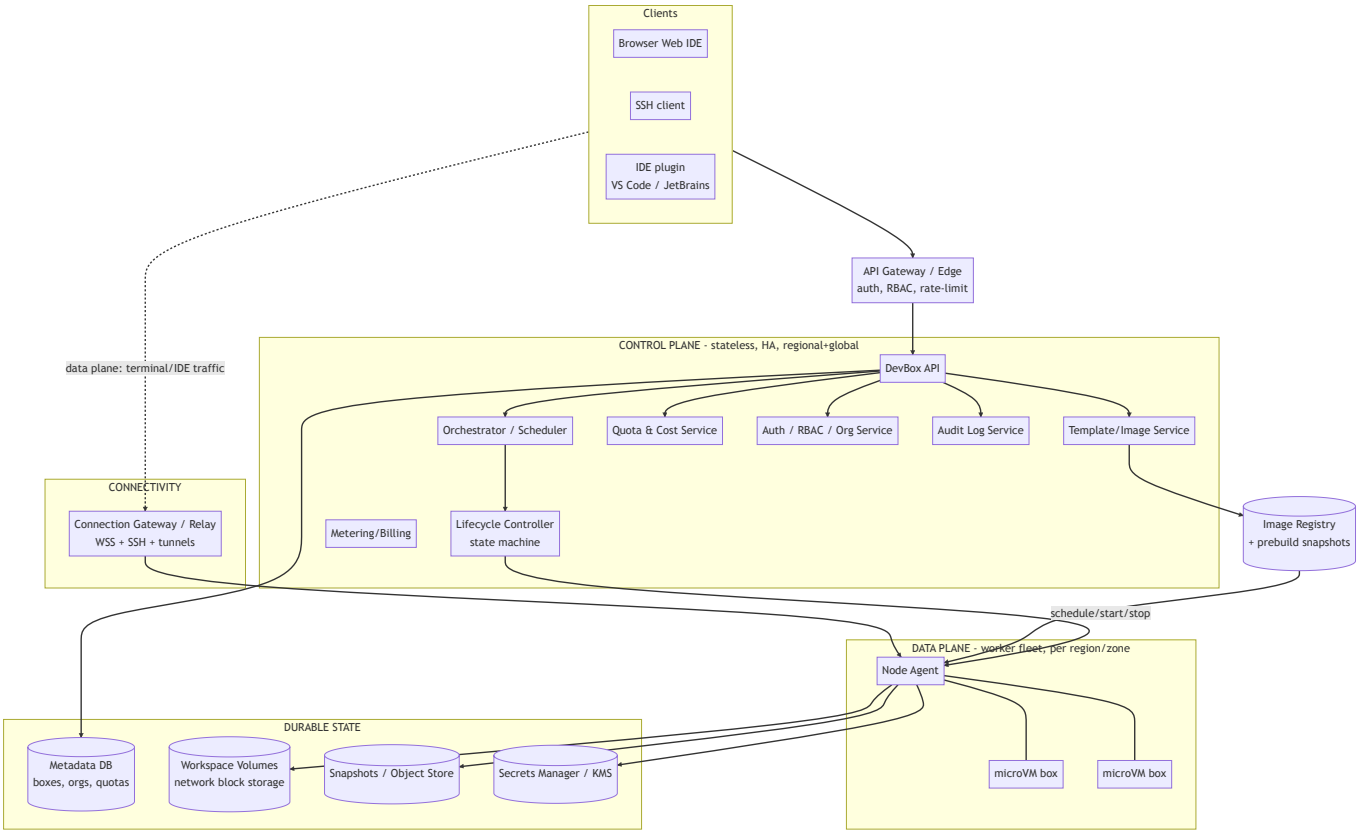
Takeaways that drive the design:

1. **Hundreds of provisions/sec at peak** → cold-build is too slow → **warm pools + prebuilt snapshots**.
2. **600 TB live RAM, 40 PB disk** → separate **compute (ephemeral)** from **storage (durable)**; never co-fate them.
3. **Most boxes idle** → aggressive **auto-pause/hibernate** + bin-packing reclaims huge cost.
4. **Many tenants per host** → **microVM isolation** is non-negotiable.

---

4. Architecture overview — control plane vs data plane

The single most important cut: a **stateless, globally-aware control plane** that orchestrates, and a **regional data plane** of worker hosts that actually run boxes.



Why this split?

- **Control plane is stateless & HA** — it holds *desired state* in a metadata DB and reconciles the fleet (Kubernetes-operator pattern). It can restart/scale without touching running boxes.
- **Data plane runs the workloads** — worker hosts with a **node agent** that launches microVMs, mounts volumes, fetches images, and reports health.
- **Data-plane traffic bypasses the control plane** — terminal/IDE bytes flow client ↔ **connection gateway** ↔ box, so user latency doesn't depend on (and isn't bottlenecked by) the orchestrator.

Core components

- **DevBox API** — CRUD + lifecycle commands; validates against quota/RBAC.
- **Orchestrator/Scheduler** — places boxes on hosts (bin-packing, GPU/zone affinity), manages **warm pools** and node autoscaling.
- **Lifecycle Controller** — drives the box **state machine**; reconciles desired vs actual.
- **Template/Image Service** — builds/stores templates, **prebuilds**, and snapshots.
- **Node Agent** — per-host daemon: start/stop microVMs, attach storage, stream health/metrics, enforce cgroups.
- **Connection Gateway/Relay** — authenticated tunnel terminating browser WSS, SSH, and IDE connections; routes to the right box (even behind NAT).
- **Quota/Cost, RBAC/Org, Audit, Metering/Billing** — the multi-tenant control surfaces.

---

5. Data model & storage choices (polyglot persistence)

Principle: separate compute from state. The box's *compute* is cattle (recreatable); its *workspace* is the pet that must persist.

Data	Store	Why
Boxes, orgs, teams, templates, quotas	Relational (Postgres, HA)	Strong consistency, transactions, relational RBAC/quota
Box runtime state / placement	Postgres + in-mem cache (or etcd)	Fast reconcile loop; source of truth for desired state
Workspace / home dir	**Network block volume** (EBS-like) per box	Durable, detach/reattach across hosts, snapshot-able
Base images & prebuilds	OCI **image registry** + content-addressed layer cache on nodes	Fast pulls, dedupe layers, cache near compute
Snapshots (disk ± memory)	Object store (S3/GCS)	Cheap, durable, for pause/resume & fast clone
Secrets	Secrets manager + **KMS envelope encryption**	Never in images or the DB in plaintext; short-lived injection
Audit logs	Append-only log → immutable store / SIEM	Tamper-evident, queryable, retention
Metrics/Logs/Traces	TSDB (Prometheus) + log store (Loki/ELK) + tracing	Observability + cost metering

Box record (metadata)

```
box {
  id, org_id, team_id, owner_id,
```

```
  template_id, state,               // CREATING/RUNNING/STOPPED/PAUSED/...
  resources { vcpu, mem_gb, disk_gb, gpu },
  host_id (nullable when not running),
  volume_id, snapshot_id,
  repo { url, ref, checkout_path },
  created_at, last_active_at, auto_stop_after, cost_to_date
}
```

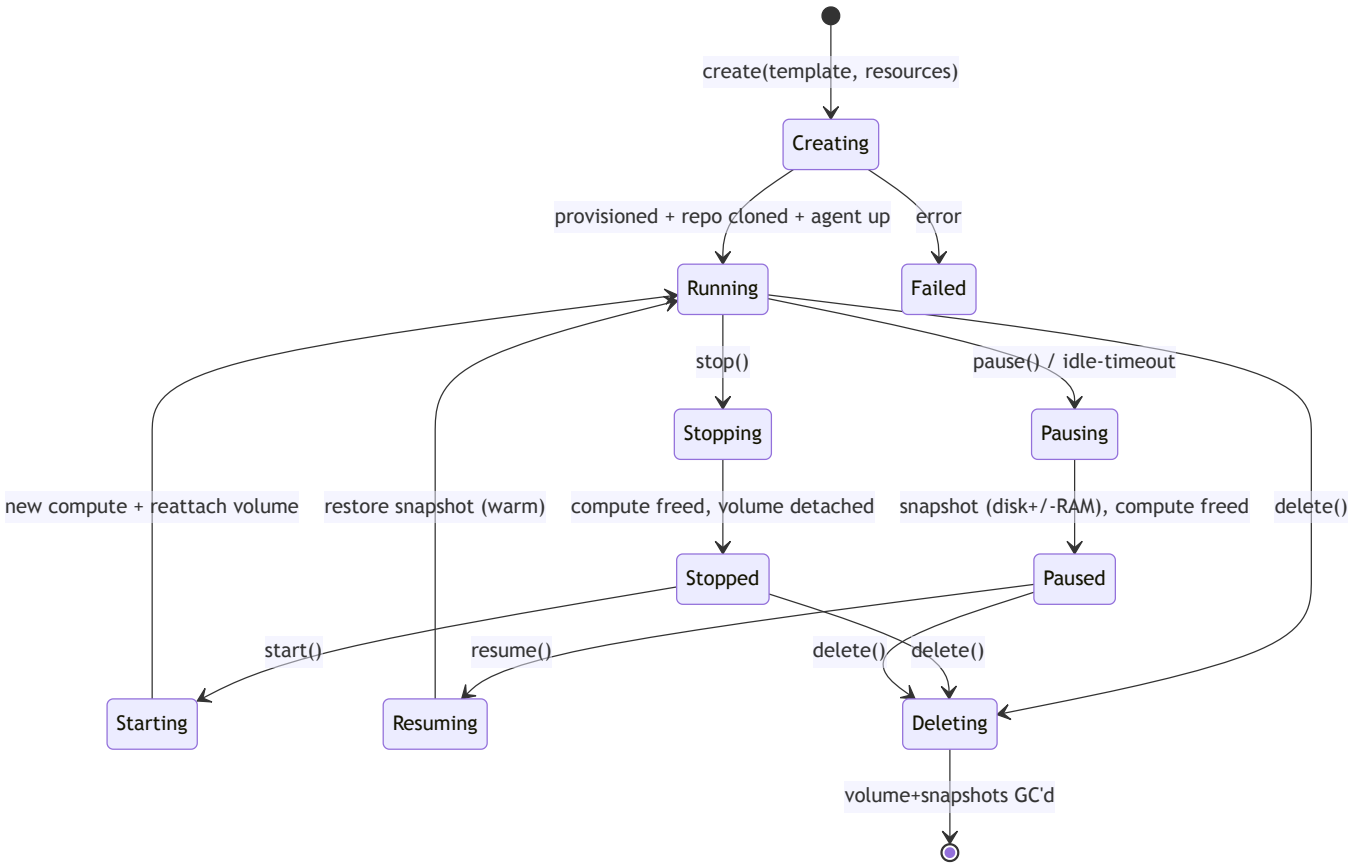
Workspace volume layout (what persists)

```
/home/<user>      ← user home: dotfiles, shell history, configs   (PERSIST)
/workspace        ← repo checkout + uncommitted work               (PERSIST)
/var, /usr, OS    ← from base image, reconstructable               (EPHEMERAL)
/tmp, build cache ← scratch                                        (EPHEMERAL, maybe cached)
secrets           ← injected at runtime via KMS, not on disk       (NEVER persisted in clear)
```

The durable volume holds `/home` + `/workspace`. The OS/toolchain come from the (immutable, cached) base image. This separation is what makes **stop = detach volume + kill compute** and **start = new compute + reattach the same volume** safe and fast.

---

6. Lifecycle — the box state machine

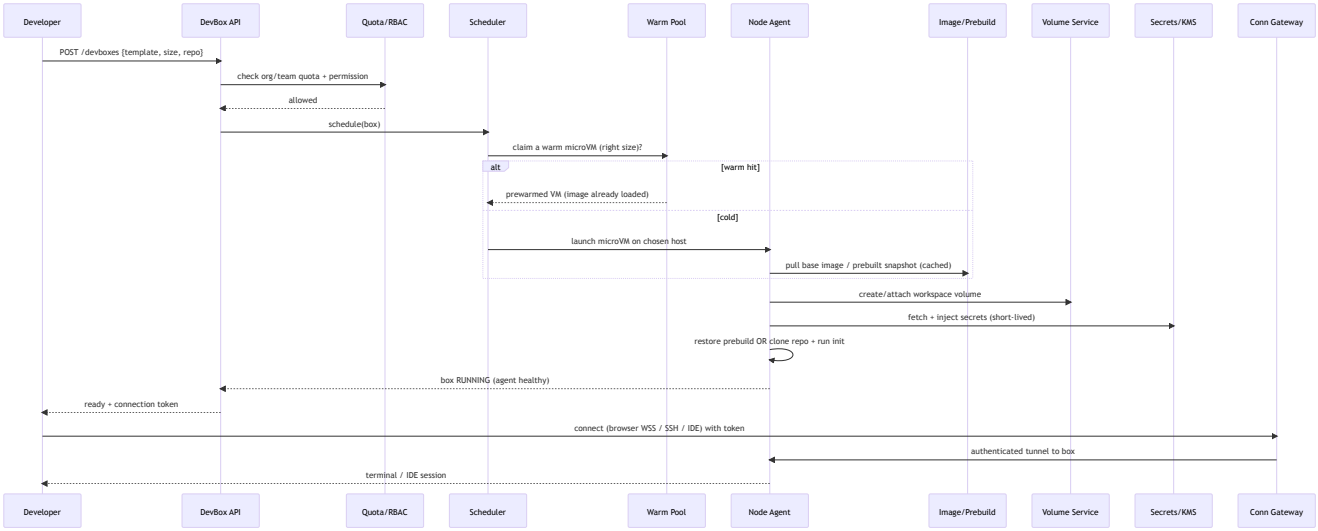


Action	Compute	Workspace volume	RAM state	Cost	Resume speed
Stop	Freed	**Detached, kept**	Lost	Storage only	Seconds (boot + reattach)
Pause	Freed	Kept (or snapshotted)	**Snapshotted**	Storage only	**Sub-second–seconds** (restore)
Resume	New host	Reattach	Restored	Resumes compute	Fast
Delete	Freed	**GC'd**	—	Stops	n/a

- **Stop** = graceful: flush disk, **detach** the network volume, release the host. Cheap to keep around.
- **Pause/hibernate** = snapshot the running state (disk and optionally **memory**), so resume drops you back exactly where you left off (open files, running servers). Memory snapshots make resume feel instant but cost more storage.
- **Idle auto-pause** is driven by the node agent watching CPU/tty/IDE activity → the biggest cost saver.

---

7. Data flow: create → provision → connect



The provisioning critical path (and how we shrink each step):

1. **Schedule placement** — pick a host (bin-pack, zone/GPU affinity) → ms.
2. **Get compute** — **warm pool** hit = already-booted microVM → ~0; cold = launch microVM (Firecracker boots in ~125 ms).
3. **Get the filesystem** — **prebuilt snapshot** of the template (deps already installed) instead of `apt/npm install` at runtime → seconds vs minutes.
4. **Get the code** — repo already in the prebuild, or a fast shallow clone; cache the git mirror near the region.
5. **Inject secrets** — pulled from KMS at boot, mounted in-memory (tmpfs), never baked into the image.
6. **Signal ready + connect** — hand back a connection token; traffic flows through the gateway.

---

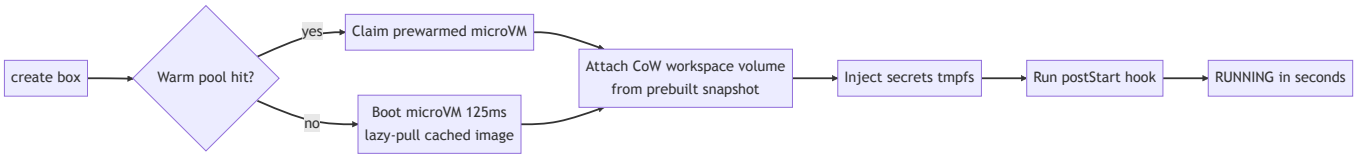
8. Fast provisioning (deep dive — a stated goal)

Cold "boot OS → install toolchain → clone repo → npm install" = minutes. Users expect **seconds**. Layered techniques:

1. **Prebuilds (the big one).** Build the template *ahead of time* — OS + toolchain + dependencies + repo + `postCreate` hooks — and store the result as a **snapshot/image**. Provisioning becomes *restore a snapshot*, not *run a build*. Rebuild prebuilds on template/branch changes (CI-style), so the common case is always warm.
2. **Warm pools.** Keep a buffer of **already-booted** generic microVMs per popular size/template. A create *claims* one and personalizes it (attach volume, inject secrets) instead of booting cold. The orchestrator autoscaler keeps the pool sized

to demand (predictive: weekday-morning surge).

3. **microVM fast boot. Firecracker** microVMs boot in ~125 ms with a minimal kernel; pair with **snapshot/restore** to resume a pre-initialized VM state directly.
4. **Image/layer caching on nodes.** Content-addressed OCI layers cached on each host (and a regional pull-through cache) → near-zero pull for shared bases. **Lazy-pull** image layers (stream blocks on first access) so the box starts before the whole image is local.
5. **Copy-on-write storage.** Clone a box/template volume via **CoW snapshots** (overlay/ZFS/btrfs/EBS fast-snapshot) — instant "copy" of a 20 GB workspace without copying bytes.
6. **Local git mirror/cache.** Maintain a regional mirror of popular repos; shallow/partial clone (`--depth=1`, sparse checkout) to minimize fetch.



One-liner: *"I make the common path a **restore, not a build** — prebuilt template snapshots + a warm pool of Firecracker microVMs + copy-on-write volumes turn minutes of `npm install` into a couple of seconds."*

---

9. Strong isolation (deep dive — a stated goal)

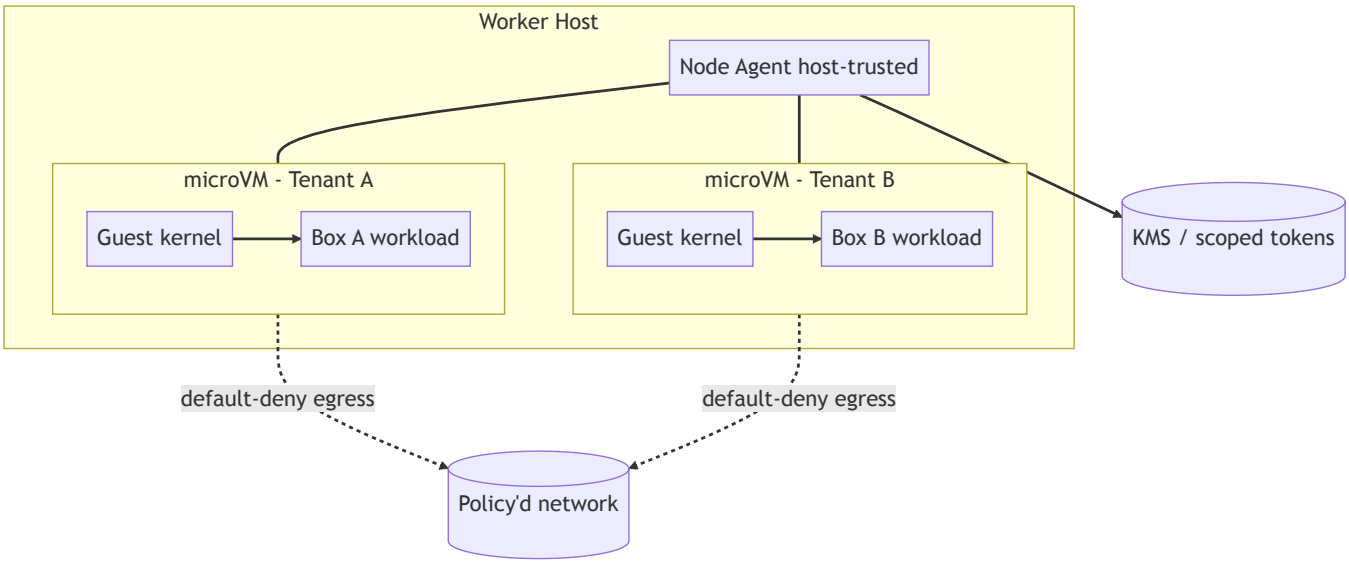
We run **untrusted code from many tenants on shared hosts**. Containers alone share the host kernel → a kernel exploit escapes to other tenants. Options:

Isolation	Boundary	Startup	Density	Security	Use
Plain containers (namespaces/cgroups)	Shared kernel	~ms	Highest	**Weak** for untrusted multi-tenant	Trusted workloads only
gVisor (userspace kernel)	Syscall interception	~ms	High	Medium-strong	Good middle ground
Kata / microVM (Firecracker) ✅	**Own guest kernel + VM**	~125 ms	High	**Strong (VM-grade)**	**Multi-tenant DevBoxes**
Full VM	Hypervisor	~10s+	Low	Strong	GPU/heavy, dedicated

Recommendation: microVMs (Firecracker/Kata). Each box gets its **own kernel** inside a lightweight VM — VM-grade isolation with near-container speed and density. This is what Codespaces/Fly/AWS Lambda use for exactly this reason.

Defense in depth around the microVM:

- **Per-box network isolation** — its own veth/VLAN/overlay; **default-deny egress** with allowlists (block metadata endpoints, SSRF to cloud IAM, lateral movement). Boxes can't see each other.
- **Resource caps** — cgroups/VM limits on CPU, memory, **disk + network IO** to stop noisy-neighbor and fork-bomb DoS.
- **No host credentials in the guest** — the node agent (host side) handles cloud APIs; the box only gets **scoped, short-lived** tokens.
- **Seccomp/least-privilege** on the agent; read-only base image; secrets in tmpfs, wiped on stop.
- **Per-tenant encryption** — volumes and snapshots encrypted with tenant-scoped KMS keys.



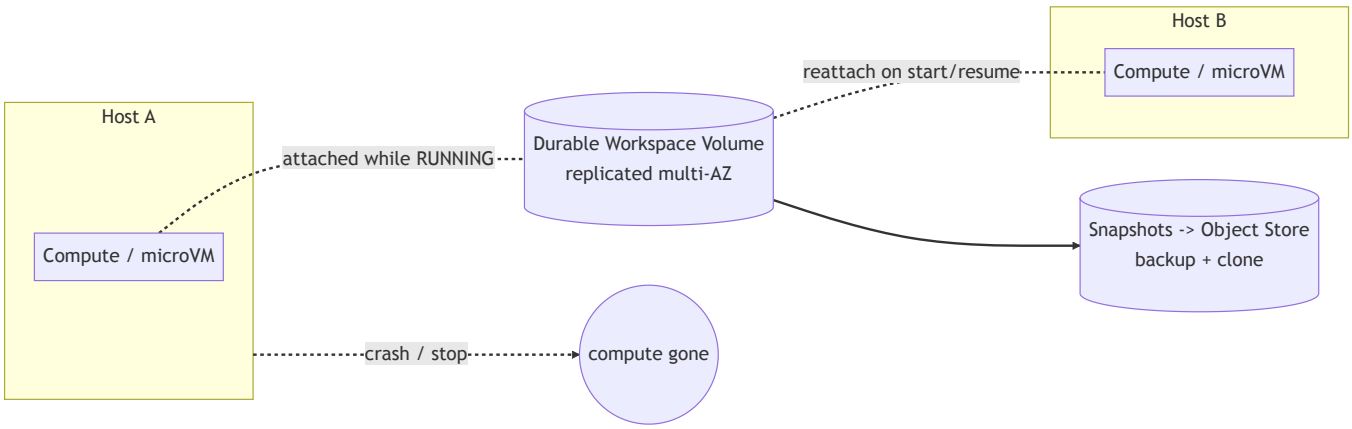
One-liner: *"Each box is a Firecracker microVM with its **own kernel**, default-deny egress, cgroup IO caps, and only short-lived scoped tokens — so a `sudo rm -rf` or a kernel exploit stays inside one disposable VM."*

---

10. Reliable persistence (deep dive — a stated goal)

A developer's uncommitted work **must not vanish** on stop, pause, host crash, or rescheduling.

- **Decouple storage from compute.** The workspace lives on a **network block volume** (EBS-like), independent of the host. Compute is disposable; the volume **detaches on stop and reattaches on start**, possibly on a *different* host. Host dies → reattach the volume elsewhere; no data loss.
- **Persist the right paths** — `/home` + `/workspace` on the durable volume; OS/toolchain come from the immutable image (reconstructable).
- **Snapshots** for pause/hibernate, fast cloning, and point-in-time backup → object store. Memory snapshot (optional) for instant resume.
- **Durability of the volume itself** — replicated block storage (multi-AZ) so a single disk/AZ failure doesn't lose data; periodic snapshot backups for DR.
- **Crash consistency** — `fsync`/journaling + filesystem-consistent snapshots; flush on graceful stop. Warn users uncommitted work isn't a substitute for `git push`.
- **Garbage collection** — on delete, GC volume + snapshots after a grace/retention window (undo-delete window). Reap orphaned volumes from failed creates.

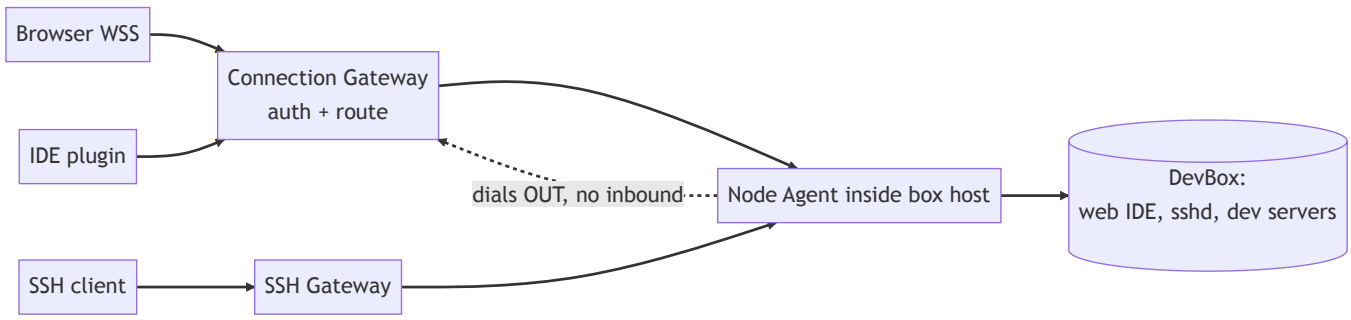


One-liner: *"Compute is cattle, the workspace is a pet — the home dir and repo live on a replicated network volume that detaches and reattaches across hosts, so a host crash costs you a reschedule, not your code."*

11. Connectivity — browser, SSH, IDE (the data plane)

How does a user reach a box that's on some random host, possibly behind NAT, securely?

- **Connection Gateway / Relay** — a globally reachable, authenticated entry point. The **node agent dials *out*** to the gateway and holds a persistent tunnel (WebSocket/QUIC), so boxes need **no public IP / inbound ports** (NAT-friendly, smaller attack surface).
- **Browser (web IDE)** — HTTPS/WSS to the gateway → routed to the box's agent → web IDE server (e.g., code-server) and terminal over a multiplexed channel.
- **SSH** — an **SSH gateway/proxy** terminates auth (short-lived certs / keys mapped to identity) and proxies to the box. Users get `ssh box-name@platform`.
- **IDE plugins** — VS Code Remote / JetBrains Gateway run a **remote agent inside the box** and tunnel through the same relay; the local IDE is just a thin client.
- **Port forwarding** — expose a dev server port via a per-box subdomain `https://3000-box-id.platform.dev`, auth-gated by RBAC.
- **Identity-aware** — every connection is authenticated and authorized (Zero-Trust); the gateway checks RBAC before bridging, and all sessions are audited.



12. Multi-tenancy: orgs, RBAC, quota, audit, cost

Org / team / RBAC

- Hierarchy: **Org** → **Teams** → **Users**, with **roles** (Owner, Admin, Member, Viewer) and **permissions** (create box, use template, manage quota, view others' boxes).
- Enforce at the API gateway on **every** lifecycle call. RBAC also gates **connection** (can this identity attach to this box / forwarded port?).

Quota enforcement

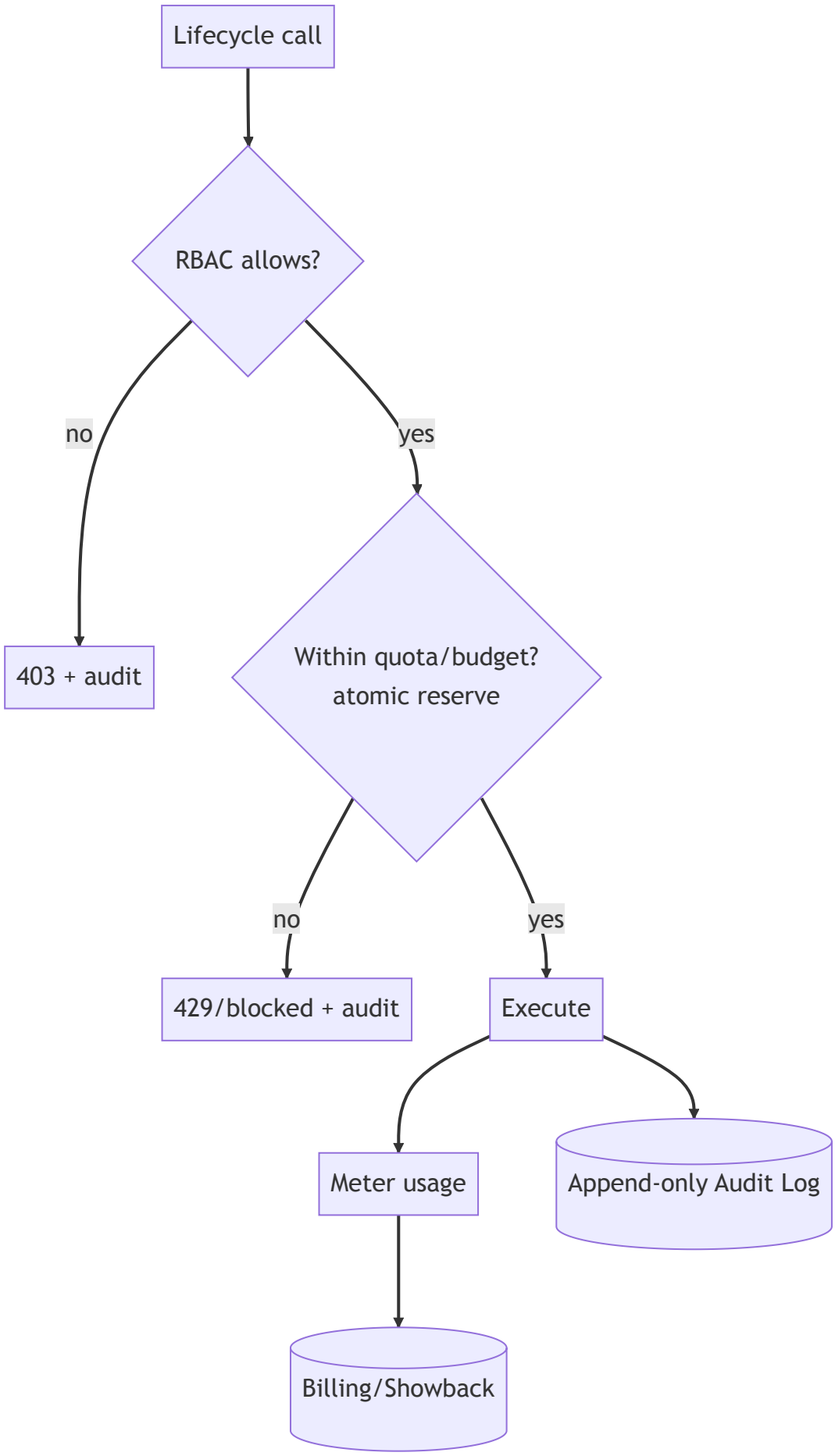
- Limits per org/team/user: **max concurrent boxes, total vCPU/RAM/GPU, storage GB, monthly spend**.
- Checked **transactionally at create/start/resize** (reserve → commit) so two concurrent creates can't both pass the last slot (the read-modify-write race — use atomic reserve).
- Soft (warn) vs hard (block) limits; templates can pin allowed sizes per tier.

Audit logs

- **Append-only, immutable** record of who did what: create/start/stop/delete, **connect/SSH sessions**, secret access, role changes, quota overrides. Tamper-evident, exported to SIEM, with retention. Essential for security and compliance.

Cost controls (huge given idle ratios)

- **Auto-stop/auto-pause** idle boxes after N minutes (agent-detected inactivity) — the #1 lever.
- **Auto-delete** disposable boxes after TTL; expire stopped boxes after a retention window.
- **Budgets & alerts** per org/team; **metering** every box's vCPU·hr, GB·hr, GPU·hr → showback/chargeback and billing.
- **Right-sizing** suggestions; **bin-packing** + node autoscaler scales the fleet down off-hours; **spot/preemptible** hosts for disposable boxes (with checkpointing).


Lifecycle call
RBAC allows?
no
yes
403 + audit
Within quota/budget?
atomic reserve
no
yes
429/blocked + audit
Execute
Meter usage
Append-only Audit Log
Billing/Showback

13. Observability (a stated goal)

- **Metrics** — control-plane (provision latency p50/p99, success rate, warm-pool hit rate, scheduler queue) and per-box (CPU/mem/disk/net, idle state) in a TSDB; SLOs + alerting.
- **Logs** — structured agent/box/control-plane logs centralized; per-tenant scoping.
- **Tracing** — distributed trace of the **provisioning path** (API → scheduler → agent → ready) to find the slow step.
- **Health & self-healing** — agent heartbeats; the lifecycle controller reconciles (restart crashed boxes, reschedule off dead hosts, reap orphans).
- **Cost observability** — usage metering doubles as the billing/showback signal.
- **Golden signal**: **"time-to-ready"** (create→connected) is the product's headline SLO.

---

14. Failure scenarios — *"what if X fails?"*

Failure	Impact	Mitigation
Worker host crashes	Running boxes on it die	Volume is network-attached → **reschedule** box on a healthy host + reattach; RAM-only state lost (recover from last snapshot)
Control plane down	Can't create/manage boxes	CP is stateless+HA (multi-replica, multi-AZ); **running boxes keep running** (data-plane independent)
Connection gateway down	Can't reach boxes	Multiple gateways, health-checked, client reconnect; boxes redial
Image registry slow/down	Cold provisions stall	Node-local + regional cache; warm pool absorbs; lazy-pull
Volume/storage AZ failure	Risk to persistence	Multi-AZ replicated volumes + snapshot backups (DR restore)
Quota race (double-spend)	Over-allocation	**Atomic reserve** (transactional/conditional) at create/start
Noisy neighbor / fork bomb	Host degraded	cgroup CPU/mem/IO caps per microVM; per-box network limits
Tenant escape attempt	Cross-tenant breach	microVM (own kernel) + default-deny egress + no host creds in guest
Provisioning storm (Mon 9am)	Latency spike	Predictive warm-pool scaling + node autoscaler + admission queue
Stuck/orphaned resources	Cost leak	Reconcile loop GCs orphans; idle auto-stop; TTLs

Guiding principle: the **data plane must outlive the control plane**, and **storage must outlive compute**. A control-plane outage should never disconnect a coding developer or lose their work.

---

15. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Isolation vs density/speed	Full VM (strong, slow, low density)	Container (fast, dense, weak)	**microVM** (Firecracker) — VM-grade isolation at near-container speed ✅
Provision speed vs cost	Big warm pool (instant)	No pool (cheap, slow)	Pool sized to predicted demand; CoW + prebuilds get speed without huge idle cost
Pause: disk-only vs disk+RAM	Stop (cheap, slower resume)	Hibernate (instant resume, more storage)	Offer both; default stop, hibernate for premium/active boxes
Persistence vs cost	Keep all volumes forever	Aggressive GC	Retention window + snapshot-to-object-store for cold boxes
Multi-tenant hosts vs dedicated	Shared (cheap, dense)	Dedicated per org (isolation, cost)	Shared + microVM by default; dedicated for high-security tiers
Spot vs on-demand hosts	Spot (cheap, preemptible)	On-demand (stable)	Spot for **disposable** boxes w/ checkpoint; on-demand for persistent
Consistency vs availability	Strong CP everywhere	Eventual reconcile	Strong for quota/RBAC/billing; eventual reconcile for fleet state

CAP/PACELC framing: the **metadata/quota/RBAC** store is **CP** (must be correct — no double-spend, no privilege bypass). The **fleet reconciliation** is **eventually consistent** (controllers converge desired→actual). The **data plane** is designed to **stay available** even when the control plane is partitioned away.

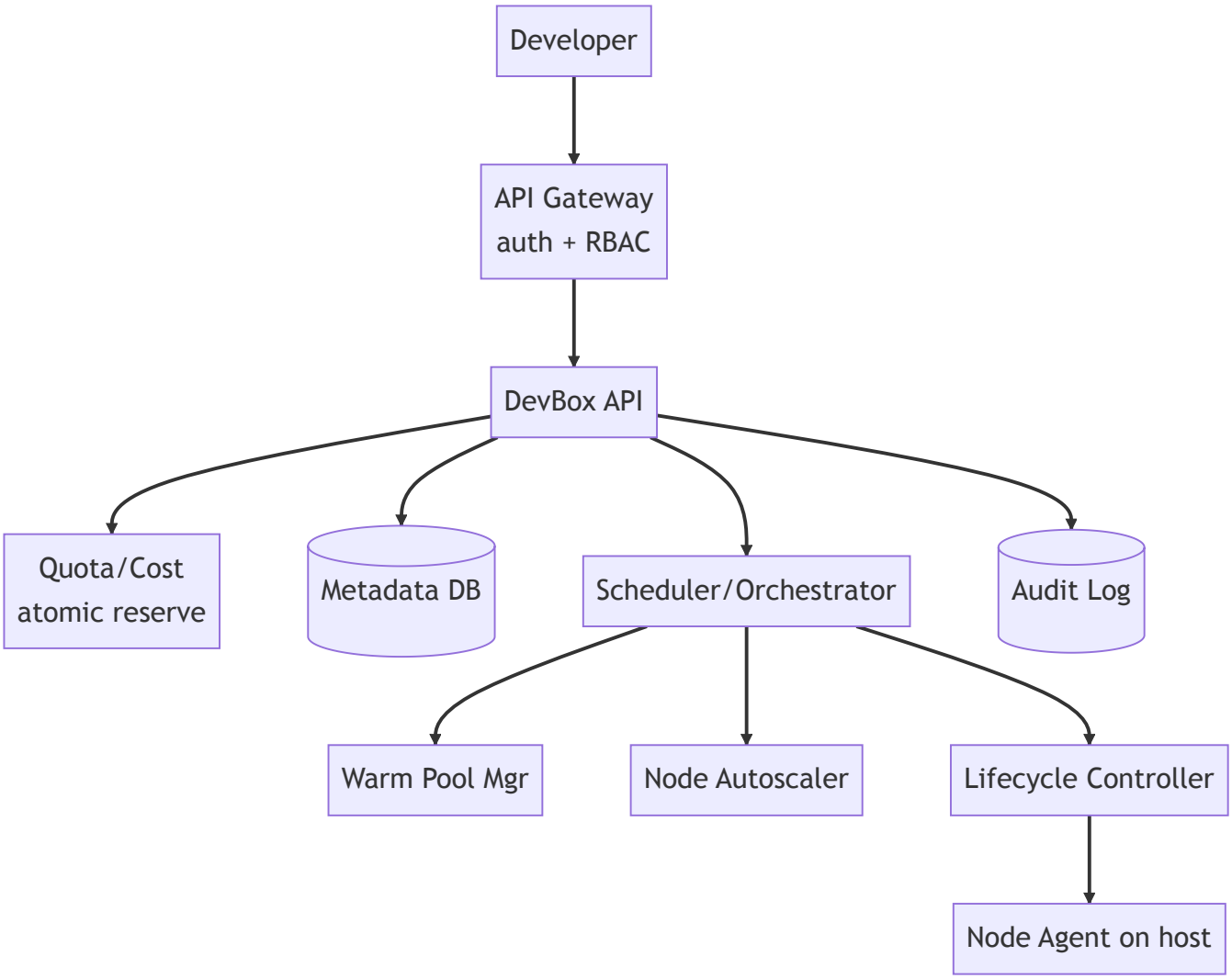
One-liner to say out loud: *"I'd split a **stateless HA control plane** (API, scheduler, lifecycle controller, quota/RBAC/audit on Postgres) from a **regional data plane** of worker hosts running each box as a **Firecracker microVM** for strong isolation. Fast provisioning comes from **prebuilt template snapshots + warm pools + copy-on-write volumes**, turning minutes into seconds. The **workspace lives on a replicated network volume** decoupled from compute, so stop/pause/host-failure never lose work — compute is cattle, the volume is the pet. Users connect through an **outbound-dialed connection gateway** (browser/SSH/IDE) so boxes need no inbound ports. Quotas are enforced with **atomic reserves**, everything is **audited append-only**, and **idle auto-pause + bin-packing** controls cost."*

---

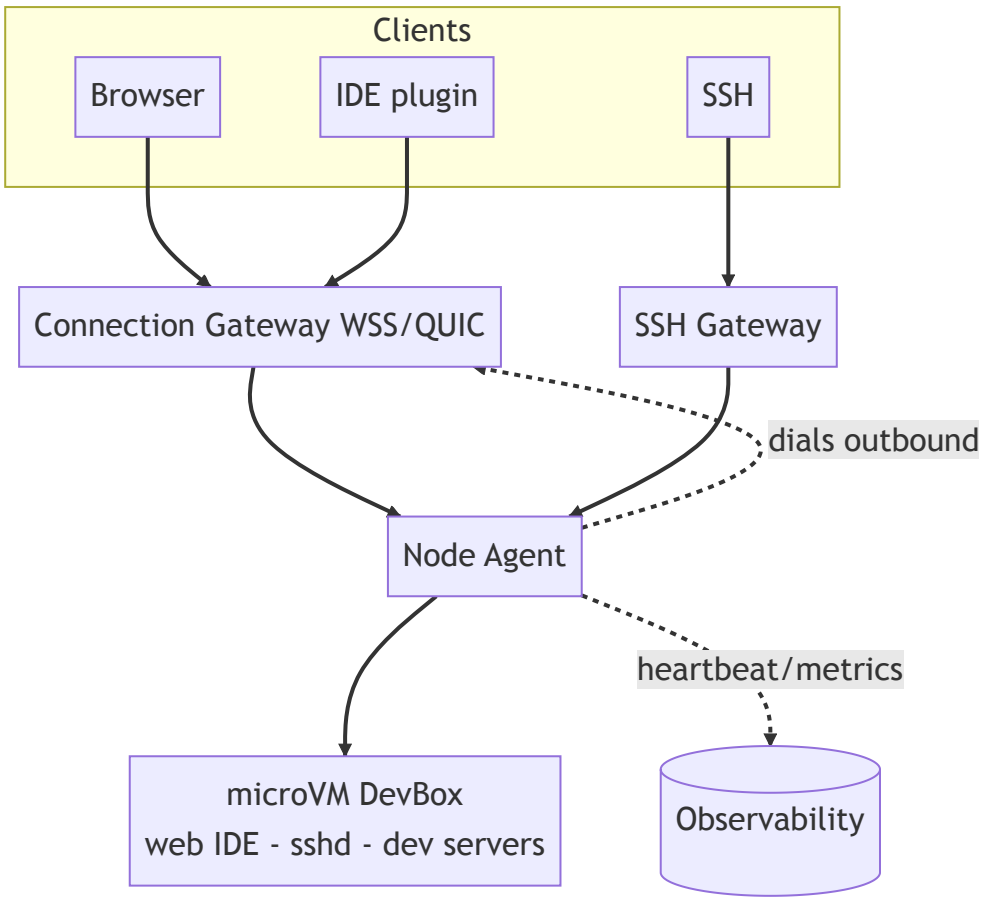
16. Full system design (detailed)

End-to-end, split into **(A)** the provisioning/control path, **(B)** the running/connect data path, and **(C)** the persistence & lifecycle path.

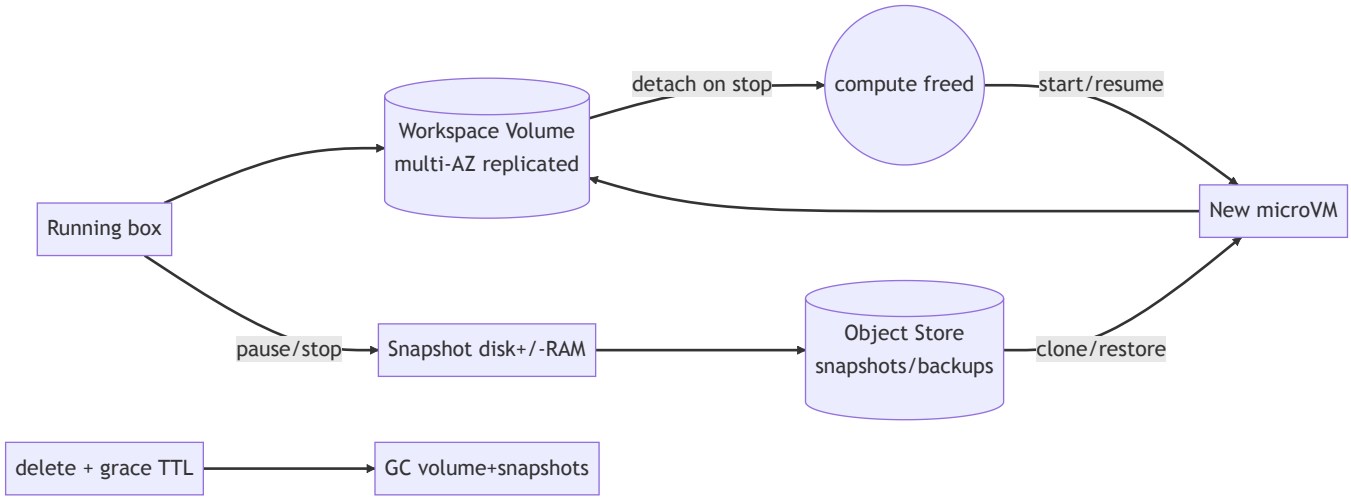
16A. Control path — create & schedule



16B. Data path — running box & connectivity



16C. Persistence & lifecycle path



---

17. Networking, security & performance best practices

Networking

- **Outbound-dialed connection gateway** — boxes hold a tunnel out; **no inbound ports** (NAT-friendly, smaller attack surface).
- **WSS/QUIC** for browser/IDE traffic; **SSH via a proxy** with short-lived certs mapped to identity.

- **Per-box network namespace/overlay**; **default-deny egress** + allowlist; **block the cloud metadata endpoint** (SSRF/IAM-credential theft).
- **Co-locate** compute with the workspace volume's AZ to minimize attach latency; per-box auth-gated subdomain port-forwarding.

Security

- **microVM per box** (own kernel); **no host/cloud creds in the guest** — the agent holds them and hands out short-lived scoped tokens.
- **Secrets from KMS** injected to tmpfs, wiped on stop; **per-tenant volume/snapshot encryption**.
- **Identity-aware / Zero-Trust gateway** — RBAC checked before bridging; **all sessions audited**.
- **Read-only base image**, **signed images** (cosign/provenance), seccomp + least-privilege agent.
- **cgroup CPU/mem/IO caps** to contain noisy neighbors and abuse (crypto-mining).

Performance

- **Prebuilt snapshots + warm pools + CoW volumes + lazy image pull** → seconds to ready ("restore, not build").
- **Firecracker snapshot/restore** for instant resume; node-local + regional **image layer cache**.
- **Idle auto-pause + bin-packing** for cost/perf; right-size suggestions.

---

18. Staying current — modern & emerging approaches

- **Isolation: Firecracker** / **Kata Containers** / Cloud Hypervisor microVMs; **gVisor** for syscall-level isolation.
- **Reference systems:** GitHub **Codespaces** (devcontainers), **Gitpod**, **Coder**, AWS (Lambda/Fargate on Firecracker).
- **Env as code:** `devcontainer.json` (Dev Container spec), **Nix/Nixpkgs** for reproducible toolchains.
- **Networking:** WireGuard/**Tailscale**-style mesh, **Cloudflare Tunnel**, relay-based reverse tunnels.
- **Storage:** CSI drivers, EBS multi-attach / fast snapshots, **ZFS/btrfs** copy-on-write.
- **IDE protocols:** VS Code Remote, JetBrains Gateway, SSH; browser IDE via code-server.
- **How I stay current:** Firecracker/Kata release notes, KubeCon talks, Codespaces/Gitpod engineering blogs.

---

19. Likely follow-up questions (rehearse these)

- microVM vs container vs gVisor — why microVM for untrusted code? *(own kernel = VM-grade isolation)*
- Host dies mid-session — does the user lose work? *(no — workspace is a network volume, reattach elsewhere)*
- How do you get sub-second provisioning? *(prebuilt snapshot + warm pool + CoW volume)*
- Pause vs stop — difference? *(pause snapshots RAM for instant resume; stop is disk-only)*
- How does a browser reach a box behind NAT? *(box dials out to the gateway; no inbound)*
- Stop a tenant stealing node IAM creds? *(block metadata endpoint; no host creds in guest)*
- Quota double-spend on concurrent creates? *(atomic/transactional reserve)*
- Cost control for idle boxes? *(idle auto-pause, TTL auto-delete, spot for disposable)*

---

20. Summary checklist (whiteboard recap)

- **Control plane vs data plane** — stateless HA orchestrator + regional worker fleet; data plane outlives the control plane.
- **microVM isolation** (Firecracker) — own kernel per box, default-deny egress, no host creds in guest, IO caps.
- **Fast provisioning** — prebuilt snapshots + warm pools + CoW volumes + lazy image pull = seconds, not minutes ("restore, not build").
- **Reliable persistence** — workspace on a **replicated network volume** decoupled from compute; snapshots for pause/clone/DR; compute=cattle, volume=pet.

- **Lifecycle state machine** — stop (detach volume), pause (snapshot RAM), resume (restore), delete (GC).
- **Connectivity** — outbound-dialed gateway for browser/SSH/IDE; no inbound ports; identity-aware + audited.
- **Multi-tenancy** — Org/Team/RBAC, **atomic quota reserves**, append-only **audit**, metering → billing.
- **Cost controls** — idle auto-pause, TTL auto-delete, bin-packing + autoscaling, spot for disposable.
- **Observability** — provision-latency/time-to-ready SLO, per-box metrics, tracing the provisioning path, self-healing reconcile.
- **CAP** — CP for quota/RBAC/billing; eventual reconcile for fleet; available data plane under CP partition.

Designing an Instagram-like Photo/Video Sharing App (Feed-Focused)

> A complete, interview-ready walkthrough: requirements → estimates → APIs → data model → data flow → **feed generation (push/pull/hybrid)** → celebrities → read scaling → failure handling → trade-offs. Use the headings as your whiteboard agenda. The **feed** is the heart of this problem — spend most of your time there.

---

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements (don't assume).
2. **Estimate** scale (DAU, posts/sec, feed reads/sec, fan-out, storage) to justify choices.
3. **Define the APIs** and core components.
4. **Walk the data flow**: post creation → media pipeline → storage → feed build → delivery.
5. **Pick storage** per data type (posts, media, follow graph, feeds) and justify.
6. **Feed strategy**: push (fan-out-on-write) vs pull (fan-out-on-read) vs **hybrid** — trade-offs.
7. **Handle celebrities** (millions of followers) — the fan-out bomb.
8. **Scale reads**: caching, replicas, precomputation; avoid feed-read bottlenecks.
9. **Handle failure** and summarize **trade-offs**.

Keep saying *"here's the trade-off…"* — that's what's being graded.

---

1. Problem & motivation

Build a photo/short-video sharing service where users **post media**, **follow** other users, and consume a **home feed** of recent + relevant posts from accounts they follow.

What makes it hard:

- **Read-heavy**: feed reads dominate writes by ~100:1. Feeds must load in tens of ms.
- **Write fan-out**: one post by a popular account must reach millions of followers' feeds.
- **Media-heavy**: photos/videos need transcoding, multi-resolution storage, and global CDN delivery.
- **Skewed graph**: follower counts span from 10 to 100M+ (celebrities) — no single strategy fits all.
- **Relevance**: not just chronological — ranking drives engagement.

The central tension: **do work at write time (precompute feeds) or read time (assemble on demand)?** The answer is *both*, chosen per-account — that's the hybrid design.

---

2. Requirements

Functional

- Users **create posts** (1+ photos or a short video, caption, location, tags).
- Users **follow / unfollow** other users (asymmetric — like Twitter, unlike Facebook).
- **Home feed**: recent and relevant posts from followed accounts, paginated, infinite scroll.
- Users can **like / comment** (engagement signals feed ranking).

- View a **user profile** (their own posts, reverse-chronological).

Out of scope (state it, then park it)

Stories, DMs, Reels recommendation/discovery (non-followed content), search, ads. Mention them so the interviewer knows you know, then focus on the feed.

Non-functional

- **Low feed latency**: p99 feed load < ~200 ms.
- **High availability**: feed must always render (stale-but-available > unavailable).
- **Scalable**: 500M DAU, billions of feed reads/day, 100M+ posts/day.
- **Eventual consistency is OK**: a new post appearing in followers' feeds a few seconds late is fine. (No need for read-after-write on *others'* feeds.)
- **Durability**: never lose a user's uploaded media or post.

Clarifying questions to ask the interviewer

- **Chronological or ranked** feed? (Assume ranked, with recency as a strong signal.)
- **Follower distribution** — do we need to support celebrity accounts (100M+)? (Yes → hybrid.)
- Read-after-write for the **author's own** feed/profile? (Yes — they must see their post instantly.)
- **Media limits** — max video length, max photo size, supported formats?
- **Global** or single-region? (Global → CDN + regional replicas.)
- Do we need **edit/delete** of posts? (Yes — affects cache invalidation.)

---

3. Back-of-the-envelope estimation

Assume Instagram-ish scale:

Quantity	Assumption	Result
DAU	—	500M
Posts/day	~20% of DAU post, ~1 each	100M/day ≈ **~1.2K/sec** avg, **~5–10K/sec peak**
Feed reads/day	each DAU opens feed ~10–20×	~~5–10B/day ≈ **~~~~100K/sec** avg, **~500K–1M/sec peak**
Read : Write ratio	—	**~100 : 1** (read-optimize everything)
Avg followers	—	~200 (median far lower; mean skewed by celebs)
Fan-out writes	100M posts × 200 followers	**~20B feed-inserts/day ≈ ~230K/sec avg, ~1M/sec peak**
Media size	photo ~2 MB × multi-res ≈ 3 MB; video larger	100M × ~~3 MB ≈ **~~~~300 TB/day** ingested → ~**100 PB/year**
Follow graph	1B users × ~200 edges	**~200B edges** (~3–5 TB raw, more with indexes)
Precomputed feed cache	500M active × ~500 post-IDs × ~16 B	**~4 TB** in RAM (sharded Redis)

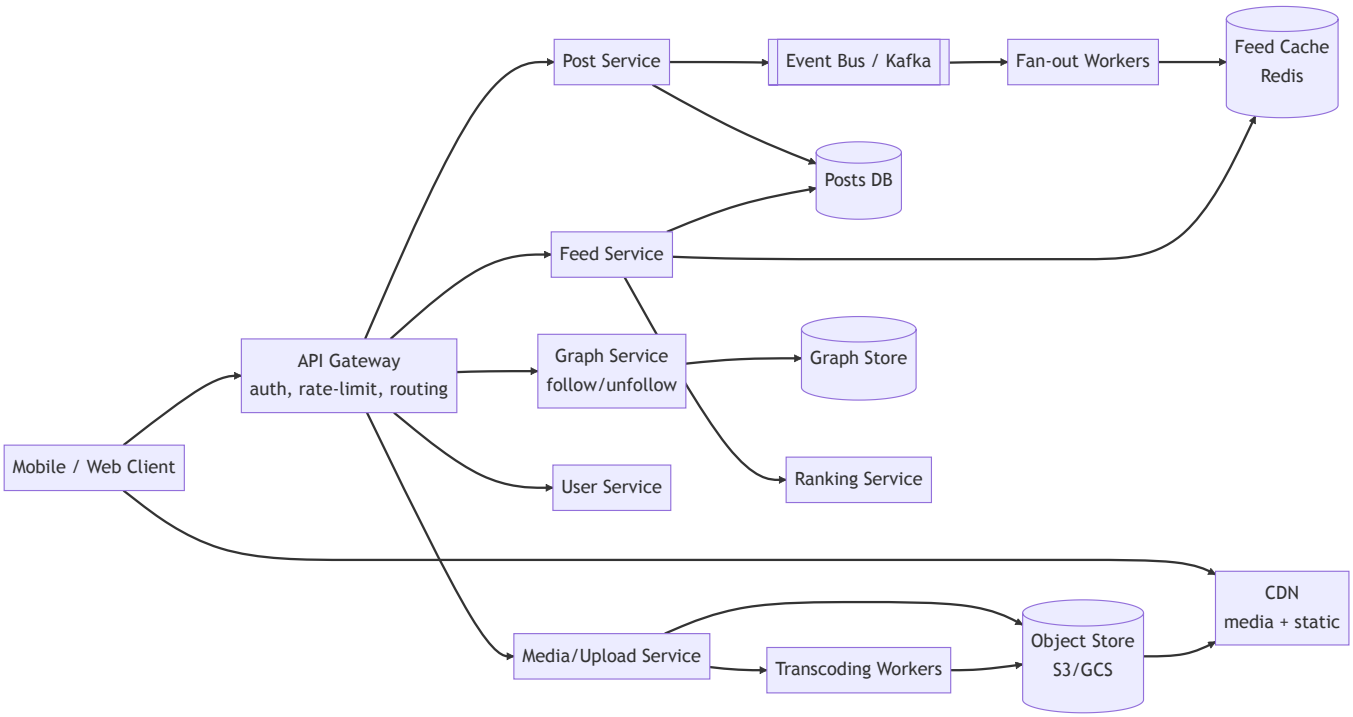
Takeaways that drive the design:

1. **100:1 read:write** → precompute and cache feeds aggressively.

2. **Fan-out peak ~1M/sec** with celebrity spikes of **100M inserts for a single post** → can't naively fan-out everyone → **hybrid**.
3. **300 TB/day media** → object storage + CDN, never in the DB.

---

4. Core components & APIs



Component responsibilities

- **API Gateway** — auth (OAuth/JWT), TLS, rate limiting, routing.
- **User Service** — profiles, auth, settings.
- **Graph Service** — follow/unfollow, "who follows X", "who does X follow"; serves the fan-out and pull paths.
- **Media/Upload Service** — issues pre-signed upload URLs, validates, triggers transcoding.
- **Transcoding Workers** — generate multiple resolutions/formats (thumbnail → 4K, HLS/DASH for video).
- **Post Service** — creates the post record, emits a `post.created` event.
- **Fan-out Workers** — consume `post.created`, write post-IDs into followers' feed caches (push path).
- **Feed Service** — assembles a user's home feed (merge precomputed + pulled celebrity posts → rank → paginate).
- **Ranking Service** — orders candidate posts by relevance.
- **Event Bus (Kafka)** — decouples writes from fan-out; absorbs spikes; enables replay.

Key APIs (REST-ish; gRPC internally)

```
# Create a post (media already uploaded → returns media IDs)
POST /v1/posts
{ "media_ids": ["m_123","m_124"], "caption": "sunset 🌅",
  "location": "SF", "tags": ["#nofilter"] }
→ 201 { "post_id": "p_987", "created_at": 1717612800 }

# Get home feed (cursor pagination — never OFFSET)
GET /v1/feed?limit=20&cursor=eyJ0cyI6MTcx...   # opaque cursor
→ 200 { "items": [ {post...}, ... ], "next_cursor": "eyJ0cyI6..." }
```

```
# Follow / unfollow
POST   /v1/users/{id}/follow
DELETE /v1/users/{id}/follow

# Request a pre-signed upload URL (client uploads media directly to object store)
POST /v1/media/upload-url
{ "type": "image/jpeg", "bytes": 2097152 }
→ 200 { "media_id":"m_123", "upload_url":"https://s3...&sig=...", "expires_in":600 }

# Profile feed (author's own posts, reverse-chronological)
GET /v1/users/{id}/posts?limit=20&cursor=...
```

Why cursor pagination, not `page/OFFSET`? Feeds are append-mostly and huge. `OFFSET` scans and skips rows (slow, O(N)); a **cursor** (e.g., last `(ranking_score, post_id)` seen) is O(1) to resume and stable as new posts arrive at the top.

---

5. Data model & storage choices

Principle: polyglot persistence — match each data shape to the right store.

Data	Store	Why
Posts (metadata)	Wide-column (Cassandra) or sharded SQL	Massive write volume, simple key access by `post_id` / `user_id`, time-ordered
Media blobs	Object store (S3/GCS) + **CDN**	Cheap, durable, infinitely scalable; **never** store blobs in a DB
Follow graph	Sharded KV / wide-column (or graph DB)	200B edges, need fast "followers of X" and "followees of X"
Feeds (precomputed)	**Redis** (in-memory)	Sub-ms reads, sorted by score, TTL/trim to bound size
Counters (likes/comments)	Redis + async durable rollup	High-frequency increments; eventual durability
Social/derived analytics	Data lake / OLAP (later)	Offline ranking model training

Posts table (Cassandra-style, partitioned by author for profile reads)

```
posts_by_user (
  user_id      partition key,
  created_at   clustering key DESC,   -- newest first
  post_id, media_ids, caption, location, tags, like_count, comment_count
)
posts_by_id (post_id PK → full post)   -- point lookups for feed hydration
```

Follow graph (store BOTH directions — denormalize for read speed)

```
followers_of (user_id  → set/rows of follower_ids)   -- drives FAN-OUT (push)
```

```
following_of (user_id  → set/rows of followee_ids)   -- drives PULL + profile-of-followees
```

Storing both directions doubles writes on follow/unfollow (rare) to make reads cheap (constant). Classic **read-optimization via denormalization**.

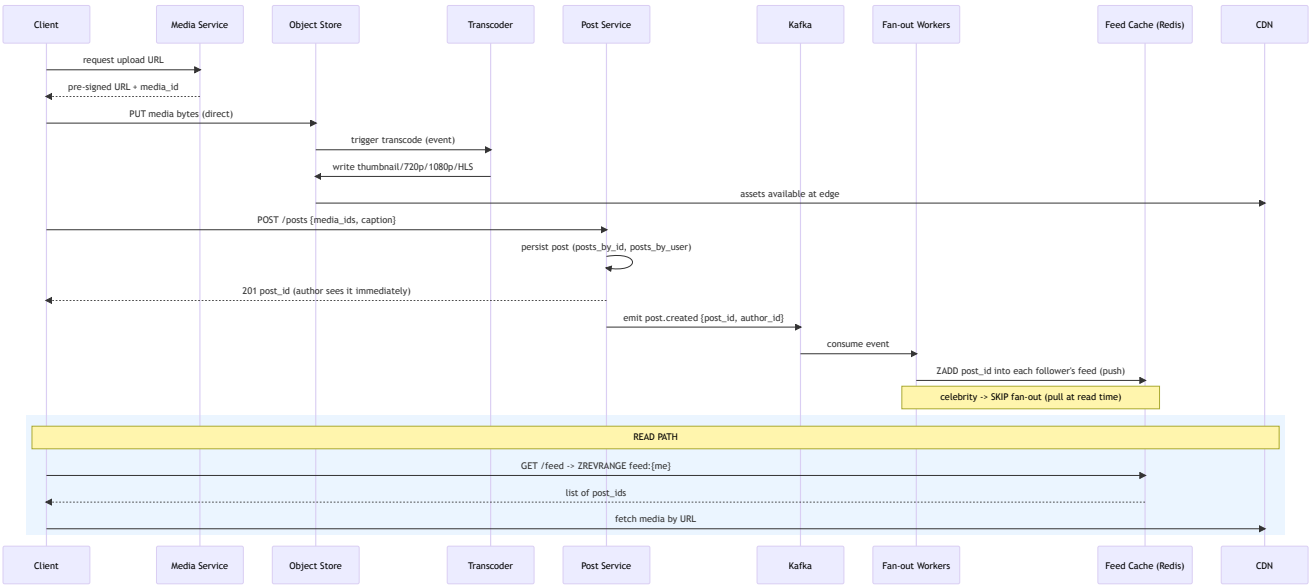
Feed cache (Redis Sorted Set per user)

```
feed:{user_id}  →  ZSET of  member=post_id , score=ranking_score (or timestamp)
ZADD feed:{u} <score> <post_id>          # fan-out insert
ZREVRANGE feed:{u} 0 19                   # top 20 for the home feed
ZREMRANGEBYRANK feed:{u} 0 -1001          # trim to newest/best ~1000
```

We store **only post-IDs** (pointers) in the feed, then **hydrate** full post + media URLs from `posts_by_id` + cache. Keeps the feed cache tiny and avoids duplicating post data per follower.

---

6. Data flow: post creation → storage → feed → delivery



Step-by-step:

1. **Upload first, post second.** Client gets a pre-signed URL and uploads media **directly to the object store** (bypasses our app servers — saves bandwidth/compute). Returns `media_id`.
2. **Transcode asynchronously.** Object-store event triggers workers that produce a thumbnail, multiple resolutions, and HLS/DASH renditions for video. Assets land back in the object store, fronted by the **CDN**.
3. **Create post.** `POST /posts` writes metadata to the Posts DB and returns `201` immediately — **the author sees their own post right away** (read-after-write for self).
4. **Emit event.** Post Service publishes `post.created` to **Kafka** (decouples the slow fan-out from the user-facing request).
5. **Fan-out (push).** Workers look up the author's followers and `ZADD` the `post_id` into each follower's `feed:{user_id}` Redis ZSET — **unless** the author is a celebrity (see §8).
6. **Delivery (read).** On feed open, Feed Service reads the top-N post-IDs from Redis, **merges in** any followed-celebrity posts (pull), **ranks**, hydrates post/media, and returns. Media itself streams from the **CDN**, not our servers.

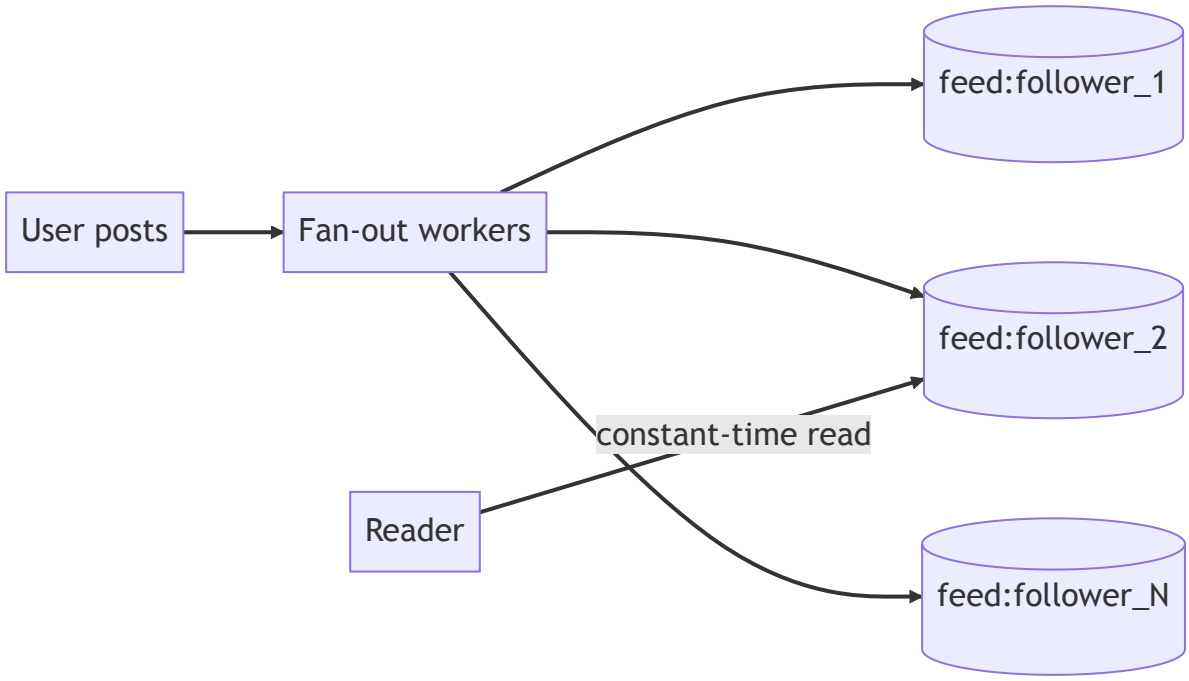
---

7. Feed generation strategies — push vs pull vs hybrid (the core)

This is where to spend the most time. Three approaches:

7.1 Push — fan-out-on-write (precompute feeds)

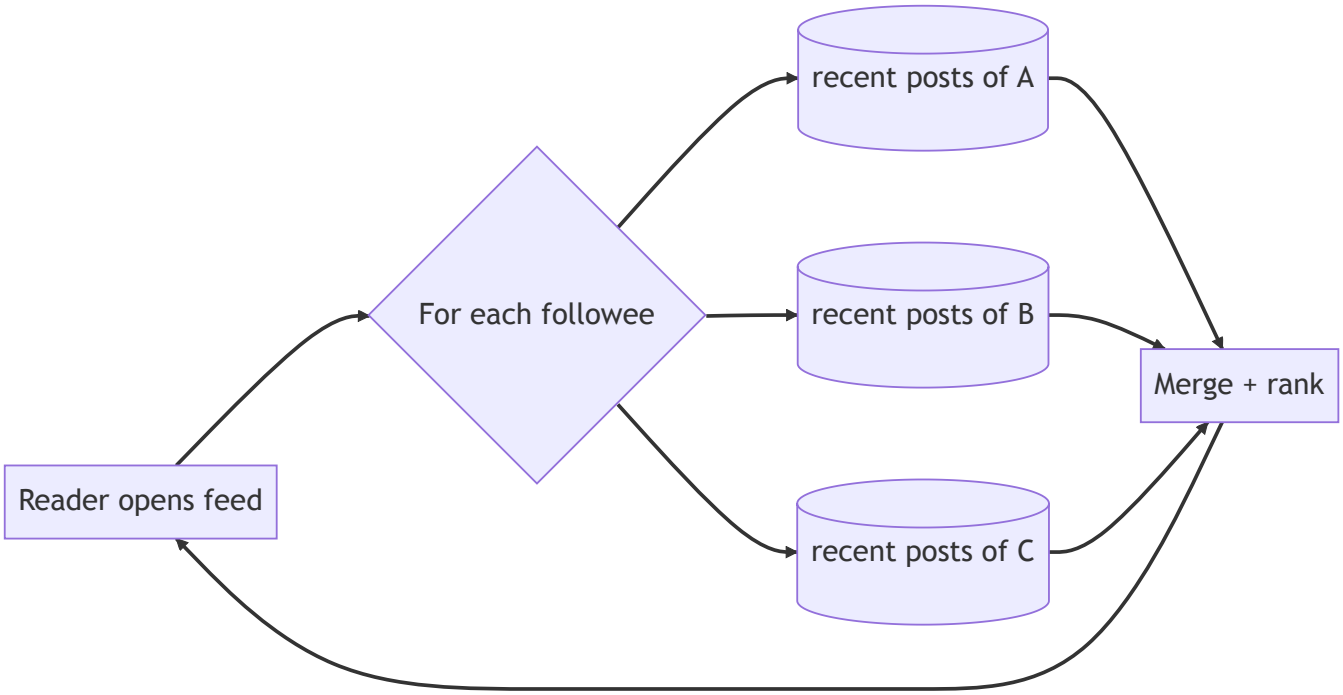
On post creation, **write the post-ID into every follower's feed** immediately.



- ✅ **Reads are trivially fast** — feed is already materialized (one `ZREVRANGE`). Perfect for the 100:1 read-heavy reality.
- ✅ Read load is predictable and cache-friendly.
- ❌ **Write amplification**: 1 post → up to *millions* of writes. A celebrity with 100M followers = 100M Redis writes per post (the **fan-out bomb**).
- ❌ Wastes work for **inactive followers** (we precompute feeds nobody reads).
- ❌ New follow/unfollow needs backfill/cleanup.

7.2 Pull — fan-out-on-read (assemble on demand)

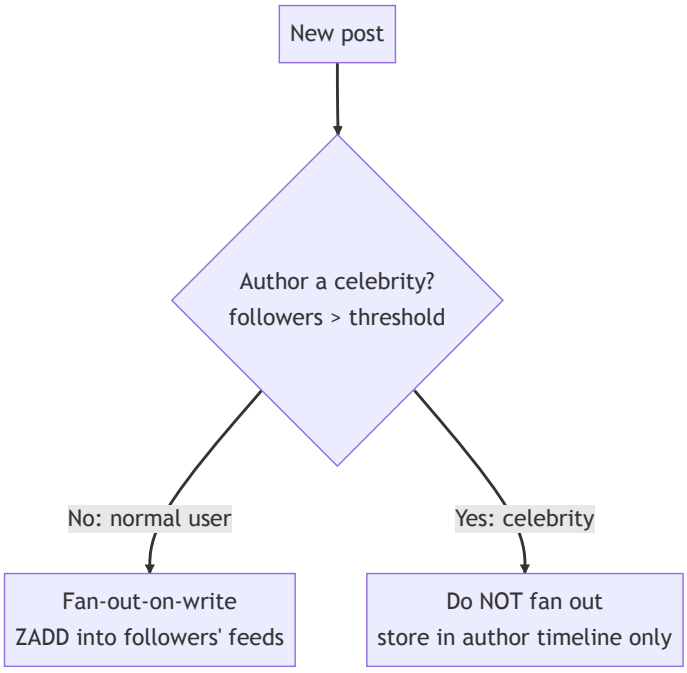
Store nothing per-follower. On feed open, **fetch recent posts from everyone the user follows**, merge, rank.

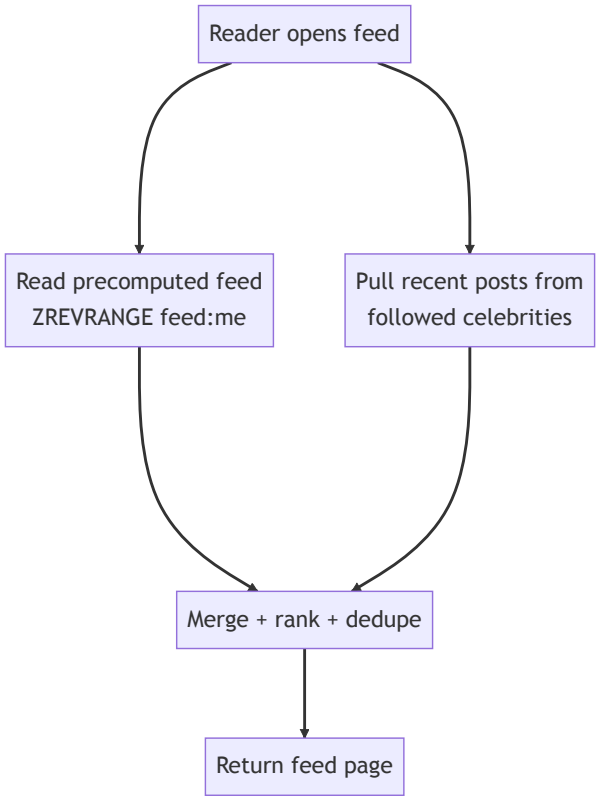


- ✅ **No write amplification** — posting is cheap; great for celebrities.
- ✅ No wasted precomputation for inactive users.
- ✅ Always fresh; easy to re-rank.
- ❌ **Reads are expensive**: a user following 1,000 accounts triggers many queries + a merge **on every feed open** → high latency, heavy DB load at 100K+ reads/sec.
- ❌ Hard to cache; "thundering herd" on popular followees.

7.3 Hybrid — the real-world answer ✅

Push for the masses, pull for celebrities. Split accounts by follower count.





- **Normal authors (followers < ~10K–100K): push** — fan out to followers' feeds. Cheap fan-out, instant reads for the long tail.
- **Celebrity authors (followers > threshold): don't fan out**. Their posts live in their own timeline; followers **pull** them at read time and merge.
- **Read path** = precomputed feed (push) ⊕ small pull from the handful of celebrities each user follows, then rank.

Because a user typically follows only a **few** mega-accounts, the pull side is small and **cacheable** (a celebrity's recent posts are read by millions → cache once, serve everyone).

Strategy	Write cost	Read cost	Best for	Used by (real world)
Push	High (fan-out)	**Very low**	Most users; read-heavy systems	Instagram/Twitter baseline
Pull	**Very low**	High	Celebrities; inactive users	Celebrity timelines
Hybrid ✅	Balanced	Low	**Everything at scale**	Instagram, Twitter, Tumblr

Extra optimizations layered on the hybrid:

- **Don't fan out to inactive users.** Skip precompute for users who haven't opened the app in N days; lazily build their feed on next login (pull). Saves a large fraction of fan-out writes.
- **Active-follower fan-out.** Prioritize fanning out to recently-active followers first; backfill the rest async.

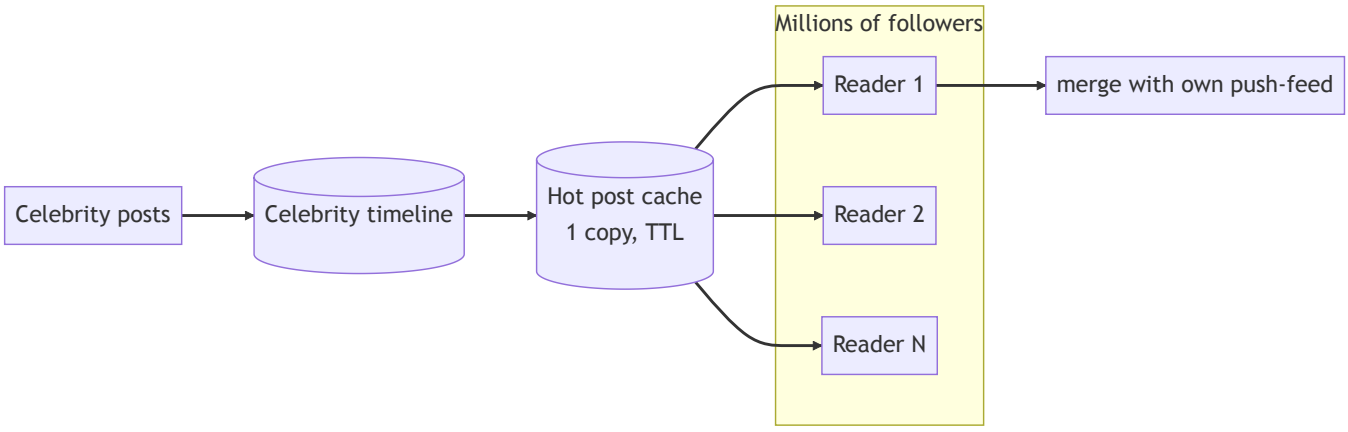
---

8. Celebrities / creators with millions of followers

The single hardest part. One post by a 100M-follower account naively = **100M feed writes**, spiking Kafka, workers, and Redis — a self-inflicted DoS.

Mitigations (stack them):

1. **No fan-out for celebrities (pull instead).** Their posts are fetched at read time and merged. This is the primary fix — turns 100M writes into ~0 writes + cached reads.
2. **Cache the celebrity's recent posts hard.** Their latest posts are requested by millions → cache in Redis/CDN with short TTL; **read once, serve all**. Pull cost becomes a single cache hit per reader.
3. **Async, throttled, prioritized fan-out (if any).** For "borderline" accounts, fan out gradually: active followers first, the rest drained from Kafka at a controlled rate so we don't saturate Redis.
4. **Per-key hot-shard handling.** A celebrity's follower list and post cache are hot keys → replicate across many Redis replicas / CDN nodes; split the follower set across shards for parallel fan-out if we *do* push.
5. **Dedupe + merge at read.** Reader's feed = precomputed (push from normal followees) merged with pulled celebrity posts, deduped, then ranked.



Threshold tuning: the push/pull cutoff (e.g., 50K–100K followers) is a knob. Below it, push (cheap fan-out, fast reads). Above it, pull (avoid the write bomb). Some systems also flip an account to pull dynamically when it suddenly goes viral.

One-liner: *"I never fan out a celebrity's post to 100M feeds — I leave it in their timeline and pull it at read time from a single hot cache that millions share."*

---

9. Media pipeline (upload, transcode, deliver)

- **Direct-to-object-store upload** via pre-signed URLs → app servers never touch raw bytes (saves bandwidth/CPU).
- **Async transcoding** → thumbnail, 720p/1080p/4K, and **HLS/DASH** adaptive bitrate for video. Store all renditions in the object store.
- **CDN in front of everything** → media served from edge POPs near the user; our origin handles only cache misses. This offloads the vast majority of read bytes.
- **Image/video IDs are content-addressable** and immutable → cache forever (`Cache-Control: immutable`); edits create new IDs.
- **Process out of band**: a post can be created with media "processing"; renditions attach when ready.

This keeps the **feed path metadata-only and fast** — media bytes flow client ↔ CDN, never through the feed services.

---

10. Ranking / relevance (recent and relevant)

A pure reverse-chronological feed is simplest; "relevant" implies **ranking**:

- **Candidate generation**: the merged set (push feed ⊕ pulled celebrity posts).
- **Scoring signals**: recency, affinity (how much you interact with the author), predicted engagement (like/comment/dwell probability), media type, diversity (don't show 5 posts from one account).
- **Two-stage**: cheap heuristic score at read time for the top page; heavier ML model offline/near-line writing a `ranking_score` used as the Redis ZSET score.
- **Keep recency dominant** so the feed feels live; blend relevance on top.

Store the score as the **ZSET score** so `ZREVRANGE` returns ranked order directly. Re-rank the small merged set per request for freshness.

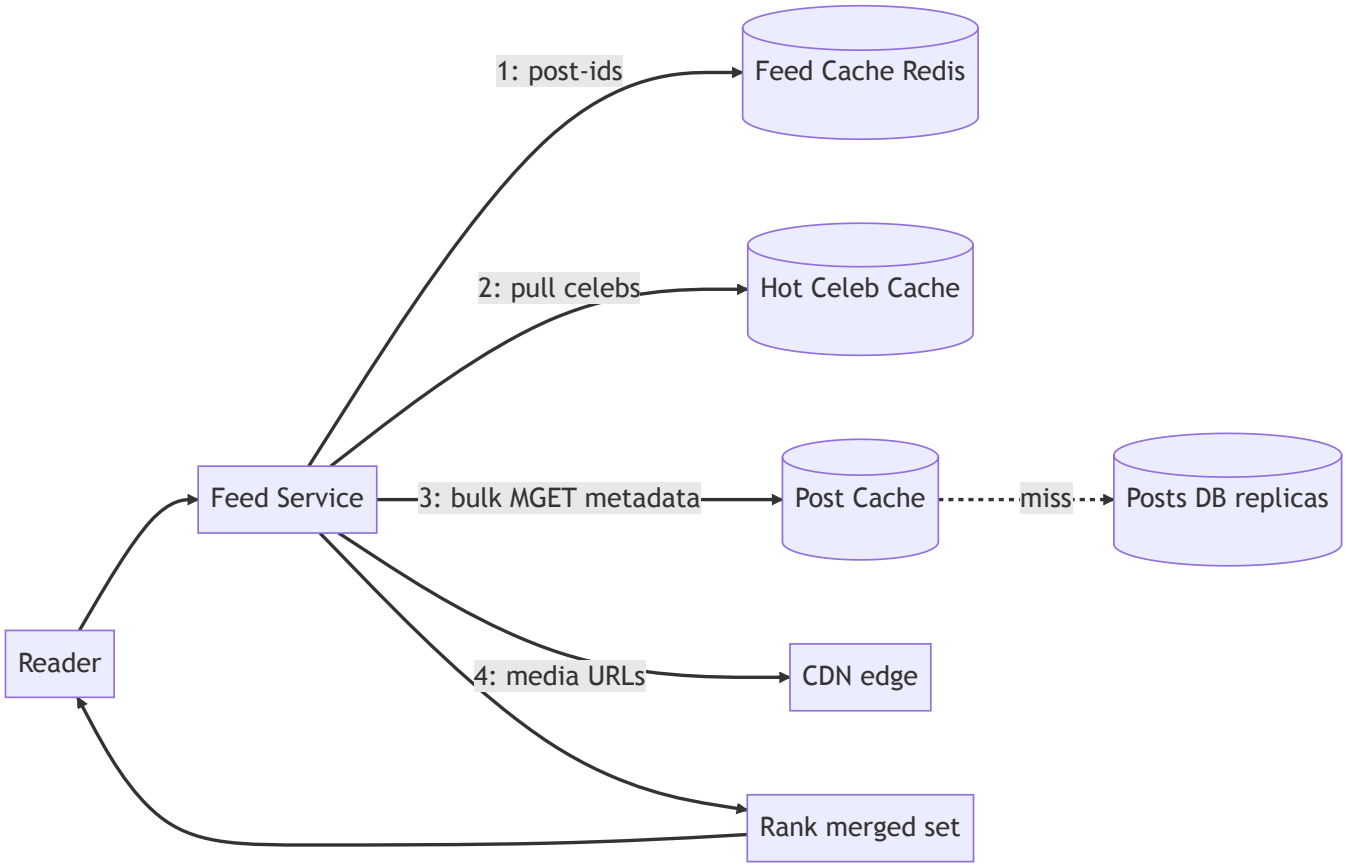
---

11. Scaling reads & avoiding feed-read bottlenecks

Reads are 100× writes — this section is where the system lives or dies.

1. **Precomputed feeds in RAM (push).** The home feed read is a single `ZREVRANGE` from Redis → sub-ms. This is the #1 defense against read load.
2. **Multi-layer caching:**
    - **CDN** — all media + some static feed fragments.
    - **Feed cache (Redis)** — materialized post-ID lists per user.
    - **Post/object cache (Redis)** — hydrated post metadata by `post_id` (read once, reused across all feeds containing it).
    - **Hot-content cache** — celebrity posts shared by millions.
3. **Read replicas + sharding** for the Posts DB; route reads to replicas, writes to primary. Shard by `user_id`/`post_id`. Accept **replica lag** (eventual consistency is fine for others' feeds).
4. **Hydrate in bulk.** Feed returns N post-IDs → one **multi-get** (`MGET`) for all post metadata, not N round-trips. Avoids the N+1 query problem.
5. **Cursor pagination** (not OFFSET) → O(1) page resumption, stable under inserts.

6. **Trim feeds.** Keep only the newest/best ~500–1,000 IDs per feed (`ZREMRANGEBYRANK`). Bounds memory and read cost; older history comes from the profile/DB on demand.
7. **Avoid hot keys.** Replicate hot Redis keys across replicas; for a celebrity follower-list/post, fan reads across many replicas or the CDN.
8. **Backpressure & async.** Fan-out via Kafka absorbs write spikes so reads aren't starved; consumers scale independently.
9. **Prefetch / pagination on client** — load page 1 fast, prefetch page 2 during scroll.



---

12. Failure scenarios — *"what if X fails?"*

Failure	Impact	Mitigation
Feed cache (Redis) node down	Some users' feeds missing	Replicas + failover; on miss, **rebuild from DB (pull)** — degraded but available
Fan-out worker backlog	New posts slow to appear in feeds	Kafka buffers; autoscale consumers; prioritize active followers; eventual consistency is acceptable
Celebrity post spikes Kafka	Fan-out lag for everyone	Celebrities are **pull-only** (no fan-out); throttle/borderline-account draining
Object store / CDN miss storm	Slow media	Multi-tier CDN, origin shielding, pre-warm popular assets
Posts DB primary down	Writes blocked	Failover to replica; writes retried via Kafka; reads served from replicas/cache
Hot key (celebrity)	One shard saturated	Replicate hot keys, CDN the post, split follower set across shards
Replica lag	Slightly stale feed	Acceptable for others' posts; route the **author's own** reads to primary/cache for read-after-write

Guiding principle: the feed must **always render**. Prefer **stale-but-available** over unavailable — fall back from cache → replica → pull-rebuild rather than erroring.

---

13. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Push vs Pull	Fan-out-on-write (fast reads, costly writes)	Fan-out-on-read (cheap writes, costly reads)	**Hybrid**: push for the long tail, pull for celebrities
Consistency vs Latency	Strong (read-after-write everywhere)	Eventual (feeds lag a few s)	Eventual for *others'* feeds; strong only for **author's own** view
Precompute vs On-demand	Materialize all feeds	Build at read time	Precompute active users; **skip inactive** users (lazy build)
Storage cost vs Read speed	Store feeds per user (RAM)	Recompute each read	Trade RAM for speed — it's a read-heavy system; **trim** to bound cost
Freshness vs Cost	Re-rank every read	Cache ranked feed	Cache base, re-rank only the small merged top page
Blob in DB vs Object store	DB (simple)	Object store + CDN	**Always** object store + CDN for media

CAP framing: the feed system leans **AP** — under partition we keep serving (possibly stale) feeds rather than blocking. Strong consistency is reserved for the narrow case of the **author seeing their own post** immediately.

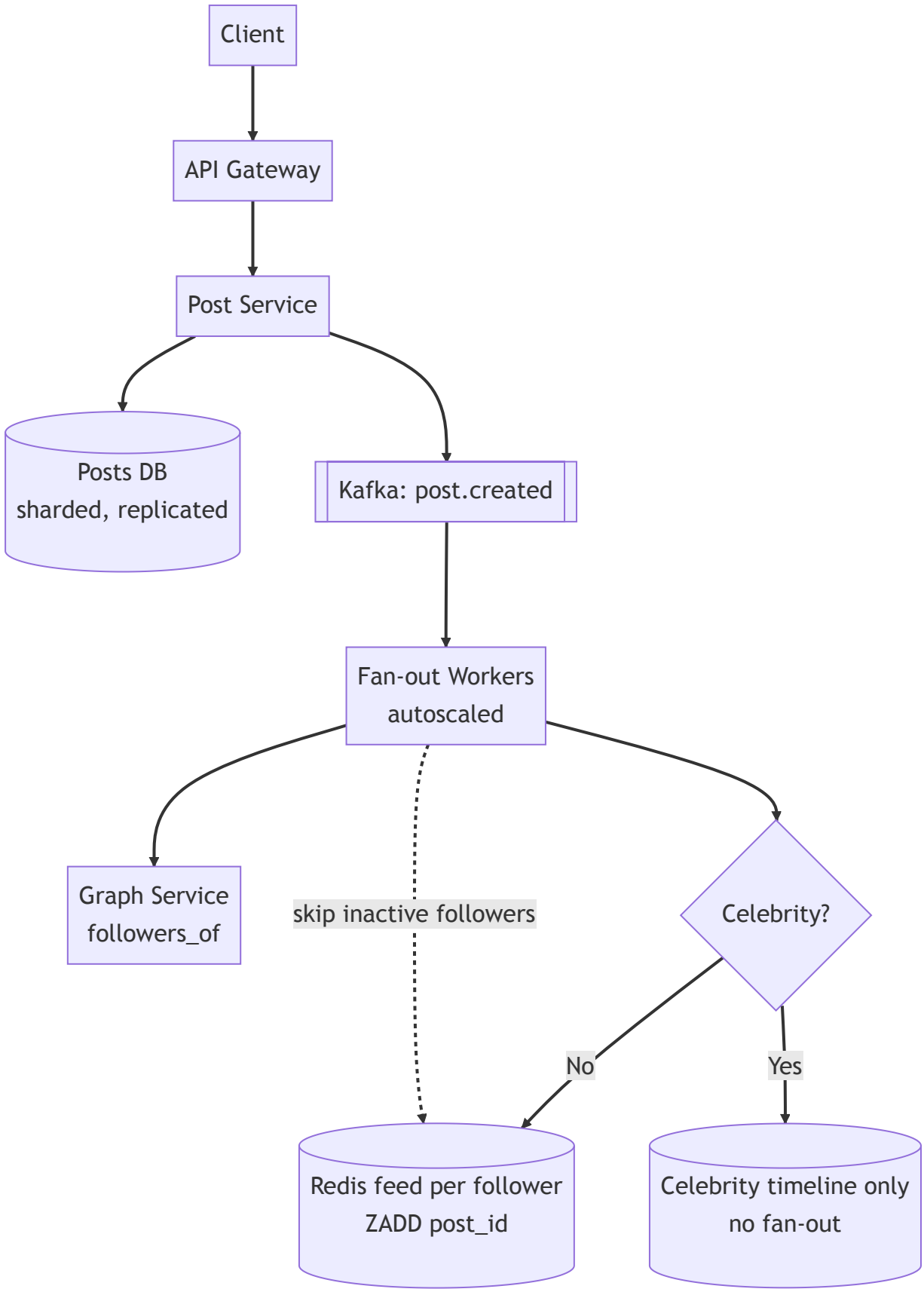
One-liner to say out loud: *"I'd use a **hybrid fan-out**: push posts from normal accounts into followers' precomputed Redis feeds for sub-ms reads, but **never** fan out celebrities — their posts stay in their timeline and are pulled from a shared hot cache at read time. Media goes to object storage behind a CDN, post metadata to a sharded wide-column store, the follow graph denormalized both directions, and Kafka decouples the write path from fan-out so read traffic is never starved. I skip precomputing feeds for inactive users and trim feeds to the newest ~1,000 to bound memory."*

---

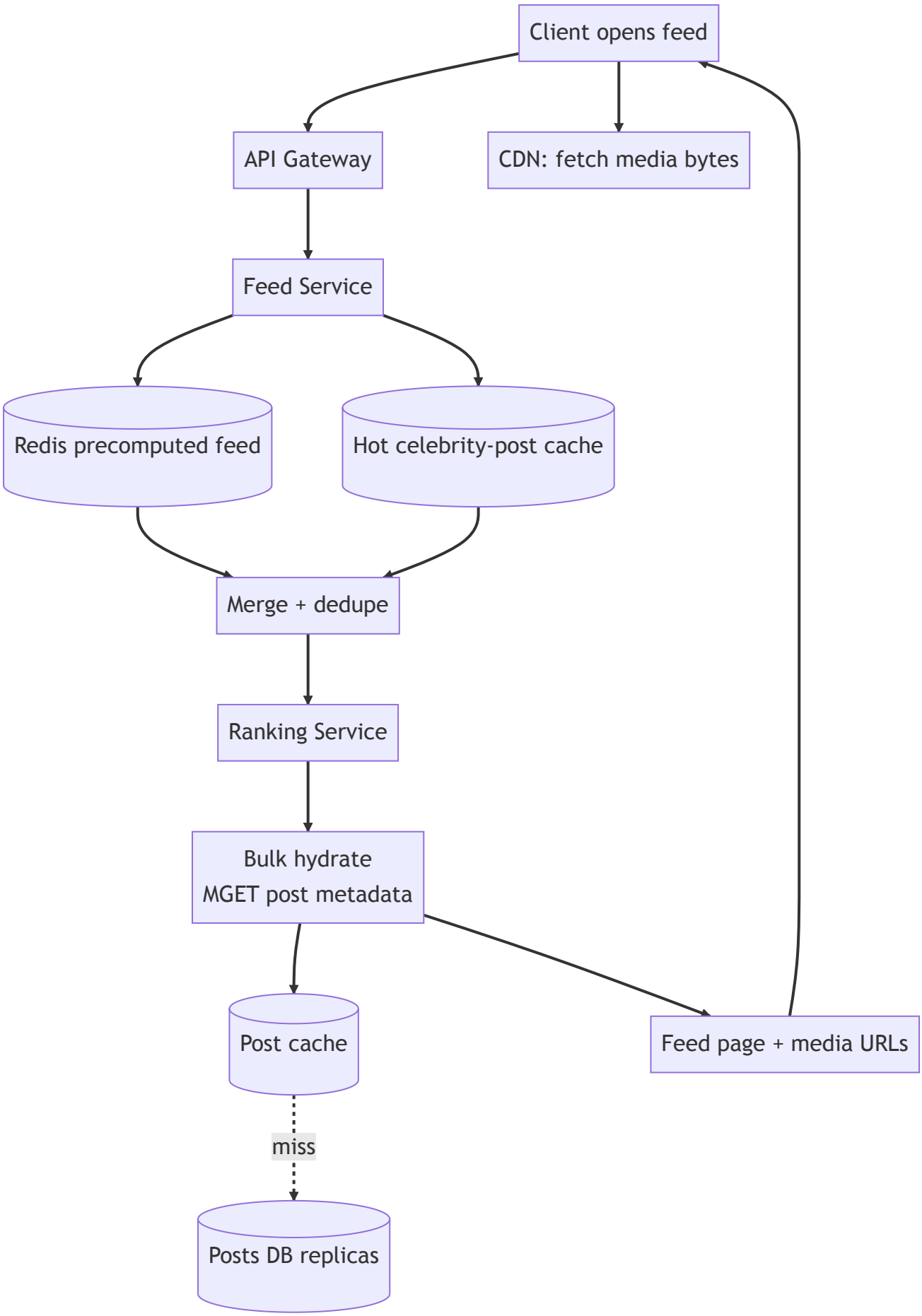
14. Full system design (detailed)

End-to-end, split into **(A)** write path (post creation + fan-out), **(B)** read path (feed delivery), and **(C)** media path.

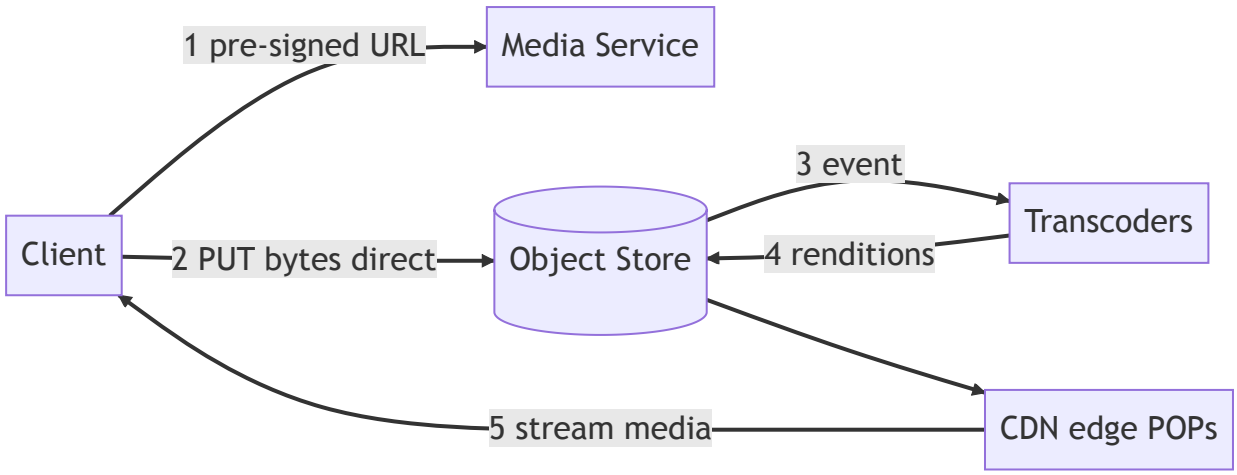
14A. Write path — post creation & fan-out



14B. Read path — feed delivery



14C. Media path — upload & delivery



---

15. Networking, security & performance best practices

Networking

- **CDN for all media** — photos/videos served from edge POPs; origin hit only on cache miss. **Adaptive bitrate (HLS/DASH)** for video, modern codecs (H.265/AV1).
- **TLS at the edge/LB**; **HTTP/3 (QUIC)** for multiplexed, low-latency feed + media fetches on flaky mobile networks; **gRPC** between internal services.
- **Direct-to-object-store uploads** via pre-signed URLs — raw media bypasses app servers (saves bandwidth/CPU).
- **Cursor pagination** on the wire (compact, stable under inserts) — never `OFFSET`.
- **Connection pooling / keep-alive** between services; co-locate feed cache with feed services.

Security

- **AuthZ on every read** — enforce private accounts, blocks, and channel membership before returning posts; never trust the client.
- **Signed, expiring media URLs** so private media isn't hotlinkable; per-object access checks.
- **Sanitize uploads in a sandbox** — strip EXIF/GPS, re-encode to kill polyglot/malware files, virus-scan before publish.
- **Rate-limit writes** (post/like/follow) and run **abuse/bot detection** on the fan-out path to blunt spam.
- **Avoid enumeration leaks** (follower lists, user existence); defend against scraping.

Performance

- **Precomputed feeds + multi-layer cache** (CDN → feed cache → post cache) → sub-ms reads.
- **Bulk hydrate** (`MGET`) to kill N+1; store only post-IDs in the feed.
- **Trim feeds** to the newest ~1,000; **skip inactive** users.
- **Thumbnails first**, lazy-load full-res; **client prefetch** of the next page during scroll.

---

16. Staying current — modern & emerging approaches

- **Stores:** Cassandra/**ScyllaDB** for posts, **Redis** for feeds, **Kafka** for fan-out — the industry-standard stack at Meta/Twitter scale; Facebook's **TAO** for the social graph.
- **Media:** HLS/DASH ABR, **AV1/HEVC** codecs, per-title encoding; CDNs (CloudFront/Fastly/Akamai); on-the-fly image resizing at the edge.
- **Transport: HTTP/3/QUIC** for mobile; gRPC internally.

- **Ranking: two-tower retrieval** + GBDT/DNN ranking, **feature stores** (Feast), near-line feature computation; embeddings for relevance.
- **Reference systems:** Instagram feed/Stories architecture, Twitter Timelines + Manhattan store, LinkedIn feed.
- **How I stay current:** engineering blogs (Meta, Discord, Twitter/X, Pinterest), papers (TAO, f4), and prototyping + load-testing before adopting.

---

17. Likely follow-up questions (rehearse these)

- Push vs pull fan-out — when each? *(read-heavy → push; celeb write-bomb → pull)*
- How do you keep a celebrity's post out of 100M feeds? *(no fan-out; pull from one shared hot cache)*
- How does the author see their own post instantly? *(read-after-write to primary/cache for self)*
- Feed cache (Redis) dies — what now? *(rebuild from DB via pull; degrade, don't fail)*
- "Relevant" ranking without killing latency? *(precompute score as the ZSET score; re-rank only the small merged top page)*
- A normal post suddenly goes viral — how? *(dynamically flip the author to pull)*
- Storing a 200B-edge follow graph? *(denormalize both directions for O(1) reads)*
- Hot key on a celebrity's follower list? *(replicate across replicas, shard, CDN the post)*

---

18. Summary checklist (whiteboard recap)

- **Read-heavy (100:1)** → precompute + cache feeds; single `ZREVRANGE` read.
- **Hybrid fan-out** → push normal accounts, pull celebrities; merge + rank at read.
- **Celebrities** → no fan-out, shared hot cache, pull at read (kills the write bomb).
- **Storage** → posts in wide-column, media in object store + CDN, graph denormalized both ways, feeds in Redis ZSETs.
- **Kafka** decouples write from fan-out; absorbs spikes; lets consumers scale.
- **Skip inactive users**, **trim feeds**, **bulk-hydrate**, **cursor paginate** → bound cost and latency.
- **Eventual consistency** everywhere except the **author's own** read-after-write.
- **Always render** the feed — degrade (cache → replica → pull) rather than fail.

Designing an In-Memory Key–Value Database (Redis-like)

> A complete, interview-ready walkthrough: requirements → estimates → APIs → in-memory structures → on-disk schemas → **concurrency, transactions, TTL, eviction** → **durability & crash recovery** → **replication & sharding** → **consistency** → **backpressure** → failure modes → trade-offs. Use the headings as your whiteboard agenda. The hard parts are **the concurrency model, durability vs latency, and consistency under replication/sharding** — spend your time there.

> Think Redis / Memcached / Aerospike. RAM-resident data for microsecond reads/writes, O(1) average ops, optional persistence (AOF + snapshots), and horizontal scale via sharding + replication.

---

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements and the access pattern.
2. **Estimate** scale (QPS, keys, value sizes, RAM, network).
3. **Define the client API / wire contract.**
4. **Pick in-memory structures** (hash table + expiry index + eviction metadata) → O(1).
5. **Choose the concurrency model** (single-threaded event loop vs locks/sharded threads).
6. **Add durability** (AOF + snapshots) and the **crash-recovery** flow — and its latency trade-off.
7. **Scale out**: sharding + replication; state the **consistency** model.
8. **Handle overload** (backpressure) and **failure modes**; summarize **trade-offs**.

Keep saying *"here's the trade-off…"* — that's what's being graded.

---

1. Problem & motivation

A KV store that keeps data in **RAM** for high-throughput, low-latency `get/set/delete` with **O(1)** average cost, optional **persistence**, **TTL**, **CAS**, atomic multi-key ops within a shard, and horizontal scale.

Why in-memory: RAM access is ~~100 ns vs ~100 µs for SSD vs ~10 ms for disk seek~~ **1000× faster**. For caches, sessions, counters, rate-limit buckets, leaderboards, and queues, microsecond latency at high QPS is the whole point.

What makes it hard:

- **O(1) at p99**, not just average — avoid rehash stalls, GC pauses, lock contention, big-key blowups.
- **Durability vs latency** — every write to disk is safe but slow; the AOF fsync policy is the central trade-off.
- **Concurrency** — a hash table mutated by millions of ops/sec; single-thread simplicity vs multi-core throughput.
- **Memory is finite** — eviction policy + expiry must reclaim space without stalling.
- **Scale-out consistency** — replication lag and sharding break the "one box, one truth" simplicity.
- **Crash recovery** — rebuild state fast and correctly from snapshot + log.

---

2. Requirements

Functional

- **Core ops**: `set(key, value[, ttl])`, `get(key)`, `delete(key)`.
- **CAS** — conditional update by **version** or **expected value** (optimistic concurrency).
- **TTL expiration** — keys auto-expire.
- **Atomic multi-key operations within a shard** (mini-transactions / `MULTI…EXEC`).

Non-functional

- **O(1)** average reads/writes; tight p99.
- **Expiration tracking** (timing wheel or min-heap).
- **Memory management + eviction** (LRU/LFU).
- **Durability** — append-only log (AOF) + periodic **snapshots**.
- **Concurrency control** — single-threaded event loop *or* fine-grained locks/sharded threads.
- **Crash recovery** procedure.
- **Replication + sharding** for scale.
- **Consistency** guarantees (stated explicitly).
- **Backpressure** under overload.
- **Observability** — logs, metrics, tracing.

Clarifying questions to ask the interviewer

- **Durability bar** — can we lose the last ~1 s of writes on crash (cache), or must every ack be durable (system of record)? *(Sets the fsync policy.)*
- **Consistency** — strong (sync replication) or eventual (async, like Redis)? Read-your-writes?
- **Value size** — small (counters, sessions) or large blobs? Affects memory + eviction.
- **Eviction** — is it a **cache** (evict under pressure) or a **store** (reject writes when full)?
- **Multi-key transactions** — within a shard only, or cross-shard (distributed txn)?
- **Single region or geo-distributed?**

---

3. Back-of-the-envelope estimation

Quantity	Assumption	Result
Target QPS	high-throughput cache	**~1M ops/sec per node** (Redis does ~100k–1M+ on one core)
Keys/node	—	~100M keys
Avg entry	~64 B key + ~256 B value + ~80 B overhead	~~400 B → 100M × 400 B ≈ **~~40 GB** data
RAM/node	data + replication buffers + fragmentation (~1.5×)	**~64 GB** box
Value sizes	small-object dominated	keep big values rare; chunk/externalize blobs
Network	1M ops × ~400 B	~~400 MB/s ≈ **~~3.2 Gbps** → 10/25 GbE NIC
AOF write rate	~300k writes/sec × ~50 B record	~15 MB/s → trivial sequential SSD; **fsync cadence** is the real cost
Cluster	1 TB dataset / 40 GB per shard	**~25 shards** (× replicas)

Takeaways that drive the design:

1. **~1M ops/sec/core** → a **single-threaded event loop** per shard avoids lock overhead; scale by **adding shards/cores**, not threads on one map.
2. **RAM is the budget** → eviction + expiry must be cheap and incremental (no stop-the-world).
3. **fsync is the latency knob** → batch/group-commit; choose per-write vs per-second durability.
4. **1 TB > one box** → **shard**; replicate each shard for HA.

---

4. Client API (wire contract)

Simple request/response over TCP (a compact binary protocol like RESP; pipelining for throughput).

```
SET key value [EX ttl_seconds] [NX|XX]      → OK | (nil)
GET key                                       → value | (nil)
DEL key                                       → 1 | 0
TTL key                                       → seconds | -1 (no ttl) | -2 (missing)
EXPIRE key ttl                                → 1 | 0

# CAS — optimistic concurrency
CAS key expected_value new_value              → OK | CAS_MISMATCH
CASV key expected_version new_value           → OK,new_version | CAS_MISMATCH   # version-based
INCR key                                       → new_int        # atomic counter

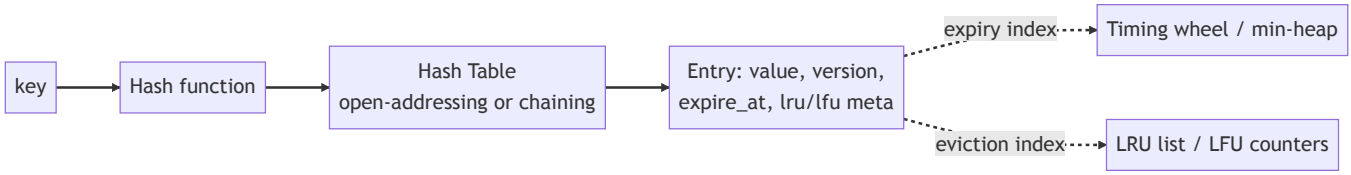
# Atomic multi-key (within a shard)
MULTI                                          # begin
  SET a 1
  INCR b
EXEC                                           → [results] | ABORT   # all-or-nothing
# optimistic variant:
WATCH k1 k2 ; MULTI ; ... ; EXEC               → nil if a watched key changed
```

Design notes:

- **CAS** returns a typed mismatch (not a generic error) so clients can retry the read-modify-write loop.
- **Versioned CAS** — each key carries a monotonically increasing **version**; `CASV` succeeds only if the version matches → lock-free optimistic updates, ABA-safe.
- **Pipelining** — clients batch many commands per round-trip (the main throughput lever at 1M QPS).
- **Idempotency** — `SET`/`DEL` are naturally idempotent; client request IDs let proxies dedup retries.

---

5. In-memory data structures & algorithms (O(1))



Primary index — hash table (O(1) avg)

- A **hash table** maps key → entry; average O(1) `get/set/delete`. Use a good hash (xxHash/SipHash — SipHash resists hash-flooding DoS).

- **Incremental rehash** (Redis-style two-table) — on growth, migrate buckets a few at a time on each op so resizing never stalls the world (protects p99). Load factor target ~0.7.
- **Entry** holds: `value`, **version** (for CAS), **expire_at** (0 = none), and **eviction metadata** (LRU timestamp / LFU counter).

Expiration tracking

Approach	How	Trade-off
Lazy expiry	Check `expire_at` on access; if past, delete + return nil	Free, but dead keys linger in RAM until touched
Min-heap by `expire_at`	Pop due keys	O(log n) per op; exact next-expiry
Timing wheel ✅	Buckets by time slot; advance a hand each tick	**O(1)** insert/expire; great for many TTLs
Active sampling (Redis)	Periodically sample N keys with TTL, evict expired	Probabilistic, bounded CPU

Recommended: lazy expiry + active background sweep (timing wheel or sampled scan). Lazy handles the hot path for free; the background sweep reclaims cold expired keys so they don't waste RAM — bounded per tick to avoid spikes.

Eviction policy (when memory is full)

- **LRU** (approximate, Redis-style sampling) — evict least-recently-used; cheap, good default. Exact LRU = an intrusive doubly-linked list (move-to-head on access).
- **LFU** — least-frequently-used (with decay) for skewed/hot-key workloads.
- **TTL-aware** (`volatile-lru`) or **random** as policies; **noeviction** = reject writes when full (store, not cache).
- Maintain eviction metadata **inline** in the entry → O(1) update on access.

Values

- Small values inline; **large values chunked**/externalized to avoid copy/allocation spikes and big-key stalls. Specialized structures (lists, sets, sorted sets) as needed, each O(1)/O(log n).

---

6. On-disk schemas — AOF & snapshot (durability formats)

Persistence is **optional** but, when on, two complementary mechanisms:

Append-Only File (AOF) — the write log

Every mutating command is appended as a record (the operation, not the whole dataset).

```
AOF record (binary, length-prefixed):
┌─────────┬──────────┬─────────┬──────────┬──────────┬──────────┬──────────┐
│ magic   │ rec_len  │ lsn     │ op (1B)  │ key_len  │ key …    │ val …    │
│         │          │ (u64)   │ SET/DEL/ │          │          │ +ttl     │
│         │          │         │ EXPIRE/  │          │          │ +ver     │
│         │          │         │ CAS      │          │          │          │
└─────────┴──────────┴─────────┴──────────┴──────────┴──────────┴──────────┘
        + CRC32 per record (detect torn/corrupt tail)
```

- **Append-only, sequential** → fast on SSD; the **LSN** (log sequence number) orders writes and drives replication + recovery.
- **fsync policy** (the durability knob): `always` (every write — safest, slowest), `everysec` (group-commit each second — Redis default, lose ≤1 s), `no` (OS flushes — fastest, least safe).
- **AOF rewrite/compaction** — periodically rewrite the log to the minimal set of commands reproducing current state (drop overwritten/expired keys) so it doesn't grow unbounded.

Snapshot (point-in-time dump)

A compact serialization of the whole keyspace at an instant (Redis RDB-style).

```
Snapshot file:
[ header: magic, version, created_at, last_lsn ]
[ repeated: key_len, key, type, value, expire_at, version ]
[ footer: count, CRC ]
```

- Produced by **fork + copy-on-write** (snapshot the child's frozen memory while the parent keeps serving) or a background thread → no long stall.
- **`last_lsn`** records where the snapshot ends so recovery knows which AOF tail to replay.

AOF + snapshot together: snapshot = fast bulk restore; AOF = the durable tail since the snapshot. Best of both: quick recovery + minimal data loss.

---

7. Concurrency, transactions & atomic multi-key (focus area)

The model: single-threaded event loop per shard ✅

- **One thread owns a shard's hash table**, processing commands from an **event loop** (epoll/kqueue) over many connections.
- **No locks, no races, no deadlocks** on the data — each command runs to completion atomically. This is why Redis is fast *and* simple: removing lock contention often beats multi-threading for in-RAM ops.
- **Scale across cores** by running **many shards (one event-loop thread each)**, partitioning the keyspace → linear scale without shared-memory locking. I/O (network read/parse, TLS) can use helper threads; the **data mutation stays single-threaded per shard**.

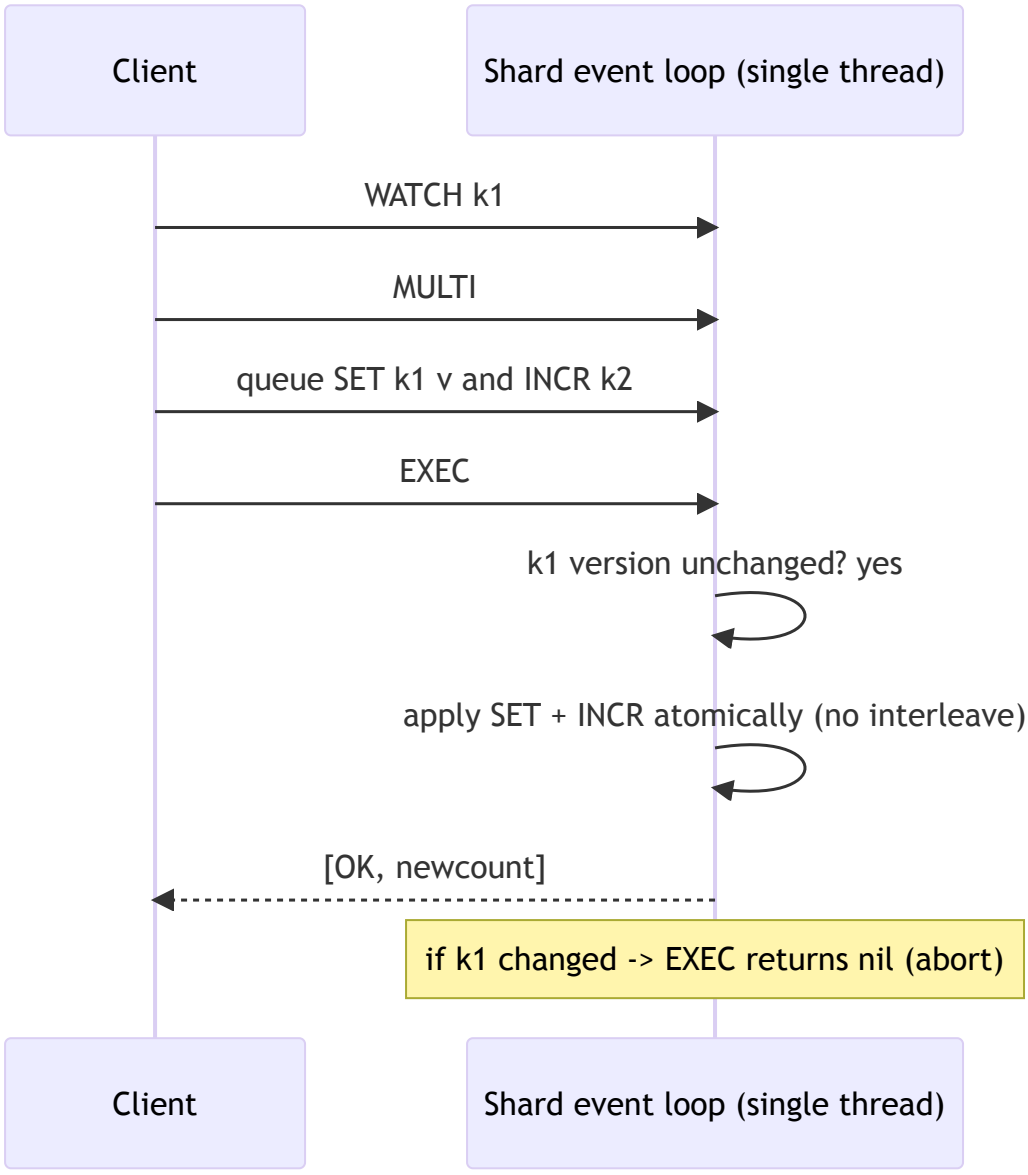
Model	Pros	Cons
Single-threaded event loop ✅	No locks, simple, atomic ops free, great p99	One core per shard → scale via sharding
Multi-threaded + fine-grained locks	Uses all cores on one keyspace	Lock contention, deadlocks, complex; cache-line bouncing
Sharded threads (lock per shard)	Parallel, bounded contention	Cross-shard ops need coordination

Recommendation: single-threaded per shard, scale by sharding. It makes **atomicity, CAS, and multi-key EXEC trivially correct** (they just run start-to-finish on the owning thread).

Atomic multi-key (within a shard)

Because one thread owns the shard, a `MULTI…EXEC` block **queues commands then executes them as one uninterrupted unit** → all-or-nothing, no interleaving.

- **Optimistic `WATCH`** — watch keys, build the transaction, `EXEC` aborts if any watched key changed since `WATCH` (compare versions) → lock-free optimistic concurrency for read-modify-write across keys.
- **CAS** is the single-key special case: succeed iff `version/value` matches, then bump version — all on the owning thread, so no lost updates.
- **Constraint:** atomic multi-key requires the keys be **on the same shard**. Use **hash tags** (`{user1}:profile`, `{user1}:cart`) so related keys co-locate → multi-key txns stay shard-local.

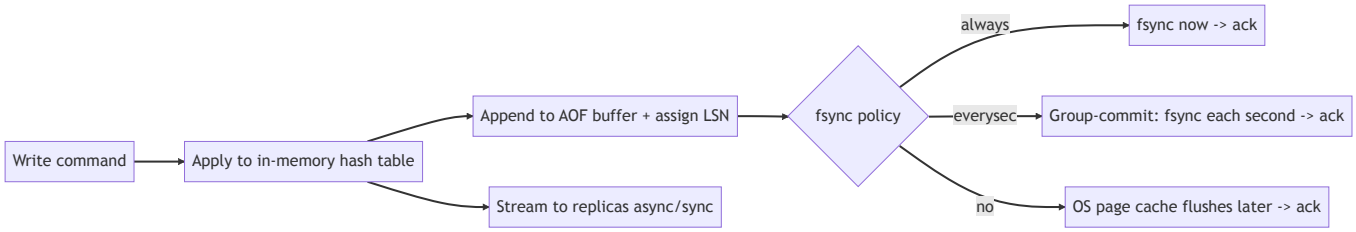


Cross-shard transactions (if required) need **2-phase commit / Raft-replicated txn coordinator** — expensive; call out the trade-off and prefer keeping a transaction within one shard via hash tags.

---

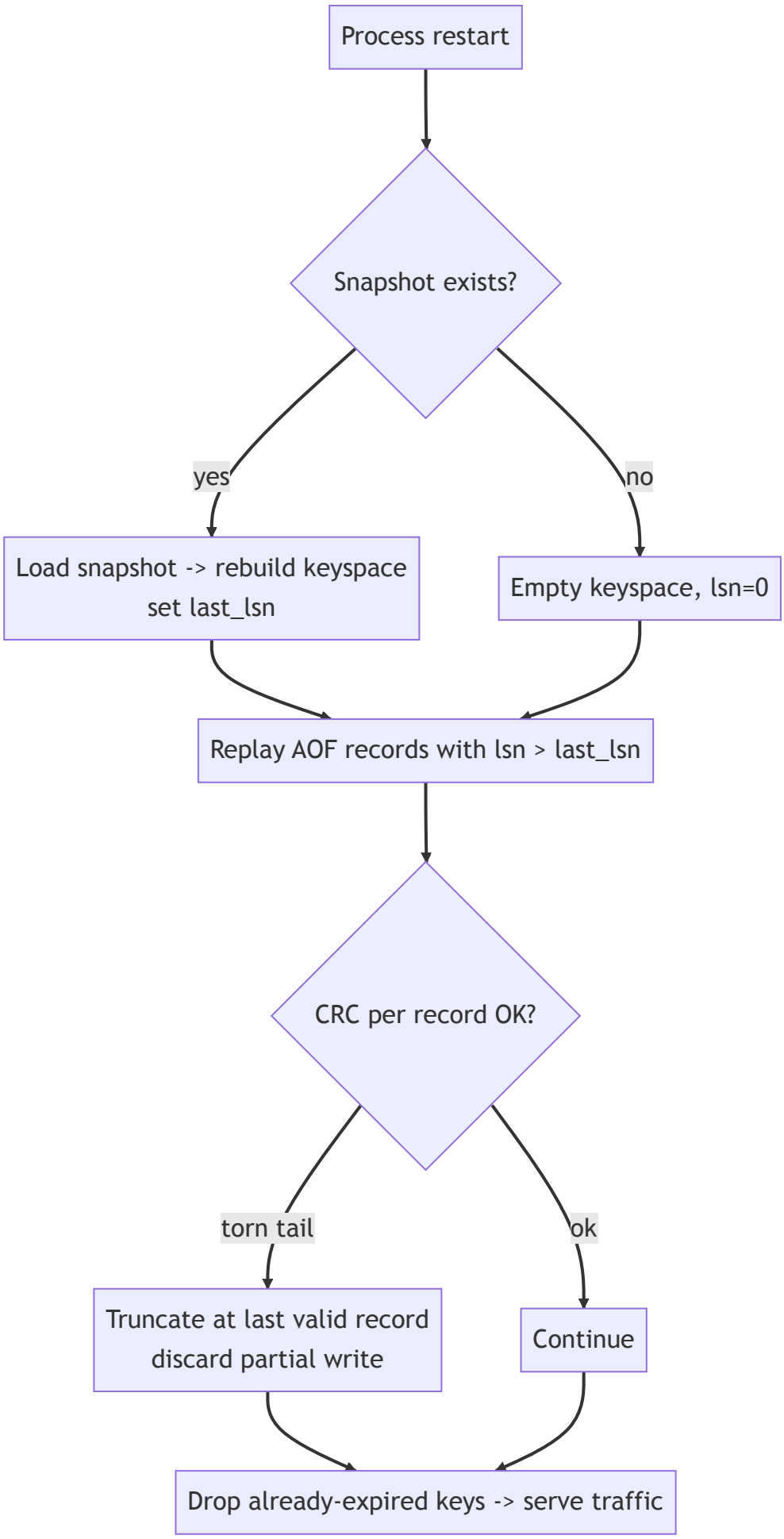
8. Durability & crash recovery (focus area)

Write path with durability



- **Group commit** — batch many writes into one fsync (amortize the ~ms fsync over thousands of ops) → durability with high throughput.
- **Order**: apply in memory → append to AOF → (per policy) fsync → ack. With `always`, ack only after fsync (durable); with `everysec`, ack early (fast, ≤1 s loss window).

Crash-recovery flow


Process restart
Snapshot exists?
yes
no
Load snapshot -> rebuild keyspace
set last_lsn
Empty keyspace, lsn=0
Replay AOF records with lsn > last_lsn
CRC per record OK?
torn tail
ok
Truncate at last valid record
discard partial write
Continue
Drop already-expired keys -> serve traffic

1. **Load the latest snapshot** (fast bulk restore) → keyspace + `last_lsn`.
2. **Replay the AOF tail** (records with `lsn > last_lsn`) to reapply writes since the snapshot.
3. **CRC-check each record**; a crash mid-write leaves a **torn tail** → truncate at the last valid record (that write was never acked under `always`, or is within the ≤1 s loss window under `everysec`).
4. **Drop expired keys** (TTL passed during downtime), then accept traffic.

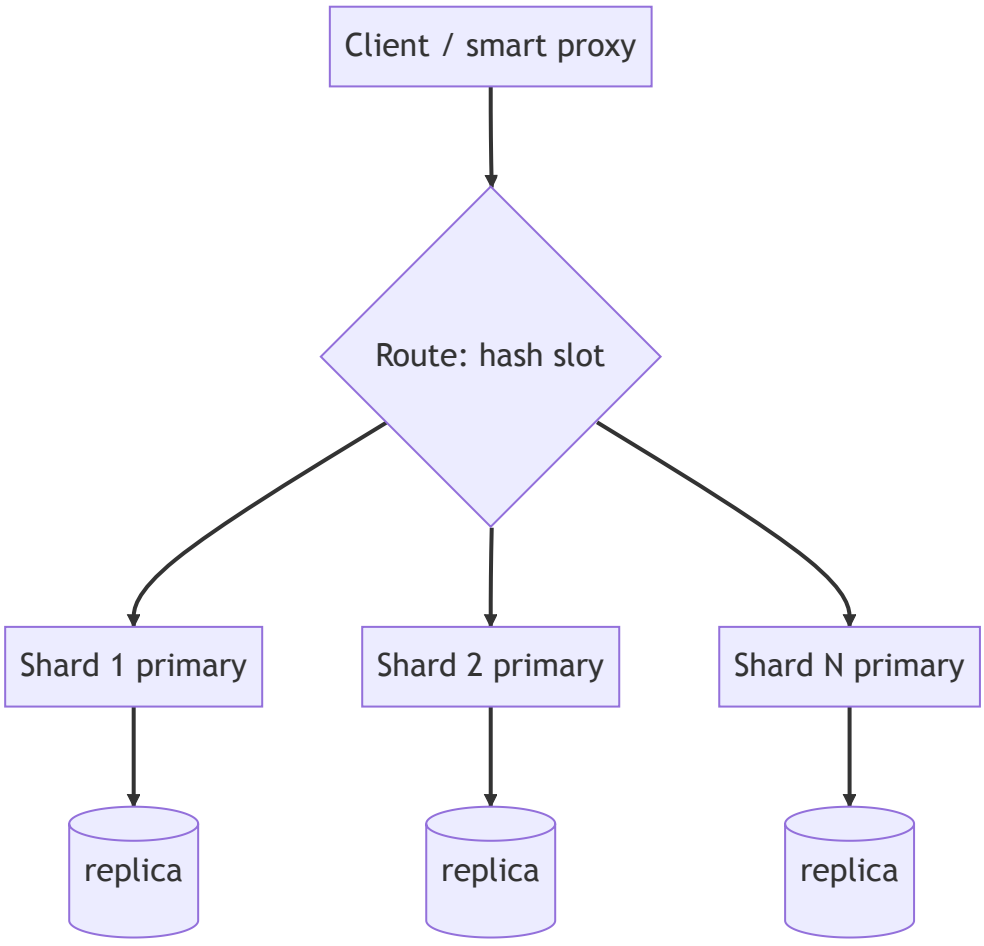
- **Recovery time** ∝ snapshot size + AOF tail length → frequent snapshots keep the tail (and recovery) short.

---

9. Replication & sharding (focus area)

Sharding (horizontal scale, partition the keyspace)

- **Hash partitioning** — `shard = hash(key) mod N`, or better **consistent hashing** / **hash slots** (Redis Cluster = 16,384 slots) so adding/removing nodes moves a **minimal** fraction of keys.
- Each shard is an independent single-threaded keyspace → cores/nodes scale linearly.
- **Hash tags** keep related keys on one shard for atomic multi-key ops.
- **Resharding** — move slots between nodes with live migration (copy + delta + cutover); clients follow **MOVED/ASK** redirects or use a routing proxy.



Replication (HA per shard)

- **Primary–replica per shard.** Primary streams its write log (LSN-ordered) to replicas; replicas apply in order.
- **Async (default, Redis-like)** — ack the client immediately, replicate in background → **low latency**, but a primary crash can lose the un-replicated tail → **eventual consistency**.
- **Semi-sync / sync** — ack only after ≥1 replica acks → **stronger durability**, higher latency. Tunable (`WAIT n`).

- **Failover** — a **sentinel/cluster manager** (or Raft) health-checks primaries; on failure, **promote a replica** and repoint clients. Replicas also **serve reads** (read scaling) at the cost of staleness.
- **Quorum option** — for strong consistency, replicate via **Raft/Paxos** (majority ack) — used by strongly-consistent KV stores (etcd, TiKV); higher latency, no lost acked writes.

---

10. Consistency guarantees & trade-offs (focus area)

- **Within a shard (single thread): linearizable** — ops are totally ordered and atomic; CAS/EXEC are correct. This is the strong island.
- **Across replicas (async): eventual consistency** — a replica may lag; reads from replicas can be stale; a failover can lose the un-replicated tail. Default for cache-like deployments (favor latency/availability).
- **Across shards: no global transaction/order** by default — each shard is its own consistency domain. Cross-shard atomicity needs 2PC/Raft (opt-in, costly).
- **Tunable knobs:**
  - **Read-your-writes** → read from primary (or wait for replica LSN ≥ write LSN).
  - **Stronger durability** → `WAIT`/semi-sync or Raft quorum.
  - **Lower latency** → async replication + `everysec` fsync.

CAP/PACELC framing: a single shard is **CP** (linearizable, but unavailable during failover). The replicated system is typically run **AP/eventual** (async replication: stay available + fast, tolerate staleness) — or **CP** if you choose quorum replication. **PACELC:** *else* (no partition) you still trade **Latency vs Consistency** via the fsync + replication-ack settings.

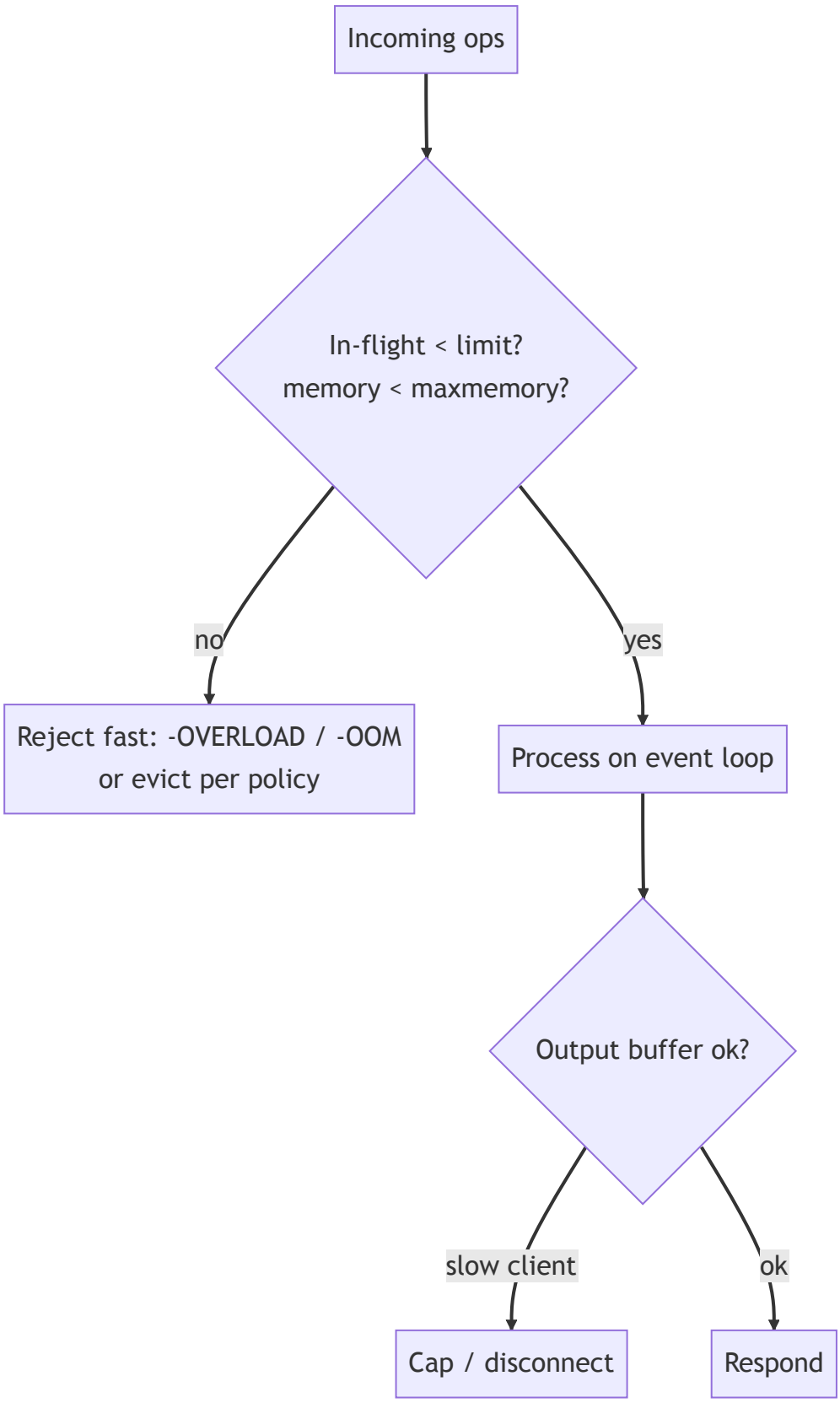
Want	Setting	Cost
Lowest latency	async replication, `fsync no/everysec`	may lose recent writes; stale replica reads
No lost acked writes	sync/quorum replication, `fsync always`	higher write latency
Read-your-writes	read from primary or wait LSN	less read scaling
Strong cross-shard txn	2PC / Raft coordinator	much higher latency/complexity

---

11. Backpressure under overload (focus area)

At 1M QPS, overload **will** happen — degrade gracefully, never melt down.

- **Bounded queues + admission control** — cap the per-connection command pipeline and global in-flight; when full, **reject fast** (busy/`-OOM`/`-OVERLOAD`) instead of unbounded buffering.
- **Memory limit + eviction/`noeviction`** — at `maxmemory`, either evict (cache) or **reject writes** (store) with a clear error; never OOM-kill the process.
- **Client output-buffer limits** — slow consumers (e.g., replicas, pub/sub) get capped buffers; disconnect abusers rather than ballooning RAM.
- **Connection/request rate limits** + **max-clients** cap; **slowlog** to catch expensive commands (big `KEYS`, large-value ops).
- **Flow control to replicas** — if a replica lags, throttle or signal the primary; don't let replication backlog exhaust memory.
- **Load shedding** — under sustained overload, shed low-priority traffic (e.g., reads) to protect writes/replication; **circuit-break** at the client/proxy.
- **Avoid p99 stalls** — incremental rehash, incremental expiry, fork-based (CoW) snapshots, no synchronous big-key deletes (lazy/async free) so one fat operation can't block the loop.



---

12. Observability (a stated requirement)

- **Metrics** — ops/sec by type, **p50/p99/p999 latency**, hit/miss ratio, **memory used vs maxmemory + fragmentation**, evictions/sec, expired keys/sec, keyspace size, **replication lag (LSN delta)**, AOF fsync time, connected clients, rejected/over-limit ops → TSDB + SLOs/alerts.

- **Logs** — slowlog (commands over a threshold), rewrites/snapshots, failovers, evictions, errors; structured.
- **Tracing** — propagate a request/trace id through proxy → shard → replica for cross-node latency attribution; sample to bound overhead.
- **Health/introspection** — `INFO`-style stats, latency histograms, hot-key/big-key detection, per-command stats.
- **Golden signals** — p99 latency, hit ratio, memory headroom, replication lag.

---

13. Failure modes — *"what if X fails?"*

Failure	Impact	Mitigation
Process crash	In-RAM data gone	Recover from **snapshot + AOF replay**; replica takes over meanwhile
Primary node dies	Shard write-unavailable	**Failover**: promote replica, repoint clients; async tail may be lost (eventual)
fsync/disk slow or full	Write stalls / AOF can't flush	Group commit; alert on fsync latency; switch to `everysec`; cap AOF, alert on disk
Torn AOF write (crash mid-append)	Corrupt tail	**CRC per record** → truncate at last valid record on recovery
Memory pressure / maxmemory	Risk of OOM	Eviction policy or reject writes; bounded buffers; never OOM the process
Hot key	One shard/core saturated	Client-side cache, key splitting, replica reads, LFU
Big key / big op	Event-loop stall (p99 spike)	Chunk large values; **lazy/async free**; incremental ops
Replica lag	Stale reads, weak durability	Monitor LSN delta; read-from-primary for RYW; throttle on backlog
Network partition	Split-brain risk	Quorum/sentinel decides primary; minority steps down (CP)
Hash-flood DoS	Degraded to O(n) buckets	**SipHash** seeded per-process; key/size limits
Resharding	Migration errors	Live slot migration + MOVED/ASK redirects; idempotent moves

Guiding principle: a single shard is a **strongly-consistent, recoverable island**; the cluster trades some consistency for availability and latency, and **degrades (evict/shed/redirect) rather than crashes**.

---

14. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Concurrency	Single-thread event loop (simple, no locks)	Multi-thread + locks (all cores)	**Single-thread per shard**, scale by sharding ✅
Durability	`fsync always` (safe)	`everysec`/`no` (fast)	`everysec` group-commit default; `always` for system-of-record
Persistence	AOF (small loss window, slower restart)	Snapshot (fast restore, more loss)	**Both** — snapshot for bulk + AOF tail for recency
Replication	Async (low latency, may lose tail)	Sync/quorum (no loss, slower)	Async for cache; quorum for strong durability
Consistency	Linearizable (CP, quorum)	Eventual (AP, async)	Per shard CP; cluster usually AP — tunable
Eviction	Evict (cache)	noeviction/reject (store)	Match to whether it's a cache or source of truth
Expiry	Lazy (free, lingers)	Active sweep/wheel (reclaims, CPU)	**Both** — lazy + bounded background sweep
Memory vs accuracy	Exact LRU (linked list)	Approx LRU (sampling)	Approximate — near-exact, far cheaper

One-liner to say out loud: *"I'd build it as **single-threaded event-loop shards** over a **hash table with incremental rehash** for O(1) ops with tight p99, **lazy + background TTL expiry**, and **approximate-LRU eviction**. Durability is **AOF with group-commit fsync (everysec by default, always for systems of record) plus fork-CoW snapshots**, recovered as *snapshot → replay AOF tail → truncate torn record*. Scale is **hash-slot sharding** (hash tags keep multi-key txns shard-local, where the single thread makes MULTI/EXEC and CAS trivially atomic) with **primary–replica replication per shard — async for latency, quorum for strong durability**. A single shard is **linearizable (CP)**; the cluster is **tunably AP/eventual** via the fsync + replication-ack knobs, and it **sheds load (bounded queues, eviction, output caps) rather than melting** under overload."*

---

15. Networking, security & performance best practices

Networking

- **Compact binary protocol** (RESP-like) over TCP; **pipelining** to amortize round-trips at high QPS; optional **RDMA/kernel-bypass (DPDK)** for extreme throughput.
- **TLS** with session resumption; co-locate clients/proxy/nodes in one AZ to keep RTT μs-low.
- **Smart client / proxy routing** (slot-aware) to hit the owning shard directly; connection pooling + keep-alive.

Security

- **AuthN** (ACL users/passwords, mTLS); **per-key/command ACLs**; disable dangerous commands (`FLUSHALL`, `KEYS`) in prod.
- **SipHash-seeded** hashing to stop **hash-flooding** DoS; key/value **size limits**.
- **Network isolation** (private subnet, no public exposure — the classic unauthenticated-Redis breach); encrypt snapshots/AOF at rest.

Performance

- **O(1) structures**, incremental rehash/expiry, **fork-CoW** snapshots, **lazy/async free** of big keys → no event-loop stalls.
- **Group-commit fsync**; **approximate LRU/LFU**; per-core sharding for linear scale.

- **NUMA-aware** pinning; jemalloc/tcmalloc to curb fragmentation.

---

16. Staying current — modern & emerging approaches

- **Reference systems: Redis** (+ Redis Cluster, AOF/RDB), **Memcached** (multi-threaded, slab), **Aerospike** (SSD-optimized), **DragonflyDB** (multi-threaded, shared-nothing), **KeyDB** (multi-threaded Redis fork).
- **Strongly-consistent KV: etcd**, **TiKV**, **FoundationDB** (Raft/Paxos, distributed txns).
- **Memory:** jemalloc/tcmalloc, **NUMA**, huge pages; persistent memory tiers; **tiered RAM+SSD** (Redis on Flash, Aerospike).
- **Concurrency:** io_uring, DPDK/kernel-bypass, lock-free/shared-nothing per-core designs (DragonflyDB, Seastar).
- **Consistency:** Raft/Paxos, CRDTs (Redis active-active / geo), leases.
- **How I stay current:** Redis/Dragonfly/Aerospike engineering blogs, the Raft & FoundationDB papers, USENIX/VLDB talks, and benchmarking before adopting.

---

17. Likely follow-up questions (rehearse these)

- Single-threaded but fast — how? *(in-RAM O(1) + no lock contention; scale via sharding)*
- How do you keep p99 low during a resize? *(incremental rehash — migrate a few buckets per op)*
- `fsync always` vs `everysec`? *(durability vs throughput; ack-after-fsync vs ≤1 s loss window)*
- Crash mid-write to the AOF? *(CRC per record → truncate the torn tail on recovery)*
- Atomic update across two keys? *(same shard via hash tags; MULTI/EXEC on the single owning thread)*
- Lost-update protection? *(versioned CAS / WATCH optimistic concurrency)*
- Primary dies with un-replicated writes? *(async = may lose the tail (eventual); use quorum/WAIT for none)*
- Memory full — then what? *(evict per policy for a cache; reject writes for a store)*
- Hot key saturating one core? *(replica reads, client cache, key splitting, LFU)*
- Strong consistency across the cluster? *(Raft/quorum per shard + 2PC across shards — call out the latency cost)*

---

18. Summary checklist (whiteboard recap)

- **O(1) ops** — hash table with **incremental rehash**; entry holds value + version + expire_at + eviction meta.
- **TTL** — **lazy expiry + bounded background sweep** (timing wheel / sampling).
- **Eviction** — **approximate LRU/LFU**; `noeviction` for a store, evict for a cache.
- **Concurrency** — **single-threaded event loop per shard** → atomic ops, CAS, MULTI/EXEC free; scale by sharding.
- **Atomic multi-key** — shard-local via **hash tags**; `WATCH`/version for optimistic txns.
- **Durability** — **AOF (group-commit fsync) + fork-CoW snapshots**; LSN orders writes.
- **Recovery** — **snapshot → replay AOF tail (lsn>last) → CRC-truncate torn record → drop expired → serve**.
- **Scale** — **hash-slot sharding** + **primary–replica per shard**; async (latency) vs quorum (durability).
- **Consistency** — per-shard **linearizable (CP)**; cluster **tunably AP/eventual** (PACELC: latency vs consistency).
- **Backpressure** — bounded queues, eviction/reject, output-buffer caps, load shedding → degrade, don't melt.
- **Observability** — p99 latency, hit ratio, memory headroom, **replication lag (LSN)**, fsync time, evictions.

Designing a Distributed Rate Limiter

> A complete, interview-ready walkthrough: requirements → estimates → API → algorithms → architecture → distributed challenges → failure handling → trade-offs. Use the headings as your whiteboard agenda.

---

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements (don't assume).
2. **Estimate** scale (QPS, keys, memory) to justify choices.
3. **Define the API / contract** and where the limiter lives.
4. **Pick an algorithm** and explain why (trade-offs vs alternatives).
5. **Go distributed**: shared state, atomicity, race conditions, latency.
6. **Handle failure**: Redis down, hot keys, network partitions.
7. **Summarize trade-offs** (consistency vs availability vs latency).

Keep saying *"here's the trade-off…"* — that's what's being graded.

---

1. Problem & motivation

A **rate limiter** controls how much traffic a client/service may send within a window. Excess traffic is **throttled** (HTTP `429 Too Many Requests`).

Why we need it:

- **Protect** backends from overload / cascading failures.
- **Security**: blunt brute-force, credential stuffing, scraping, L7 DoS.
- **Cost control**: cap usage of expensive downstreams (3rd-party APIs, LLM tokens, DB).
- **Fairness / QoS**: stop one noisy tenant from starving others.
- **Monetization**: enforce plan tiers (Free = 100/min, Pro = 10k/min).

---

2. Requirements

Functional

- Throttle requests that exceed a configured limit (e.g., `100 req/min per user`).
- **Configurable rules** per dimension: user ID, API key, IP, endpoint, tenant, plan tier.
- Support **multiple limits at once** (e.g., 10/sec **and** 1000/hour).
- Return a clear signal when throttled (`429` + `Retry-After`).
- Rules are **hot-reloadable** without redeploy.

Non-functional

- **Low latency**: add only single-digit ms to each request.
- **High availability**: the limiter must not become a single point of failure that takes down the app.
- **Scalable**: millions of req/sec, 100M+ unique keys.
- **Accurate enough**: small over-count at boundaries is usually acceptable.

- **Distributed-correct**: consistent decisions across many limiter instances.

Clarifying questions to ask the interviewer

- Per-user, per-IP, per-API-key, or global limits? (Often **all**, layered.)
- Hard limit (reject) or soft limit (queue/delay)?
- Is **approximate** counting acceptable, or must it be exact?
- Single region or **multi-region**? Global budget or per-region?
- On limiter failure, do we **fail-open** (allow) or **fail-closed** (deny)?

---

3. Back-of-the-envelope estimation

Assume **1B requests/day**:

- Average ≈ `1e9 / 86,400` ≈ **~12k req/sec**; peak (5×) ≈ **~60k req/sec**.
- Each check = 1–2 Redis ops (read + atomic increment).
- **Memory**: ~~100M active keys × (≈16B key + 8B counter + overhead ≈ 100B) ≈ **~~10 GB** → fits in a small Redis cluster, fully in-memory.
- Counters are short-lived (TTL = window), so memory stays bounded.

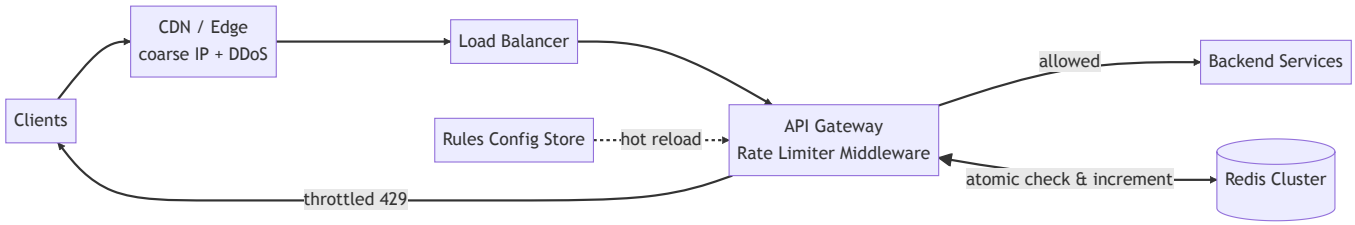
Takeaway: an in-memory store (Redis) easily handles this; the real challenges are **atomicity, latency, and failure handling**, not raw storage.

---

4. Where does the rate limiter live?

Placement	Pros	Cons
Client-side	No server cost	Untrusted, easily bypassed/tampered — never the only defense
In-app middleware	Full request context (user, route)	Couples limiter to every service; duplicated logic
API Gateway / dedicated edge ✅	Centralized, language-agnostic, also does auth/TLS/routing	Extra hop; must be HA

Recommendation: a **dedicated middleware at the API Gateway** (Envoy, Kong, NGINX, AWS API Gateway, Cloudflare). Push coarse IP/DDoS limits to the **CDN/edge**; do fine-grained per-user/key limits at the gateway.



---

5. API & response contract

Config / rules API (control plane):

```
POST /v1/rules
{
  "key": "plan:pro",
  "dimension": "api_key",
  "limit": 10000,
  "window_seconds": 60,
  "algorithm": "token_bucket",
  "burst": 2000
}
```

Enforcement (data plane) is transparent — the limiter inspects each request and either forwards it or returns:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1717612800
```

Returning `X-RateLimit-*` headers on **successful** responses too lets well-behaved clients self-throttle.

---

6. Algorithms (know all five + trade-offs)

Algorithm	Memory	Burst handling	Accuracy	Notes / used by
Token Bucket	Low (2 fields/key)	Allows bursts up to bucket size	Good	Smooth, intuitive. *Stripe, AWS*
Leaky Bucket	Low (queue)	Smooths to constant out-rate	Good	Adds latency; can starve new reqs. *Shopify*
Fixed Window	Lowest (1 counter)	**Bad** — 2× burst at edges	Coarse	Simplest; boundary spike problem
Sliding Window Log	**High** (timestamp/req)	Exact	**Best**	Stores every timestamp; costly memory
Sliding Window Counter ✅	Low	Smooths edges	~99.99%	Weighted prev+curr window. *Cloudflare*

Token Bucket (recommended default)

- Bucket of capacity `B`, refilled at `R` tokens/sec. Each request consumes 1 token; empty ⇒ reject.
- **Pros:** memory-efficient, allows controlled bursts, easy to reason about.
- **Cons:** two parameters (`B`, `R`) to tune.

Fixed Window — the boundary problem

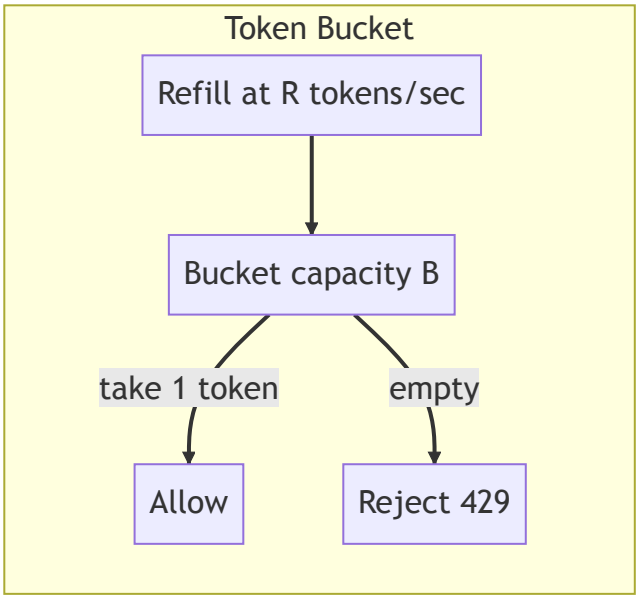
With `100/min`, a client can send 100 at `00:00:59` and another 100 at `00:01:00` → **200 in ~1s**. This is why we prefer sliding approaches.

Sliding Window Counter (recommended when accuracy matters)

Approximate the rolling window by weighting the previous fixed window:

```
count = curr_window_count
      + prev_window_count * (overlap_fraction_of_prev_window_still_in_view)
allow if count < limit
```

Smooths the edge spike, uses ~2 counters per key, and Cloudflare measured error well under 1%.



---

7. Distributed design — the hard part

With many limiter instances behind a load balancer, **counter state must be shared and updated atomically**.

7.1 Where to keep counters

- **Centralized Redis (cluster)** — single source of truth; atomic ops; TTL eviction.
- Use **Redis Cluster** sharded by key for horizontal scale; replicas for HA.

7.2 Race conditions (the classic bug)

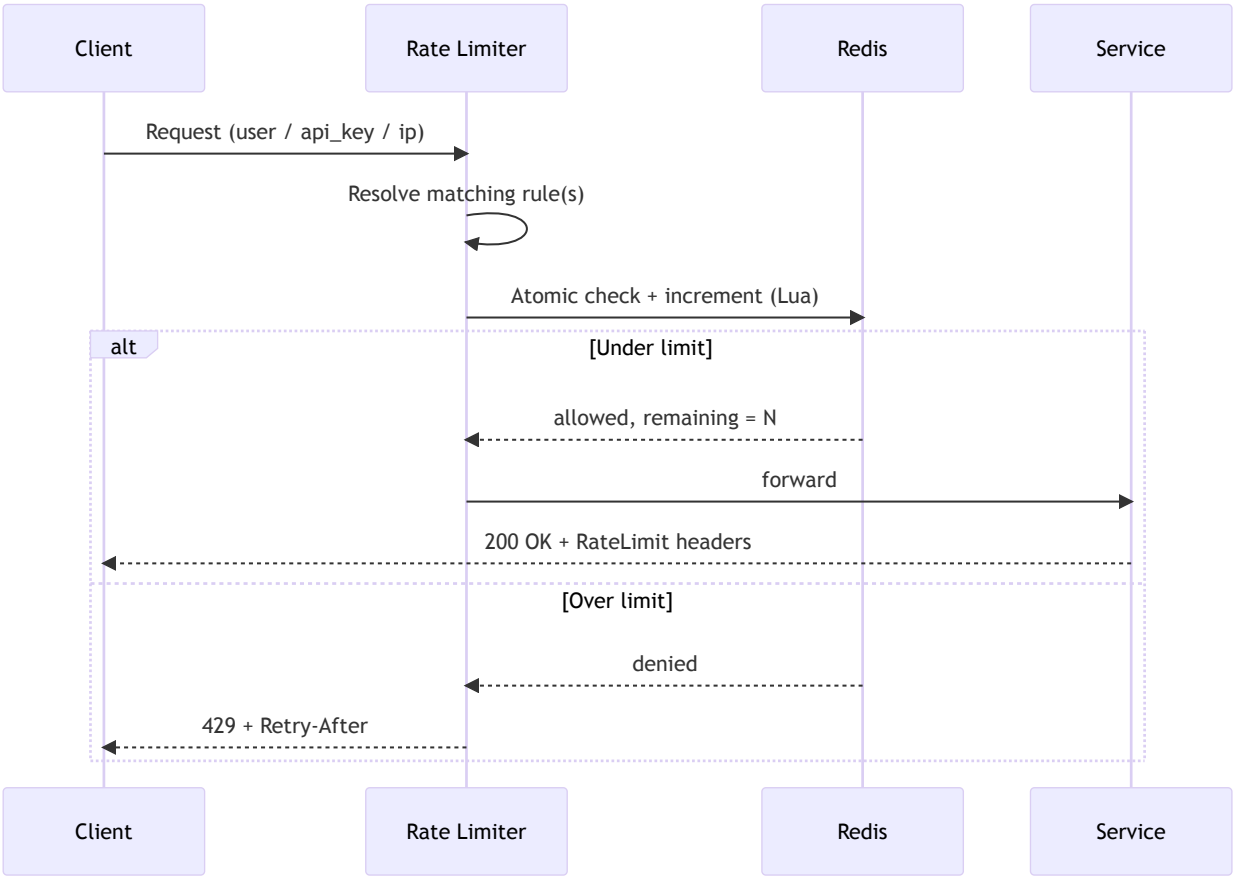
Naïve `GET` → check → `SET` is a **read-modify-write race**: two instances both read `99`, both allow, count becomes `101`.

Fixes (pick atomic):

- **`INCR` + `EXPIRE`** (fixed window) — `INCR` is atomic.
- **Lua script** — runs atomically on the Redis server (best for token/sliding-window logic).
- **Sorted sets** (`ZADD`/`ZREMRANGEBYSCORE`/`ZCARD`) for sliding-window log.
- Avoid distributed **locks** — correct but slow; they kill throughput.

7.3 Latency

- **Co-locate** Redis with limiters (same AZ/region) to keep RTT ~sub-ms.
- For ultra-low latency, allow a **local in-memory cache** with periodic sync (trade accuracy for speed).



---

8. Reference implementations (Redis)

Fixed window (simple, atomic)

```
key = "rl:{user}:{window_start_epoch}"
count = INCR key
if count == 1:        # first hit in this window
    EXPIRE key window_seconds
if count > limit:
    reject (429)
else:
    allow
```

Token bucket (atomic via Lua — avoids races)

```
-- KEYS[1] = bucket key
-- ARGV: capacity, refill_rate, now_ms, requested(=1)
local b   = redis.call("HMGET", KEYS[1], "tokens", "ts")
local cap = tonumber(ARGV[1])
local rate= tonumber(ARGV[2])      -- tokens per ms
local now = tonumber(ARGV[3])
local req = tonumber(ARGV[4])

local tokens = tonumber(b[1]) or cap
```

```
local ts     = tonumber(b[2]) or now
tokens = math.min(cap, tokens + (now - ts) * rate)   -- refill

local allowed = tokens >= req
if allowed then tokens = tokens - req end

redis.call("HMSET", KEYS[1], "tokens", tokens, "ts", now)
redis.call("PEXPIRE", KEYS[1], math.ceil(cap / rate))
return allowed and 1 or 0
```

Lua guarantees the **refill + check + decrement** happen as one atomic step.

---

9. Failure scenarios — *"what if X fails?"*

Failure	Impact	Mitigation
Redis node down	Can't read counters	Replicas + **Sentinel/Cluster** failover; **circuit breaker** → local fallback
Redis cluster unreachable	Global enforcement lost	Decide policy: **fail-open** (availability) vs **fail-closed** (protection); usually fail-open with a local cap
Hot key (one viral user/IP)	Single shard overloaded	Local pre-aggregation, key splitting/sharding, edge IP limits
Limiter instance dies	Capacity loss	Stateless instances behind LB; just add more
Network partition	Split-brain counts	Accept eventual consistency / over-count; reconcile after heal
Clock skew	Wrong windows	Use Redis server time (`TIME`) as source of truth

Fail-open vs fail-closed: for most user-facing APIs, **fail-open** (allow traffic if the limiter is down) preserves availability — losing rate limiting briefly is better than an outage. For abuse-sensitive endpoints (login, payments), **fail-closed**.

---

10. Scaling the system

- **Limiter tier:** stateless → scale horizontally behind the LB.
- **Redis: Cluster** sharded by key; add shards as keys grow; replicas for reads/HA.
- **Hot keys:** two-tier counting — buffer locally per node, flush deltas to Redis every few ms (trades a little accuracy for huge throughput).
- **Multi-region:**
  - *Independent regional limits* (simple, low latency, but global budget can be exceeded) — usually fine.
  - *Global budget* (strong, but needs cross-region coordination → latency) — only if strictly required.
- **Layered limits:** global → per-service → per-tenant → per-user → per-IP, evaluated cheapest-first.

---

11. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Consistency vs Latency	Centralized Redis (accurate)	Local cache + async sync (fast, approximate)	Most APIs accept slight over-count → favor latency
Availability vs Protection	Fail-open	Fail-closed	Fail-open for general APIs; fail-closed for auth/payments
Accuracy vs Memory	Sliding-window **log**	Sliding-window **counter**	Counter ≈ same accuracy, far less memory → preferred
Burst tolerance vs Smoothness	Token bucket (bursts ok)	Leaky bucket (constant rate)	Token bucket for APIs; leaky for steady downstreams
Simplicity vs Correctness	Fixed window	Sliding window	Sliding avoids 2× boundary burst

One-liner to say out loud: *"I'd start with a token-bucket limiter at the API gateway, counters in a co-located Redis Cluster updated atomically via Lua, fail-open on Redis outage with a conservative local cap, and move to local pre-aggregation only if hot keys or latency force it."*

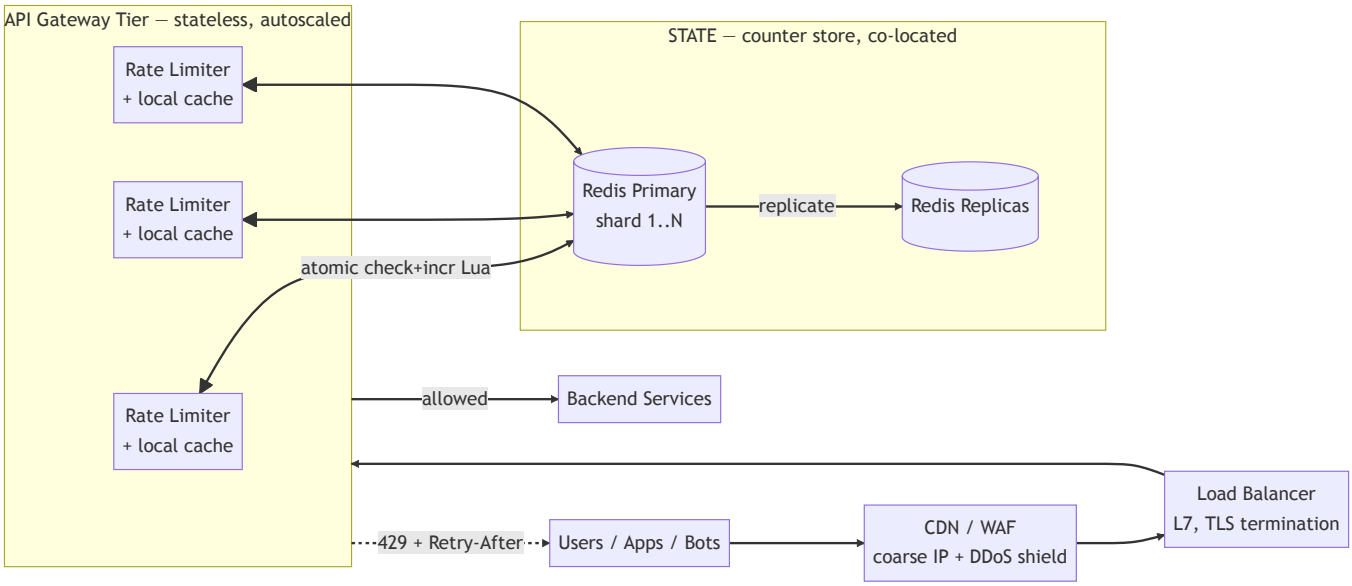
CAP framing: the counter store forces a choice under partition — **fail-open = AP** (stay available, risk over-admitting) vs **fail-closed = CP** (stay correct, risk rejecting valid traffic). Most rate limiters lean **AP**: availability of the protected service usually matters more than perfectly exact counts.

---

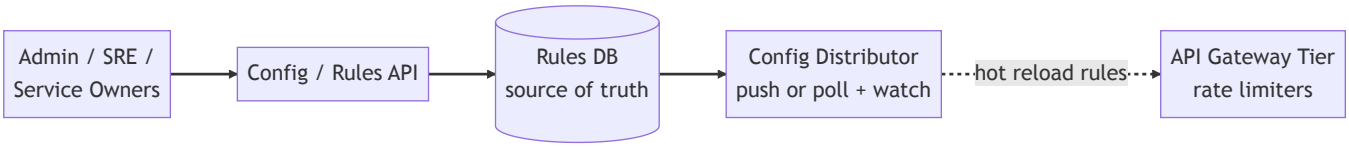
12. Full system design (detailed)

End-to-end view, split into three diagrams for clarity: **(A)** the hot request path (data plane + state), **(B)** the control plane (rule management), and **(C)** observability.

12A. Data plane — the hot request path

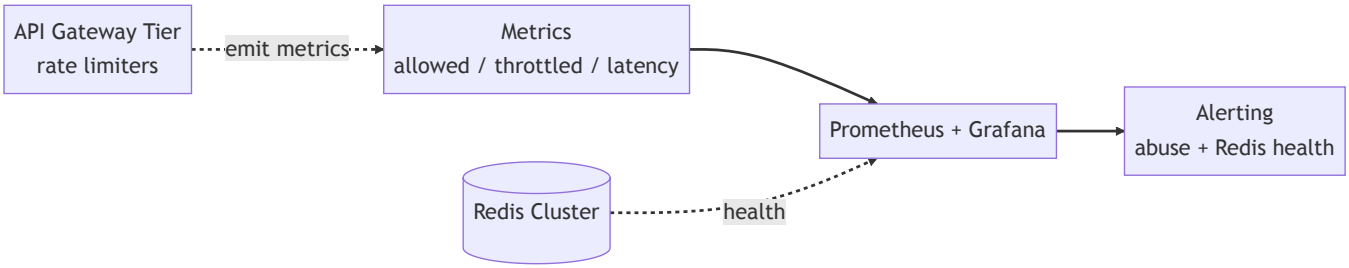


12B. Control plane — rule management (cold path)



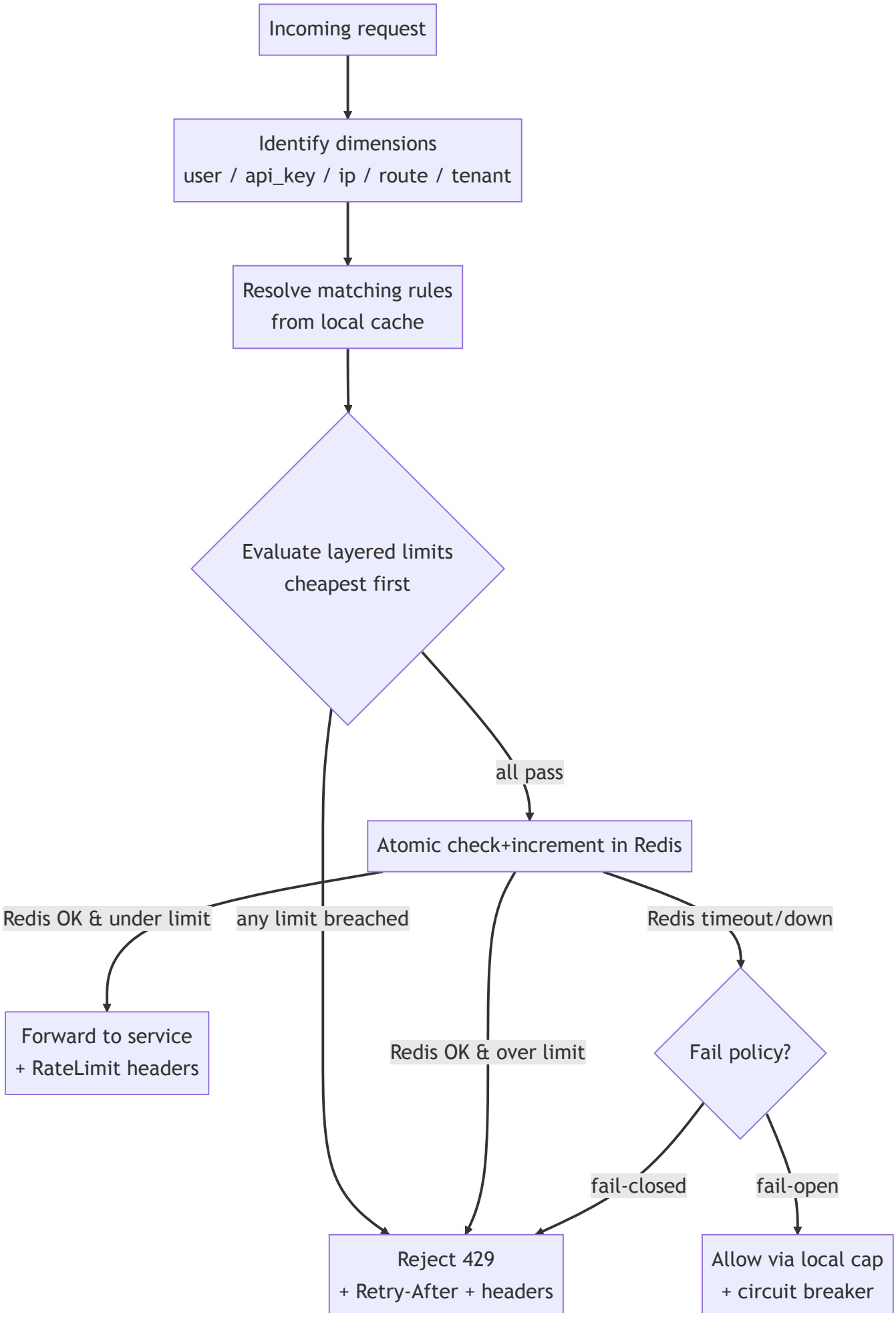
The Rules API + DB are the source of truth; the distributor pushes (or limiters poll + watch) so rules hot-reload into every limiter with **no redeploy**.

12C. Observability



Per-decision metrics (allowed / throttled / latency) drive dashboards and abuse alerts; Redis health feeds the same monitoring so on-call sees failover early.

Request decision logic (inside each limiter)


Incoming request
Identify dimensions
user / api_key / ip / route / tenant
Resolve matching rules
from local cache
Evaluate layered limits
cheapest first
all pass
Atomic check+increment in Redis
Redis OK & under limit
any limit breached
Redis timeout/down
Forward to service
+ RateLimit headers
Redis OK & over limit
Fail policy?
fail-closed
fail-open
Reject 429
+ Retry-After + headers
Allow via local cap
+ circuit breaker

Components recap

- **CDN / WAF (edge):** coarse IP-based + DDoS protection before traffic hits us.
- **Load Balancer:** L7 routing + TLS, spreads load across limiter instances.
- **Rate Limiter middleware:** stateless, evaluates layered rules, allow/deny, emits metrics.
- **Redis Cluster:** atomic counters, TTL eviction, sharded by key + replicated for HA.
- **Control plane:** Rules API + DB are the source of truth; a distributor hot-reloads rules into limiters (push or poll-and-watch), so no redeploy is needed.
- **Observability:** per-decision metrics drive dashboards, abuse alerts, and Redis health checks.

---

13. Networking, security & performance best practices

Networking

- **Terminate TLS at the edge/LB**; keep limiter→Redis traffic on a private network in the same AZ to minimize RTT.
- **Connection pooling + keep-alive** to Redis — reuse connections instead of reconnecting per request.
- **Pipeline / `MULTI` / single Lua call** to batch ops and cut round-trips (one atomic call beats several).
- **Tight timeouts** on the Redis call (e.g., 5–10 ms) so a slow store can't stall the request path; then apply the fail-open/closed policy.
- **Retries with capped exponential backoff + jitter** — never unbounded; retry storms amplify outages.
- **Anycast/CDN** to absorb volumetric attacks close to the client.

Security

- **Don't trust client-supplied identity.** Derive the IP from the real connection or a *validated* `X-Forwarded-For` — the header is spoofable, so attackers rotate it to dodge per-IP limits.
- **Secure the control plane.** AuthN + RBAC + audit on the rules API so limits can't be tampered with or silently disabled.
- **Layered defense.** Coarse IP/DDoS at the edge/WAF + fine-grained per-identity limits at the gateway; stricter **fail-closed** limits on auth/payment endpoints (brute force, credential stuffing).
- **The limiter must be cheap.** Reject over-limit traffic *before* expensive work (auth, DB) so the limiter isn't itself a DoS amplifier.
- **Avoid info leaks.** Don't let `RateLimit-*` headers or error messages reveal whether an account/key exists (enumeration).

Performance

- **O(1) atomic ops**, no distributed locks (see §7.2).
- **L1 local cache** for hot keys + async flush (see §10) to shed Redis load.
- **Pre-resolve rules** from a local cache; never hit the control plane on the hot path.
- **TTL every key** so memory stays bounded; **sample** metrics to keep observability overhead low.

---

14. Staying current — modern & emerging approaches

- **Managed / cloud-native:** AWS API Gateway usage plans & throttling, Cloudflare Rate Limiting, Azure API Management, GCP Apigee, Kong/Tyk — offload limiting to the platform when it fits.
- **Service mesh:** Envoy's **global rate-limit service (RLS)** backed by Redis, deployed via Istio — limiting as a sidecar concern with no app changes.
- **Battle-tested algorithms/libraries: GCRA** (Generic Cell Rate Algorithm) via the **`redis-cell`** module; app-side libs like **bucket4j**, **resilience4j**, Guava `RateLimiter` (Java), `golang.org/x/time/rate` (Go).
- **Emerging: eBPF/XDP** for L3/L4 limiting at kernel/line speed; **WASM** filters in Envoy; **CRDT-based counters** for multi-region eventual consistency without coordination.

- **Standards:** the IETF **`RateLimit` header fields** draft is standardizing `RateLimit-Limit/Remaining/Reset` — prefer it over ad-hoc `X-RateLimit-*`.
- **How I stay current:** engineering blogs (Cloudflare, Stripe, AWS Builders'), conference talks, and prototyping + benchmarking before adopting.

---

15. Likely follow-up questions (rehearse these)

- Token bucket vs sliding window — when each? *(burst tolerance vs strict accuracy)*
- How do you make the counter update atomic? *(`INCR`/Lua, not GET+SET)*
- Redis goes down — what now? *(fail-open + circuit breaker + local cap)*
- One user sends 1M req/sec to one key — hot key mitigation? *(edge limit + local pre-agg + key split)*
- Multi-region global limit — how? *(coordination cost vs per-region budgets)*
- How do you limit on multiple dimensions at once? *(evaluate layered rules, reject on first breach)*
- Exactly-once vs approximate counting trade-off?

Distributed Rate Limiter — One-Page Cheat Sheet

> Print: open in VS Code → Markdown preview → ⌘P / Print → Save as PDF. Dense by design.

Talk track (order on whiteboard)

Clarify → Estimate → Placement → Algorithm → Distributed/atomicity → Failure → Trade-offs

Requirements

- **Functional:** throttle over-limit traffic; configurable rules per user/IP/API-key/tenant/route; multiple simultaneous limits (10/s **and** 1k/h); return `429`; hot-reloadable rules.
- **Non-functional:** single-digit-ms latency; HA (not a SPOF); millions req/s, 100M+ keys; approximate counting OK.
- **Ask:** per-user vs IP vs global? hard vs soft? exact vs approx? multi-region? **fail-open vs fail-closed?**

Estimation (1B req/day)

- ~12k req/s avg, ~60k peak (5×). Check = 1–2 Redis ops.
- 100M keys × ~~100B ≈ **~~10 GB** → small Redis cluster, in-memory, TTL-bounded.
- *Conclusion:* storage is easy; **atomicity + latency + failure** are the hard parts.

Placement

Client (untrusted) ✗ · In-app middleware (coupled) ~ · **API Gateway (dedicated)** ✅ · Edge/CDN for coarse IP+DDoS.

Algorithms

Algo	Mem	Burst	Accuracy	Use
Token bucket ✅	Low	Allows bursts	Good	Default for APIs (Stripe/AWS)
Leaky bucket	Low	Constant out-rate	Good	Smooth downstreams; adds latency
Fixed window	Lowest	**2× edge spike**	Coarse	Simplest only
Sliding log	High	Exact	Best	Costly (timestamp/req)
Sliding counter ✅	Low	Smoothed	~99.99%	Accuracy w/o memory (Cloudflare)

- **Token bucket:** capacity `B`, refill `R`/s; take 1/req; empty ⇒ `429`. Memory-light, controlled bursts.
- **Fixed-window bug:** 100 at `00:59` + 100 at `01:00` = 200 in ~1s → use sliding.

Distributed correctness (the hard part)

- **Shared state:** centralized **Redis Cluster** (sharded by key, replicas, TTL eviction).
- **Race:** `GET→check→SET` lets two nodes both read 99 → over-count. **Fix = atomic:**
  - `INCR+EXPIRE` (fixed window) · **Lua** (token/sliding, atomic refill+check+decrement) · `ZADD/ZCARD` (sliding log).

    - **Avoid distributed locks** (kill throughput).
- **Latency:** co-locate Redis with limiters (same AZ); optional local cache + async sync (trade accuracy for speed).

Atomic fixed-window (Redis)

```
key = rl:{user}:{window_start}
c = INCR key
if c == 1: EXPIRE key window
allow if c <= limit else 429
```

Failure — "what if X fails?"

Failure	Mitigation
Redis node down	Replicas + Sentinel/Cluster failover; circuit breaker → local fallback
Cluster unreachable	**Fail-open** (avail) vs **fail-closed** (protect); usually fail-open + local cap
Hot key	Edge IP limit + local pre-aggregation + key splitting
Instance dies	Stateless behind LB → add more
Partition	Accept eventual over-count; reconcile after heal
Clock skew	Use Redis `TIME` as source of truth

Fail-open general APIs (availability > brief loss of limiting) · **Fail-closed** login/payments.

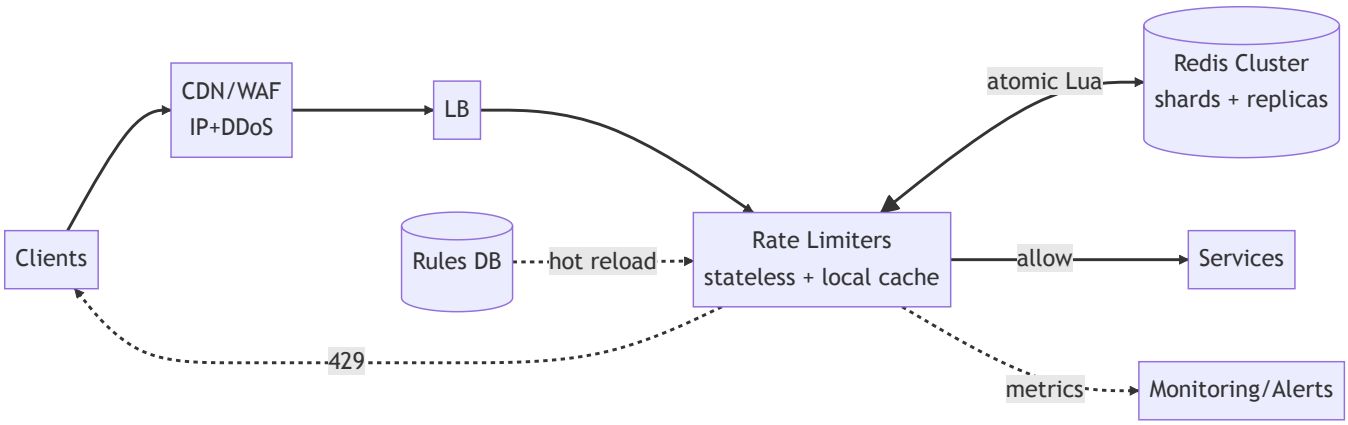
Scaling

- Limiter tier **stateless** → scale horizontally. Redis **Cluster** sharded; add shards as keys grow.
- **Hot keys:** two-tier — buffer locally, flush deltas every few ms.
- **Multi-region:** per-region budgets (simple, fast) vs global budget (coordination → latency).
- **Layered limits:** global → service → tenant → user → IP, cheapest-first, reject on first breach.

Trade-offs (say "here's the trade-off…")

Axis	A	B	Pick
Consistency↔Latency	Central Redis	Local+async	Latency (slight over-count OK)
Availability↔Protection	Fail-open	Fail-closed	Open for APIs / closed for auth
Accuracy↔Memory	Sliding **log**	Sliding **counter**	Counter
Burst↔Smooth	Token	Leaky	Token for APIs
Simple↔Correct	Fixed	Sliding	Sliding

Architecture (one glance)



Closing one-liner

"Token-bucket limiter at the API gateway, counters in co-located Redis Cluster updated atomically via Lua, fail-open with a conservative local cap on Redis outage, and local pre-aggregation only if hot keys or latency demand it."

Designing a Slack-like Team Messaging System (Real-Time Focus)

> A complete, interview-ready walkthrough: requirements → estimates → entities → APIs → architecture → **real-time delivery, missed-message recovery, ordering, fan-out** → notifications → cold start → storage/scaling → failure handling → trade-offs. Use the headings as your whiteboard agenda. The hard parts are **real-time delivery, ordering, and recovering missed messages across reconnects** — spend your time there.

> Think Slack / Microsoft Teams / Discord. Persistent connections deliver messages to channels and DMs in real time; clients must never lose or reorder messages, even across disconnects and offline periods.

---

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements.
2. **Estimate** scale (users, concurrent connections, messages/sec, fan-out).
3. **Model entities** (users, workspaces, channels, DMs, messages) + their keys.
4. **Define the APIs** (send/receive) and the **real-time transport** (WebSocket).
5. **Walk the send path**: client → gateway → persist → fan-out → deliver.
6. **Nail ordering** (per-channel sequence) and **missed-message recovery** (cursor + backfill).
7. **Notifications** + the latency/correctness/UX trade-offs.
8. **Cold start** after offline.
9. **Storage, queues, fan-out, scale**; then **failure + trade-offs**.

Keep saying *"here's the trade-off…"* — that's what's being graded.

---

1. Problem & motivation

Real-time team chat: users in **workspaces** exchange messages in **channels** (group) and **DMs** (1:1 / small group), delivered to all members' devices **instantly**, durably, and **in order** — surviving disconnects, offline periods, and multi-device usage.

What makes it hard:

- **Persistent connections at scale** — millions of always-on WebSockets, not request/response.
- **Fan-out** — one message to a 50k-member channel must reach every online member's device(s) fast.
- **Ordering** — everyone in a channel must see messages in the **same order**; no gaps, no duplicates.
- **Reliability across disconnects** — flaky networks are the norm; a client must **recover exactly the messages it missed**.
- **Multi-device** — the same user on phone + laptop must stay in sync (read state, delivery).
- **Cold start** — opening the app after hours offline must sync efficiently without downloading everything.

The central tension: a message must be **durably stored, globally ordered within its channel, and pushed in real time** — then **reconcilable** when a client was away.

---

2. Requirements

Functional

- **Send/receive messages** in channels and DMs; edits, deletions, threads, reactions, attachments.
- **Real-time delivery** to all online members' devices (persistent connection).
- **Missed-message recovery** — fetch everything missed while disconnected.
- **Ordering** — consistent per-channel order across all clients.
- **Presence & typing indicators**; read receipts / unread counts.
- **History** — scrollback / search of past messages.
- **Notifications** — push to offline/background devices (mobile/desktop/email).

Non-functional

- **Low latency** — p99 end-to-end delivery < ~200 ms for online users.
- **High availability** — chat must stay up; degrade gracefully.
- **Durability** — an acked message is **never lost**.
- **Scalable** — 10M+ concurrent connections, millions of msgs/sec at peak fan-out.
- **Ordered & exactly-once *to the UI*** — no gaps, no dupes after dedup.
- **Eventual consistency** acceptable for presence/read-state; **strong ordering** within a channel.

Clarifying questions to ask the interviewer

- **Channel size** — small teams or **giant** channels (50k+)? (Changes fan-out strategy.)
- **Ordering guarantee** — strict per-channel total order, or causal? (Assume **per-channel total order**.)
- **Delivery semantics** — at-least-once + client dedup (→ exactly-once UI), acceptable? (Yes.)
- **Retention** — forever, or N days? Affects storage + search.
- **Multi-device** sync required? (Yes.)
- **E2E encryption**? (Assume server-side encryption at rest, not E2E, like Slack — say so.)
- **Global** users → multi-region? (Yes → regional gateways, home-region per workspace.)

---

3. Back-of-the-envelope estimation

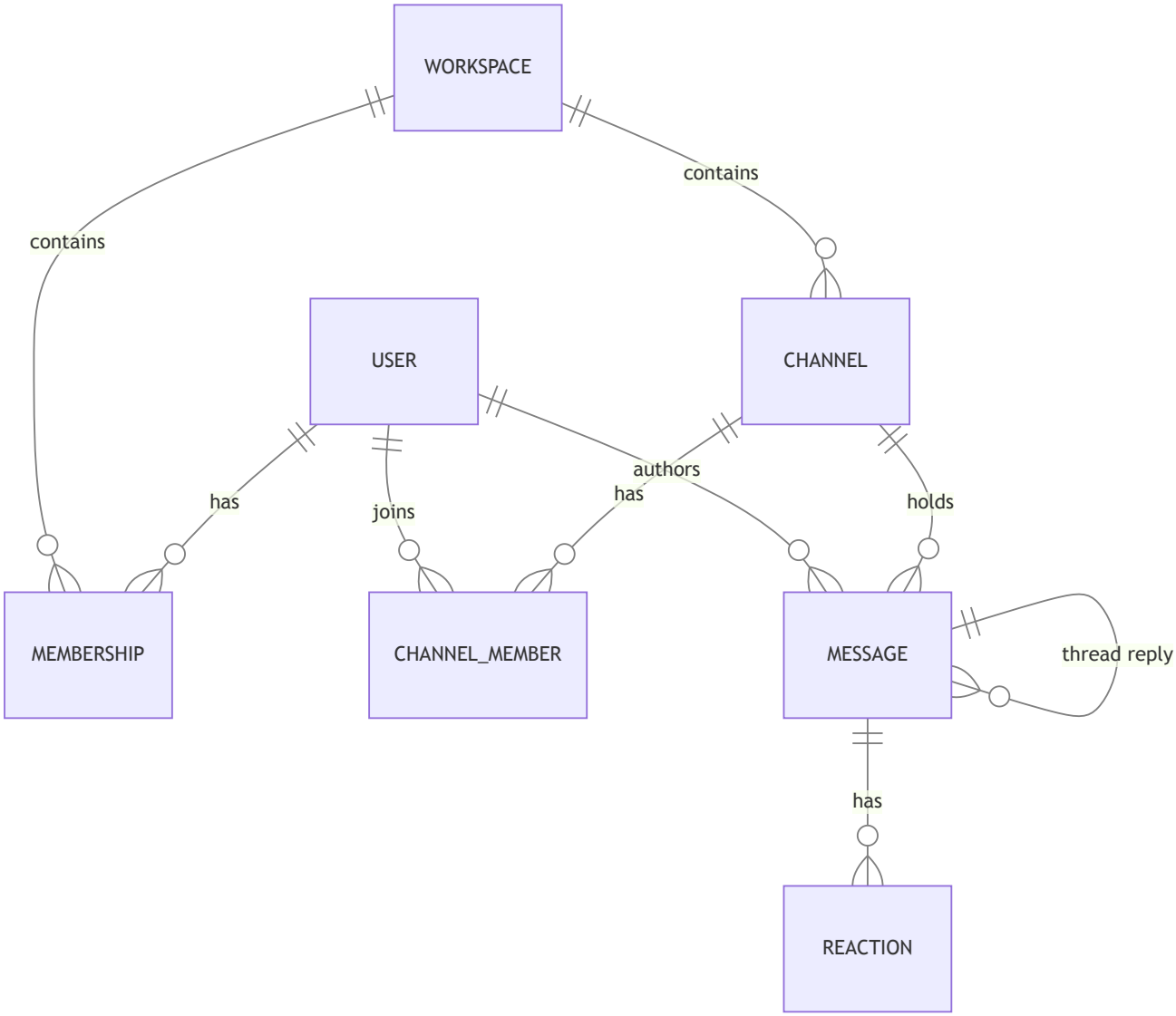
Quantity	Assumption	Result
DAU	—	50M
Concurrent connections	~40% online, ~1.5 devices	**~30M live WebSockets**
Messages sent/day	~20 msgs/active user	~~1B/day ≈ **~~12K/sec avg, ~50–100K/sec peak**
Fan-out factor	avg ~50 recipients/msg (channels)	**~50B deliveries/day ≈ ~600K/sec avg, several M/sec peak**
Connections per gateway	~100K WS/node	**~300 gateway nodes** for 30M conns
Message size	~1 KB metadata + text	~1 TB/day text; attachments to object store
Storage	1B msgs/day × 1 KB × retention	**~PB-scale** message store (sharded, tiered)
Read:write	history reads + delivery >> sends	delivery (fan-out) dominates → optimize the **push path**

Takeaways that drive the design:

1. **30M persistent connections** → a dedicated, horizontally-scaled **connection/gateway tier** decoupled from business logic.
2. **Fan-out is the bottleneck** (millions of deliveries/sec) → a **message queue + fan-out service**, not synchronous loops.
3. **Per-channel ordering** at this rate → assign a **monotonic sequence per channel** at persist time.
4. **PB-scale history** → wide-column store partitioned by channel + time; attachments in object store + CDN.

---

4. Core entities & data model



Entity	Key fields	Notes
User	`user_id`, name, avatar, status	Global identity
Workspace	`workspace_id`, name, plan	Tenant boundary; users belong via membership
Membership	`(workspace_id, user_id)`, role	Workspace-level RBAC
Channel	`channel_id`, workspace_id, type (public/private), name	DMs modeled as a channel with type=`dm`
ChannelMember	`(channel_id, user_id)`, last_read_seq	**last_read_seq drives unread + recovery**
Message	`message_id`, channel_id, **seq**, sender_id, ts, body, type, thread_id, edited, deleted	`seq` = per-channel monotonic order
Reaction	`(message_id, user_id, emoji)`	

DMs = channels. A 1:1 DM is just a channel with two members (type `dm`); group DM = small private channel. This **unifies the send/deliver/order/recover paths** — you implement them once.

Message identity & ordering keys

```
message_id : globally unique, client-generated UUID (idempotency/dedup)
seq        : per-channel monotonically increasing integer (ORDERING)
ts         : server timestamp (display + tie-break, NOT ordering)
```

- **`message_id`** (client UUID) → **idempotency**: retries don't duplicate; clients dedup on it.
- **`seq`** (server-assigned, per channel) → **the ordering and recovery cursor**. "Give me everything in channel C with `seq > my_last_seq`."

---

5. APIs

Real-time transport (WebSocket — the primary path)

Client holds a persistent **WebSocket** to a gateway; messages flow both ways as framed events.

```
// client → server: send a message
{ "type":"send", "channel_id":"C1", "client_msg_id":"u-9f3a", "body":"deploying now 🚀" }

// server → client: ack with assigned order
{ "type":"ack", "client_msg_id":"u-9f3a", "message_id":"m_77", "channel_id":"C1", "seq":3412, "ts":...

// server → client: a new message in a subscribed channel
{ "type":"message", "channel_id":"C1", "message_id":"m_78", "seq":3413, "sender":"U2", "body":"..." }

// client → server: heartbeat / cursor checkpoint
{ "type":"ping", "cursors":{ "C1":3413, "C2":901 } }

// server → client: presence / typing
```

```
{ "type":"presence", "user":"U2", "state":"online" }
{ "type":"typing", "channel_id":"C1", "user":"U2" }
```

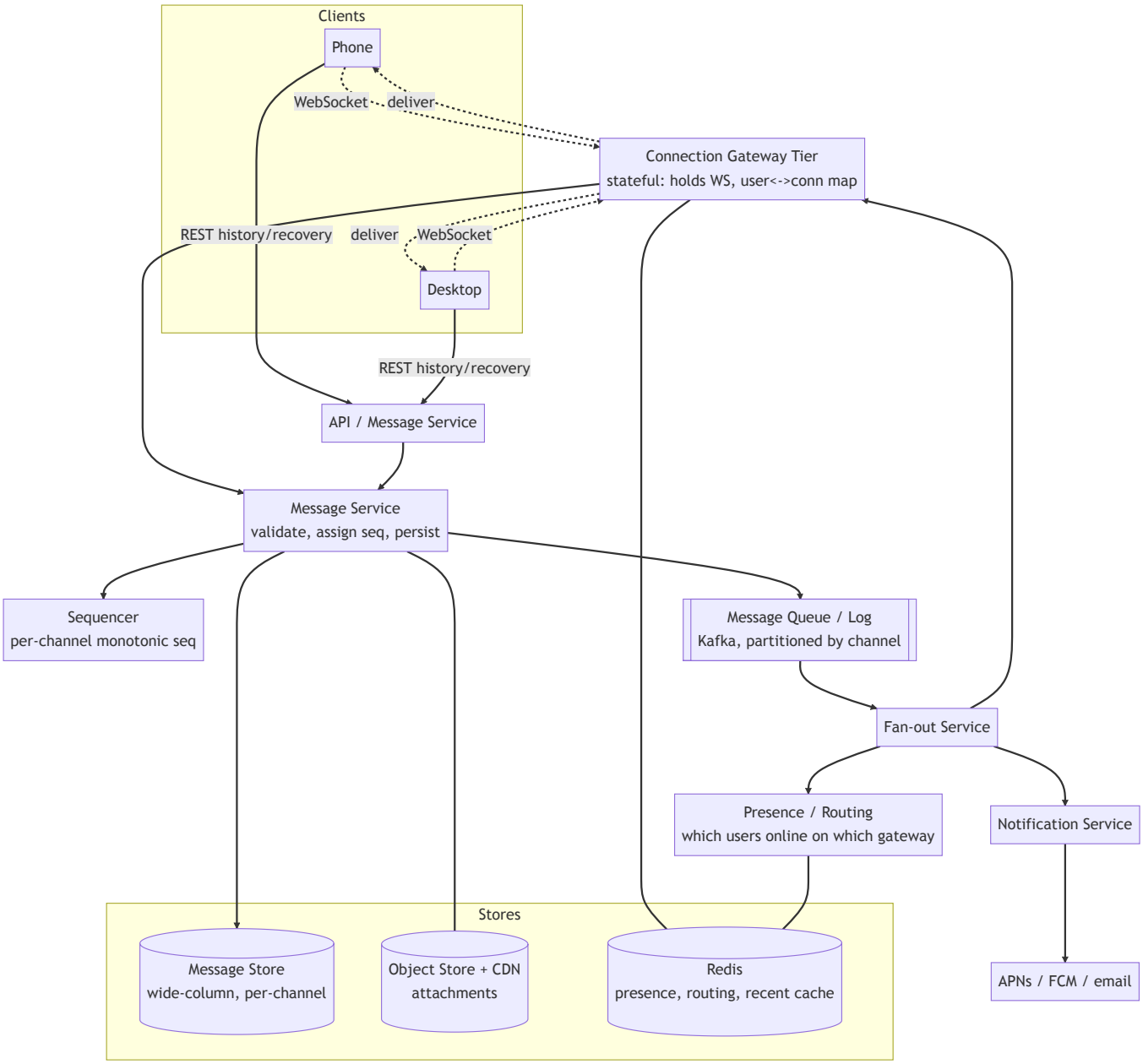
REST/HTTP (history, recovery, management — not the hot push path)

```
POST /v1/channels/{id}/messages        {client_msg_id, body}      # send (fallback if WS down)
GET  /v1/channels/{id}/messages?after_seq=3413&limit=50          # backfill missed / scrollback
GET  /v1/channels/{id}/messages?before_seq=3000&limit=50         # history (older)
POST /v1/channels/{id}/read            {up_to_seq:3413}          # advance read cursor
GET  /v1/sync?since=<cursor_token>                               # cold-start multi-channel delta
POST /v1/channels {workspace_id, type, members}                  # create channel / DM
```

Why both WS and REST? WebSocket carries the **live** push (low latency); REST carries **bulk history & recovery** (cursor-paginated, cacheable, retry-friendly). Sends can go over either — WS when connected, REST as a fallback — both idempotent via `client_msg_id`.

---

6. Architecture overview

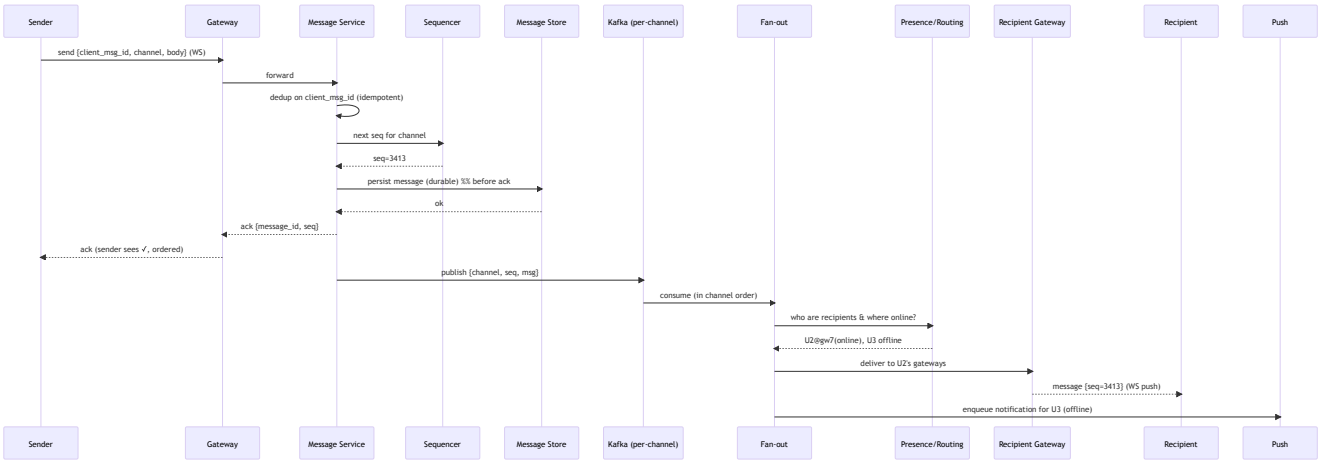


Component responsibilities

- **Connection Gateway** — terminates WebSockets; maintains the **user → connection(s) → gateway** routing map (in Redis); pushes events to connected devices. *Stateful but disposable* — on disconnect the client reconnects (possibly to another gateway) and recovers.
- **Message Service** — validates, **assigns per-channel `seq`**, persists the message (durability **before** ack), publishes to the queue. The point of truth for "the message exists."
- **Sequencer** — hands out monotonic per-channel sequence numbers (see §8).
- **Message Queue/Log (Kafka)** — durable, **partitioned by channel_id** → preserves per-channel order and decouples persist from fan-out; absorbs spikes; replayable.
- **Fan-out Service** — for each message, find recipients, look up who's online & on which gateway (presence/routing), and deliver; enqueue notifications for offline users.
- **Presence/Routing** — tracks online users and their gateway(s) in Redis (TTL heartbeats).
- **Notification Service** — push (APNs/FCM), desktop, email for offline/background users.
- **Stores** — message store (durable history), Redis (presence/routing/recent-message cache), object store + CDN (attachments).

---

7. Data flow: send → persist → fan-out → deliver



Key points:

1. **Persist before ack.** The sender isn't told "delivered/ordered" until the message is **durable** and has a `seq`. Guarantees an acked message is never lost.
2. **Idempotent send.** `client_msg_id` dedup means a retried send (flaky network) doesn't create duplicates.
3. **Kafka partitioned by channel** → fan-out consumes a channel's messages **in seq order**, so delivery preserves order.
4. **Fan-out is async + decoupled** → the sender's ack isn't blocked on delivering to 50k recipients.
5. **Online → push via gateway; offline → notification queue.**

---

8. Message ordering (focus area)

Goal: every client in a channel sees the same order, with no gaps or duplicates. Wall-clock timestamps are *not* reliable ordering (clock skew, concurrent sends) — use a **per-channel monotonic sequence**.

Assign a per-channel `seq` at persist time

- The **Message Service** (via the Sequencer) assigns `seq = last_seq(channel) + 1` atomically when persisting.
- **Total order per channel**; `seq` is the contract for both **display order** and **recovery cursor**.
- Cross-channel order is **not** guaranteed (and not needed) — ordering is *per channel*, which is the natural unit.

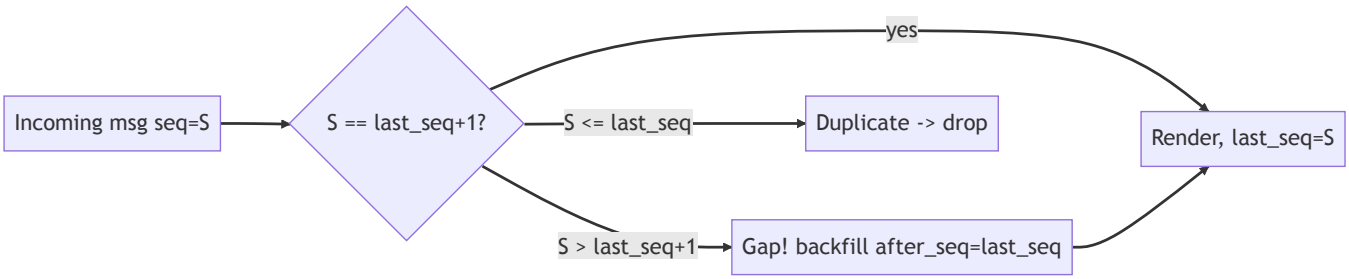
How to implement the sequencer (options)

Approach	How	Trade-off
DB atomic counter / conditional write	`UPDATE channels SET last_seq=last_seq+1 RETURNING` or Cassandra LWT	Simple, strongly consistent; the channel row is a hot spot for very busy channels
Single-partition Kafka per channel	Kafka offset = order	Order = partition offset for free; ties ordering to the log
Per-channel owner (sharded actor)	One service instance owns a channel, assigns seq in memory, persists	Fast, no DB contention; needs ownership/failover (consistent hashing)
Logical clock (Lamport/snowflake)	Monotonic ID	Globally unique + roughly ordered; for strict per-channel, still reconcile via seq

Recommended: a **per-channel sequencer** (sharded by `channel_id` via consistent hashing so one owner serializes a channel) writing to a **Kafka partition keyed by channel_id**. Order is then preserved end-to-end: assign → log → fan-out → deliver, all per-channel ordered.

Client-side ordering & gap detection

- Client tracks `last_seq` per channel. Incoming `seq` should be `last_seq + 1`.
- **Gap** (`seq > last_seq + 1`) → client missed messages → **trigger backfill** `GET ?after_seq=last_seq` (§9).
- **Duplicate/old** (`seq <= last_seq`) → dedup/ignore (at-least-once delivery → **exactly-once at the UI**).



One-liner: *"Order is a **per-channel monotonic `seq`** assigned at persist time by a channel-sharded sequencer over a channel-keyed Kafka partition — timestamps are display-only. Clients detect gaps when `seq` skips and backfill; duplicates are dropped, giving exactly-once at the UI."*

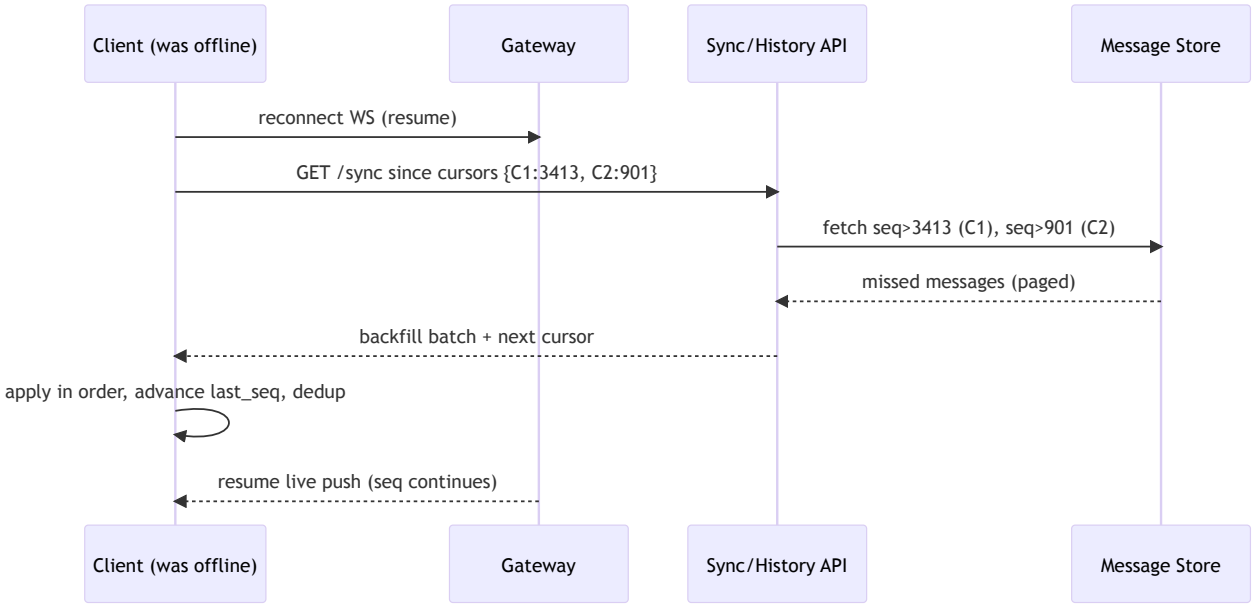
---

9. Missed-message recovery (focus area)

Disconnects are constant (subway, sleep, app background). A reconnecting client must get **exactly what it missed** — efficiently.

The mechanism: per-channel cursors + backfill by `seq`.

1. **Client persists `last_seq` per channel** (the highest contiguous seq it has rendered).
2. **On reconnect**, the client sends its cursors (per channel, or a compact bundle) — or calls `GET /v1/sync?since=cursor`.
3. **Server computes the delta**: for each channel, return messages with `seq > last_seq` (paginated).
4. **Client applies** them in order, advances `last_seq`, then resumes live push.
5. **Live + backfill reconciliation** — messages arriving live during backfill are buffered and **deduped by `seq`/`message_id`** so nothing is lost or doubled.



Design choices & guarantees:

- **At-least-once delivery + idempotent client** (dedup on `message_id`/`seq`) → **no loss, no duplicates** at the UI.
- **Server doesn't need per-client outboxes** for history — the **durable message store + seq cursor** *is* the recovery log. (For brief blips, a short per-connection replay buffer in the gateway avoids a DB hit.)
- **Bounded backfill** — page the delta; if a client is *way* behind (offline for days), fall back to **cold start** (§11) rather than streaming millions of messages.
- **Acks both ways** — server acks sends (durable); clients ack/advance read cursors so the server knows what's delivered/read (drives unread counts + notification suppression).

One-liner: *"The durable per-channel message log **is** the recovery mechanism — clients keep a `last_seq` cursor and on reconnect pull `seq > last_seq`, dedup by id, then resume live. No message is lost because we persist before ack; none is duplicated because the client dedups."*

---

10. Fan-out & delivery at scale (focus area)

One message → potentially tens of thousands of recipient **devices**. Naive "loop and send synchronously" doesn't scale.

Fan-out strategies (mirror the social-feed push/pull trade-off)

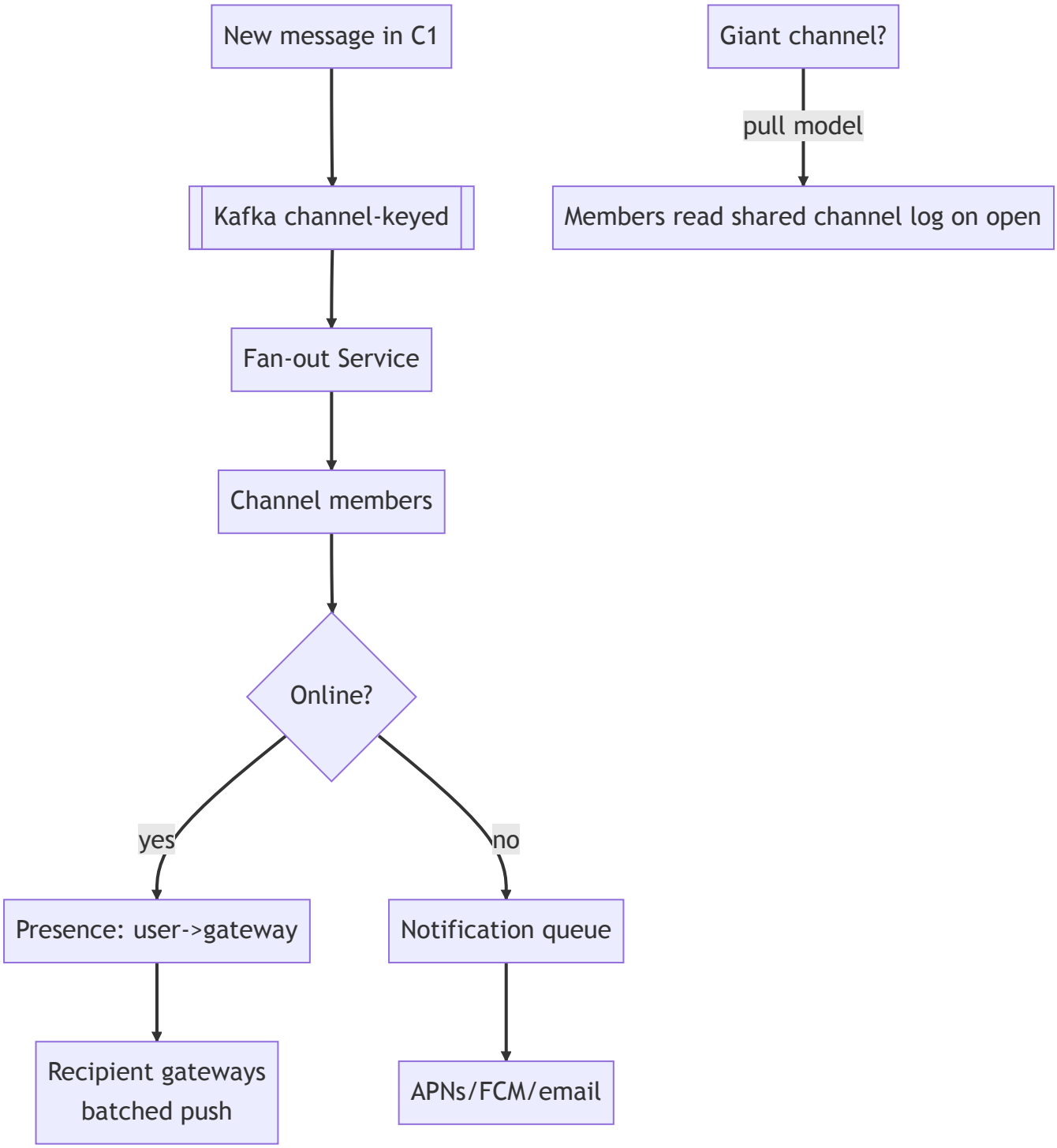
Strategy	How	Best for
Fan-out on write (push)	On send, deliver to every **online** member's gateway now	Normal channels; real-time feel ✅
Fan-out on read (pull)	Store once; members read the channel log on open	**Huge channels** (50k+), mostly-offline members
Hybrid ✅	Push to online members; **offline** → **notification + pull on cold start**; giant channels → pull	Real-world default

- **Online members** → push immediately via their gateway(s) (look up presence/routing in Redis).
- **Offline members** → **don't push**; enqueue a **notification** and let them **pull** (recover) on return. Saves the vast majority of wasted deliveries.

- **Giant channels** → avoid fan-out write amplification: members **read the shared channel log** (pull) rather than copying the message into 50k inboxes. The channel's recent messages are cached once (Redis) and served to many — the **celebrity/hot-channel** pattern.

Delivery path mechanics

- **Presence/routing table** (Redis): `user → {gateway, conn_ids}` with heartbeat TTL. Fan-out asks "who's online and where," then sends each gateway a batched delivery for its connected recipients.
- **Per-gateway batching** — one network call carries many recipients on the same gateway.
- **Backpressure** — if a client is slow, buffer briefly then drop to "reconnect & backfill" rather than blocking fan-out.
- **Multi-region** — workspace has a **home region**; gateways are regional; cross-region delivery goes via replicated Kafka.



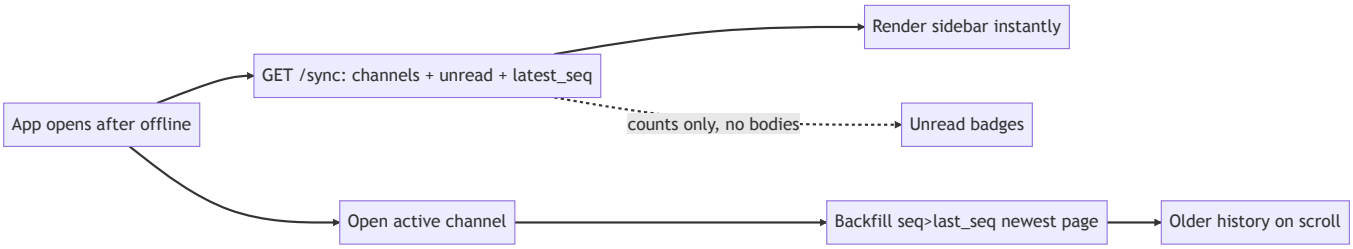
---

11. Cold start (focus area)

User opens the app after being offline for hours/days. Goal: **fast, bounded sync** — usable UI in a moment, not a multi-GB download.

Strategy: sync state, lazy-load history.

1. **Bootstrap** — fetch the **workspace/channel list + per-channel unread counts and latest seq** in one `GET /v1/sync` (compact: "C1 has 14 unread, latest seq 3500; your last_seq 3486"). Cheap; renders the sidebar immediately.
2. **Prioritize visible/active channels** — backfill **only the channel the user opens first** (and maybe DMs / @-mentions), `seq > last_seq`, newest page first. Other channels load on demand (when opened) or lazily in the background.
3. **Bounded recovery** — if a channel is thousands behind, **don't stream all of it**; show "you're caught up to here," load the most recent page, and page older on scroll. Unread **counts** come from `latest_seq - last_read_seq` without fetching bodies.
4. **Notifications already summarized** the important stuff (mentions/DMs) → cold start confirms & clears them.
5. **Snapshot/caching** — recent messages per channel cached in Redis/CDN so cold-start reads are fast and don't hammer the message store.



Trade-off: unread **counts** (cheap, from seq math) vs full message **bodies** (lazy). Compute badges from `latest_seq - last_read_seq`; fetch bodies only when a channel is opened. Avoids the "download everything" cold-start tax.

One-liner: *"Cold start syncs **metadata + unread counts from seq cursors** instantly, then **lazily backfills** only the channel the user actually opens — counts are `latest_seq - last_read_seq`, so badges are free and history is paid for on demand."*

---

12. Notifications & the latency/correctness/UX trade-offs (focus area)

Notifications target **offline/background** devices (the online path is the WebSocket push). The interesting part is the **trade-offs**.

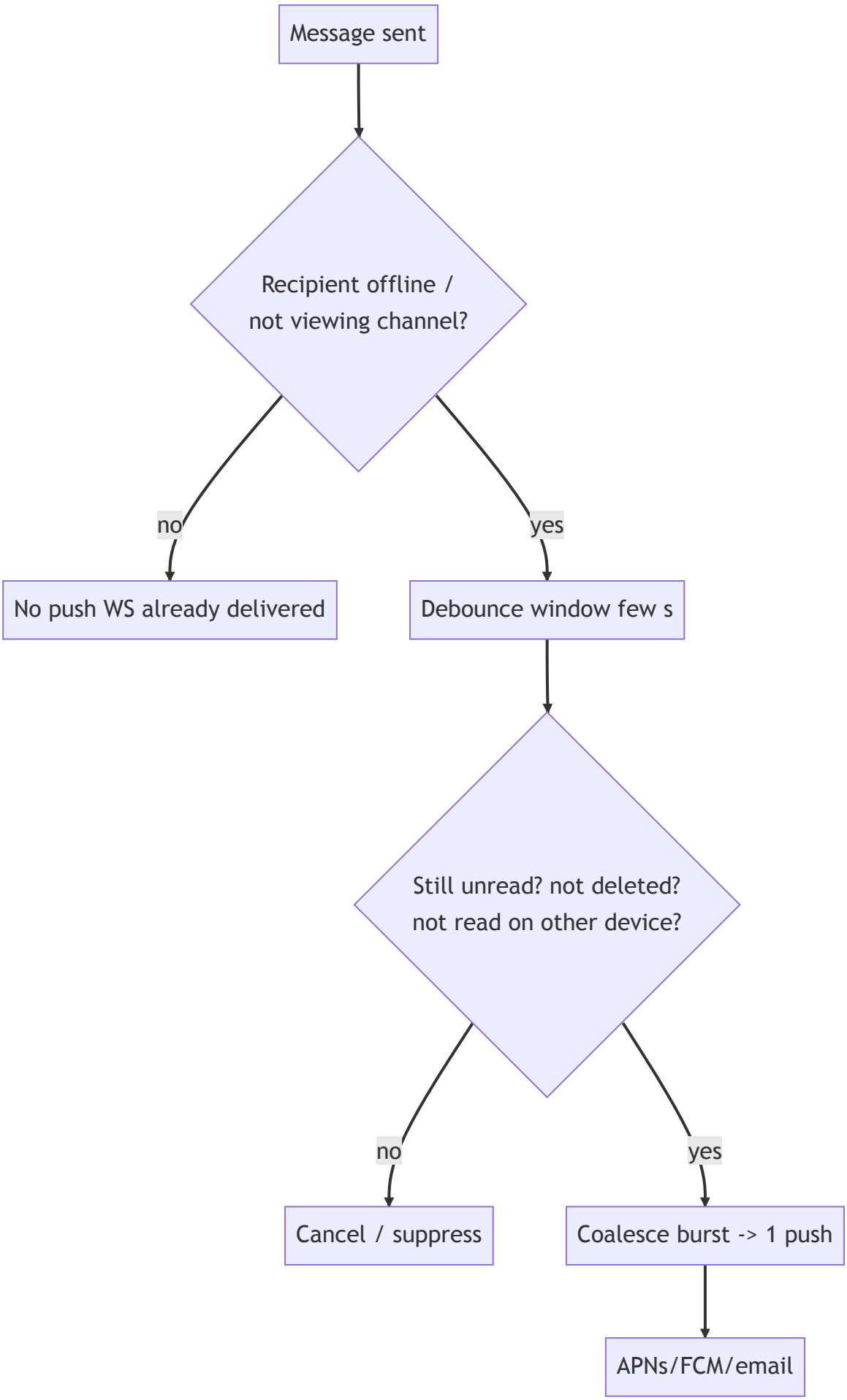
- **Who gets notified** — only if the user is **not actively viewing** that channel on an online device (suppress if they're already reading it). Respect **mentions-only / muted / DND** preferences.
- **Multi-device coordination** — if delivered+read on the laptop, **suppress/clear** the phone push. Requires read-state sync (read cursors broadcast to the user's devices).
- **Channels** — APNs/FCM (mobile), desktop notifications, email digests for long-offline.

The three-way tension

Want	Pull	Tension
Low latency	Fire push immediately on send	May notify for a message the user already saw elsewhere, or that later gets deleted/edited → **incorrect/annoying**
Correctness	Wait to confirm still-unread + not-viewed-elsewhere + not-deleted	Adds delay (debounce window) → **higher latency**
UX	Batch/debounce ("3 new messages"), respect DND, dedup across devices	Coalescing adds delay; too much → feels laggy; too little → notification spam

Resolution — a short debounce + suppression window:

- Hold a brief window (e.g., a few seconds) before pushing: lets **read-state from another device** suppress it, **edits/deletes** cancel it, and **bursts coalesce** ("Alice sent 3 messages"). Costs a little latency for much better correctness/UX.
- **DMs / @mentions** → **near-immediate** (latency matters most); **noisy channels** → **batched/digested** (UX matters most). Tune per type.
- **Idempotent notifications** (dedup key per message+device) so retries don't double-buzz.


Message sent
Recipient offline / not viewing channel?
no
yes
No push WS already delivered
Debounce window few s
Still unread? not deleted? not read on other device?
no
yes
Cancel / suppress
Coalesce burst -> 1 push
APNs/FCM/email

One-liner: *"Online delivery goes over the socket; **notifications are for offline/background only**, gated by a short debounce-and-suppress window so reads on another device, edits, and deletes can cancel them and bursts coalesce — DMs/mentions fire fast (latency), noisy channels batch (UX), all idempotent (correctness)."*

---

13. Storage choices

Data	Store	Why
Messages (history)	Wide-column (Cassandra/HBase/Bigtable), partition `channel_id`, cluster by `seq`	Massive write volume, ordered range scans per channel (`seq > X`), horizontal scale
Channel/user/membership metadata	Relational (Postgres) or wide-column	Relational RBAC, transactions for seq counter
Recent messages / hot channels	Redis cache	Sub-ms reads for live + cold-start + giant-channel pull
Presence / routing	Redis (TTL heartbeats)	Ephemeral, fast `user→gateway` lookups
Attachments	Object store (S3) + CDN	Blobs never in the message DB; edge delivery
Message log / pipeline	Kafka (partition by channel)	Durable ordered log; decouples persist↔fan-out; replay
Search index	Elasticsearch (async from Kafka)	Full-text history search
Notifications outbox	Queue + store	Reliable async push

Message store schema (wide-column):

```
messages_by_channel (
  channel_id   partition key,
  seq          clustering key ASC,     -- ordered scans: seq > last_seq
  message_id, sender_id, ts, body, type, thread_id, edited, deleted
)
```

Partitioning by `channel_id` + clustering by `seq` makes **both** hot paths cheap: deliver/recover = a forward range scan `seq > cursor`; history = a backward scan. Very large/old partitions are bucketed by time and tiered to cold storage.

---

14. Scalability considerations

- **Connection tier scales independently** — ~100k WS/node; add nodes behind an L4/L7 LB; sticky enough that a user's devices land predictably, but **any gateway can serve any user** (routing via Redis) so failover is cheap.
- **Stateless business services** — Message/Fan-out/Notification scale horizontally; state lives in Kafka/DB/Redis.
- **Kafka partitioning by channel** — parallelism across channels while preserving per-channel order; rebalance partitions as load grows.
- **Hot/giant channels** — switch to **pull** (shared cached log) to avoid fan-out write amplification; replicate the hot channel cache across Redis replicas.
- **Sharding** — message store sharded by `channel_id`; presence sharded by `user_id`.

- **Multi-region** — regional gateways for low RTT; **workspace home region** owns ordering; cross-region via replicated Kafka; presence is regional with a global directory.
- **Backpressure & graceful degradation** — queues absorb spikes; under extreme load, shed non-critical (typing/presence) before messages; slow clients fall back to reconnect+backfill.
- **Read scaling** — recent-message cache + read replicas; cold start uses counts (seq math) not bulk bodies.

---

15. Failure scenarios — *"what if X fails?"*

Failure	Impact	Mitigation
Client disconnects	Misses live messages	Reconnect + **backfill by `seq` cursor**; brief gateway replay buffer for blips
Gateway node dies	Its connections drop	Clients reconnect to another gateway (routing via Redis) + recover; stateless-ish tier
Message Service down	Sends fail	Client retries (idempotent `client_msg_id`); HA replicas; persist-before-ack means no half-sent state
Kafka lag/backlog	Delivery delayed	Consumers autoscale; ordering preserved (log); eventual delivery; live feel degrades gracefully
Sequencer/owner failover	Risk of seq gap/dup	Persist seq with the message + conditional writes; new owner resumes from `max(seq)` in store
Duplicate delivery (at-least-once)	Client sees dupes	**Dedup on `message_id`/`seq`** → exactly-once at UI
Presence stale	Push to a dead conn	TTL heartbeats; failed push → mark offline → notification path
Hot channel storm	Fan-out overload	Pull model + cached shared log + replicas
Region outage	Workspace unreachable	Replicated Kafka + store; failover home region; clients reconnect & recover
Notification storm	Spam / double-buzz	Debounce/coalesce + idempotent dedup key + DND

Guiding principle: persist before ack (never lose an acked message) + **at-least-once + client dedup** (exactly-once UI) + **cursor recovery** (heal any gap). The system favors **durability and ordering**; presence/typing are best-effort.

---

16. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Delivery semantics	Exactly-once on the wire (hard, slow)	At-least-once + client dedup	**At-least-once + dedup** → exactly-once at the UI ✅
Ordering	Global total order (bottleneck)	**Per-channel** order	Per-channel `seq` — the natural unit, scalable ✅
Fan-out	Push to all (fast, costly)	Pull on read (cheap, slower)	**Hybrid**: push online, pull for giant/offline
Recovery	Per-client server outbox	**Durable log + seq cursor**	Cursor on the shared log — no per-client state ✅
Notification	Instant (low latency)	Debounced (correct/UX)	Debounce + suppress; DMs fast, channels batched
Consistency	Strong everywhere	Eventual for presence/read-state	Strong **ordering**; eventual presence/typing
Connection	Long-poll (simple)	**WebSocket** (live, bidirectional)	WebSocket for push; REST for history/recovery
Timestamp vs seq	Order by `ts` (skew bugs)	Order by **`seq`**	`seq` orders; `ts` is display-only

CAP framing: chat leans **AP for delivery/presence** (stay available; reconcile via backfill) but needs **CP-ish ordering within a channel** (the per-channel sequencer is the consistency anchor). Acked-message **durability is non-negotiable**.

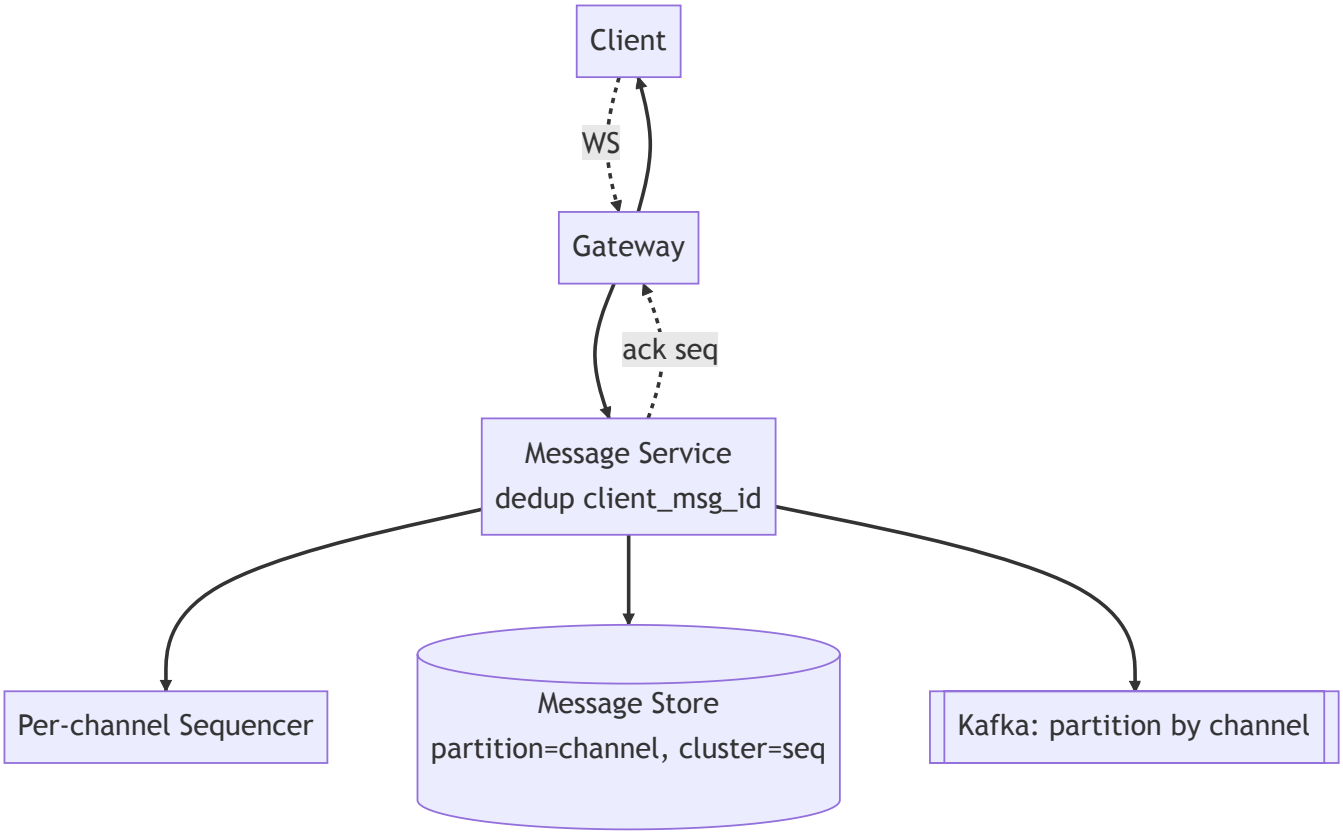
One-liner to say out loud: *"Clients hold **WebSockets** to a scalable gateway tier; a send is **persisted and assigned a per-channel `seq` before it's acked**, published to a **channel-partitioned Kafka log**, and **fanned out** to online members' gateways (offline → notifications). **Ordering** is the per-channel `seq`; **recovery** is a `seq` cursor — on reconnect a client pulls `seq > last_seq` from the durable log and dedups by `message_id`, giving no loss and exactly-once at the UI. Giant channels switch to a **pull** model, cold start syncs **unread counts from seq math** and lazy-loads bodies, and notifications use a **debounce-and-suppress** window to trade a little latency for correctness and UX."*

---

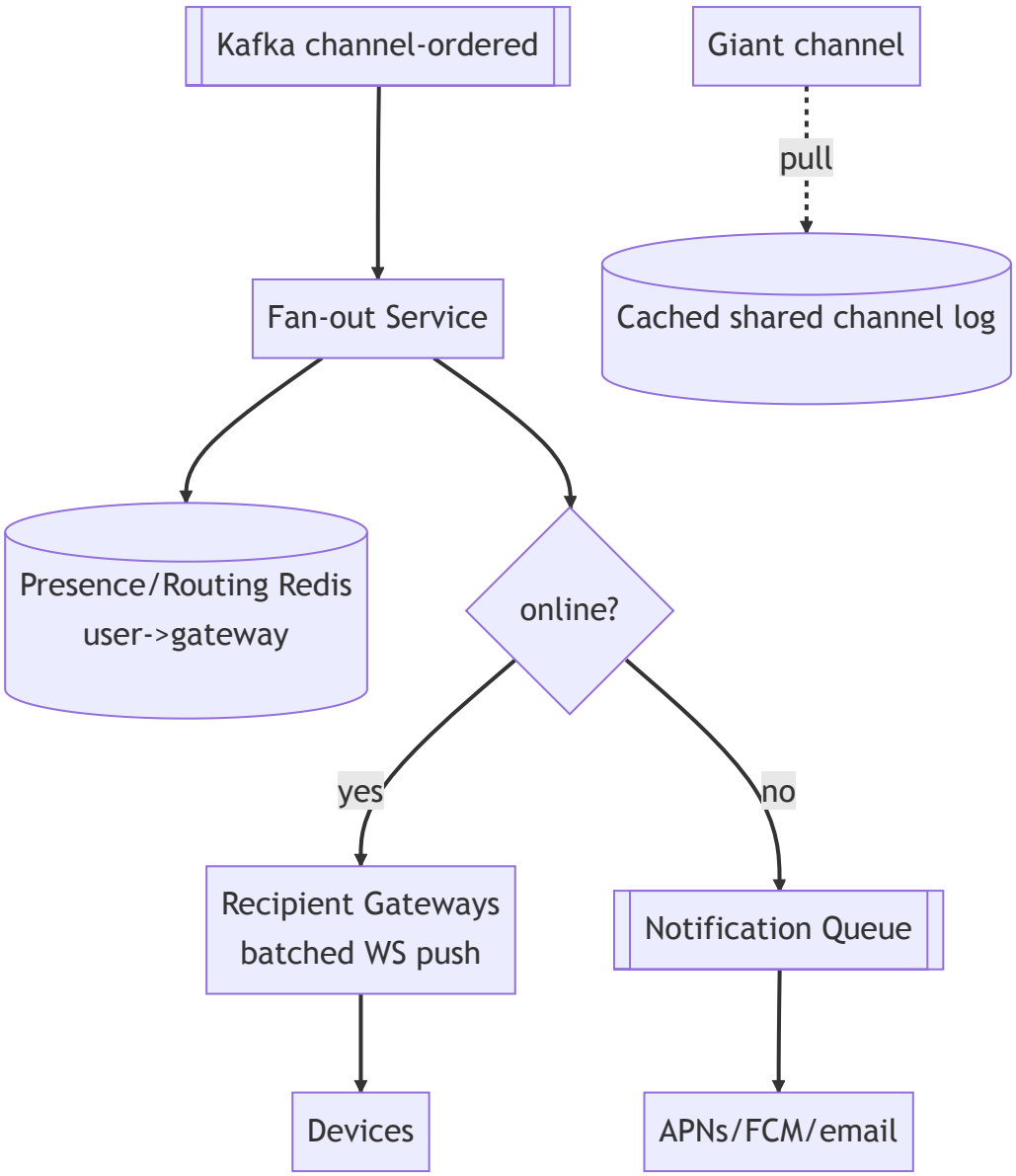
17. Full system design (detailed)

End-to-end, split into **(A)** the send/persist/order path, **(B)** the fan-out/delivery path, and **(C)** reconnect/recovery + notifications.

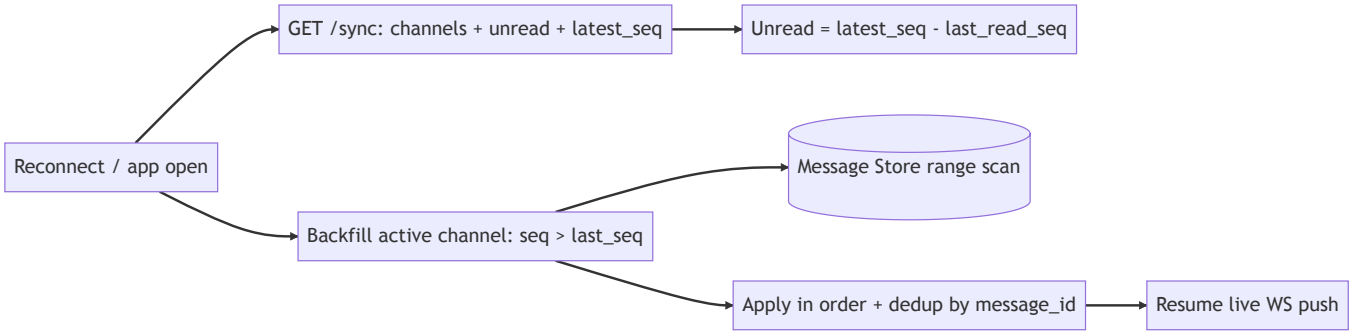
17A. Send → persist → order



17B. Fan-out → delivery



17C. Reconnect, recovery & cold start



---

18. Networking, security & performance best practices

Networking

- **Persistent WebSocket per device** with heartbeats + idle timeout; **auto-reconnect with backoff + jitter**.

- **Independent gateway tier** (~100k conns/node) behind an L4/L7 LB; **routing via Redis** so any gateway can serve any user (cheap failover).
- **TLS at the edge**; **HTTP/3 (QUIC)** for mobile resilience; **gRPC** internal; **per-gateway batched fan-out** (one call, many recipients).
- **Multi-region** — regional gateways for low RTT; workspace **home region owns ordering**; **replicated Kafka** cross-region.

Security

- **AuthN/Z on connect and per-channel** (RBAC: can this user read/post this channel?).
- **Server-side encryption at rest**; **per-workspace data isolation** (tenant boundary).
- **DLP / content + attachment scanning**; **signed expiring URLs** for files; malware scan on upload.
- **Rate-limit sends** + abuse/spam detection; avoid presence/user enumeration leaks.
- **Audit** security-relevant events (channel access changes, admin actions).

Performance

- **Per-channel `seq` + channel-keyed Kafka** → ordered, parallel fan-out.
- **Hybrid fan-out**: push to online members, **pull a cached shared log** for giant channels.
- **Recent-message cache** (Redis); **cold start = unread counts from `seq` math**, lazy-load bodies.
- **Client dedup** on `message_id`/`seq` → exactly-once at the UI; **bounded** cursor backfill.

---

19. Staying current — modern & emerging approaches

- **Reference systems: Slack** (channel servers + Flannel edge cache), **Discord** (Elixir/Erlang gateways, **ScyllaDB**), **WhatsApp/Signal** (Erlang/BEAM).
- **Stores:** Cassandra/**ScyllaDB**/Bigtable for messages, **Redis** for presence, **Kafka/Pulsar** for the ordered log.
- **Transport:** WebSocket, **HTTP/3/QUIC**, **MQTT** for mobile push, gRPC streaming.
- **Push: APNs/FCM**, Web Push; silent/background push for sync.
- **Patterns: CRDTs** for offline edits, **event sourcing**, the **transactional outbox** for reliable fan-out.
- **How I stay current:** Discord/Slack/Signal engineering blogs, Erlang/Elixir at-scale talks, distributed-systems papers.

---

20. Likely follow-up questions (rehearse these)

- How do you guarantee per-channel ordering? *(monotonic `seq` at persist time, not wall-clock timestamps)*
- Client offline for a day — recover missed messages? *(`seq` cursor backfill; fall back to cold start if far behind)*
- At-least-once delivery — how do you avoid duplicates? *(dedup on `message_id`/`seq` → exactly-once UI)*
- A 50k-member channel — fan-out? *(switch to pull: members read one cached shared log)*
- Notification correctness vs latency? *(short debounce + suppress window; DMs fast, channels batched)*
- A gateway node dies — do users drop messages? *(reconnect to another gateway via Redis routing + backfill)*
- Multi-device read-state sync? *(broadcast read cursors to the user's devices)*
- Sequencer hot spot on a busy channel? *(channel-sharded owner / Kafka partition offset as order)*

---

21. Summary checklist (whiteboard recap)

- **Entities** — users, workspaces, channels, DMs(=channels), messages; **DMs unified as channels**.
- **Transport** — WebSocket for live push; REST for history/recovery; sends idempotent via `client_msg_id`.
- **Ordering** — **per-channel monotonic `seq`** assigned at persist time (channel-sharded sequencer + channel-keyed Kafka); timestamps display-only.

- **Recovery** — durable log + **`seq` cursor**; reconnect pulls `seq > last_seq`, dedups by id → no loss, exactly-once UI.
- **Persist before ack** — an acked message is never lost; **at-least-once + client dedup**.
- **Fan-out** — hybrid: push to online via presence/routing, offline → notifications, **giant channels** → **pull** a cached shared log.
- **Cold start** — sync metadata + **unread counts from seq math**, lazy-load bodies on open.
- **Notifications** — offline/background only; **debounce + suppress** (read-elsewhere/edit/delete cancel; bursts coalesce); DMs fast, channels batched.
- **Storage** — wide-column messages (partition=channel, cluster=seq), Redis presence/cache, Kafka log, object store + CDN attachments, ES search.
- **Scale** — independent connection tier (~100k WS/node), stateless services, channel-partitioned Kafka, multi-region home-region ordering.
- **CAP** — AP for delivery/presence; per-channel sequencer is the consistency anchor; durability non-negotiable.

Designing an Online Chess Platform (Real-Time, Timed Games)

> A complete, interview-ready walkthrough: requirements → estimates → entities → APIs → architecture → **matchmaking, server-authoritative move validation, the game clock, real-time delivery, game end & ratings, persistence/replay** → scaling → failure handling → trade-offs. Use the headings as your whiteboard agenda. The hard parts are the **server-authoritative clock (who flags first under network lag), move validation, and rating-based matchmaking** — spend your time there.

> Think Lichess / Chess.com. Two human players are matched by strength, join a real-time session, exchange moves over a persistent connection; the **server is the sole authority** on legality, game state, and the clock. Finished games are persisted as PGN for later viewing.

---

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements (timed 1v1, rating-based matchmaking, server-authoritative).
2. **Estimate** scale (concurrent players, concurrent games, moves/sec, storage).
3. **Model entities** (players, games, moves, clocks, ratings) + their keys.
4. **Define the APIs** (matchmaking REST + the **real-time move transport** over WebSocket).
5. **Walk the game lifecycle**: queue → match → session → move → validate → clock → broadcast → end → persist.
6. **Nail the clock** (server-authoritative time, flagging, lag compensation) and **move validation** (server-side rules engine).
7. **Matchmaking** (rating windows) + **reconnection** + rating updates.
8. **Persistence & replay** (PGN), then **storage, scaling, failure + trade-offs**.

Keep saying *"here's the trade-off…"* — that's what's being graded. The signature line: **"the server is authoritative for legality, state, and time; clients are just views."**

---

1. Problem & motivation

A real-time online chess service: players request a **timed** game, get **matched by rating**, join a **live session**, and alternate **moves**. The server **validates every move**, maintains the **authoritative board and clocks**, decides the **end** (checkmate / resign / draw / timeout), and **persists** the finished game for replay.

What makes it hard:

- **Server-authoritative clock** — deciding *who ran out of time first* is subtle under network latency. A move sent "on the buzzer" may arrive after the server thinks time expired. Flag the wrong player and the game is ruined.
- **Server-authoritative move validation** — never trust the client. Full chess rules (castling, en passant, promotion, check, checkmate, stalemate, threefold repetition, 50-move rule, insufficient material) must run on the server to prevent illegal-move cheating.
- **Rating-based matchmaking** — pair players of similar strength quickly, per time control, widening the search as they wait.
- **Real-time, consistent state** — both players (and spectators) must see the **same** board and clocks with low latency, surviving reconnects.
- **Fairness & anti-cheat** — legality is enforced structurally; engine-assistance cheating needs statistical detection.

The central tension: the game must feel **instant** to both players, yet the **server alone** decides legality, state, and time — and it must resolve the **flag race** fairly when the network is slow.

---

2. Requirements

Functional

- **Start a timed game** — player picks a time control (e.g., 1+0 bullet, 3+2 blitz, 10+0 rapid) and joins a queue.
- **Rating-based matchmaking** — pair with an opponent of similar rating in that time-control pool.
- **Real-time game session** — both players connected; moves delivered live to the opponent.
- **Server-side move validation** — only legal moves accepted; illegal moves rejected.
- **Authoritative clocks** — per-player countdown; increment support; **timeout ⇒ loss** (unless already decided).
- **Game end** — checkmate, resignation, draw (stalemate / threefold / 50-move / insufficient material / agreement), timeout, abort.
- **Rating update** — recompute both players' ratings on completion.
- **Persistence & replay** — store the finished game (moves + clocks + result) as **PGN** for later viewing.
- **(Extensions)** spectating, draw offers / takebacks, reconnection, rematch, anti-cheat.

Non-functional

- **Low latency** — p99 move round-trip < ~100 ms for co-located players; the clock must feel exact.
- **Correctness** — the authoritative game state and clock are **never wrong**; a legal game is never corrupted by a bad move or a mis-flag.
- **High availability** — an in-progress game must survive a server restart (don't lose a game people are playing).
- **Durability** — a finished, rated game is **never lost**; ratings update atomically.
- **Scalable** — millions of concurrent players, hundreds of thousands of concurrent games.
- **Fairness** — clock decisions are lag-compensated and reproducible from server timestamps.

Clarifying questions to ask the interviewer

- **Only human vs human**, or also vs bots / puzzles? (Assume **human 1v1**.)
- **Time controls** — bullet/blitz/rapid only, or classical/correspondence (days per move)? (Assume real-time bullet→rapid; note correspondence differs.)
- **Rating system** — Elo or **Glicko-2**? (Assume Glicko-2 like Lichess; Elo is the simple fallback.)
- **Spectators** — required? (Yes, read-only; affects fan-out.)
- **Reconnection policy** — does the clock keep running while a player is disconnected? (Yes, with a small grace window.)
- **Anti-cheat depth** — structural legality only, or engine-detection too? (Legality now; statistical detection as an async subsystem.)
- **Global** — multi-region for latency? (Yes → regional game servers, pair by region.)

---

3. Back-of-the-envelope estimation

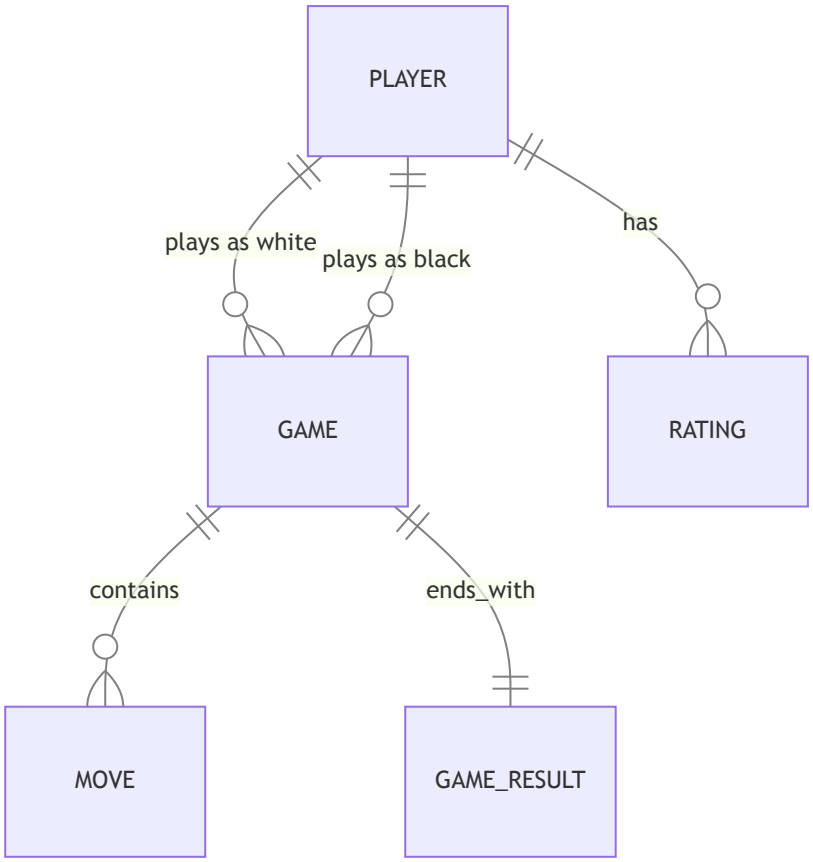
Quantity	Assumption	Result
DAU	—	10M
Concurrent players (peak)	~10% online & playing	**~1M in games**
Concurrent games	2 players/game	**~500K live games**
Games/day	~5 games/active user	**~50M games/day**
Move rate	blitz ≈ 40 full moves over ~6 min ⇒ ~0.2 plies/s/game	**~100–200K moves/sec peak**
Move validation cost	rules engine per move	**microseconds** — CPU-cheap
Live game state	board + clocks + history	**~a few KB/game** ⇒ ~GBs total in memory
Persisted record	moves + clocks + metadata as PGN	**~5–10 KB/game** ⇒ ~0.5 TB/day
Matchmaking pool	waiting players per time control	thousands, sorted by rating

Takeaways that drive the design:

1. **Games are tiny and CPU-light**, but **stateful and long-lived** (minutes). → a **game-server tier that owns games in memory** (one authoritative owner per game), not a stateless request/response service.
2. **Real-time move exchange** for 1M connections → a **connection/gateway tier** with persistent WebSockets, decoupled from game logic.
3. **The clock is the hard correctness problem**, not throughput → invest in **server-authoritative timing + lag compensation**.
4. **Persistence is modest** (~0.5 TB/day) and **append-only** → cheap to store every game as PGN for replay.

---

4. Core entities & data model



Entity	Key fields	Notes
Player	`player_id`, name, rating, rating_deviation, country	Identity + strength; `rating` per time-control category
Rating	`(player_id, category)`, value, RD, volatility, games	Glicko-2 state per bucket bullet/blitz/rapid
Game	`game_id`, white_id, black_id, `time_control`, `status`, start_ts, region	Owned by one game server while live
GameState *(in-memory)*	FEN, side_to_move, castling, ep_target, halfmove_clock, position_hashes	Authoritative live state; rebuildable from moves
Clock *(in-memory)*	white_ms, black_ms, increment_ms, turn_started_ts	Server-authoritative remaining time
Move	`(game_id, ply)`, uci, san, clock_ms_after, ts	Ordered by **ply** (half-move index) — the ordering key
GameResult	`game_id`, winner, reason, white_rating_delta, black_rating_delta	checkmate / resign / draw / timeout / abort

Ordering key = `ply`. Chess is strictly turn-based, so each game has a single monotonic **half-move index (`ply`)**. Move `ply = n` is only legal from the position after `ply = n−1`. This makes ordering trivial (one writer per game) — unlike chat, there's no fan-out sequencing problem.

Time-control notation

```
base + increment   (minutes + seconds/move)   e.g., 5+3 = 5 min base, +3 s per move
category:  bullet (< 3 min)  ·  blitz (3–8)  ·  rapid (8–25)  ·  classical (> 25)
```

Rating is tracked **per category** (a strong bullet player may be an average rapid player). Matchmaking pools are keyed by exact time control.

---

5. APIs

Matchmaking & lifecycle (REST/HTTP)

```
POST /v1/matchmaking/queue      {time_control:"5+3"}            # enter the pool, returns ticket
DELETE /v1/matchmaking/queue    {ticket_id}                    # cancel search
GET  /v1/games/{id}             -> {fen, clocks, moves, status} # snapshot (resume/replay)
POST /v1/games/{id}/resign                                     # resign
POST /v1/games/{id}/draw        {action:"offer|accept|decline"} # draw negotiation
GET  /v1/players/{id}/games?before=<cursor>                    # finished-game history (replay list)
```

Real-time transport (WebSocket — the hot path)

Once matched, each player holds a persistent **WebSocket** to the game's server; moves and clock updates flow as framed events.

```
// server -> both: game starts, seats + clocks assigned
{ "type":"game_start", "game_id":"g_42", "color":"white", "opponent":{"name":"Mia","rating":1587}, "ti

// client -> server: submit a move (from-square, to-square, optional promotion)
{ "type":"move", "game_id":"g_42", "ply":7, "uci":"e7e8q", "client_ts": 1721659200123 }

// server -> both: accepted move + AUTHORITATIVE clocks after applying it
{ "type":"move_applied", "ply":7, "uci":"e7e8q", "san":"e8=Q+", "fen":"...", "clocks":{"white":297400,

// server -> the mover: rejected (illegal or not your turn) -> client rolls back
{ "type":"move_rejected", "ply":7, "reason":"illegal_move" }

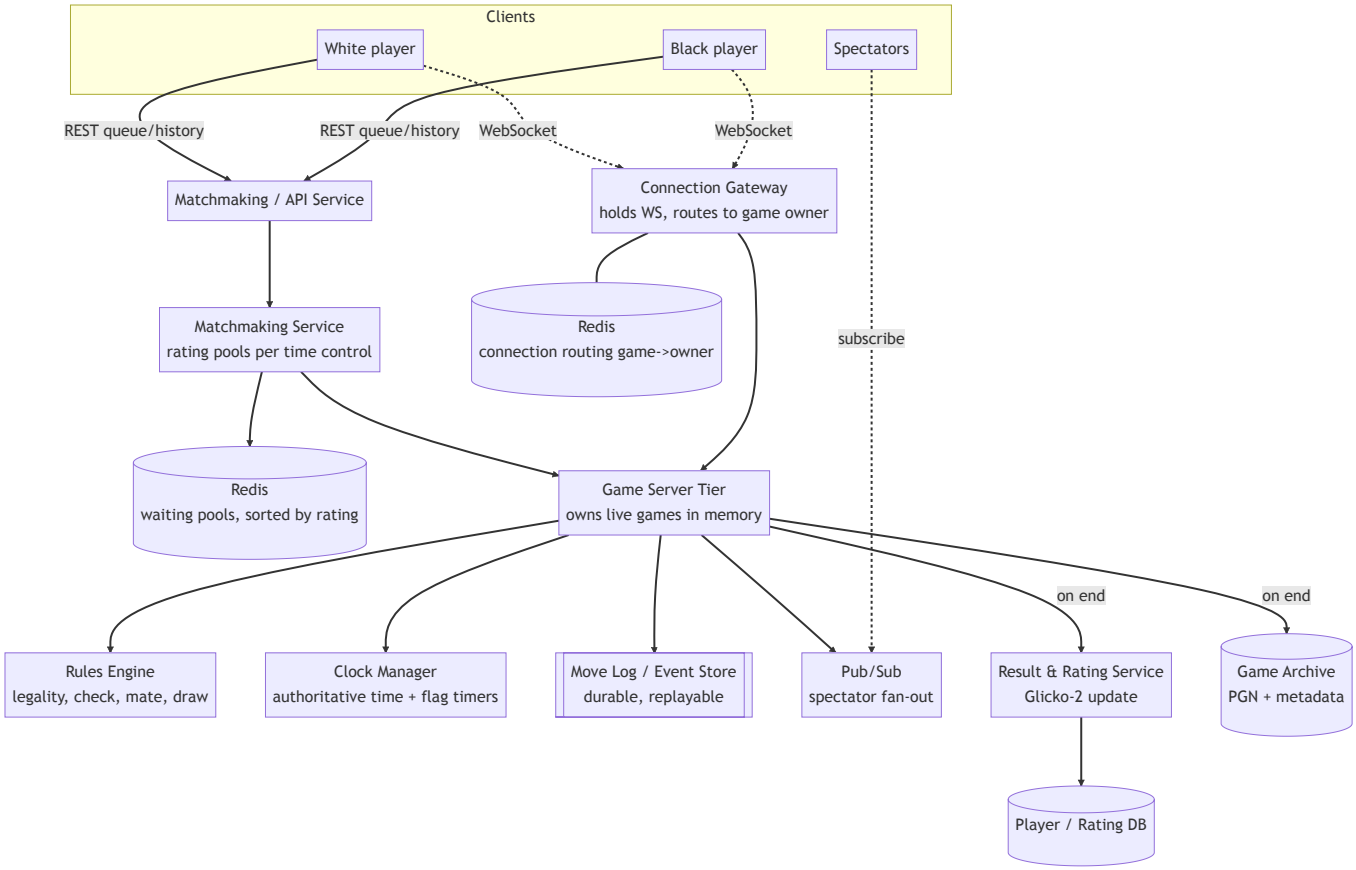
// server -> both: periodic authoritative clock sync (client interpolates between)
{ "type":"clock", "clocks":{"white":295800,"black":301300}, "turn":"white", "server_ts": 1721659203000

// server -> both: game over
{ "type":"game_over", "result":"black_wins", "reason":"timeout", "rating_delta":{"white":-8,"black":+8
```

Why WebSocket for moves, REST for the rest? Moves and clocks need **push, low latency, bidirectional** → WebSocket. Matchmaking, snapshots, history, and replay are **request/response, cacheable, retry-friendly** → REST. Moves carry a **`ply`** so the server can reject stale/duplicate submissions idempotently.

---

6. Architecture overview

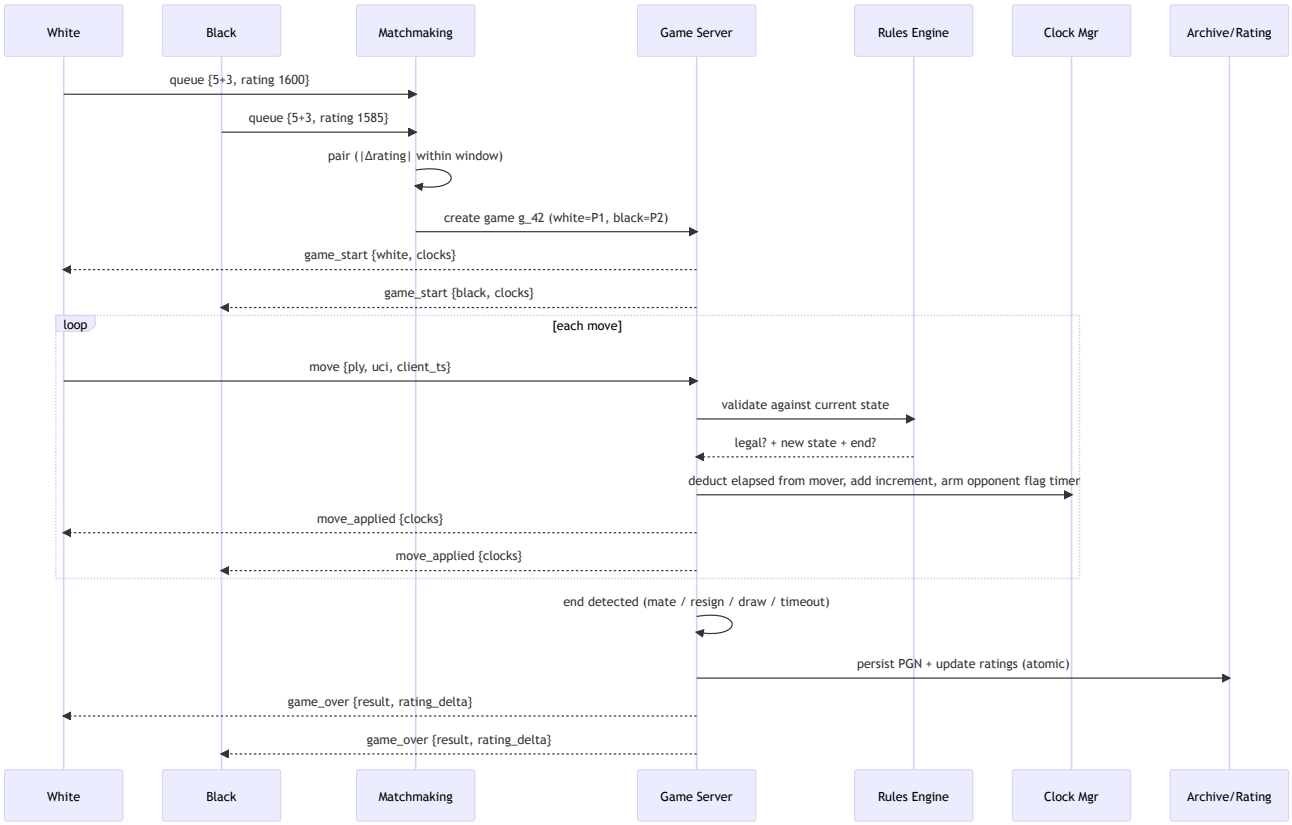


Component responsibilities

- **Connection Gateway** — terminates player WebSockets; routes each player's frames to the **game-owner** node (via Redis `game_id → owner`). Stateful but disposable: on disconnect, the client reconnects (any gateway) and resumes.
- **Matchmaking Service** — maintains per-time-control **waiting pools sorted by rating** (Redis sorted sets); pairs players and asks a Game Server to create the game.
- **Game Server (the core)** — **owns the authoritative game** in memory: applies moves through the rules engine, drives the clock, broadcasts to both players + spectators, appends to the durable move log, and finalizes on end. **One owner per game** (single-writer ⇒ trivial move ordering).
- **Rules Engine** — pure chess logic: legal-move generation/validation, check/checkmate/stalemate detection, draw conditions.
- **Clock Manager** — per-game authoritative clocks; sets **flag timers** and resolves timeouts with lag compensation (§10).
- **Move Log / Event Store** — durable append-only log of moves so a crashed game is **reconstructable**.
- **Result & Rating Service** — computes Glicko-2/Elo deltas and updates both players atomically.
- **Game Archive** — finished games as **PGN + metadata** for replay; served via cache/CDN.
- **Pub/Sub** — fan-out live moves to spectators (read-only), keeping the hot player path lean.

---

7. Game lifecycle: queue → match → play → end → persist



Key points:

1. **Single-writer game owner** processes moves serially → no race between the two players' moves; `ply` increments cleanly.
2. **Validate before applying** — an illegal or out-of-turn move is rejected and never mutates state.
3. **Clock updated on every move** — deduct the mover's elapsed, add increment, re-arm the opponent's flag timer.
4. **End detection is server-side** — the engine reports mate/stalemate/draw; the clock reports timeout.
5. **Persist + rate atomically on end** — the finished game is durable and both ratings move together.

---

8. Matchmaking (focus area)

Goal: pair two players of similar strength, in the same time control, quickly. Rating is a number (Elo/Glicko, ~0–3000); pools are separate per exact time control (you don't mix 1+0 with 10+0).

The pool structure

- One **waiting pool per time control**, holding `(player_id, rating, joined_at, region)`.
- Keep it **sorted by rating** (Redis **sorted set** `ZADD pool:5+3 rating player`) so the nearest-rated opponent is a range query away.

Pairing strategies

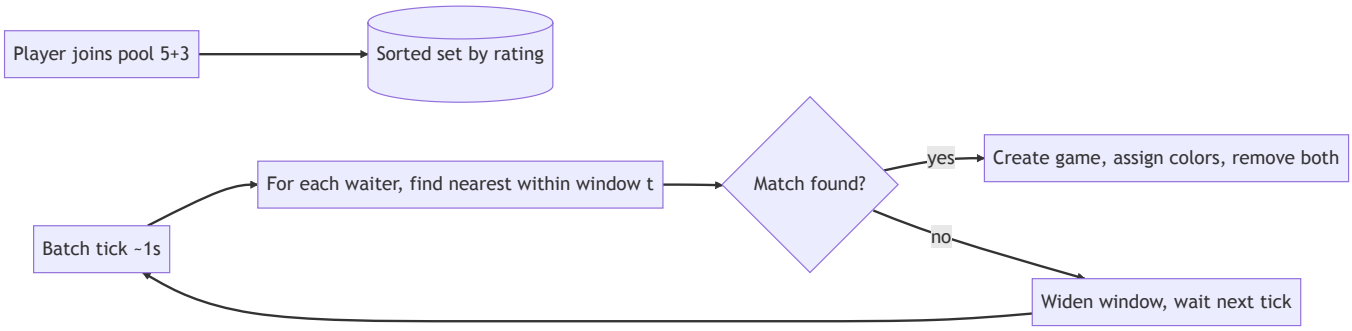
Strategy	How	Trade-off
Greedy on arrival	New player matches the closest waiting player within the window	Instant when pool is dense; can make globally sub-optimal pairs
Periodic batch (Lichess-style)	Every ~1 s, run a pass over the pool and pair globally (nearest-neighbor / min-cost matching)	Better pairings; adds up to ~1 s wait
Hybrid ✅	Try greedy immediately; if no in-window opponent, wait for the next batch tick	Fast when possible, optimal when busy

Widening window (the key idea)

Start strict, relax as the player waits so nobody waits forever:

```
window(t) = base ± (Δ0 + rate · seconds_waited)     # e.g., ±50, growing to ±100, ±200 …
```

- **Provisional players** (few games, high rating deviation) → **wider** window immediately (we're unsure of their strength).
- **Region/latency** — prefer same-region opponents for low RTT; widen to cross-region only after waiting.
- **Avoid immediate rematches** / recently-played opponents when the pool allows.



Color assignment — balance white/black over a player's recent games; otherwise random.

One-liner: *"Per-time-control pools sorted by rating in Redis; a batch tick pairs each waiter with the nearest-rated opponent inside a window that **widens with wait time** (and starts wide for provisional/high-RD players), preferring same-region — so matches are fast **and** fair."*

---

9. Move validation & game state (focus area)

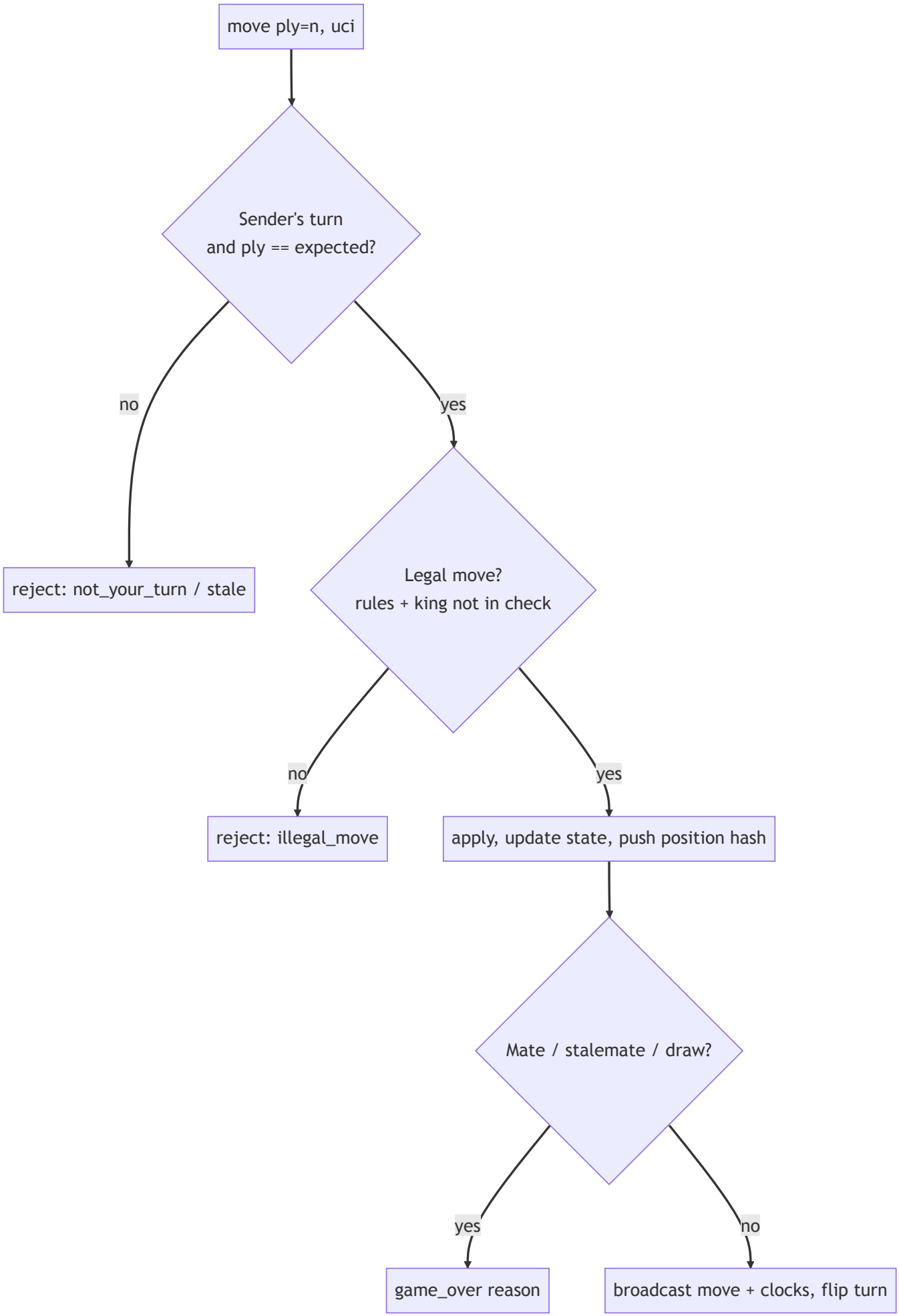
The server is the sole authority on legality — never trust the client. The client may validate locally for instant UI, but the server re-validates every move with a full rules engine.

Authoritative game state (per live game, in memory)

```
FEN / board       : piece placement (8x8, or bitboards)
side_to_move      : white | black
castling_rights   : KQkq
en_passant_target : square or none
halfmove_clock    : plies since last capture/pawn move  (50-move rule)
fullmove_number   : display
position_history  : hashes of every position            (threefold repetition)
```

Validating a move

1. **Right player, right turn** — the sender owns `side_to_move`; the submitted `ply` matches the expected next ply (else reject stale/duplicate — **idempotent**).
2. **Pseudo-legal + legal** — the piece moves correctly; **special rules** (castling through/into check, en passant, promotion) handled; the move **does not leave the mover's king in check**.
3. **Apply** — mutate the board, update castling/ep/halfmove, push the position hash, flip `side_to_move`.
4. **Detect end** — checkmate, stalemate, **threefold repetition**, **50-move rule**, **insufficient material**; report to the game server.
5. **Broadcast** the accepted move + new FEN + authoritative clocks to both players (and spectators).


move ply=n, uci
Sender's turn
and ply == expected?
no
yes
reject: not_your_turn / stale
Legal move?
rules + king not in check
no
yes
reject: illegal_move
apply, update state, push position hash
Mate / stalemate / draw?
yes
no
game_over reason
broadcast move + clocks, flip turn

Move format — prefer **UCI** `from+to(+promo)` (e.g., `e7e8q`); derive **SAN** (`e8=Q+`) for display/PGN. Unambiguous and easy to validate.

Anti-cheat — illegal moves are impossible (structural). **Engine-assistance** (a human relaying an engine's moves) is a *separate* problem: an async subsystem flags suspicious accuracy/timing patterns; out of the hot path.

One-liner: *"The server keeps the authoritative FEN + castling/ep/halfmove/position-history and re-validates every move — right turn, legal, king-safe — then detects mate/stalemate/threefold/50-move/insufficient-material. Clients are optimistic views that roll back on `move_rejected`."*

---

10. The game clock (focus area — the hard part)

The server owns time. Clients only display an estimate. The subtle problem is **who flags first under network latency** — a move sent at the buzzer may arrive after the server thinks the clock hit zero.

The model

- Each side has **remaining milliseconds**; `increment_ms` is added after each completed move (Fischer).
- The clock **only ticks for the side to move**. The server records `turn_started_ts` = the instant it broadcast the opponent's move (i.e., when it became your turn).
- On receiving your move at server time `t_recv`:

```
elapsed          = t_recv − turn_started_ts − lag_credit
remaining[mover] = remaining[mover] − elapsed + increment_ms
if remaining[mover] <= 0  before the move arrived  → the mover FLAGS (loses on time)
```

Detecting timeout when a player just sits there

- When it becomes X's turn, the Clock Manager **arms a flag timer** = `remaining[X]` ms.
- If the timer **fires before a move arrives** → declare **timeout**: X loses — *unless the opponent cannot possibly win* (e.g., opponent has only a king / insufficient material) → **draw** instead. If a mate/stalemate already occurred, that result stands.
- If a move **arrives just before** the timer, the game server (single-writer) processes it first and **cancels** the timer — the single-owner design makes this race deterministic (one thread decides order).

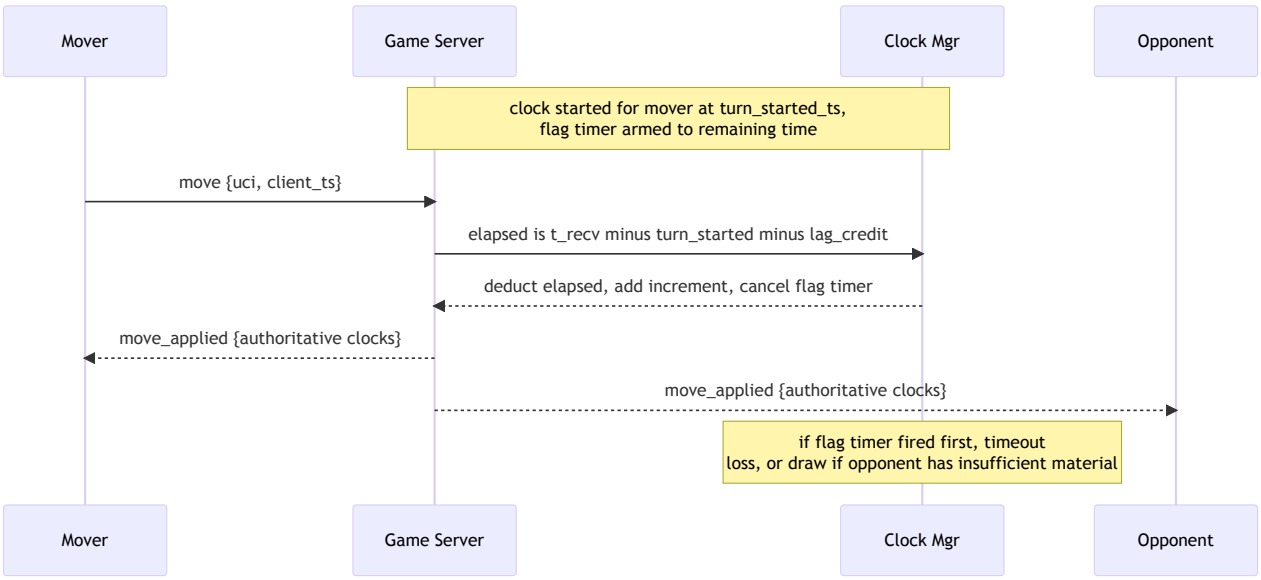
Lag compensation (fairness)

Naive "server receive time" penalizes high-latency players — unfair. Mitigations:

- **Measure per-player network lag** continuously (WebSocket ping/pong RTT) and credit back an estimated one-way lag (`lag_credit`) so a player isn't flagged for their connection.
- **Cap the credit** (e.g., ≤ a few hundred ms) so it can't be abused to gain time.
- **Grace on the very last move** — many implementations don't flag if the move was clearly *sent* before expiry (client_ts + measured lag), trusting the measured connection profile.

Keeping both clients in sync

- The server periodically pushes an **authoritative `clock` snapshot** `{white_ms, black_ms, turn, server_ts}`.
- Clients **interpolate locally** (tick down the active side) between snapshots but **snap to the server** on every update — so drift never accumulates and both players agree.



One-liner: *"The server is the clock authority: it deducts `t_recv − turn_started` from the mover, adds increment, and arms a flag timer for the side to move. Timeout ⇒ loss, unless the opponent has insufficient material ⇒ draw. Network lag is measured via ping and **credited back (capped)** so players aren't flagged for their connection; clients interpolate but always snap to the server's authoritative snapshots."*

---

11. Game session & real-time delivery (focus area)

One authoritative owner per game

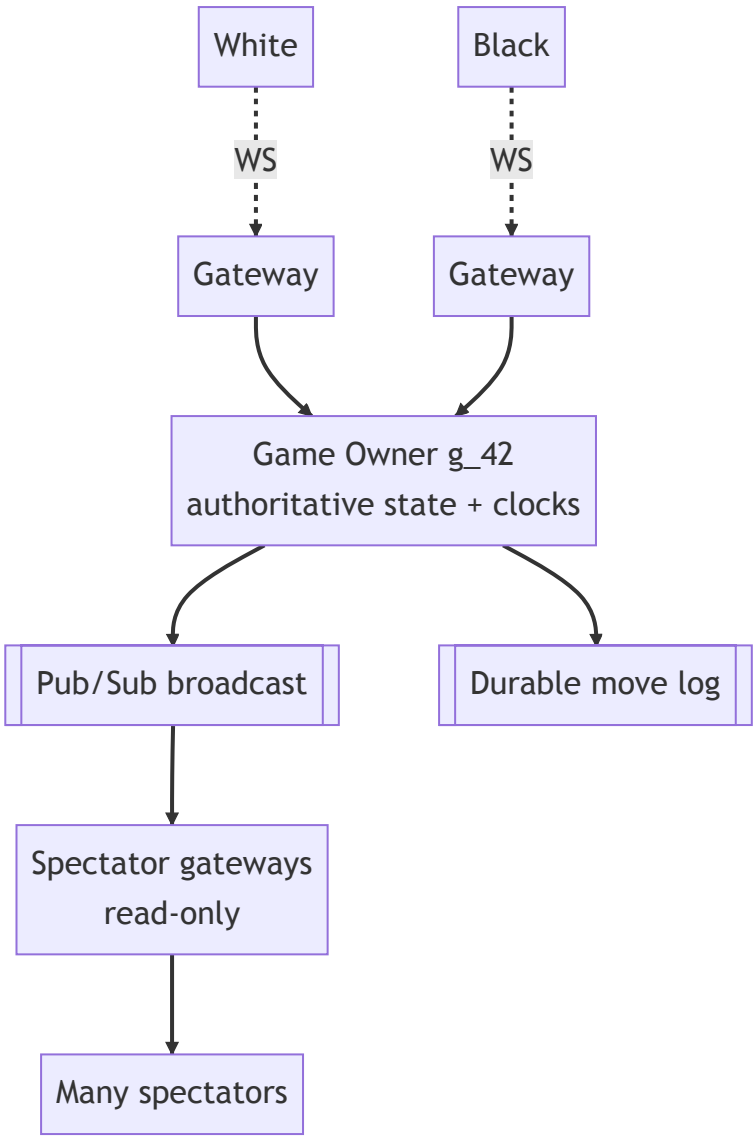
- Each live game is **owned by exactly one game-server node**, holding the state + clocks in memory (a **sharded actor**: `game_id → owner` via consistent hashing, recorded in Redis).
- **Single-writer** ⇒ moves are serialized naturally; there's no distributed consensus per move and `ply` ordering is automatic.
- Gateways forward each player's frames to the owner; the owner broadcasts results back through the gateways to both players.

Reconnection (clients drop constantly)

- Game state lives on the **server**, not the client. On reconnect, the client calls `GET /v1/games/{id}` (or a WS resume) and receives the **current FEN + clocks + move history**, then resumes live.
- **Clock keeps running** during a disconnect (you can't gain time by pulling the cable), but a small **grace window** and "opponent disconnected" indicator are shown; if you don't return before your flag, you lose on time. Optional: auto-abort if a player never makes a first move.

Spectators

- Read-only subscribers to the game's move stream via **pub/sub**; they get the same `move_applied/clock` events but can't submit moves.
- Popular games (thousands of watchers) → the owner publishes **once** to a fan-out layer (Redis pub/sub / a broadcast tier) that cached spectator gateways read — the **celebrity/hot-game** pattern, keeping the player path lean.



One-liner: *"Each game has a single authoritative owner node (sharded by `game_id`), so moves serialize without consensus. State lives server-side, so reconnects just re-fetch FEN + clocks and resume; the clock keeps running through disconnects with a grace indicator. Spectators are read-only pub/sub subscribers, and hot games fan out through a broadcast tier."*

---

12. Game end & rating updates (focus area)

End conditions

Reason	Detected by	Result
Checkmate	rules engine	mover wins
Resignation	player action	opponent wins
Timeout (flag)	clock manager	flagged player loses (draw if opponent has insufficient material)
Stalemate	rules engine	draw
Threefold repetition / 50-move	position history / halfmove clock	draw (claimable/automatic)
Insufficient material	rules engine	draw
Draw agreement	offer + accept	draw
Abort / abandonment	no first move / long disconnect	no result, usually unrated

Rating update (Glicko-2 preferred; Elo as the simple story)

On completion, the **Result & Rating Service** computes both deltas **atomically**:

- **Elo (say this first, it's simple):** expected score `E_A = 1 / (1 + 10^((R_B − R_A)/400))`; new `R_A' = R_A + K·(S_A − E_A)` with `S ∈ {1, 0.5, 0}`. `K` larger for provisional players.
- **Glicko-2 (production):** tracks rating **+ deviation (RD) + volatility**; uncertain/inactive players move more, confident players move less — better than Elo for confidence and inactivity. Ratings are **per time-control category**.
- Update **both players in one transaction** (they must move together); write a **rating-history** row for auditing/graphs. Only **rated** games count (casual/abort don't).

One-liner: *"The engine or clock reports the end; the Result service updates both players' **per-category** ratings atomically — Elo `R += K·(S − E)` as the simple model, **Glicko-2** in production for confidence/volatility — then the game is archived as PGN."*

---

13. Persistence & replay (focus area)

Two very different durability needs: **surviving a crash mid-game** vs **archiving finished games**.

Live durability (don't lose a game in progress)

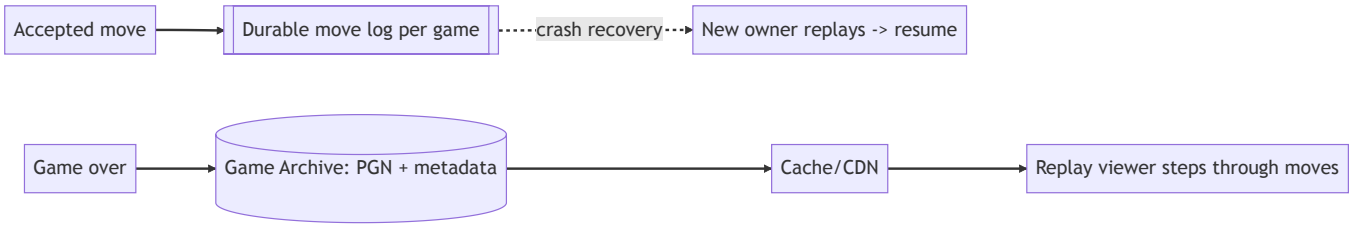
- The authoritative state is in memory (fast), but every accepted move is **appended to a durable move log** (event store) keyed by `game_id`, with the post-move clocks.
- If the owner node crashes, a new owner **replays the move log** to rebuild the exact position + clocks and resumes. (The log **is** the source of truth; the in-memory board is a cache.)

Finished-game archive (for later viewing)

- On end, write a single immutable record: **metadata** (players, ratings, time control, result, timestamps) + the **move list as PGN** (with clock annotations) + final FEN.
- **PGN** is the portable standard — compact, human-readable, replayable in any viewer:

```
[White "Ada"] [Black "Mia"] [Result "0-1"] [TimeControl "300+3"]
1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 ... 0-1
```

- Serve replays from a **cache/CDN** (finished games are immutable ⇒ perfectly cacheable). A replay is just "step through the moves," reconstructing each position client-side.



One-liner: *"Every move is appended to a durable per-game log, so a crashed game is rebuilt by replay — the log is the truth, memory is the cache. On end, the game is frozen as **PGN + metadata** in an append-only archive and served immutably from a CDN for replay."*

---

14. Storage choices

Data	Store	Why
Live game state + clocks	In-memory on the game owner (+ Redis backup)	Microsecond move application; single-writer authority
Move log / event store	Append-only log (Kafka / durable log / fast KV by `game_id`)	Crash recovery by replay; ordered by `ply`
Matchmaking pools	Redis sorted sets (by rating, per time control)	Range queries for nearest-rated opponent; ephemeral
Connection routing	Redis (`game_id → owner`, `player → gateway`)	Fast routing + cheap failover
Finished games (PGN)	Object store / wide-column, partition by `game_id` or `player_id`	Massive append-only volume; immutable; replay reads
Players & ratings	Relational (Postgres)	Transactions for atomic dual-rating updates + history
Rating history	Relational / time-series	Graphs, auditing
Replay delivery	CDN / cache	Immutable finished games ⇒ edge-cacheable
Anti-cheat features	Analytics store (async from move log)	Off the hot path

Live vs archived is the key split: **hot, tiny, in-memory** while playing (backed by a replayable log); **cold, immutable, cheap** once finished (PGN in an archive + CDN).

---

15. Scalability considerations

- **Game-server tier scales by game count** — each game is a few KB and CPU-light; a node hosts **tens of thousands of games**. Shard by `game_id` (consistent hashing); rebalance as load grows.
- **Connection tier scales independently** — ~100k WebSockets/node behind an L4/L7 LB; routing via Redis so any gateway can serve any player (cheap failover).
- **Matchmaking shards by time control** (and optionally region) — each pool is independent; hot pools (blitz) get more workers.

- **Stateless supporting services** — matchmaking pairing, rating, archive are horizontally scalable; durable state lives in Redis/log/DB.
- **Hot games (spectators)** — broadcast tier + cached game state fan out to many watchers without touching the player path.
- **Multi-region** — regional game servers + gateways for low RTT; **pair players by region** first; a game's owner lives in one region (both players routed there); archives replicate globally.
- **Graceful degradation** — under load, shed non-critical (spectator streams, presence) before player moves; the clock and move path are sacred.

---

16. Failure scenarios — *"what if X fails?"*

Failure	Impact	Mitigation
Player disconnects	Can't see/make moves	State is server-side; reconnect re-fetches FEN + clocks; **clock keeps running** + grace indicator; auto-abort if no first move
Game-owner node dies	Live games on it stall	New owner **replays the move log** to rebuild exact state + clocks and resumes; routing updated in Redis
Gateway node dies	Its connections drop	Clients reconnect to another gateway (routing via Redis) and resume
Move lost / retried	Duplicate/stale submit	Idempotent on `(game_id, ply)`: re-applying the same ply is a no-op; wrong ply rejected
Clock race at the buzzer	Mis-flag risk	**Single-writer owner** orders "move vs timer" deterministically; lag credit (capped) for fairness
Client clock drift	Displayed time wrong	Authoritative `clock` snapshots; client interpolates but snaps to server
Rating update fails	Inconsistent ratings	Atomic dual-update transaction; retry from the persisted result (idempotent by `game_id`)
Archive write fails	Game not saved	Move log is already durable → re-derive and retry the PGN archive
Cheating (engine use)	Unfair games	Structural legality always enforced; async statistical detection flags accounts
Region outage	Games unreachable	Owner + both players re-homed on reconnect; replay log to resume; archives replicated

Guiding principle: the **durable move log is the source of truth** (rebuild any live game), the **single-writer owner** makes move/clock ordering deterministic, and **idempotency on `(game_id, ply)`** heals retries. Correctness of state and time is non-negotiable; spectators/presence are best-effort.

---

17. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Authority	Trust client moves/clock (fast, cheating)	**Server-authoritative** legality + time	Server authority — always ✅
Clock truth	Client-timed (unfair, hackable)	**Server-timed + lag credit**	Server clock; measure lag, credit capped ✅
Game state	Distributed/replicated per move (consensus)	**Single-writer owner in memory** + replayable log	Single owner — no per-move consensus ✅
Matchmaking	Greedy instant (sub-optimal)	Batch optimal (slower)	**Hybrid**: greedy, fall back to batch tick
Rating	Elo (simple)	**Glicko-2** (confidence/volatility)	Glicko-2 in production; Elo to explain
Move ordering	Sequencer/consensus (chat-style)	**`ply` from single writer**	Turn-based ⇒ ply is free ordering ✅
Persistence	Snapshot state periodically	**Event-sourced move log**	Log = truth + crash recovery + PGN ✅
Disconnect	Pause clock (exploitable)	**Keep clock running** + grace	Keep running; grace indicator
Spectators	Push through player path	**Separate pub/sub broadcast tier**	Isolate hot games from player latency

CAP framing: each game is a **CP island** — one authoritative owner serializes moves and time (consistency + correctness over availability *for that game*), rebuildable from a durable log after failover. Matchmaking, spectating, and presence lean **AP** (stay available, reconcile). Acked/rated results are **durable, non-negotiable**.

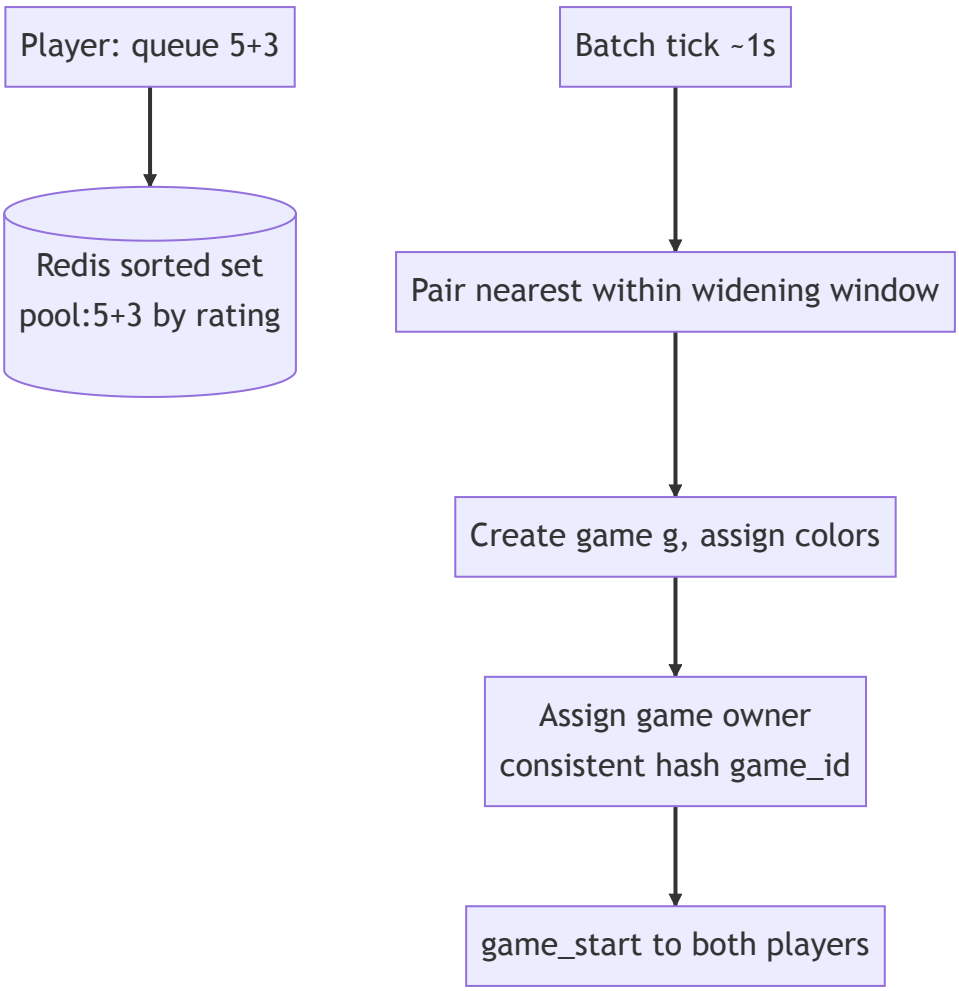
One-liner to say out loud: *"Players queue by time control and are **paired by rating** with a window that widens over time. Each game gets a **single authoritative owner** that validates every move with a rules engine and owns the clock — deducting `t_recv − turn_started` and arming a flag timer, with **capped lag credit** so nobody is flagged for their connection. Moves are ordered for free by **`ply`** (single writer), every move is appended to a **durable log** (so a crashed game is replayed and resumed), and on end both **Glicko-2 ratings update atomically** and the game is frozen as **PGN** in an immutable, CDN-served archive. Clients are optimistic views that roll back on rejection and snap to authoritative clock snapshots."*

---

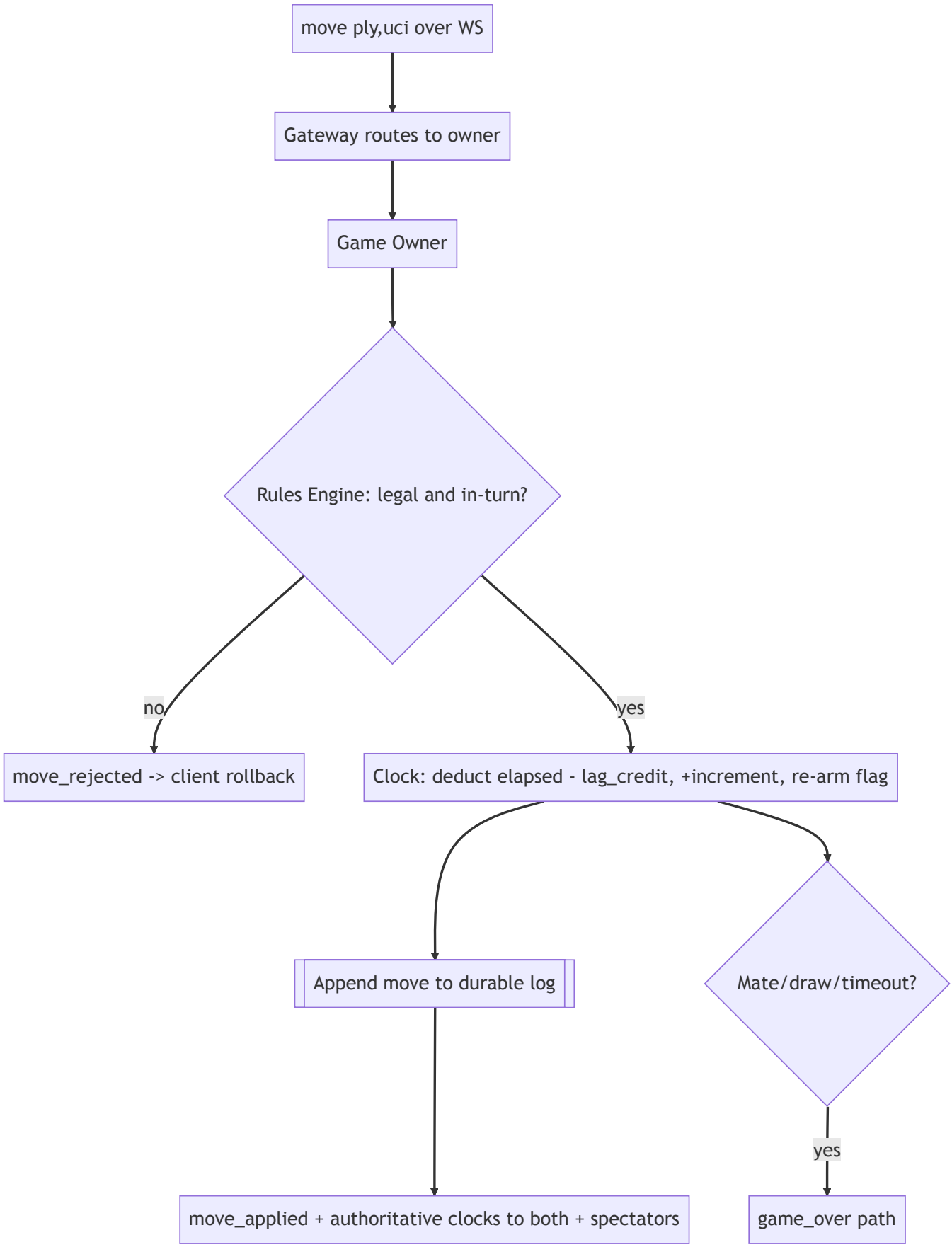
18. Full system design (detailed)

End-to-end, split into **(A)** matchmaking, **(B)** the live move/clock loop, and **(C)** end/persist/rate + replay.

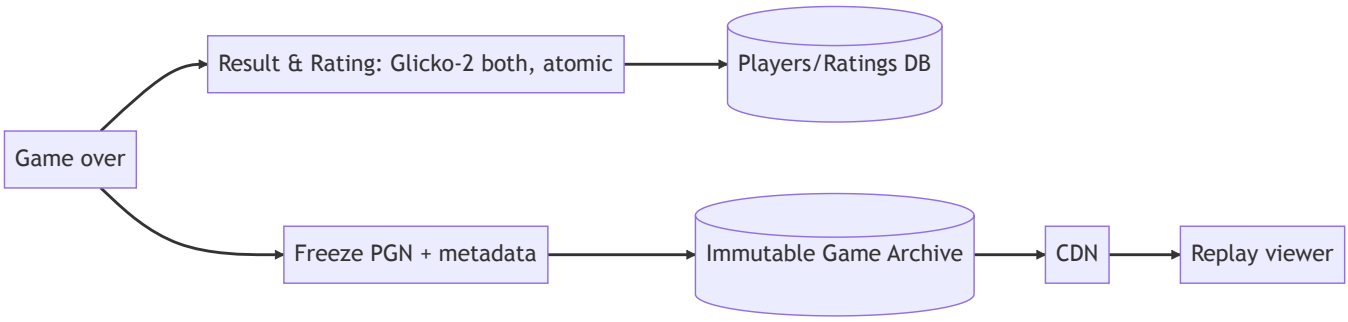
18A. Matchmaking → game creation



18B. Live move & clock loop



18C. End → persist → rate → replay



---

19. Networking, security & performance best practices

Networking

- **Persistent WebSocket per player** with ping/pong heartbeats (also the **lag measurement**), auto-reconnect with backoff + jitter.
- **Independent gateway tier** (~100k conns/node) behind an L4/L7 LB; **routing via Redis** (`game_id → owner`, `player → gateway`) for cheap failover.
- **TLS at the edge**; **HTTP/3 / QUIC** for mobile resilience; **gRPC** internal; both players of a game routed to the **same owner** (and, multi-region, the same region).

Security & fairness

- **AuthN/Z** on connect and per action (only the two seated players can move/resign; spectators are read-only).
- **Server-authoritative legality** blocks illegal-move exploits; **rate-limit** move spam and queue abuse.
- **Anti-cheat**: async statistical **engine-detection** (accuracy/timing) on the move log; flag & review — never trust client "analysis."
- **Abuse**: sandbag/boost detection via rating anomalies; fair-play bans; audit result-affecting events.

Performance

- **In-memory authoritative state** + microsecond validation ⇒ sub-ms server processing per move.
- **Single-writer owner** ⇒ no locks/consensus on the hot path; `ply` ordering for free.
- **Authoritative clock snapshots + client interpolation** ⇒ exact-feeling clocks without chatty updates.
- **Immutable finished games** ⇒ CDN-cacheable replays; **hot games** fan out via a broadcast tier.

---

20. Staying current — modern & emerging approaches

- **Reference systems: Lichess** (open-source; Scala, **Glicko-2**, actor-per-game, lag compensation) and **Chess.com**.
- **Ratings: Glicko-2** / TrueSkill over classic Elo for confidence + inactivity handling.
- **Transport:** WebSocket, **HTTP/3/QUIC**, gRPC streaming; actor models (Akka/Erlang/Elixir) for per-game ownership.
- **State:** event-sourced move logs (the game **is** its move list); Redis for pools/routing; object store + CDN for PGN.
- **Anti-cheat:** ML/statistical engine-move detection, timing analysis, centipawn-loss profiling.
- **Patterns: sharded actors** (one owner per game), **event sourcing**, **optimistic client + server reconciliation**, **lag compensation** (borrowed from real-time gaming/netcode).
- **How I stay current:** the Lichess engineering blog & open-source repo, real-time game netcode talks, distributed-systems and rating-theory papers.

---

21. Likely follow-up questions (rehearse these)

- **How do you decide who flagged first under lag?** *(server-authoritative clock, `t_recv − turn_started`, capped lag credit; single-writer owner orders move-vs-timer deterministically)*
- **How do you stop illegal-move cheating?** *(server-side rules engine validates every move; client is just a view)*
- **A player disconnects mid-game — what happens?** *(state is server-side; reconnect re-fetches FEN + clocks; clock keeps running with a grace indicator)*
- **The game server crashes — is the game lost?** *(no — replay the durable move log to rebuild exact state + clocks and resume)*
- **How do you match players fast but fairly?** *(rating-sorted pools per time control; window widens with wait; wider for provisional/high-RD)*
- **Elo vs Glicko-2?** *(Elo `R += K(S−E)` simple; Glicko-2 adds deviation + volatility → better confidence/inactivity)*
- **How is move ordering guaranteed?** *(strictly turn-based; single-writer owner increments `ply`; no sequencer needed)*
- **Timeout but the opponent only has a king?** *(insufficient material ⇒ draw, not a loss)*
- **Handling thousands of spectators on a top game?** *(broadcast tier + cached state, isolated from the player path)*

---

22. Summary checklist (whiteboard recap)

- **Entities** — players (+ per-category rating/RD), games, moves (ordered by **`ply`**), in-memory state + clocks; PGN archive.
- **Transport** — WebSocket for moves/clocks; REST for queue/history/replay; moves idempotent on `(game_id, ply)`.
- **Matchmaking** — Redis rating-sorted pools **per time control**; **window widens with wait**; wider for provisional; pair by region.
- **Move validation** — **server-authoritative** rules engine (legality, check/mate, stalemate, threefold, 50-move, insufficient material); client optimistic + rollback.
- **Clock** — **server owns time**; deduct `t_recv − turn_started`, add increment, arm flag timer; **timeout ⇒ loss (draw if insufficient material)**; **capped lag credit**; snapshots + client interpolation.
- **Session** — **one authoritative owner per game** (sharded by `game_id`, single-writer ⇒ free ordering); reconnection re-fetches state; spectators via pub/sub.
- **End & rating** — engine/clock detect end; **Glicko-2** (or Elo) update **both players atomically**.
- **Persistence** — every move appended to a **durable log** (crash recovery by replay); finished game frozen as **PGN + metadata**, served immutably via CDN.
- **Scale** — game tier by game count (tens of k/node), connection tier ~100k WS/node, matchmaking sharded by time control, multi-region pairing.
- **CAP** — each game is a **CP island** (single-writer owner, replayable log); matchmaking/spectating/presence lean AP; rated results durable and non-negotiable.

Designing a URL Shortener (Bit.ly)

> Interview-ready walkthrough based on the Hello Interview breakdown: requirements → estimates → entities → API → v1 → deep dives (**unique codes, fast redirects, scaling to 1B URLs / 100M DAU**) → failure handling → trade-offs. The hard parts are **globally unique short codes** and **extreme read skew**.

> Entry-level question, but seniors are graded on counter coordination, cache invalidation for expiry, 301 vs 302, multi-region, and failure of the counter.

---

0. How to drive the interview (talk track)

1. **Clarify**: create short URL, redirect, optional custom alias + expiry. Analytics and auth out of scope.
2. **Estimate**: 100M DAU, 1B URLs, read:write about 1000:1.
3. **Entities and API**: `POST /urls`, `GET /{code}`.
4. **v1**: client, one server, one DB.
5. **Deep dive 1**: unique code generation (prefix, hash, counter + base62).
6. **Deep dive 2**: fast redirect (index, Redis, CDN).
7. **Deep dive 3**: scale (split read/write services, Redis counter batching, HA).
8. **Close** with failure modes and trade-offs.

---

1. Problem

Convert a long URL into a short code (`short.ly/abc123`) and redirect visitors to the original. The mapping is **immutable-ish, tiny, and read about 1000x more than written**.

What makes it hard

- Codes must be **unique** with **no coordination bottleneck**.
- Codes must be **short** (7 chars of base62 is about 3.5 trillion values).
- Redirects must be **< 100 ms** and **99.99% available**.

---

2. Requirements

Functional

- Submit a long URL and get a short URL.
- Optional **custom alias** and optional **expiration date**.
- Visiting the short URL redirects to the long URL.

Out of scope: auth/accounts, click analytics, spam/malicious URL detection.

Non-functional

- Uniqueness: one short code maps to exactly one long URL.
- Redirect latency < 100 ms.

- Availability 99.99% (availability over consistency).
- Scale: 1B URLs, 100M DAU.

Clarifying questions

- Can one long URL have many short codes? (Yes: separate expiry, owner, alias.)
- Should codes be guessable/enumerable? (Usually fine; private URLs need a larger space or a permutation.)
- Is analytics needed later? (Affects 301 vs 302.)

---

3. Estimation

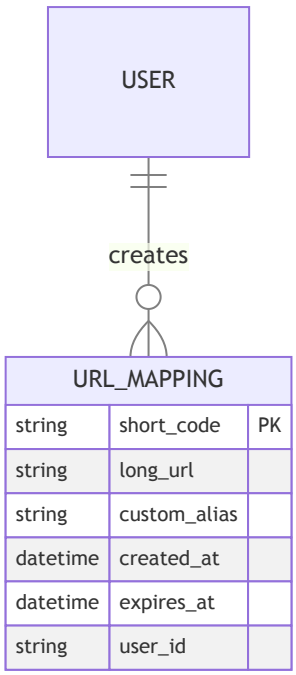
Quantity	Assumption	Result
New URLs/day	100K	about 1 write/s
Redirects/day	100M DAU x 1 click	about 1.2K/s avg, about 10K/s peak
Row size	code 8B + url 100B + ts + alias + expiry	about 200 B, round to 500 B
Total storage	1B x 500 B	**about 500 GB** (fits one SSD)
Read:write	skewed	**about 1000:1**

Consequences

1. 500 GB fits one Postgres node, so sharding is **not needed yet**; replicas for HA.
2. Writes are about 1/s, so the DB is not the write bottleneck. **Reads dominate**, so cache aggressively.
3. Hot links (viral) follow a power law, so a small cache covers most traffic.

---

4. Entities



---

5. API

```
POST /urls
{ "long_url": "https://example.com/very/long", "custom_alias": "my-link", "expiration_date": "2027-01-
-> 201 { "short_url": "http://short.ly/abc123" }
-> 409 alias already taken, 400 invalid URL

GET /{short_code}
-> 302 Location: https://example.com/very/long
-> 404 unknown code, 410 expired
```

301 vs 302

	301 permanent	**302 temporary**
Browser caches	Yes, skips our server	No
Pros	Lowest latency and load	Control: expire, edit, analytics
Cons	Cannot revoke or expire; analytics lost	Every click hits us

Choose **302** because the service is an indirection layer (expiry, takedown, future analytics).

---

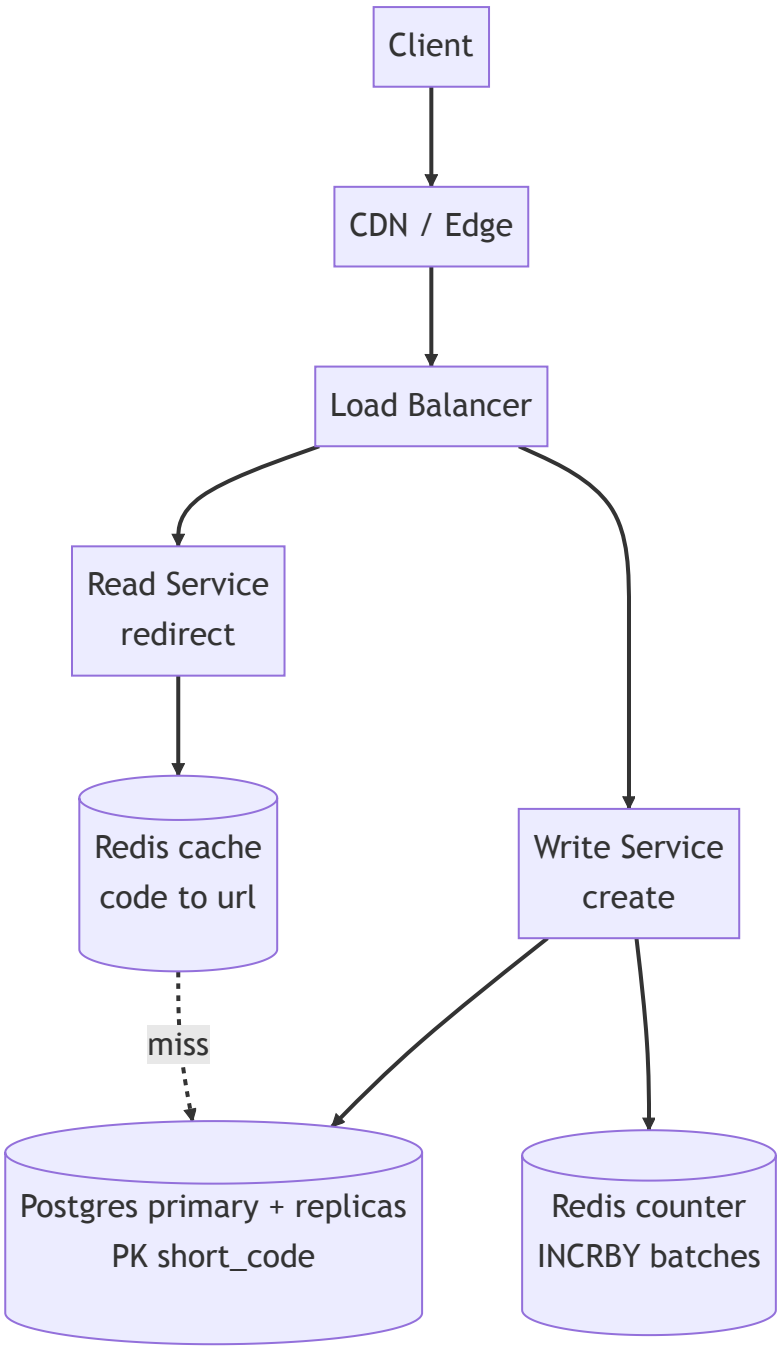
6. Architecture

v1



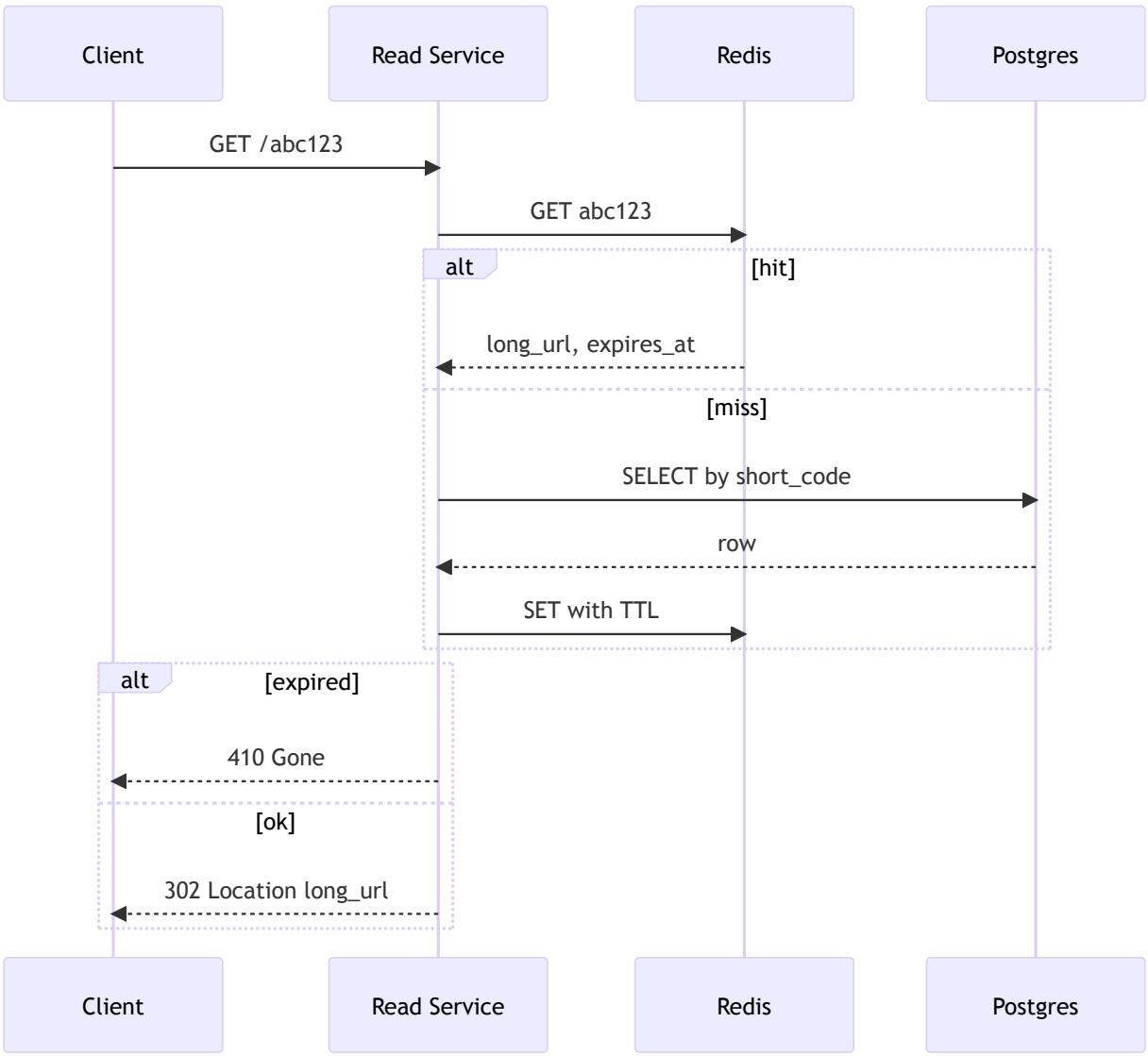
Cons: full-table lookup, one server, uniqueness unspecified.

Final design


Client
CDN / Edge
Load Balancer
Read Service
redirect
Redis cache
code to url
miss
Write Service
create
Postgres primary + replicas
PK short_code
Redis counter
INCRBY batches

---

7. Data flow



---

8. Deep dive 1: unique short codes

Approach	Verdict	Why
Long URL prefix (first N chars)	Bad	Collisions are certain; not unique
Hash (SHA-256 of long URL + salt, base62, first 7 chars)	Good	Stateless, no coordination. Truncation raises collisions (birthday paradox). Needs DB unique constraint and retry with a new salt. Same URL = same code unless salted
Counter + base62	Great	Collision-free by construction. Needs a shared counter

Base62: `[0-9a-zA-Z]`. 62^7 is about 3.5T, enough for 1B URLs for centuries.

Custom alias: check uniqueness with an insert on the PK. Keep generated codes in a separate namespace (e.g. prefix or reserved charset) so aliases never collide with counter values.

One-liner: *Counter + base62 gives guaranteed uniqueness; hashing is the stateless alternative that pays with collision retries.*

---

9. Deep dive 2: fast redirects

Layer	Effect
Index on `short_code` (PK, B-tree)	O(log n) instead of full scan; the baseline
Redis cache (cache-aside, LRU)	Sub-ms; power-law hot set is small. 20% of 1B x 500 B is large, but the **hot** few GB is what matters
CDN/edge	Cache 302s near the user; short TTL so expiry and takedown still work

TTL on cache entries <= URL expiry, so expired links evict themselves. Negative-cache unknown codes briefly to resist scanning.

One-liner: *Index, then cache-aside Redis, then edge. Keep TTLs aligned with expiry.*

---

10. Deep dive 3: scaling to 1B URLs and 100M DAU

- **Split Read and Write services**: 1000:1 skew, scale independently.
- **Storage**: 500 GB fits one node. Shard by `short_code` hash only if it grows. Replicate for HA; take backups.
- **Counter across many writers**: central **Redis `INCR`** (atomic, 100K+ ops/s). Use **counter batching**: each writer grabs a block of 1000 via `INCRBY 1000` and issues IDs locally.
- **Counter HA**: Redis Sentinel/Cluster. On failover a few values may be lost or re-issued. Uniqueness is preserved by the **DB unique constraint** (retry with a new batch). Gaps are harmless.
- **Multi-region**: disjoint ranges per region (A: 0-1B, B: 1B-2B) so no cross-region coordination.
- **Alternatives**: Snowflake IDs (no central dependency but 11-char codes), pre-allocated ranges from DB/ZooKeeper.

Bottleneck path: DB read, then Redis hit rate, then Redis counter, then DB write.

---

11. Failure handling

Failure	Impact	Mitigation
Redis cache down	Reads hit DB	Replicas, request coalescing, rate limit, controlled warm-up
Redis counter loses state	Possible duplicate codes	DB PK rejects; fetch new batch and retry; restart from max(code)
DB primary down	No writes	Promote replica; reads continue from cache/replicas
Write service crash	Unused counter block	Gaps are fine
Region outage	Region's links	Multi-region replicated DB; DNS failover
Abuse (scanning/spam)	Load	Per-IP rate limits, CAPTCHA, malicious URL checks

---

12. Trade-offs

Decision	Chosen	Alternative	Given up
Code generation	Counter + base62	Hash	Central coordination point
Redirect	302	301	Latency and server load
DB	Postgres + replicas	DynamoDB	Auto-scaling convenience (not needed at 1 write/s)
Dedup same long URL	No	Yes	Storage for per-link expiry/analytics
Code predictability	Sequential	Permuted/random	Enumeration risk (mitigate with a permutation or wider space)

---

13. What is expected at each level

- **Mid:** working create + redirect, unique code idea, 302 reasoning, index, cache when prompted.
- **Senior:** drives read-heavy analysis, hash vs counter trade-off, cache invalidation with expiry, read/write split, Redis counter scaling.
- **Staff+:** multi-region ranges, counter failover behavior, enumeration/security, expiry cleanup, product evolution (analytics, link edit).

---

14. Follow-ups

- Add click analytics (stream to Kafka, aggregate offline; forces 302).
- Custom domains per tenant.
- Link expiry cleanup (TTL index, lazy deletion).
- Prevent enumeration (format-preserving encryption of the counter).

15. Summary checklist

- [ ] Read-heavy (1000:1), 500 GB, one node + replicas
- [ ] Counter + base62, batched from Redis, DB unique PK as safety net
- [ ] 302 redirect, cache-aside Redis, CDN
- [ ] Read/write service split
- [ ] Multi-region via disjoint ranges

Designing Dropbox (File Storage and Sync)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**large uploads, sync, chunking and dedup, resumable transfers, security**) → trade-offs. The hard parts are **big files over flaky networks** and **efficient multi-device sync**.

> Think Dropbox / Google Drive. Files live in object storage, **metadata** in a database, and clients upload and download **directly** from blob storage through signed URLs.

---

0. Talk track

1. Clarify upload/download/sync/share; file size up to 50 GB.
2. Estimate: 500M users, 100M DAU, 50 PB.
3. Entities: File, FileMetadata, User, Share.
4. API: request upload URL, get download URL, sync changes.
5. HLD: metadata service + blob store + signed URLs.
6. Deep dives: large files (chunking), sync (polling/WebSocket + change feed), speed (CDN, compression, delta), security.

---

1. Problem

Users store files in the cloud, access from any device, **share** them, and keep devices **automatically synced**.

What makes it hard

- Files up to **50 GB**: must not stream through app servers.
- **Flaky networks**: uploads must **resume**.
- **Sync**: local and remote changes must converge quickly and cheaply.
- Durable (no file loss) and **available**.

---

2. Requirements

Functional

- Upload and download files from any device.
- Share files with other users.
- Auto-sync across devices.

Out of scope: in-browser editing (Docs), file versioning UI, virus scanning details.

Non-functional

- **Availability over consistency** (consistency eventual for sync).
- Files up to 50 GB; **resumable** uploads.
- Low latency uploads/downloads.
- **Highly durable** (never lose a file), secure (encrypt at rest and in transit).

3. Estimation

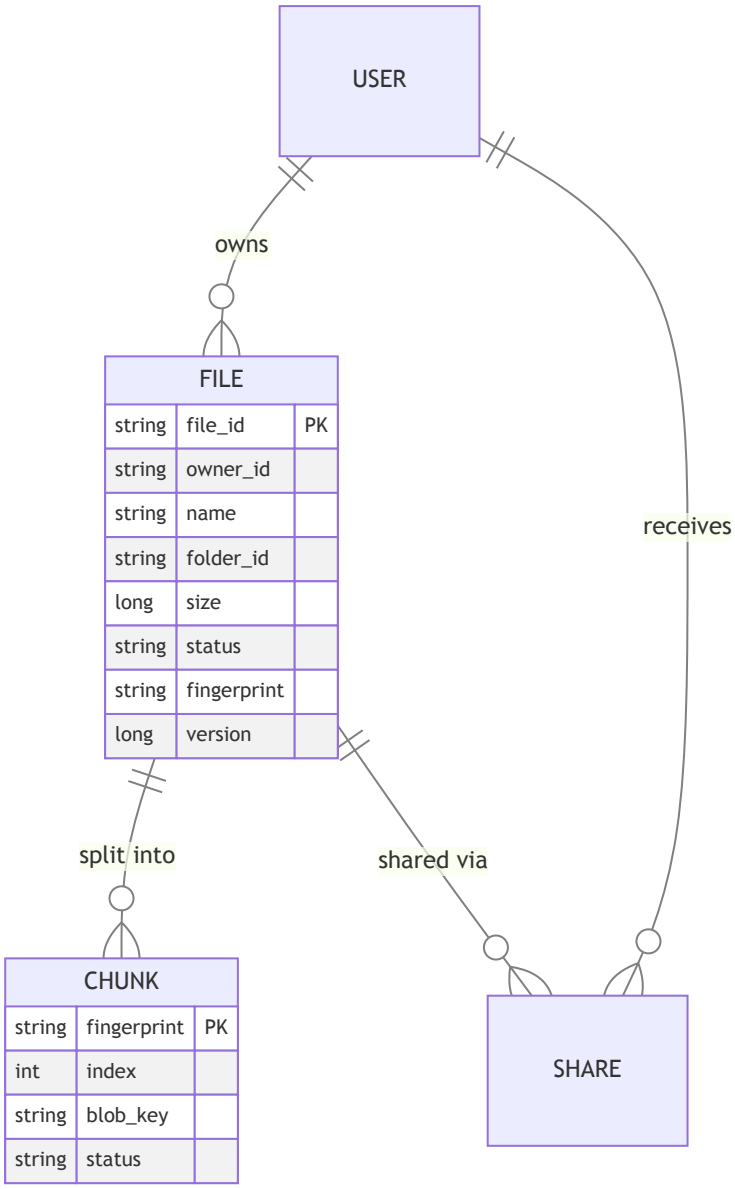
Quantity	Assumption	Result
Users	500M, 100M DAU	
Files/user	200 avg, 1 MB mean	**about 100B files, 100 PB logical** (dedup/compress reduces)
Uploads/day	100M DAU x 2	about 2K/s avg
Metadata row	about 500 B	**50 TB** metadata
Egress	downloads x 2 uploads	multi-Tbps, mostly via CDN

Consequences

- **Blob data goes straight to S3-like storage**; app servers only handle metadata.
- Metadata (50 TB) needs **sharding by user_id**.
- Chunking is essential for large files (retry, parallel, dedup).

---

4. Entities



---

5. API

```
POST /files/upload-session            # metadata first
{ "name":"a.mov", "size":8000000000, "fingerprint":"sha256...", "chunk_count": 1600 }
-> 201 { "file_id":"f1", "upload_id":"u1", "presigned_urls":[...] }

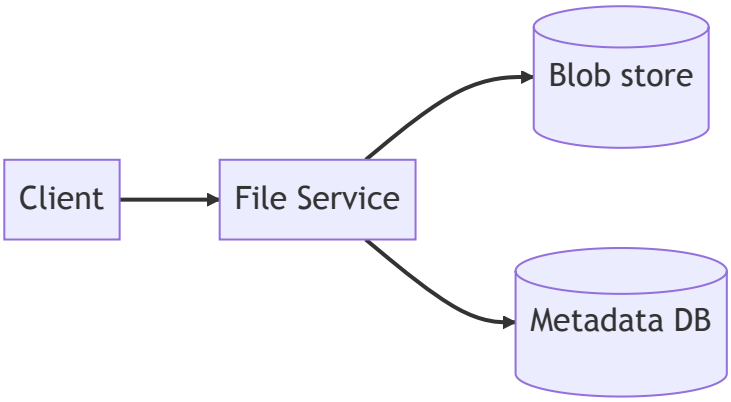
PUT <presigned S3 url for chunk i>    # client uploads DIRECTLY to blob store
POST /files/{id}/complete             # server verifies chunks, marks status=uploaded

GET  /files/{id}/download             -> 302 / signed CDN URL
POST /files/{id}/share  { "user_id":"u2", "perm":"read" }
GET  /changes?cursor=123              # sync: delta since cursor
```

---

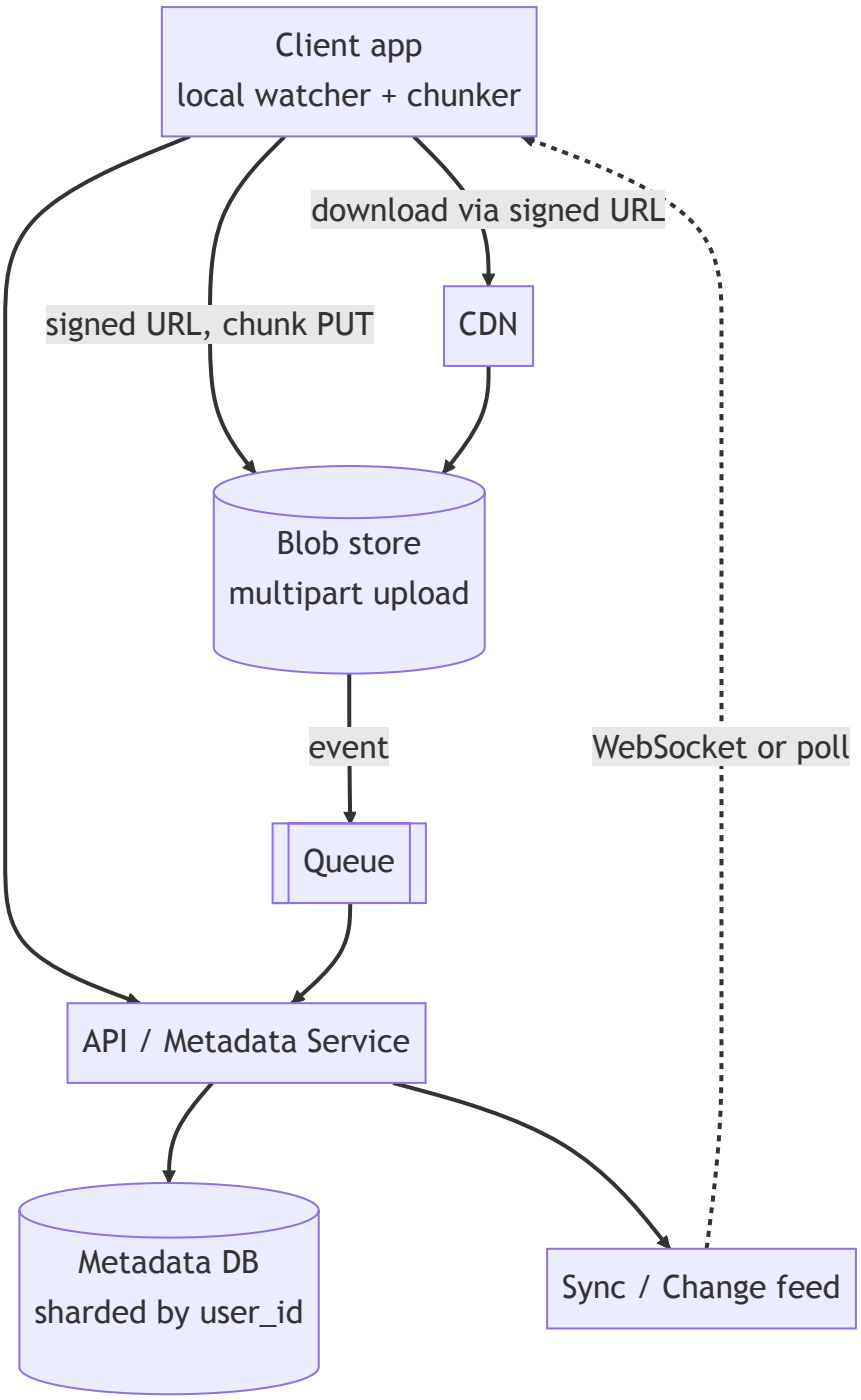
6. Architecture

v1



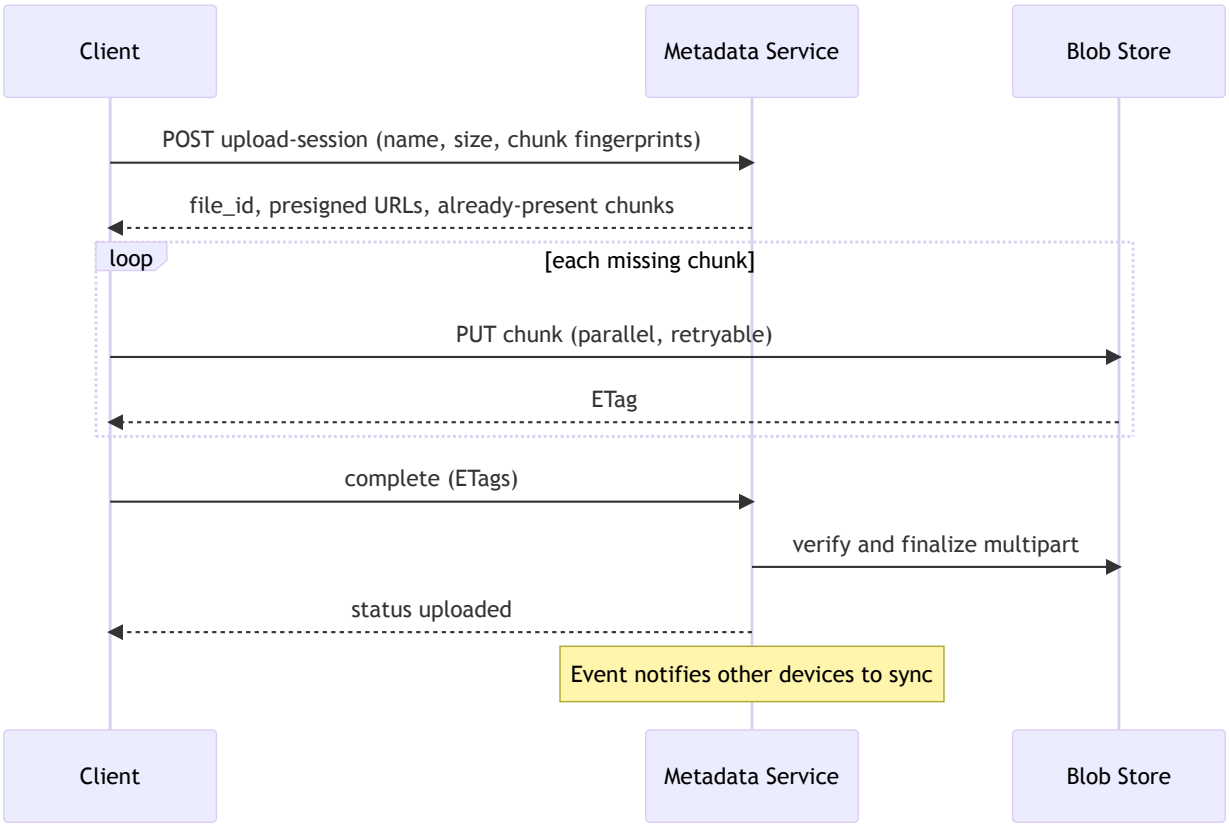
Problem: 50 GB through the app server is slow and costly; no resume.

Final design



---

7. Upload flow



---

8. Deep dive 1: uploading large files

Approach	Verdict	Why
Upload through app server	Bad	Bottleneck; server memory/bandwidth
Presigned URL direct upload	Good	Offloads bytes to blob store, but one request fails = restart
Chunked/multipart (5-10 MB chunks)	Great	Parallel, **resumable**, per-chunk retry, enables dedup

Resume: server tracks which chunks arrived (`status` per chunk or S3 `ListParts`). Client asks "what do you have?" and sends only the rest. Chunk completion is verified by hash/ETag. Files marked `uploading` until the commit; incomplete uploads garbage collected.

One-liner: *Chunk and presign: bytes bypass the servers, retries are per chunk.*

---

9. Deep dive 2: sync

Direction	Mechanism
Local to remote	OS file watcher detects change, client chunks, uploads, updates metadata
Remote to local	Client keeps a `cursor`; **long poll / WebSocket** notified of changes, then `GET /changes?cursor`

- **Hybrid**: WebSocket/push for fresh files (low latency), periodic polling as safety net.
- **Conflicts**: two devices edit the same file: keep both (`file (conflicted copy)`) and let the user merge, using version/vector.

- **Delta sync**: only changed chunks transfer (content-defined chunking so an insert at the start doesn't change every chunk).

One-liner: *Cursor-based change feed plus push notification; sync only changed chunks.*

---

10. Deep dive 3: speed and cost

- **Chunk-level dedup** by SHA-256 fingerprint: skip uploads already stored (cross-user dedup carries a side-channel privacy risk, so scope it per user or require proof of possession).
- **Compression** before upload; **CDN** for popular downloads; parallel chunk transfer.
- **Tiering**: cold files to cheap storage (S3 IA/Glacier).

---

11. Security

- Auth on API; **short-lived presigned URLs** scoped to a single object and operation.
- Encryption at rest (server-side keys, optionally per-user) and TLS in transit.
- ACL check on every metadata and download request; shares as ACL rows or signed share links with expiry.
- Malware scan on upload completion (async).

---

12. Failure handling

Failure	Mitigation
Client drops mid-upload	Resume by chunk
Metadata DB primary fails	Replica promotion; shards isolate blast radius
Blob store region outage	Cross-region replication, 11 9's durability
Orphaned chunks	Lifecycle GC for unfinished uploads
Sync storm after outage	Cursor-based catch-up, jittered reconnects, rate limit

---

13. Trade-offs

Decision	Chosen	Alternative	Given up
Upload path	Direct to S3	Via servers	Control (mitigated by signed URLs + callbacks)
Chunking	Fixed 5-10 MB	Content-defined	Simpler vs better delta dedup
Sync	Push + poll	Poll only	Connection management
Consistency	Eventual	Strong	Brief divergence across devices
Dedup	Per-user	Global	Storage savings vs privacy

---

14. Level expectations

- **Mid:** metadata + blob separation, presigned URLs, basic sync.
- **Senior:** chunked resumable uploads, change feed, conflict handling, CDN, sharded metadata.
- **Staff+:** content-defined chunking, dedup privacy, storage tiering and cost, end-to-end integrity, outage drills.

15. Follow-ups

Versioning/undo, shared folders with permissions, offline mode, selective sync, E2E encryption (kills dedup/preview).

16. Summary checklist

- [ ] Metadata DB sharded by user, blobs in object store
- [ ] Presigned multipart upload, resumable
- [ ] Change feed with cursor + push
- [ ] Chunk hashes for dedup and delta sync
- [ ] CDN downloads, encryption, ACLs

Designing Ticketmaster (Ticket Booking)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**no double-booking, seat reservation, flash-sale surge, virtual waiting room, search**) → trade-offs. The hard parts are **correctness under contention** and **burst traffic**.

> Think Ticketmaster / Eventbrite. Browsing and search are read-heavy and eventually consistent; **booking a seat is strongly consistent**.

---

0. Talk track

1. Clarify: view events, search, book tickets. Scale: 10M users per hot event.
2. Highlight the core tension: **consistency for booking vs availability for browsing**.
3. Entities and API.
4. HLD: Event service, Search, Booking service, payment.
5. Deep dives: **seat hold** (reserve before pay), distributed lock/TTL, waiting room, scaling reads, search.

---

1. Problem

Let users find events, view seat maps, **reserve** seats, pay, and receive tickets, while ensuring **a seat is sold only once** even when millions hit the same on-sale.

What makes it hard

- **No double-booking** under massive concurrency.
- **Hot events**: millions arrive at 10:00 for one stadium.
- **Hold-then-pay**: seats locked during checkout, released on abandonment.

---

2. Requirements

Functional

- View events and details.
- Search events (name, artist, city, date).
- Book tickets (select seat, pay).

Out of scope: dynamic pricing, resale marketplace, admin tools, GDPR.

Non-functional

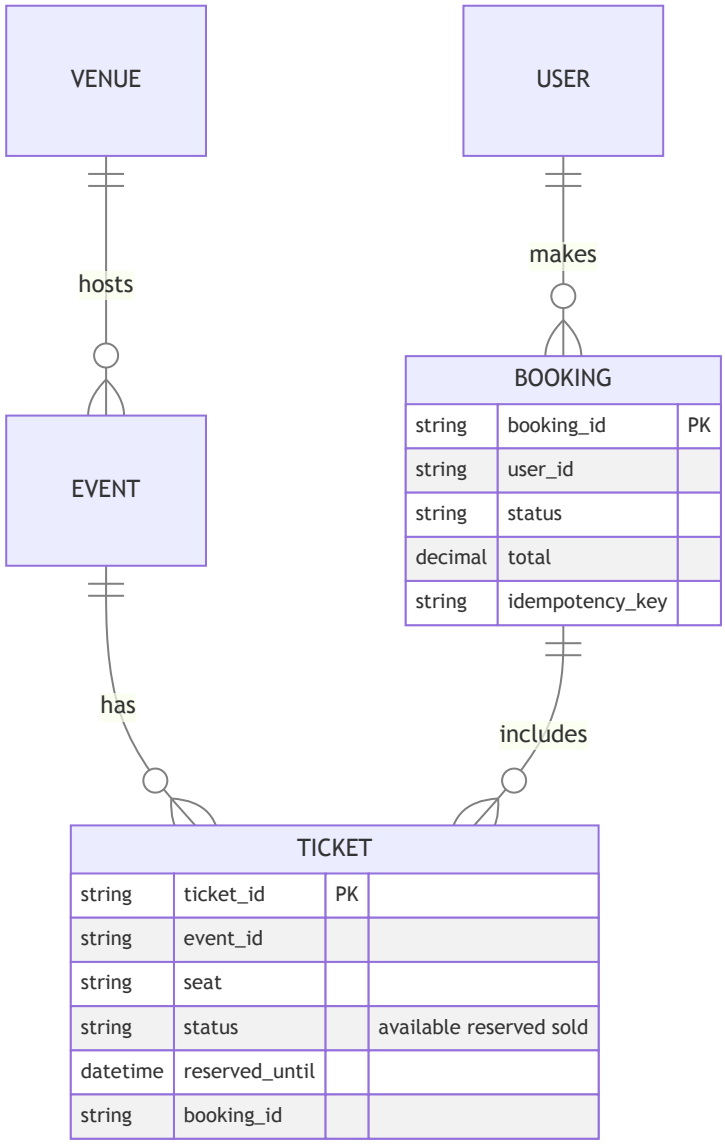
- **Strong consistency for booking** (no double-sell); **availability for browse/search**.
- Read-heavy: **100:1** browse to book.
- Scale to **10M concurrent users for a popular event**.
- Search under 500 ms; low-latency browse.
- Fault tolerant; fair queueing for hot events.

3. Estimation

Quantity	Assumption	Result
Users	100M MAU	
Hot on-sale	10M users, 50K seats	**200:1 contention** per seat
Browse QPS	10M within minutes	hundreds of K req/s peak
Bookings/s	50K seats over about 10 min	about 100/s sustained, spiky
Data	1M events, 50K seats each	tens of GB: small. **Traffic, not data, is the problem**

Consequence: a relational DB handles the data; the challenges are **surge control** and **seat contention**, not capacity.

4. Entities



5. API

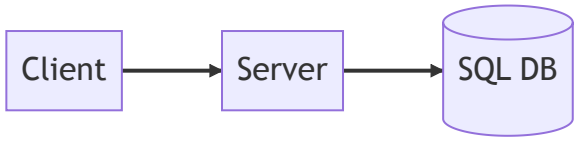
```
GET  /events/{id}                       -> event + seat availability
GET  /events/search?q=&city=&date=&cursor=
POST /bookings/reserve   { "event_id":"e1", "ticket_ids":["t7","t8"] }   -> 200 hold with expires_at
POST /bookings/confirm   { "booking_id":"b1", "payment_token":"..." }
    Idempotency-Key: <uuid>             -> 200 confirmed | 410 hold expired | 402 payment failed
```

Two-step reserve then confirm (like checkout) so payment is not done while holding a DB lock.

---

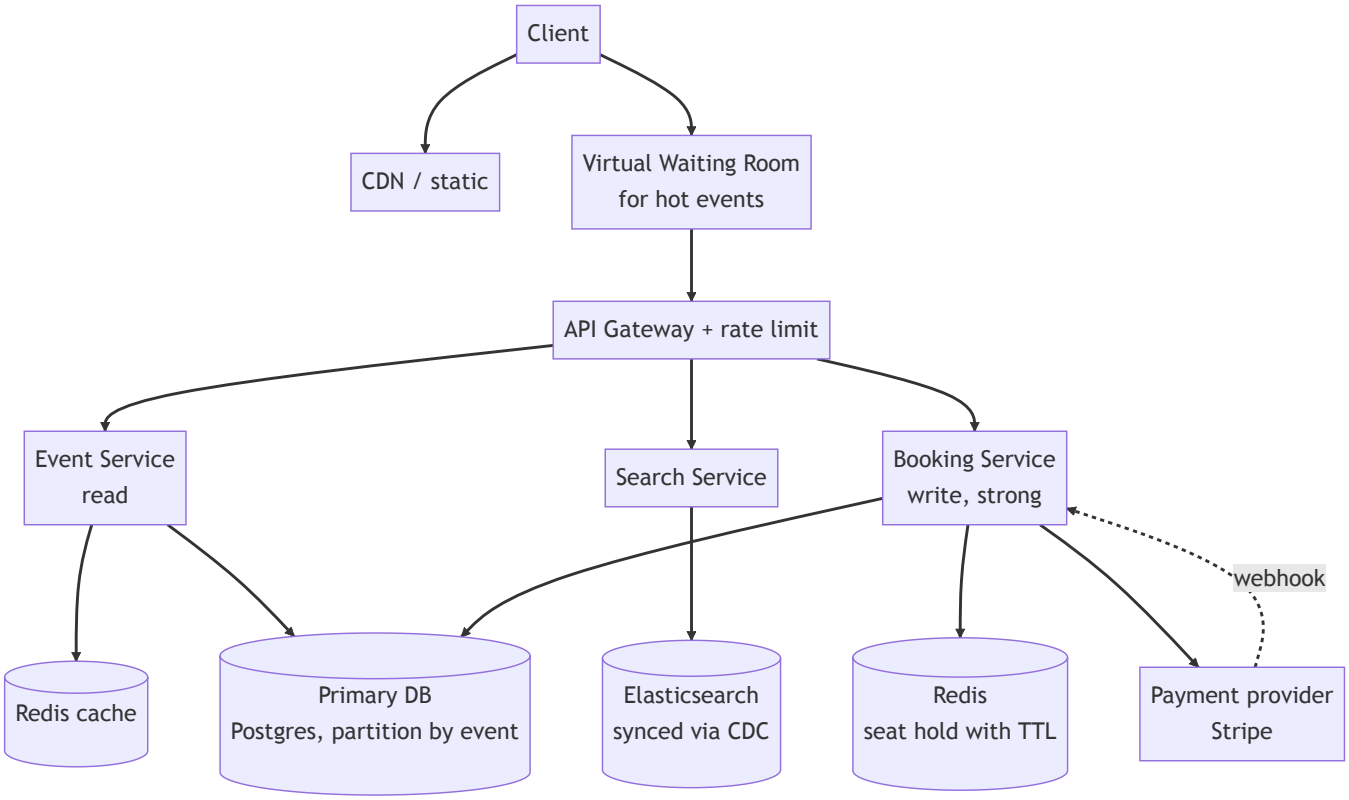
6. Architecture

v1



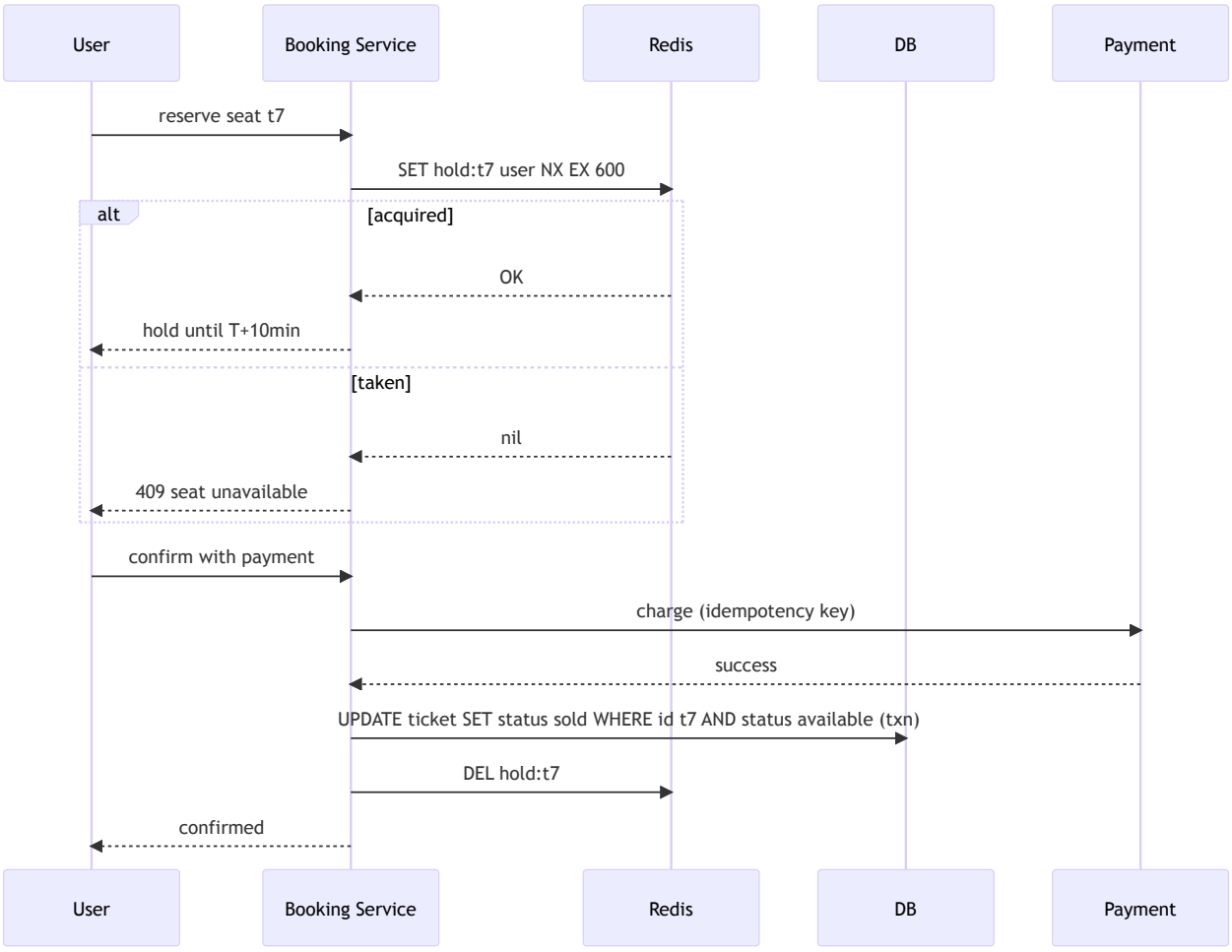
Problem: reads and bookings compete; hot event overloads everything.

Final design



---

7. Booking flow



---

8. Deep dive 1: preventing double-booking and holds

Approach	Verdict	Why
Status column + cron to expire holds	Bad	Expiry is delayed; stale holds block seats
DB row lock (`SELECT FOR UPDATE`) / optimistic version CAS	Good	Correct; but holding txn open during payment hurts throughput
Distributed lock with TTL (Redis `SET NX EX`)	Great	Seat is held without long DB txn; auto-expires on abandon

Pattern: Redis TTL holds guard the **checkout window**, and the **DB conditional update** (`WHERE status='available'`) is the final source-of-truth write. If Redis loses a hold, the DB check still prevents double-sell (**defense in depth**). If a hold expires mid-payment, return 410 and refund or void.

One-liner: *TTL hold in Redis for the UX window, atomic conditional update in the DB for truth.*

---

9. Deep dive 2: surge and fairness (virtual waiting room)

- Admit users at a rate the booking path can sustain (token bucket); others wait in a queue with an ordering token (signed, position-based).
- Waiting room state in Redis/edge; **long-poll or SSE** for position updates; admission via signed cookie.

- Bots: auth, CAPTCHA, per-account limits.
- Protect booking DB with **admission control**, not just autoscaling.

One-liner: *Meter admissions to what the booking path can take.*

---

10. Deep dive 3: scaling reads

- **Event pages** cached heavily at CDN/Redis; seat availability is **eventually consistent** (a seat map may be stale by seconds). A 409 at reserve time corrects it.
- Push live seat updates via **SSE/WebSocket** for the active event.
- **Read replicas**; partition by `event_id` (hot events isolated to their own shard).

---

11. Deep dive 4: search

Postgres `LIKE` is slow; add **Elasticsearch/OpenSearch** fed by **CDC** (Debezium) from the primary (eventually consistent; fine). Full-text, fuzzy, geo filters. Cache popular queries. Index only searchable fields.

---

12. Failure handling

Failure	Mitigation
Redis down	Fall back to DB conditional update with short app-level hold rows; degrade to lower throughput
Payment succeeded but confirm failed	Idempotency key + webhook reconciliation; refund job
Booking service crash mid-flow	Hold expires by TTL; saga compensation
DB failover	Synchronous replica for booking DB; accept brief unavailability over inconsistency
ES lag	Stale search only

---

13. Trade-offs

Decision	Chosen	Alternative	Given up
Hold mechanism	Redis TTL + DB CAS	DB lock only	Added component for scale
Seat view	Eventually consistent	Strongly consistent	Occasional 409
Surge	Waiting room	Autoscale only	Immediate access
Search	Elasticsearch	SQL	Dual-write/CDC complexity
Booking txn	Single-DB per event	Distributed txn	Cross-event atomicity (not needed)

---

14. Level expectations

- **Mid:** reserve/confirm, DB uniqueness, caching browse.
- **Senior:** TTL holds, locking comparison, search index, read scaling, payment idempotency.
- **Staff+:** waiting room design, bot defense, failure reconciliation, hot-event partitioning, SLOs.

15. Follow-ups

Dynamic pricing, resale/transfer (ownership ledger), general admission, QR ticket anti-fraud (rotating barcodes), multi-region.

16. Summary checklist

- [ ] Reserve then confirm, 10 min TTL hold
- [ ] Redis NX EX plus DB conditional update
- [ ] Virtual waiting room and admission control
- [ ] CDN-cached reads, ES via CDC
- [ ] Idempotent payments and reconciliation

Designing a News Feed (Facebook-style)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**fan-out on write vs read, hybrid for celebrities, hot-key caching, pagination**) → trade-offs. The hard part is **producing a fast feed for users who follow many accounts, some with millions of followers**.

> Sibling doc: *Instagram feed* in this folder covers photo specifics. This one is the generic text/link feed (Facebook / X) per Hello Interview.

---

0. Talk track

1. Clarify: create posts, follow, view feed (chronological/ranked), scale 2B users.
2. Core insight: **read-heavy, eventual consistency OK**, feed latency under 500 ms.
3. Entities and API (cursor pagination).
4. v1: pull model with joins.
5. Deep dives: fan-out on write, hybrid for celebrities, feed cache, hot posts, media.
6. Failures and trade-offs.

---

1. Problem

Users follow people/pages and see a **reverse-chronological (or ranked) feed** of their posts.

What makes it hard

- A user may follow **hundreds to thousands** of accounts; a feed read can't scan them all.
- **Celebrities** with 10M+ followers make fan-out on write explosive.
- Must be fast (under 500 ms) and **highly available**.

---

2. Requirements

Functional

- Create posts (text, media links).
- Follow and unfollow users.
- View a **feed** of followed users' posts (newest first, infinite scroll).

Out of scope: likes/comments, ranking ML, ads, privacy settings, notifications.

Non-functional

- **Availability over consistency** (eventual: a post may appear after up to about 1 minute).
- Feed read under 500 ms p99.
- Scale: **2B users**, follow up to 5K accounts, celebrities up to 100M followers.

---

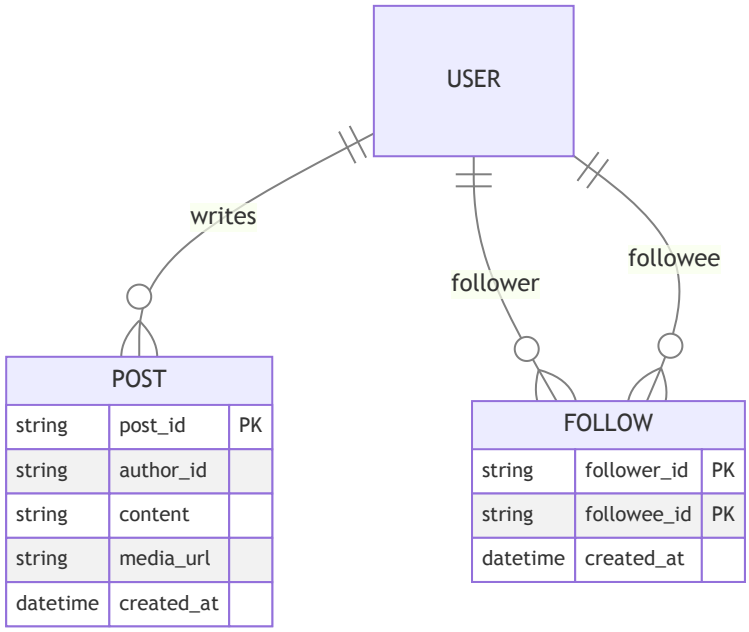
3. Estimation

Quantity	Assumption	Result
DAU	500M	
Feed reads	10 per user/day	5B/day = about 58K/s avg, about 200K/s peak
Posts	100M/day	about 1.2K/s
Read:write		**about 50:1**
Avg followers	200	fan-out 1.2K x 200 = **about 240K feed inserts/s**
Post size	about 1 KB metadata	about 100 GB/day
Feed cache	500M users x 500 post ids x 8 B	**about 2 TB** (Redis cluster)

Consequence: precompute feeds (**write fan-out**) for normal users, handle celebrities differently.

---

4. Entities



---

5. API

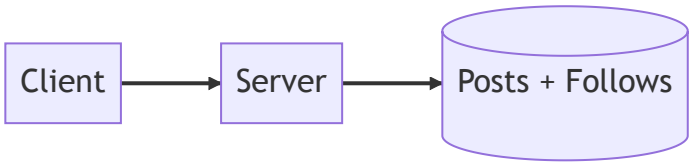
```
POST /posts      { "content":"...", "media":["..."] }          -> 201 { post_id }
POST /follows    { "followee_id":"u2" }                          -> 204
DELETE /follows/{followee_id}
GET  /feed?cursor=<post_id_or_ts>&limit=20                      -> { posts:[...], next_cursor }
```

Cursor (keyset) pagination instead of offset: stable under new inserts and O(log n).

---

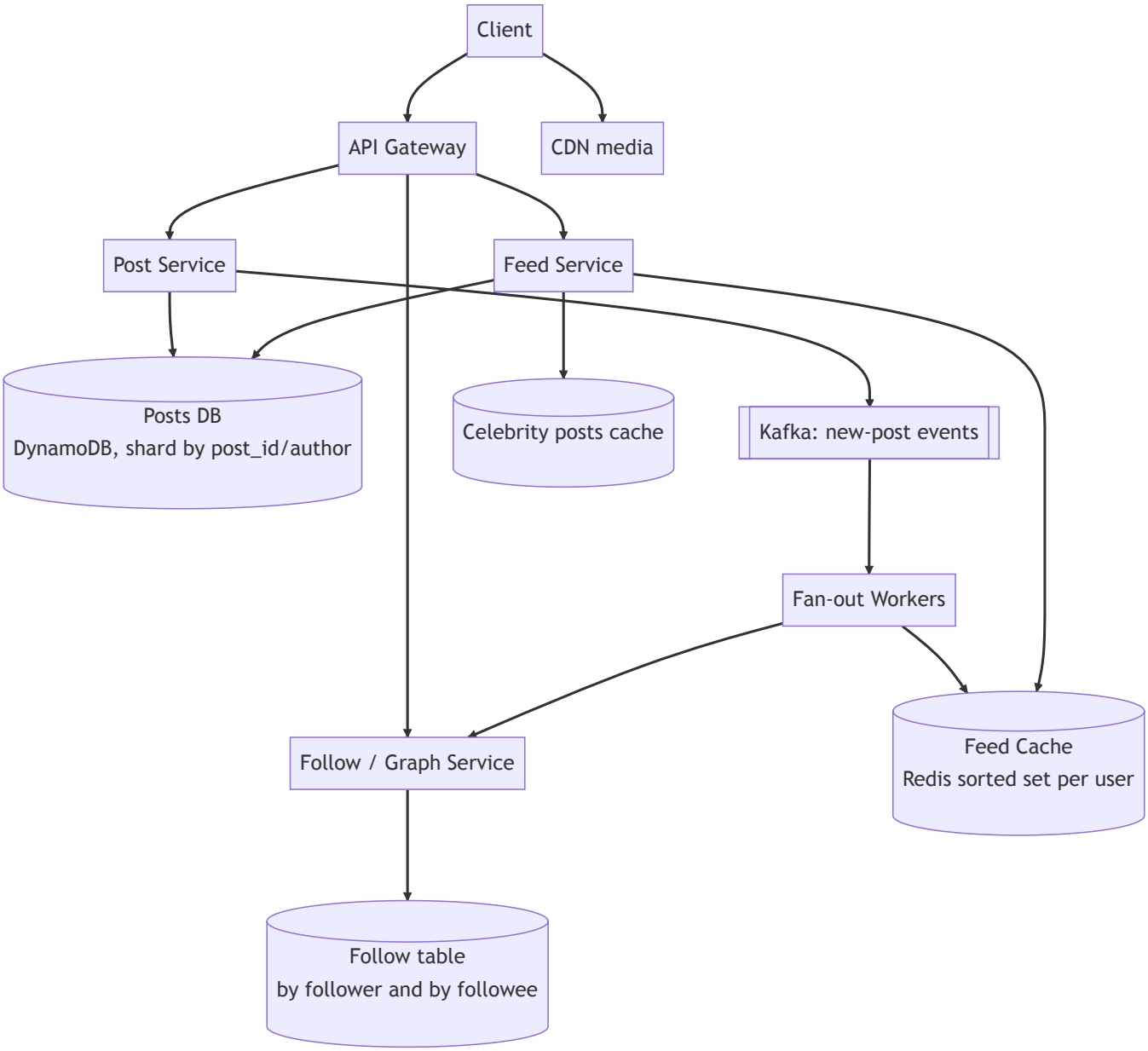
6. Architecture

v1 (pull)



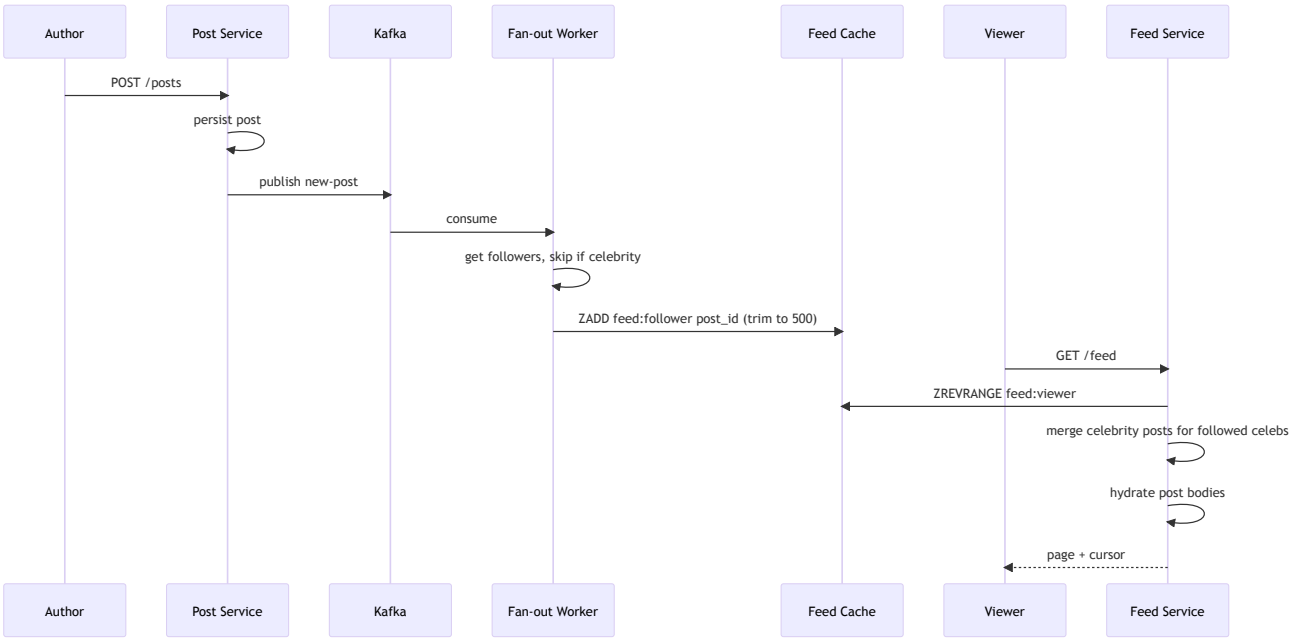
Feed query: `SELECT p FROM posts JOIN follows ... ORDER BY ts LIMIT 20`. Scans 1K+ users' posts: too slow at scale.

Final design (hybrid)



---

7. Flow



---

8. Deep dive 1: fan-out strategy

Approach	Verdict	Why
Fan-out on read (query followees' posts at read time)	Fine for small, bad at scale	Reads are 50x more frequent; N-way merge per request
Fan-out on write (push post id into each follower's feed list)	Great for normal users	O(1) read; write cost = followers
Hybrid	Best	Push for normal authors; **pull for celebrities** and merge at read

Celebrity threshold (e.g. > 100K followers): skip fan-out. At read time the feed service fetches recent posts for the followed celebrities (typically few) from a hot cache and merges by time. Inactive users: skip precompute, build on first read (saves memory).

One-liner: *Push for the many, pull for the famous, merge at read.*

---

9. Deep dive 2: feed storage and pagination

- Feed = **Redis sorted set** per user of `(timestamp, post_id)`, trimmed to about 500-1000. Older pages fall back to pull from the DB.
- Store only **ids**; hydrate post bodies from a post cache (read-through) to avoid duplicating content.
- Post ids are **time-sortable** (Snowflake / ULID), so the cursor is the last id.

---

10. Deep dive 3: hot posts and caching

- A viral post is read by millions: **post cache** (Redis) plus CDN for media. Replicate hot keys across cache nodes or use local in-process caches with short TTL.
- Counters (likes) not in scope; if added, use sharded counters to avoid hot rows.

11. Storage and scaling

Data	Store	Key
Posts	DynamoDB/Cassandra	`post_id` (index by `author_id, ts`)
Follows	Two tables	`(follower_id)` for feed build, `(followee_id)` for fan-out
Feed lists	Redis cluster	`feed:{user_id}`
Media	S3 + CDN	

Shard Redis by user_id; feed service stateless.

---

12. Failure handling

Failure	Mitigation
Fan-out lag	Queue lag metric; feed still serves older entries; eventual
Redis node loss	Rebuild feed from posts + follows on miss (pull fallback); replicas
Kafka backlog on celeb post	Celebrities bypass fan-out; priority lanes
Fan-out worker crash	At-least-once with idempotent ZADD (same member = no dup)
Thundering herd after cache loss	Request coalescing, jittered TTL, rate-limited rebuild

---

13. Trade-offs

Decision	Chosen	Alternative	Given up
Fan-out	Hybrid	Pure push / pull	Read-path complexity
Feed contents	IDs only	Full posts	Extra hydration hop
Consistency	Eventual	Strong	Few seconds of delay
Pagination	Cursor	Offset	Random jump
Cache sizing	Active users only	All	Cold-start cost on first read

---

14. Level expectations

- **Mid:** pull vs push, basic tables, pagination.
- **Senior:** hybrid fan-out, Redis feed lists, cursor pagination, hot-key mitigation.
- **Staff+:** celebrity threshold tuning, backfill/rebuild, ranking pipeline hooks, capacity math, failure drills.

15. Follow-ups

Ranked feed (candidate gen + scorer), deletion/edit propagation, blocked users, ads injection, real-time updates via SSE.

16. Summary checklist

- [ ] Hybrid fan-out, celebrity pull
- [ ] Redis sorted-set feed of ids, trimmed
- [ ] Cursor pagination, time-sortable ids
- [ ] Post cache and CDN for hot content
- [ ] Idempotent workers, lag monitoring

Designing Uber (Ride Sharing)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**location ingestion at scale, geo-indexing, driver matching without double-assign, consistency of ride state, surge**) → trade-offs. The hard parts are **write-heavy location updates** and **matching one driver to one rider exactly once**.

---

0. Talk track

1. Clarify: fare estimate, request ride, match driver, driver accept/decline, track. Out: payments, ratings.
2. Non-functional: strong consistency for **matching**, low latency, high throughput.
3. Entities and API.
4. HLD: ride service, location service, matching, notifications.
5. Deep dives: location updates (10M drivers), nearby search (geo index), matching with locks, reliability (durable workflow).

---

1. Problem

Riders request rides; the system matches the **best nearby available driver**, tracks the trip, and handles retries when a driver declines.

What makes it hard

- Millions of drivers send location every few seconds (**write-heavy**).
- "Nearby" search must be real-time over moving points.
- **Invariant:** a driver gets **one** ride at a time, and a ride gets one driver.
- Offers expire; drivers may ignore, so matching is a **long-running workflow**.

---

2. Requirements

Functional

- Rider enters pickup/destination and gets a **fare estimate**.
- Rider requests a ride; system **matches a driver**.
- Driver accepts or declines; navigates to pickup and drop-off.

Out of scope: ratings, scheduled rides, ride categories, payments.

Non-functional

- **Low latency matching** (under 1 min; ideally seconds).
- **Strong consistency** for assignment: no driver double-booked.
- **High throughput** at peak (events, rush hour): 100K requests.
- Resilient to node failures, retries.

---

3. Estimation

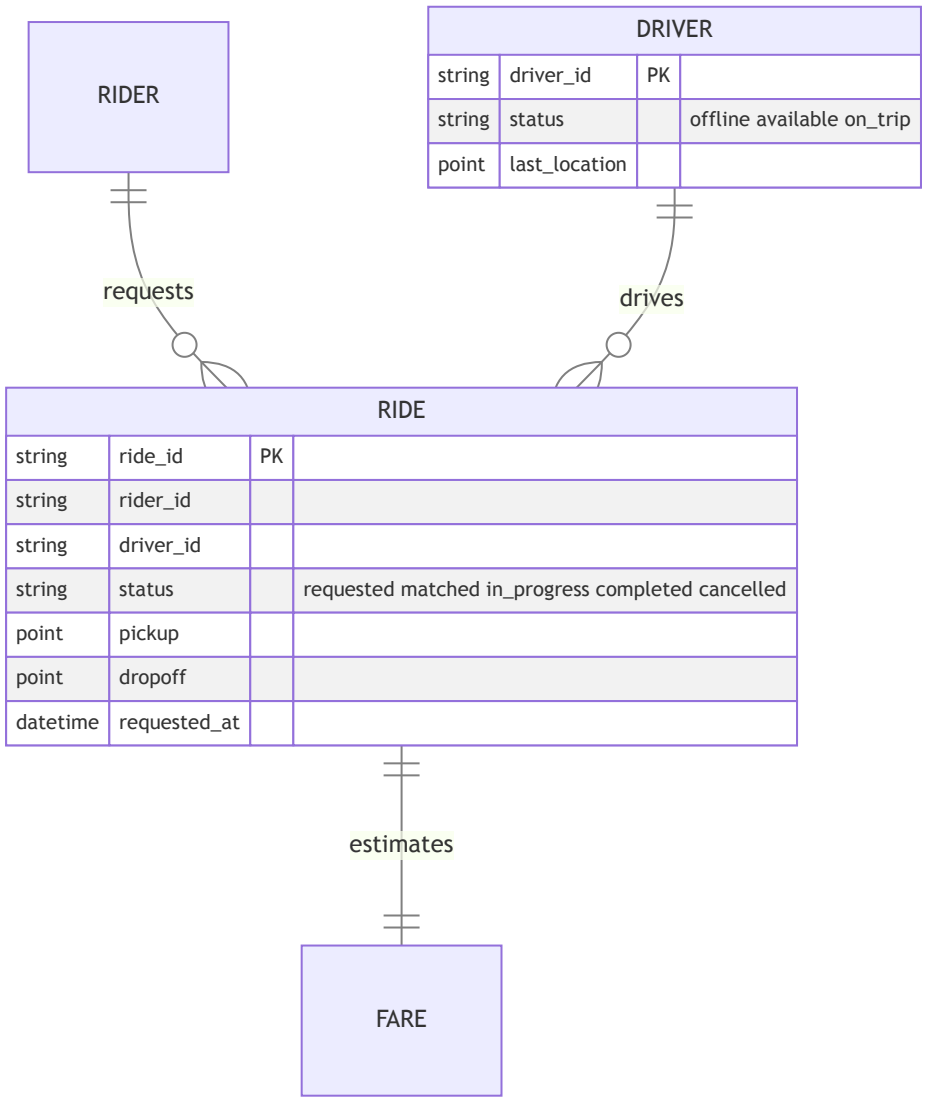
Quantity	Assumption	Result
Drivers online	10M, location every 5 s	**2M writes/s**
Ride requests	20M/day, peak 10x	about 230/s avg, about 2.5K/s peak
Location payload	100 B	200 MB/s ingress
Geo index size	10M x 100 B	**1 GB**: fits in memory

Consequences

- 2M writes/s needs **in-memory geo store**, not a SQL row update per ping; shard by region.
- Ride requests are modest; the **location firehose** dominates.
- Reduce write rate: adaptive frequency (driver idle vs en-route), batching.

---

4. Entities



---

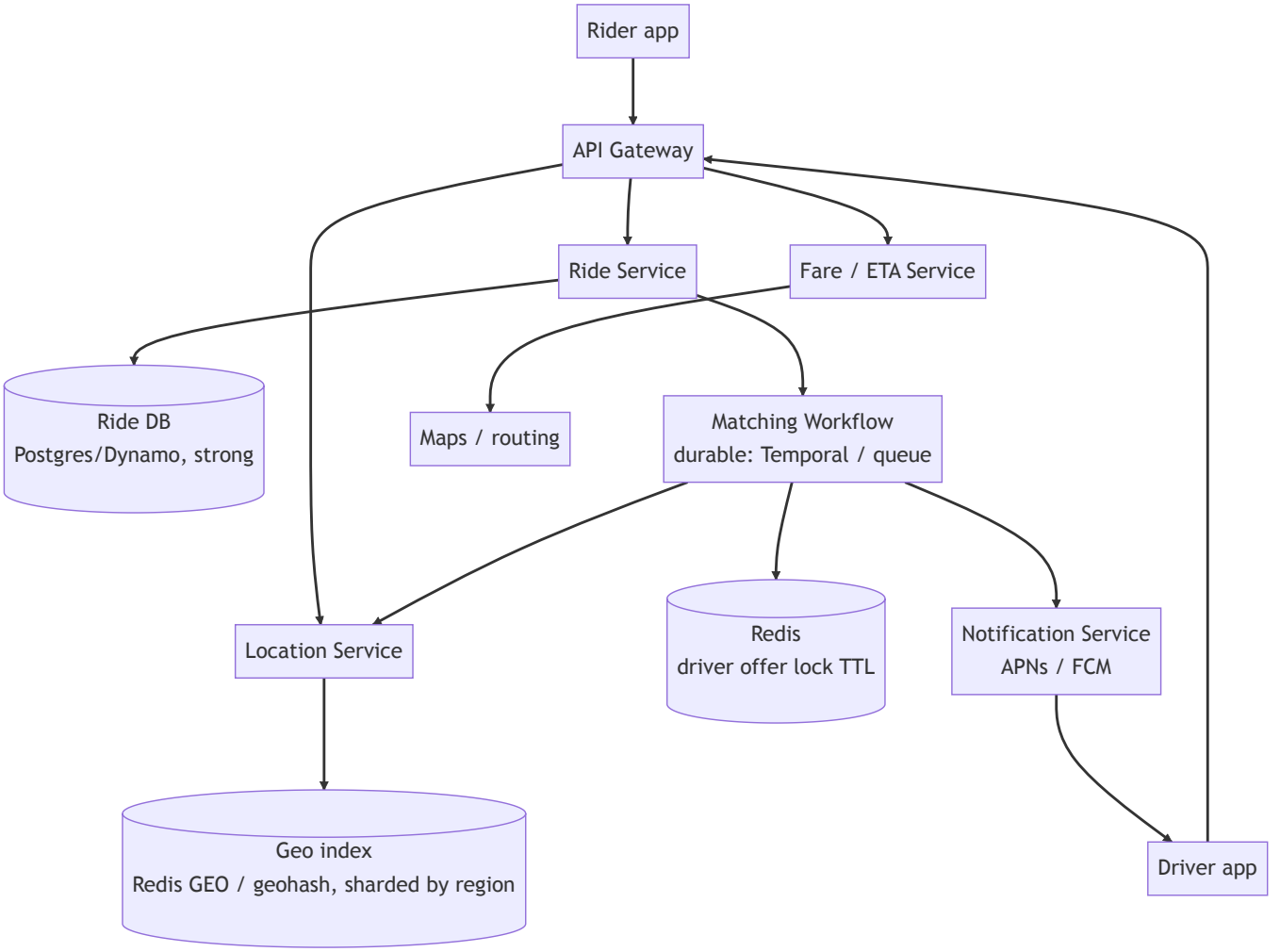
5. API

```
POST /fare            { pickup, destination }                -> { fare_id, estimate, eta }
POST /rides           { fare_id }                            -> 202 { ride_id, status: "requested" }
PATCH /rides/{id}     { "driver_response":"accept|decline" }  # driver side
POST /drivers/location { lat, lng }                          # every 5 s; driver id from JWT
GET  /rides/{id}                                             # status; or push via WebSocket/push no
```

IDs come from auth tokens, never from the request body.

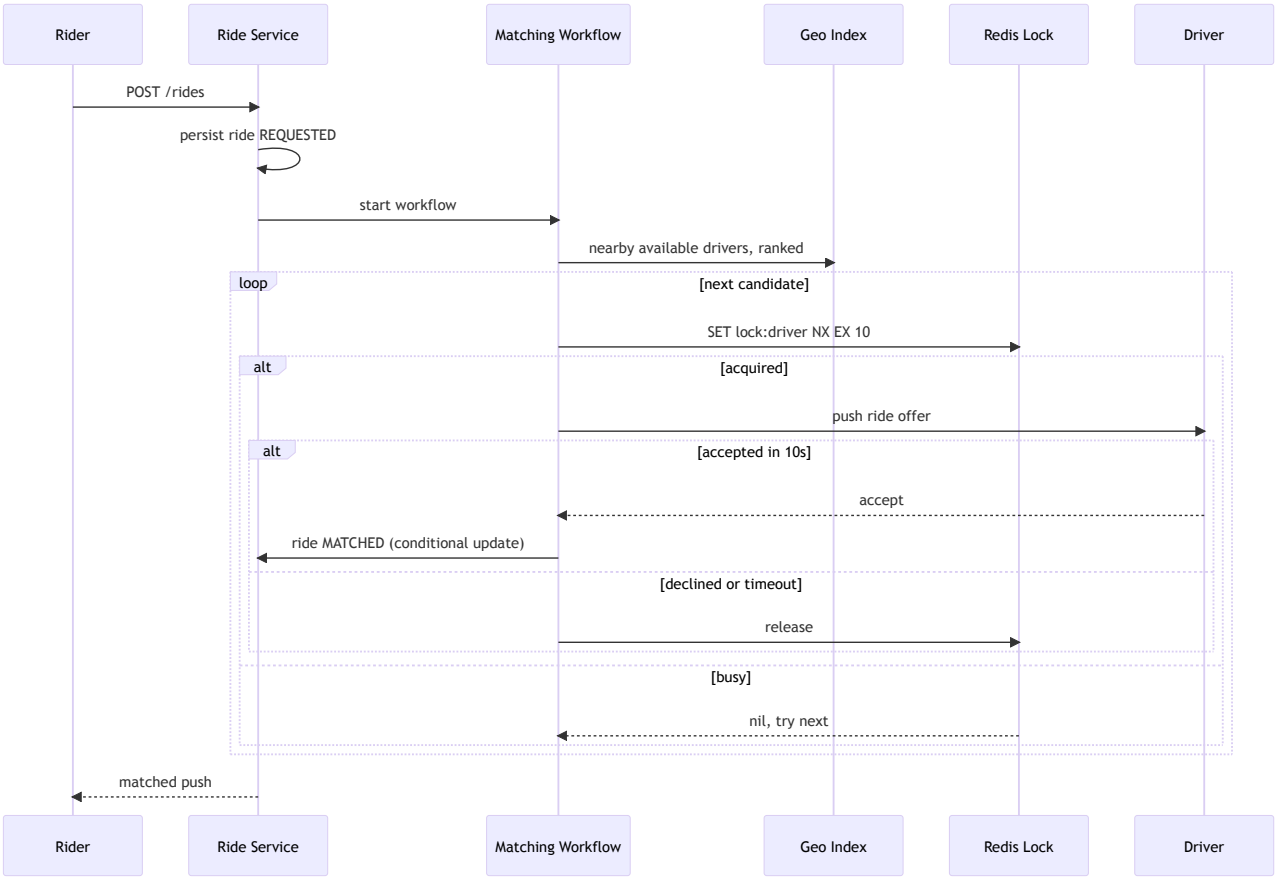
---

6. Architecture



---

7. Matching flow



---

8. Deep dive 1: location ingestion

Approach	Verdict	Why
Write every ping to SQL	Bad	2M writes/s kills the DB
Batch + in-memory geo store (Redis `GEOADD`, geohash/H3 cell index)	Great	Last-known location only: overwrite; TTL for dead drivers
Adaptive ping rate	Good add-on	Fewer writes when idle/stationary

Drivers are **sharded by region/cell** so one node handles a city. A durable history stream (Kafka) is optional for analytics, off the hot path. Eventual consistency of location is fine (a few seconds).

One-liner: *Latest-location-wins in memory, shard by geography.*

---

9. Deep dive 2: finding nearby drivers

Approach	Verdict	Why
Scan all drivers	Bad	O(N)
Geohash/H3 grid cells: look up the rider's cell plus neighbors	Great	O(drivers in neighborhood); expand ring if sparse
Quadtree	Good	Adapts to density; more complex to update at high write rates

Rank by **ETA** (routing service), not straight-line distance (rivers, one-way). Take the top K, then offer sequentially.

---

10. Deep dive 3: no double-assignment

- **Offer lock** per driver (Redis `SET NX EX`) while an offer is outstanding; driver status flips to `offered`.
- Final commit with **conditional update** on the DB: `UPDATE ride SET driver_id=? WHERE ride_id=? AND status='requested'` and driver `available -> on_trip`. Loser retries the next candidate.
- Strong consistency on the **ride and driver state**, eventual on location.

One-liner: *TTL lock for the offer window, conditional write for the commit.*

---

11. Deep dive 4: durable matching workflow

Matching spans timers (10 s per offer), retries, and failures. Use a **workflow engine** (Temporal, or queue + delayed messages) that persists state so a crash resumes mid-match. Alternative: queue with per-ride delayed retries. Handle: no drivers (expand radius, notify), rider cancel (signal workflow), driver goes offline (release).

---

12. Surge and hot zones

At peaks, cell load spikes: shard by finer cells, queue ride requests per region, **admission control**, and surge pricing to rebalance supply and demand.

---

13. Failure handling

Failure	Mitigation
Geo store node loss	Replicas; drivers repopulate within 5 s (self-healing via pings)
Matching worker crash	Workflow resumes; offers idempotent by `(ride_id, driver_id)`
Lost push notification	Driver polls/WebSocket fallback; offer TTL
Ride DB failover	Sync replication for ride table; conditional updates avoid double-match
Region outage	Regional isolation: other cities unaffected

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Location store	In-memory geo	SQL	Durability of history (stream it separately)
Match	Sequential offers	Broadcast to many	Speed vs driver fairness/double-accept races
Consistency	Strong on rides, eventual on location	All strong	Cost
Workflow	Temporal	Queue+cron	Extra platform dependency
Ranking	ETA	Distance	Routing-call cost

---

15. Level expectations

- **Mid:** ride request/match flow, geo-search idea, DB for rides.
- **Senior:** geohash/H3, location write math, locking for exclusive offers, notifications.
- **Staff+:** durable workflows, regional sharding, surge/admission control, ETA-based ranking, failure drills.

16. Follow-ups

Ride pooling, scheduled rides, driver incentives, fraud (GPS spoofing), multi-city/multi-region, post-trip payments.

17. Summary checklist

- [ ] 2M location writes/s, so in-memory geo, sharded by cell
- [ ] ETA-ranked candidates, sequential offers
- [ ] TTL offer lock plus conditional DB commit
- [ ] Durable matching workflow, push with fallback
- [ ] Regional isolation, surge control

Designing WhatsApp (1:1 and Group Messaging)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**persistent connections, delivery guarantees, offline inbox, groups, multi-device, scale to billions**) → trade-offs. Contrast with the *Slack* doc here: WhatsApp is **store-and-forward** with a **transient inbox**, not durable channel history.

---

0. Talk track

1. Clarify: 1:1 and group chat (up to 100), delivery receipts, **offline delivery**, media. Out: calls, stories.
2. Scale: 1B users, 200M+ concurrent connections.
3. HLD: WebSocket chat servers, routing, inbox.
4. Deep dives: routing across chat servers, delivery semantics, groups, multi-device, media.

---

1. Problem

Send messages in real time between users, store them **until delivered**, and handle users who are offline or have multiple devices.

What makes it hard

- **Hundreds of millions of persistent connections.**
- Message must reach the recipient **exactly once** from the UI's view, even across reconnects.
- Offline recipients: queue, then deliver on return.
- Groups and multiple devices multiply deliveries.

---

2. Requirements

Functional

- Group chats (up to 100 participants).
- Send/receive messages (1:1 and group).
- **Receive messages sent while offline** (retained up to 30 days).
- Send media (images, video).

Out of scope: voice/video calls, E2EE protocol details, status, payments.

Non-functional

- Latency under 500 ms for online delivery.
- **Deliver at least once**, dedupe at client (message id).
- **Billions of users**, high throughput.
- Minimize server-side retention (messages deleted after delivery, privacy).
- Fault tolerant.

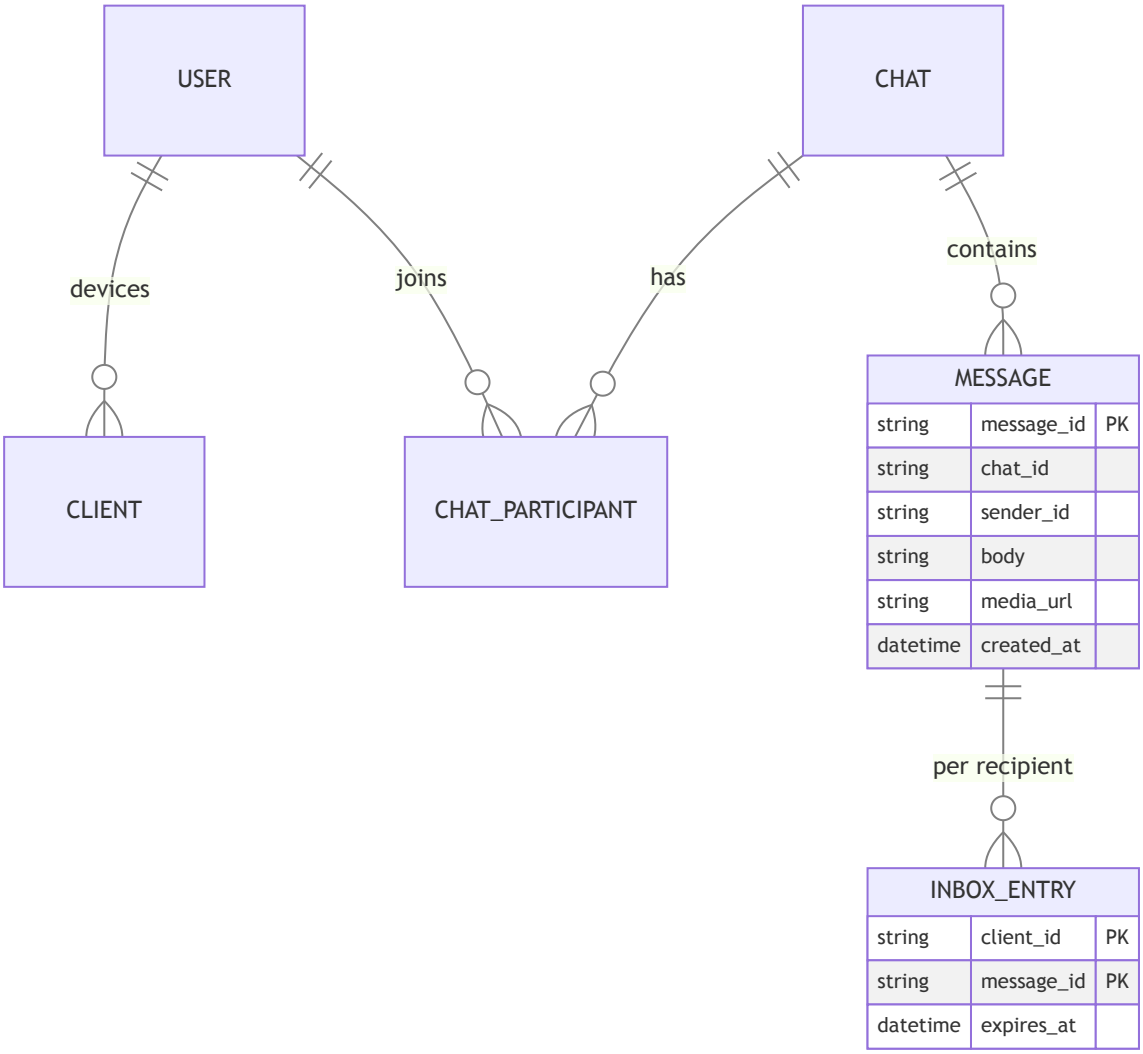
---

3. Estimation

Quantity	Assumption	Result
Users	1B, 500M DAU, 200M concurrent	**200M WebSockets**
Messages/day	100B	**about 1.2M msgs/s avg, 3M peak**
Conns per server	about 1-2M (tuned; or 100K conservative)	**100-2000 chat servers**
Inbox storage	undelivered only, 30-day cap	a small fraction of total
Media	10% of messages, 200 KB avg	about 60 GB/s at peak: **object store + CDN**, never through chat servers

---

4. Entities



---

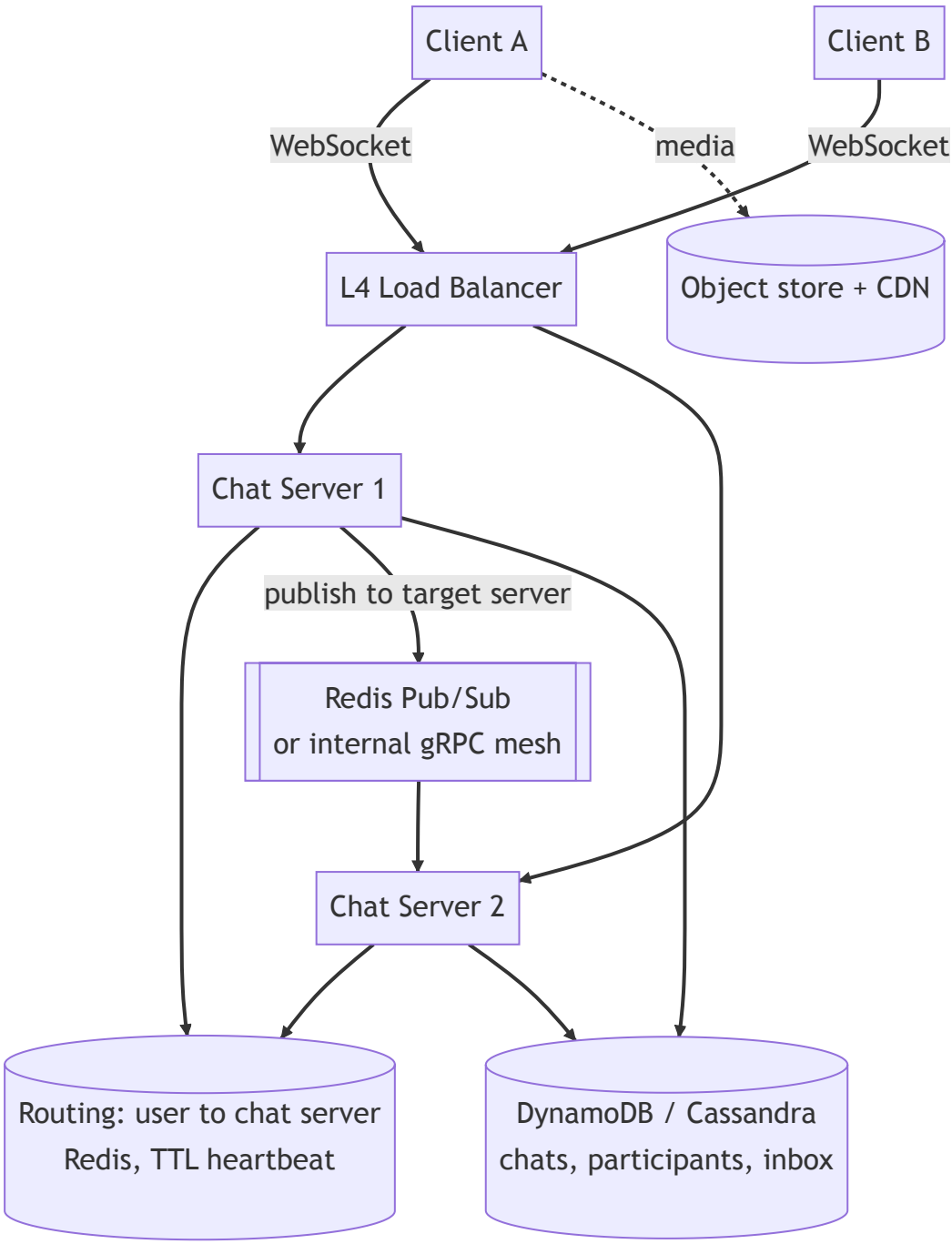
5. API (WebSocket messages)

```
// send
{ "type":"sendMessage", "chatId":"c1", "msgId":"uuid", "body":"hi", "attachments":["m1"] }
// server -> sender
{ "type":"ack", "msgId":"uuid", "status":"stored" }
// server -> recipient
{ "type":"newMessage", "chatId":"c1", "msgId":"uuid", "sender":"u1", "body":"hi" }
// recipient -> server
{ "type":"deliveryAck", "msgId":"uuid" }          // then server deletes from inbox
{ "type":"createChat", "participants":["u2","u3"] }
{ "type":"modifyParticipants", "chatId":"c1", "add":["u4"] }
```

Media: upload via presigned URL to blob store, send the attachment id in the message.

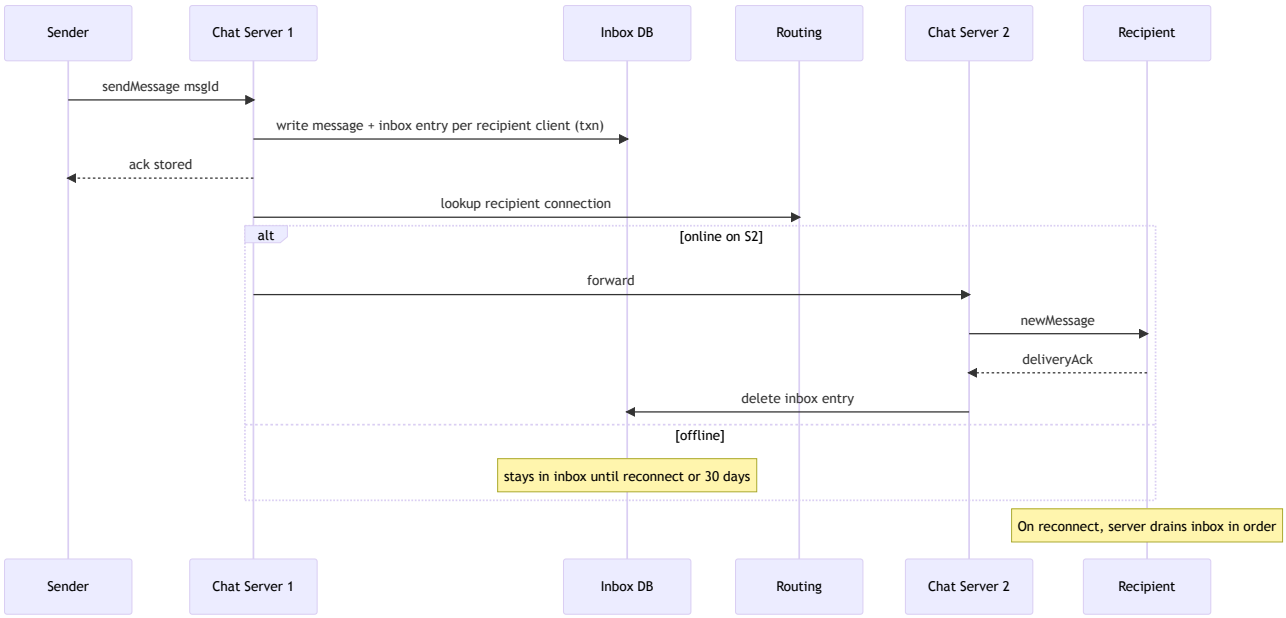
---

6. Architecture



---

7. Delivery flow



---

8. Deep dive 1: routing between chat servers

Approach	Verdict	Why
Consistent-hash users to servers	Good	No lookup table, but rebalancing disconnects users, and servers aren't the natural unit of connection
Routing table (`user -> server`, in Redis, TTL refreshed by heartbeat) plus **pub/sub per server**	Great	Any server can look up the target; stale on crash, so TTL
Redis Pub/Sub topic per user	OK	Simple; connections to pub/sub scale poorly at 200M

Server-to-server direct (gRPC) after table lookup keeps the pub/sub shallow. If the target is missing, the message stays in the inbox (store-and-forward makes routing failures harmless).

---

9. Deep dive 2: delivery guarantees

- **At-least-once**: persist **before** ack; resend until `deliveryAck`.
- **Client dedupes** on `msgId`, so UI is exactly-once.
- **Ordering** within a chat: per-chat sequence numbers (server-assigned) or sender timestamps plus client ordering; strict global order not required.
- Inbox entries have TTL (30 days). After the ack, **delete** (minimal retention).

One-liner: *Persist, ack, push, delete on client ack. Duplicates are fine, losses are not.*

---

10. Deep dive 3: groups and multi-device

- **Groups (<=100)**: sender's server writes one inbox entry **per recipient client** (fan-out on write is fine at this size). `ChatParticipant` table keyed by chat to list members; a GSI by user to list chats.
- **Multi-device**: each device has a `clientId`; messages deliver to **every** device inbox of each participant. Cap devices (e.g. 3-4) to bound fan-out.
- Large channels would instead need a pull/seq model (see Slack doc).

11. Deep dive 4: scaling connections

- Tuned L4 LBs (long-lived TCP), `epoll`-based servers (Erlang/Netty), 1M+ connections per host.
- **Graceful draining** on deploy, with jittered reconnects to avoid thundering herd.
- Heartbeats refresh routing TTL; missed heartbeat marks the user offline.
- Shard inbox DB by `clientId`.

---

12. Media

Presigned upload to object store, **encrypt client-side**, send only the URL/key and size/hash. Thumbnails generated async. CDN for downloads. Retention: delete after the retention window.

---

13. Failure handling

Failure	Mitigation
Chat server crash	Clients reconnect (backoff + jitter); inbox holds undelivered
Routing entry stale	Forward fails, message stays in inbox; client pulls on reconnect
Duplicate delivery	`msgId` dedupe
DB shard down	Replicated (quorum writes); failover; sender sees retry
Mass reconnect storm	Connection rate limiting, staged drains

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Storage	Transient inbox	Permanent history	Server-side search/backfill (client holds history)
Routing	Table + direct forward	Pure pub/sub	Table upkeep
Group fan-out	Write per client	Pull	Storage for 100-member groups, fine at that size
Delivery	At-least-once + dedupe	Exactly-once	Client logic
DB	Dynamo/Cassandra	SQL	Joins; fits key-value access

---

15. Level expectations

- **Mid:** WebSocket, store and forward, DB.
- **Senior:** routing across servers, ack semantics, offline inbox, groups, multi-device.
- **Staff+:** connection scaling and drain, ordering, E2EE implications, cost, reconnect storms.

16. Follow-ups

E2EE (Signal protocol, key distribution), presence/typing, read receipts, message edit/delete, status, large communities.

17. Summary checklist

- [ ] WebSocket per device, L4 LB
- [ ] Routing table in Redis with heartbeat TTL
- [ ] Inbox per client, delete on ack, 30-day TTL
- [ ] Persist before ack, client dedupe
- [ ] Group fan-out on write, device cap

Designing YouTube (Video Upload and Streaming)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**resumable large uploads, transcoding pipeline, adaptive bitrate streaming, CDN, metadata scale**) → trade-offs. The hard parts are the **upload/transcode pipeline** and **global video delivery**.

---

0. Talk track

1. Clarify: upload, watch (stream), search out of scope. 1M-ish uploads/day, 100M+ views.
2. Insight: video is **big and immutable**; metadata is small; delivery is bandwidth-dominated.
3. HLD: upload to blob store, async transcode DAG, store segments, serve via CDN + manifest.
4. Deep dives: chunked upload, parallel transcoding, ABR (HLS/DASH), CDN and hot content, failures.

---

1. Problem

Users upload videos; others stream them with **minimal buffering** on any device and network.

What makes it hard

- Raw uploads are GBs; need resumable transfer.
- Each video must be **transcoded into many resolutions/codecs**.
- Delivery: PBs per day; must go through **CDN**, adapt to bandwidth.
- Highly available: viewing > consistency.

---

2. Requirements

Functional

- Upload videos.
- Watch (stream) videos.

Out of scope: search, recommendations, comments/likes, subscriptions, live streaming, DRM.

Non-functional

- **High availability** (eventual consistency OK).
- Support **large videos** (up to 256 GB), resumable.
- **Low-latency playback**, minimal buffering, adaptive to network.
- Scale: **1M videos uploaded/day, 100M DAU**.

---

3. Estimation

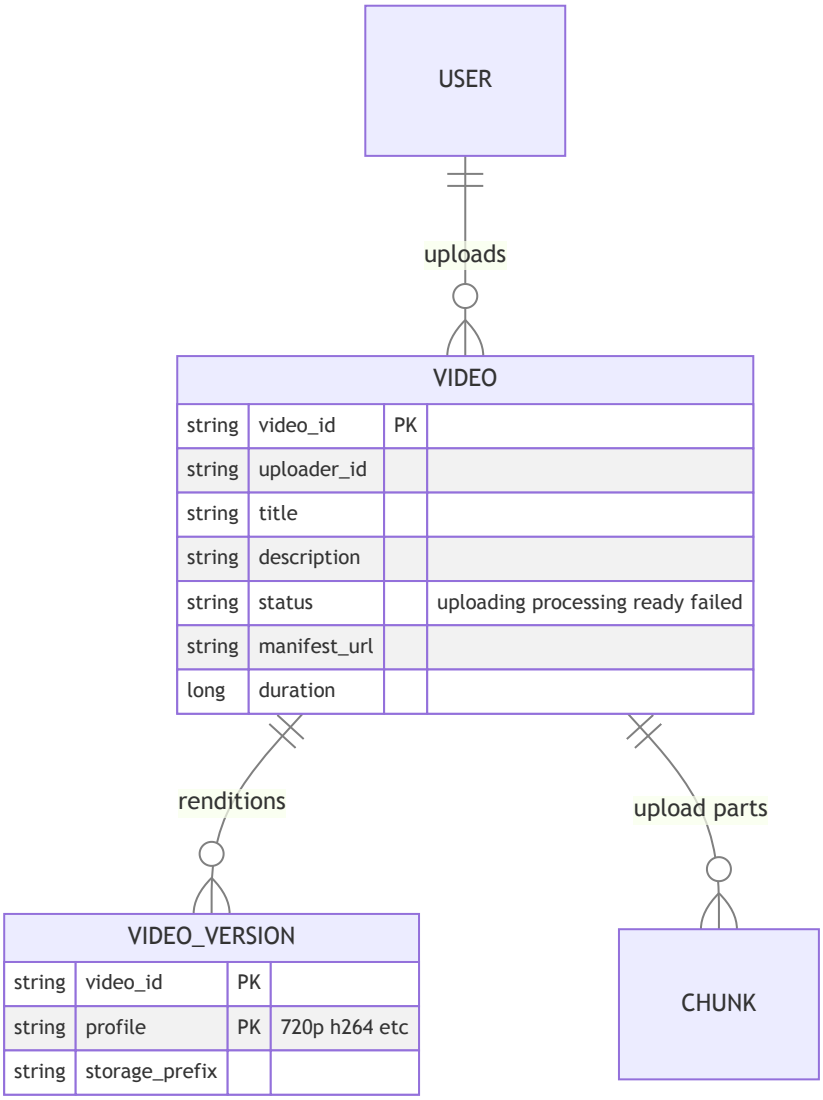
Quantity	Assumption	Result
Uploads	1M/day, 500 MB avg	**500 TB/day raw**
Transcoded copies	about 6 renditions totaling about 3x raw	**about 1.5 PB/day** stored
Views	100M DAU x 5 videos x 50 MB	**25 PB/day egress**
Peak egress		**about 1 Tbps+**, CDN essential
Metadata	1M/day x 1 KB	1 GB/day (tiny)

Consequences

- Blobs go to object storage; **never through app servers**.
- 25 PB/day egress is impossible from one origin, so a **CDN/ISP caches** are the delivery tier.
- Storage cost drives tiering (hot recent, cold old), and codec efficiency.

---

4. Entities



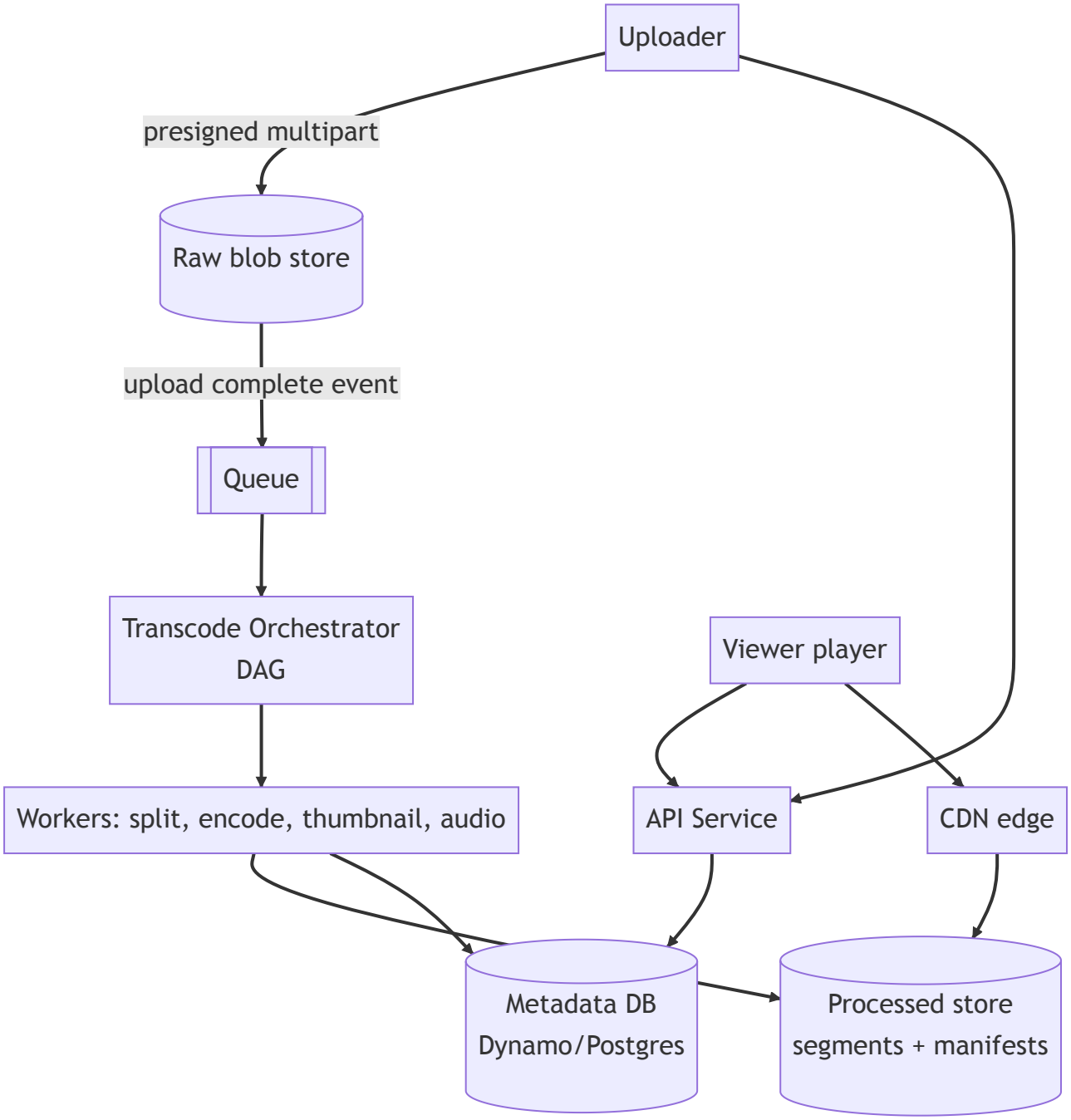
---

5. API

```
POST /videos            { title, description, size, content_type } -> { video_id, upload_id, presigned
PUT  <presigned url>    # client uploads chunk directly to S3 multipart
POST /videos/{id}/complete {parts:[etag...]}                        -> 202 processing
GET  /videos/{id}                                                   -> metadata + manifest_url (HLS/DAS
GET  <CDN>/{id}/master.m3u8  and segments  *.ts / *.m4s             # player streams
```

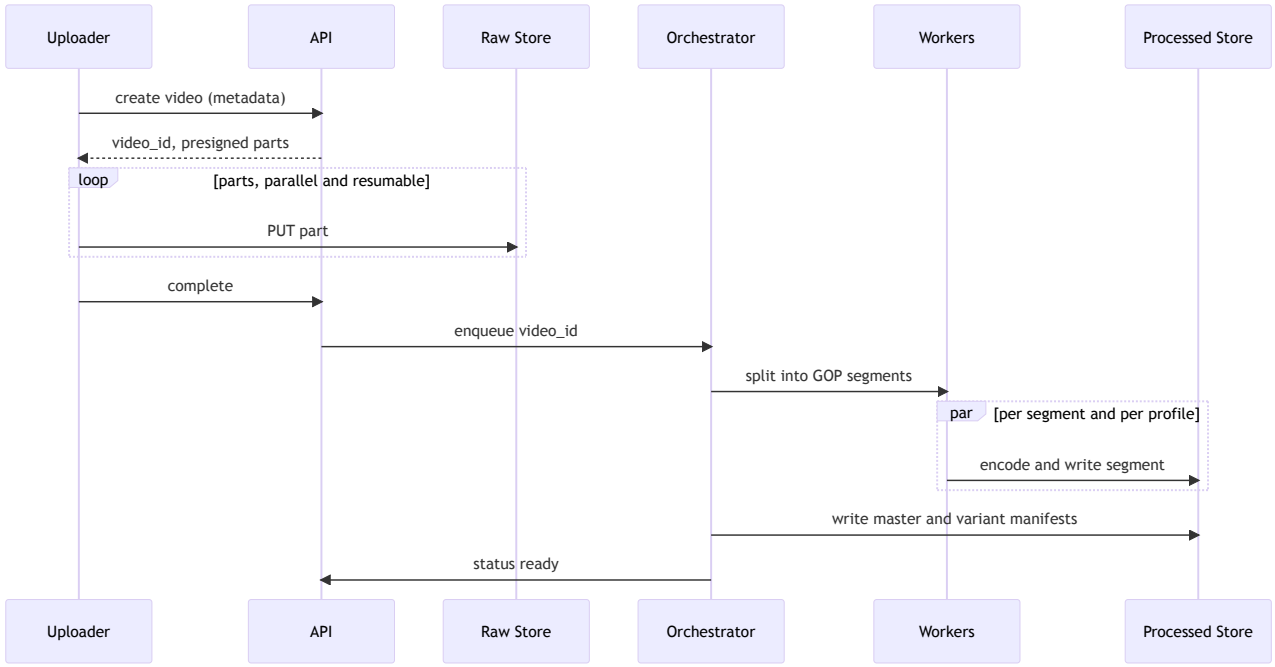
---

6. Architecture



---

7. Upload and process flow



---

8. Deep dive 1: large uploads

Presigned **multipart upload** straight to object storage: parallel parts, per-part retry, resumable via `ListParts`; server tracks `status` (see Dropbox doc). Cleanup incomplete uploads by lifecycle policy.

9. Deep dive 2: transcoding

- Split the raw file into **GOP-aligned segments** (e.g. 2-10 s), and run encode jobs **in parallel** per segment per profile (240p to 4K, H.264/VP9/AV1), plus audio, thumbnails, captions.
- Orchestrate as a **DAG** (Temporal / Step Functions / queue + workers); idempotent tasks keyed by `(video_id, segment, profile)`.
- Fault tolerance: retry failed segments only; poison inputs go to a dead-letter queue and mark the video `failed`.
- Cost: spot instances, hardware encoders; skip high-res for low-view videos; **encode lazily** for the long tail.

One-liner: *Segment, fan out encodes, stitch with a manifest.*

10. Deep dive 3: adaptive streaming

- **HLS/DASH**: each rendition cut into short segments; a **manifest** lists them. The player measures throughput and **switches bitrate per segment**.
- Segments are static immutable files, so **perfectly cacheable** on a CDN (long TTL, content-addressed paths).
- Start with a lower rendition for fast start; seek by jumping to the segment index.

11. Deep dive 4: CDN and hot content

Approach	Verdict
Serve from origin	Bad: cannot sustain egress
CDN edge caching with tiered origins (edge, regional, origin shield)	Great
Pre-position popular and new viral videos to edges / ISP caches (Open Connect style)	Great for the head

Long-tail videos are fetched on demand and may be evicted; viral spikes are protected by the origin shield (request coalescing).

12. Metadata scale

Small data; shard by `video_id` (Dynamo/Cassandra) or Postgres + replicas + cache. View counts are separate (sharded counters, async aggregation) and out of scope.

---

13. Failure handling

Failure	Mitigation
Upload interruption	Multipart resume
Encode worker dies	Task retry; idempotent output paths
Queue backlog	Autoscale workers, priority for new/popular
CDN POP down	Anycast/DNS reroute, multi-CDN
Origin down	Edge serves cached; replicas across regions
Corrupt file	Validate on probe (ffprobe), reject early

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Transcode	Parallel segments	Whole-file	Orchestration complexity
Storage	Many renditions	On-the-fly transcode	Storage cost vs CPU/latency
Delivery	CDN + ABR	Progressive download	Segment/manifest overhead
Encode all profiles	Lazy for tail	Eager	First-view latency
Consistency	Eventual	Strong	Video appears slightly later

---

15. Level expectations

- **Mid:** blob store, metadata, CDN idea.
- **Senior:** multipart upload, async DAG, HLS/DASH, segment parallelism.
- **Staff+:** egress math, ISP caching, codec/cost strategy, lazy encoding, failure and capacity planning.

16. Follow-ups

Live streaming (RTMP ingest, low-latency HLS), DRM, recommendations, view counting, content moderation, thumbnails/previews.

17. Summary checklist

- [ ] Presigned multipart direct upload
- [ ] Segmented parallel transcode DAG
- [ ] HLS/DASH manifests, ABR
- [ ] CDN with origin shield, pre-position hot
- [ ] Small sharded metadata DB

Designing a Web Crawler

> Interview-ready walkthrough: requirements → estimates → entities → HLD → deep dives (**frontier and politeness, fault-tolerant pipeline, dedup, DNS, scale to billions of pages, traps**) → trade-offs. The hard parts are **throughput within politeness constraints** and **idempotent, resumable stages**.

---

0. Talk track

1. Clarify: crawl the web from seed URLs, extract text for LLM training, respect robots.txt. 10B pages in about 5 days.
2. Estimate: bandwidth bound; need hundreds of crawlers.
3. HLD: frontier queue, fetch, parse, store, extract links.
4. Deep dives: pipeline fault tolerance, politeness/rate limiting, scaling, duplicate detection, traps.

---

1. Problem

Starting from seed URLs, repeatedly **fetch pages, extract text and outgoing links, and enqueue new URLs**, at web scale.

What makes it hard

- **Scale**: 10B pages.
- **Politeness**: do not overload any host; follow `robots.txt`.
- **Failures**: sites slow/down; crawlers die; pipeline must resume.
- **Duplicates and traps**: same content many URLs, infinite URL spaces.

---

2. Requirements

Functional

- Crawl from seed URLs.
- Extract and store text for downstream use.
- Discover and follow links.

Out of scope: JS-rendered pages (mention headless), media, indexing/search.

Non-functional

- **Fault tolerant**; resume after failure.
- **Polite** (robots.txt, per-domain rate).
- **Efficient**: finish 10B pages within about 5 days.
- **Scalable**; extensible to other content types.

---

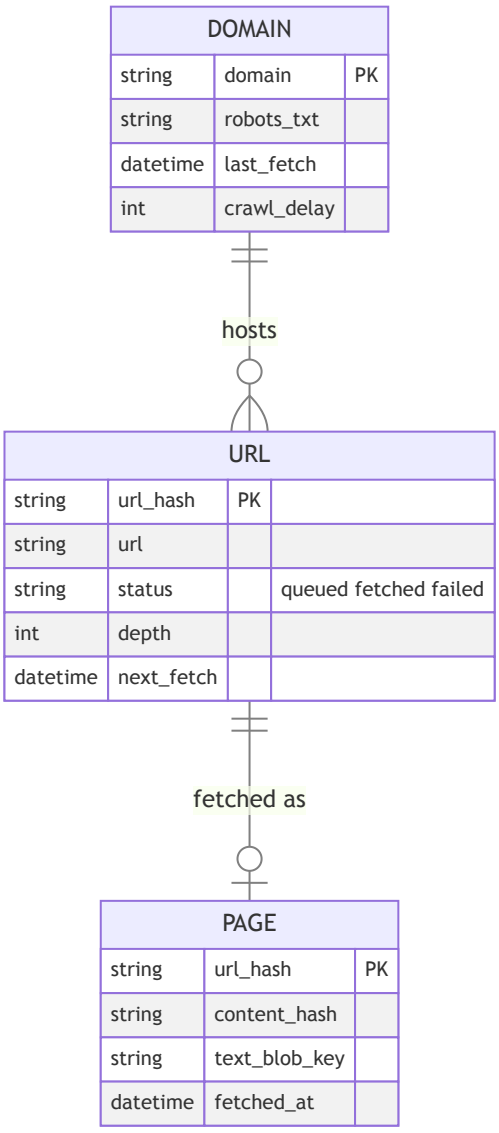
3. Estimation

Quantity	Assumption	Result
Pages	10B in 5 days	**about 23K pages/s**
Avg page	HTML only, about 100 KB	23K/s x 100 KB = **about 2.3 GB/s** (about 18 Gbps)
Bandwidth per host	10 Gbps NIC at 30% util, about 0.4 GB/s	**about 6 hosts** on bandwidth alone; DNS, TLS, parse CPU and politeness push this to **hundreds** of crawlers
Storage	10B x 100 KB text	**about 1 PB**, in object store
URL metadata	10B x 200 B	2 TB

Consequence: bandwidth is not the only limit. **Per-domain politeness (about 1 req/s/domain)** and DNS/connection overhead decide the crawler count.

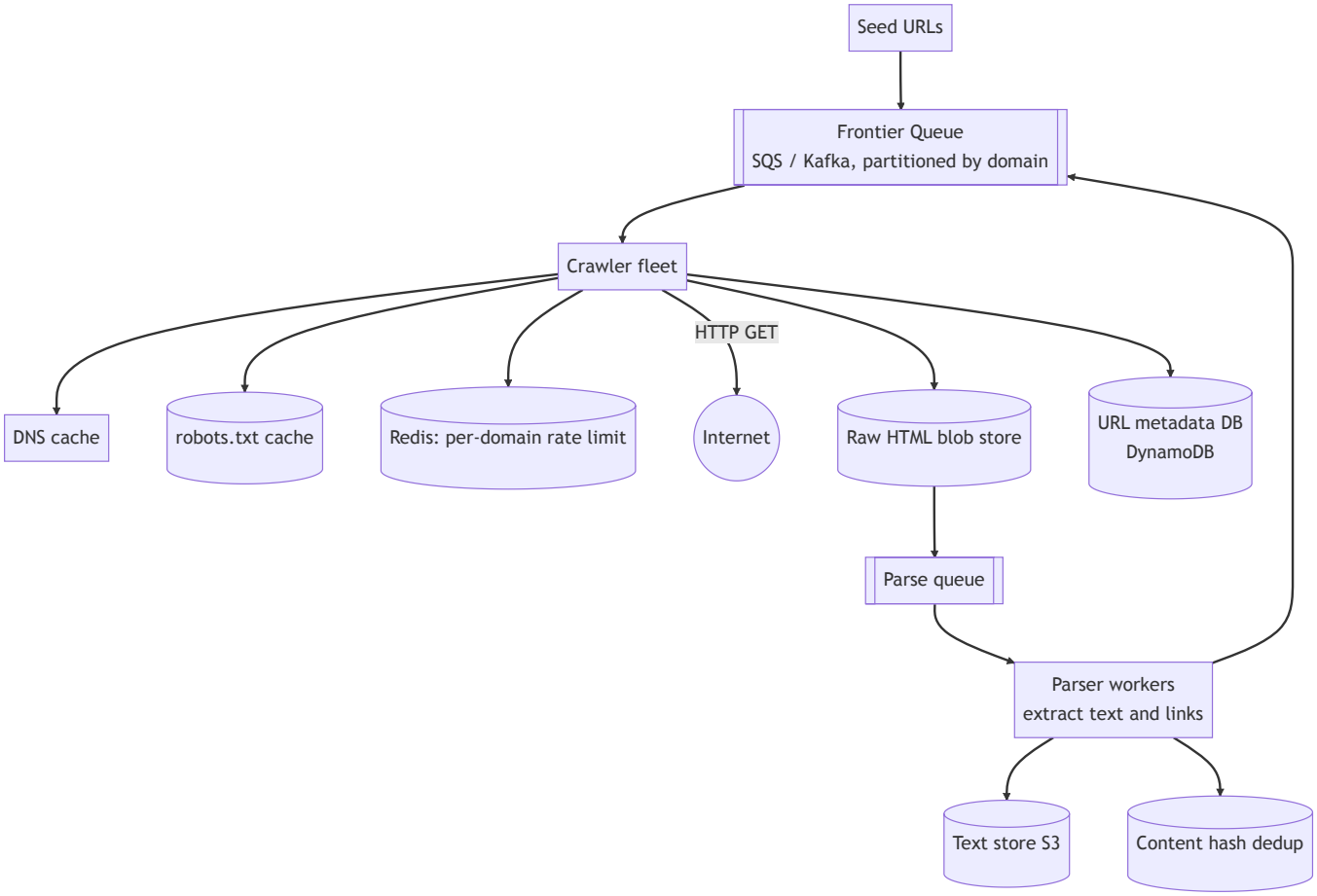
---

4. Entities



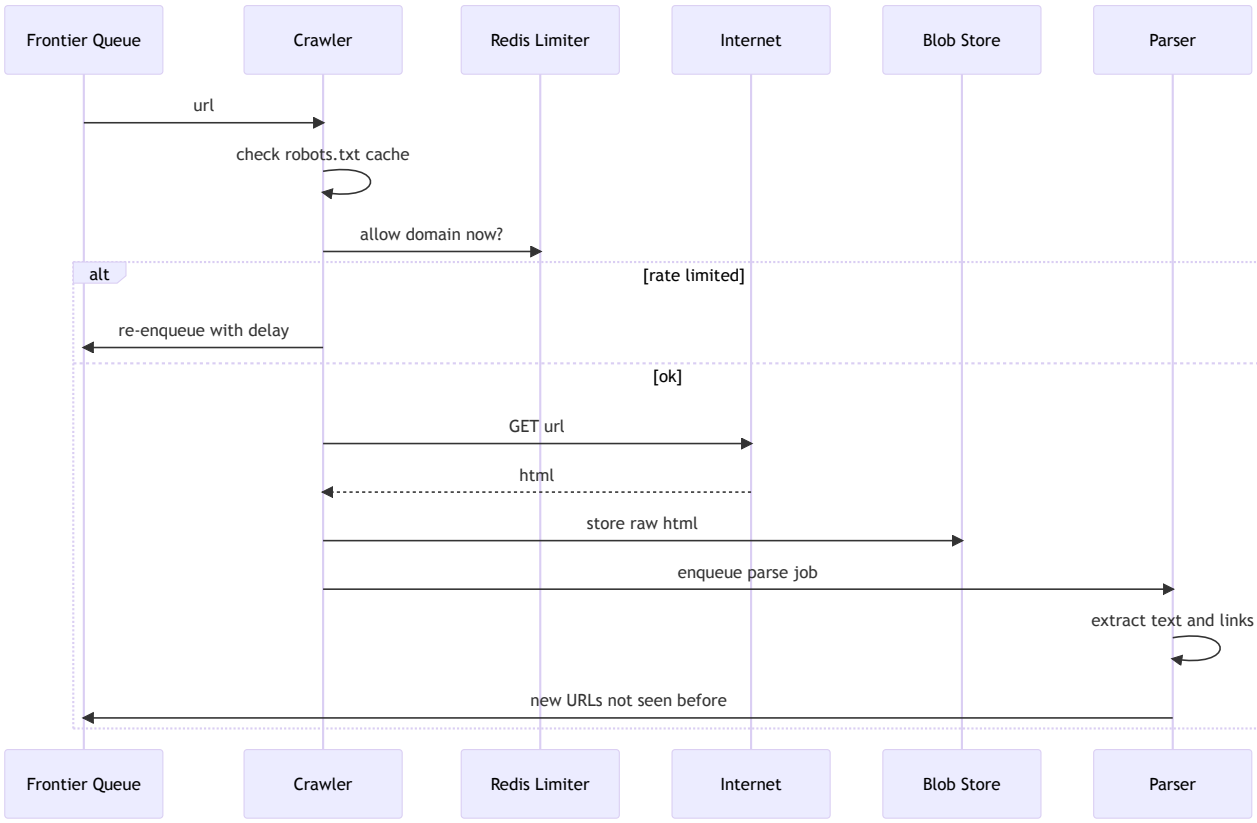
---

5. Architecture



---

6. Flow



---

7. Deep dive 1: fault-tolerant pipeline

Approach	Verdict	Why
Single crawl-and-parse process	Bad	Crash loses work; parse CPU blocks network
Split stages with durable queues (fetch, store raw, parse)	Great	Each stage retried independently; raw HTML saved first so parse can be replayed

- Queue: **SQS** with visibility timeout (retry) and a **DLQ** after N failures; or Kafka.
- **Retry with exponential backoff and jitter**; track `retry_count` per URL; permanent errors (404, DNS failure) are not retried.
- Everything **idempotent** by `url_hash`.

One-liner: *Durable queue between stages; store raw first; retry with backoff; DLQ the poison.*

8. Deep dive 2: politeness

- Fetch and cache **robots.txt** per domain (TTL about 24 h); honor `Crawl-delay` and disallow rules.
- **Per-domain rate limiter** (Redis token bucket, key = domain). If denied, re-enqueue with delay.
- Partition the frontier **by domain** so one worker owns a domain's politeness state locally, avoiding a distributed limiter for the hot path.
- Priority: score by importance/freshness (PageRank, change rate) so queue ordering is not FIFO only.
- Identify the crawler (User-Agent) and offer contact.

9. Deep dive 3: scaling to 10B pages

- Horizontal crawlers behind queues; autoscale on queue depth.
- **DNS** is a bottleneck: local resolver cache, batched prefetch, own recursive resolver.
- Reuse connections (HTTP keep-alive), compress (gzip), async I/O (many concurrent fetches per host).
- **Geo-distribute** crawlers near target sites.
- Sharded metadata (DynamoDB, key=`url_hash`) and bulk writes.

10. Deep dive 4: duplicates and traps

Problem	Technique
Seen URL	Normalize URL; `url_hash` PK in DB + **Bloom filter** front check
Same content, different URL	Content hash (SHA-256); near-dup via SimHash/MinHash
Crawler traps (calendars, session ids, infinite depth)	**Max depth**, max pages per domain, URL pattern heuristics, strip tracking params
Redirect loops	Hop limit

11. JavaScript-heavy pages

Detect via thin HTML; send to **headless browser pool** (Playwright) at much higher cost; keep it a separate, rate-limited queue.

---

12. Failure handling

Failure	Mitigation
Crawler crash mid-fetch	Message becomes visible again; idempotent redo
Site slow/timeouts	Timeouts, backoff, circuit-break the domain for a while
Blocking by site	Respect robots; reduce rate; rotate egress IPs only within policy
Queue/DB down	Replicated managed services; buffer locally
Bad content (huge/binary)	Size caps, content-type filter

---

13. Trade-offs

Decision	Chosen	Alternative	Given up
Queue	SQS/Kafka by domain	In-memory	Operational cost, but durable
Dedup	Bloom + DB	DB only	False positives (tiny)
Politeness	Domain-partitioned frontier	Global limiter	Skew for huge domains
Parse	Separate stage	Inline	Extra storage hop
JS rendering	Selective	Always	Coverage vs cost

---

14. Level expectations

- **Mid:** frontier, fetch, parse loop; robots.txt; basic dedup.
- **Senior:** staged durable queues, retry/DLQ, per-domain rate limiting, DNS and bandwidth math.
- **Staff+:** frontier prioritization/freshness, trap defense, near-dup detection, cost, geo-distribution.

15. Follow-ups

Recrawl scheduling, freshness scoring, sitemap ingestion, link graph for PageRank, indexing pipeline.

16. Summary checklist

- [ ] 23K pages/s, hundreds of crawlers
- [ ] Domain-partitioned frontier, Redis limiter, robots.txt cache
- [ ] Raw HTML first, separate parser stage, DLQ
- [ ] URL and content-hash dedup (Bloom filter)
- [ ] Depth limits for traps, DNS cache

Designing an Ad Click Aggregator

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**scale of clicks, exactly-once-style correctness, hot ads, click fraud and dedup, reconciliation**) → trade-offs. The hard parts are **stream aggregation at 10K clicks/s** and **never losing or double counting**.

---

0. Talk track

1. Clarify: record clicks, redirect user, advertiser queries aggregated metrics (per-minute), fraud prevention.
2. Insight: **write-heavy ingest, read of aggregates, durability over low-latency**.
3. HLD: click processor to stream to aggregator to OLAP store.
4. Deep dives: scaling the pipeline, hot shards, idempotency/dedup, batch reconciliation (lambda), query layer.

---

1. Problem

When a user clicks an ad, **redirect** them to the advertiser's site and **record the click**. Advertisers query **aggregated click metrics** (by ad, by time) with up to **1-minute** granularity and low latency.

What makes it hard

- **10K clicks/s average, 100K peak**; no click may be lost.
- Aggregates must be **accurate** (billing) yet **fresh** (near real time).
- **Hot ads** during events; **duplicate/fraudulent clicks** must not be billed.

---

2. Requirements

Functional

- Users click an ad and are redirected to the advertiser's site.
- Advertisers query click metrics over time at **1-minute minimum granularity**.

Out of scope: ad targeting, impression tracking, conversion attribution, cross-device tracking.

Non-functional

- Scale: **10K clicks/s** peak 100K; billions of clicks stored.
- **Low-latency analytics queries** (about 100 ms-1 s).
- **Fault tolerant and accurate**: no data loss.
- **Near real-time** (data within minutes).
- **Idempotent** click tracking (no double counting).

---

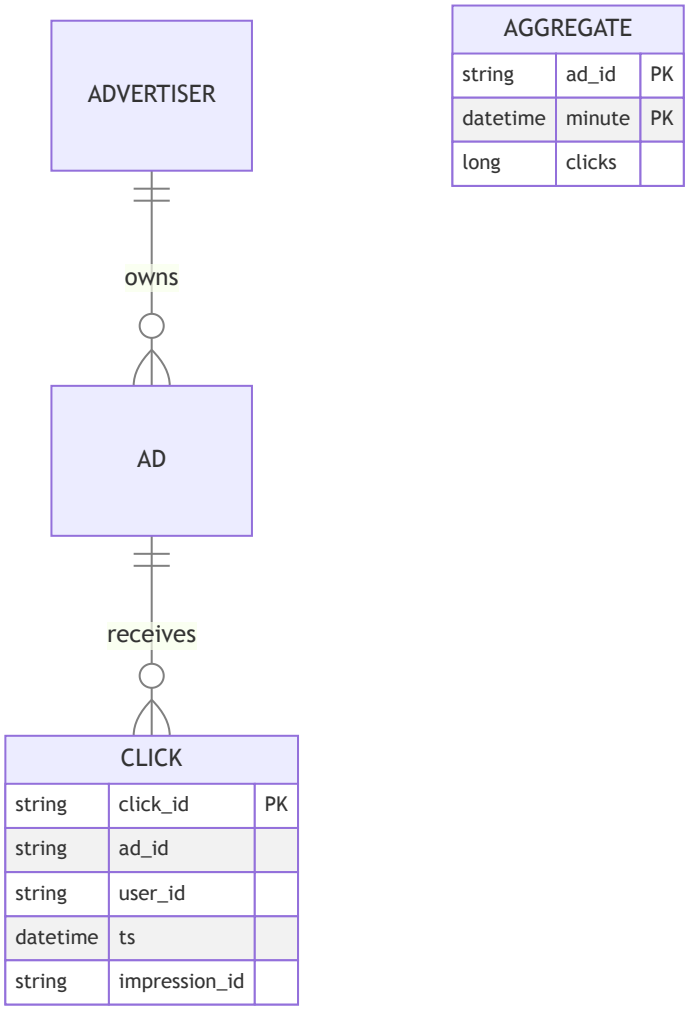
3. Estimation

Quantity	Assumption	Result
Clicks	10K/s avg	**864M/day**
Click record	about 100 B	86 GB/day raw, about 30 TB/yr
Peak	10x	100K/s: Kafka partitions needed
Aggregates	10M ads x 1/min active	much smaller; stored as rollups

Consequence: raw log in cheap storage (S3), aggregated rollups in an OLAP/time-series store. Ingest scales through **partitioned stream**.

---

4. Entities



---

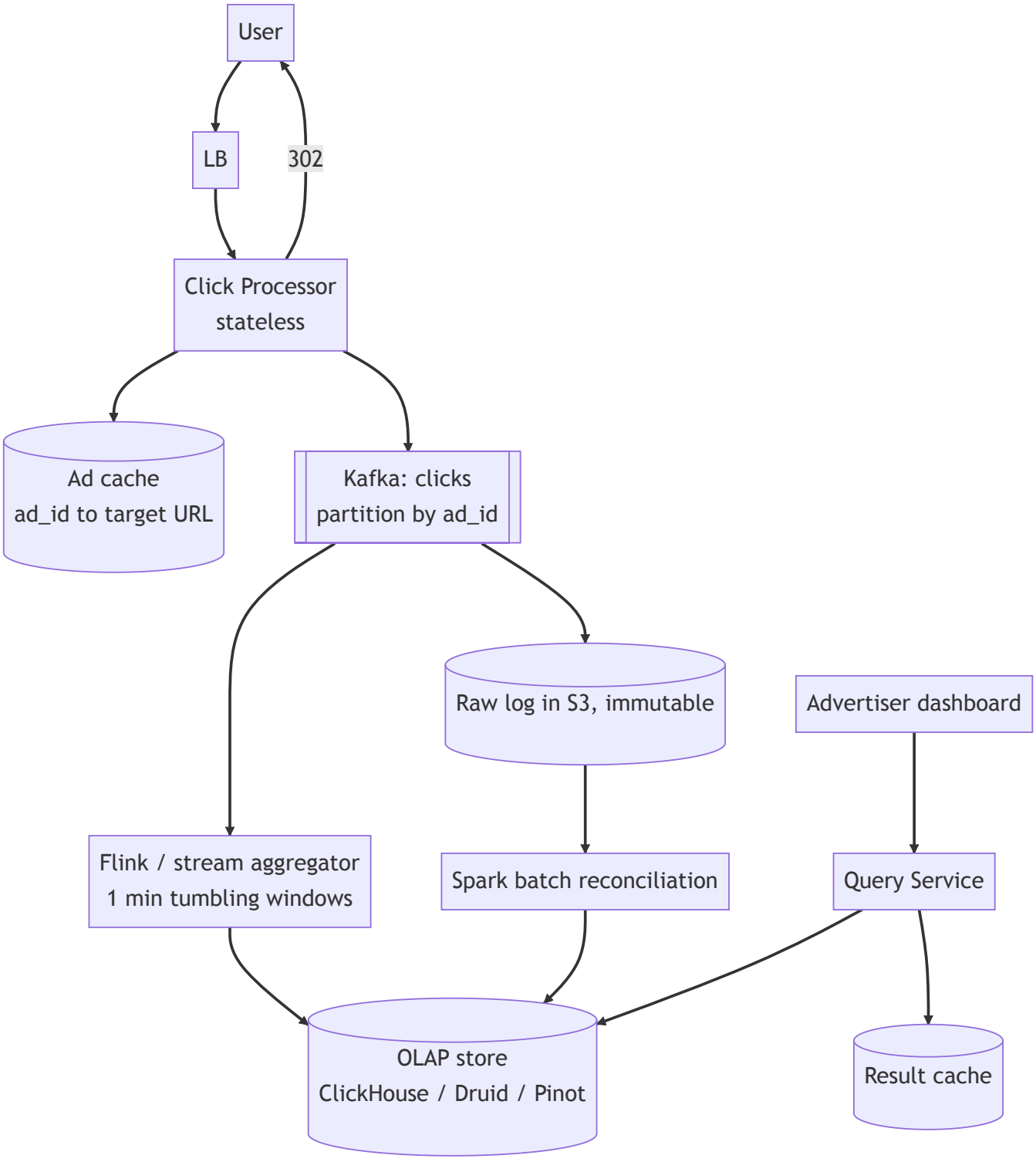
5. API

```
GET /click?ad_id=a1&impression_id=<signed token>     -> 302 Location: advertiser URL   # also records
GET /metrics?ad_id=a1&from=...&to=...&granularity=minute -> [{ts, clicks}]
```

Redirect target is **looked up server-side** (cached ad_id to URL), never taken from the client (open-redirect/fraud).

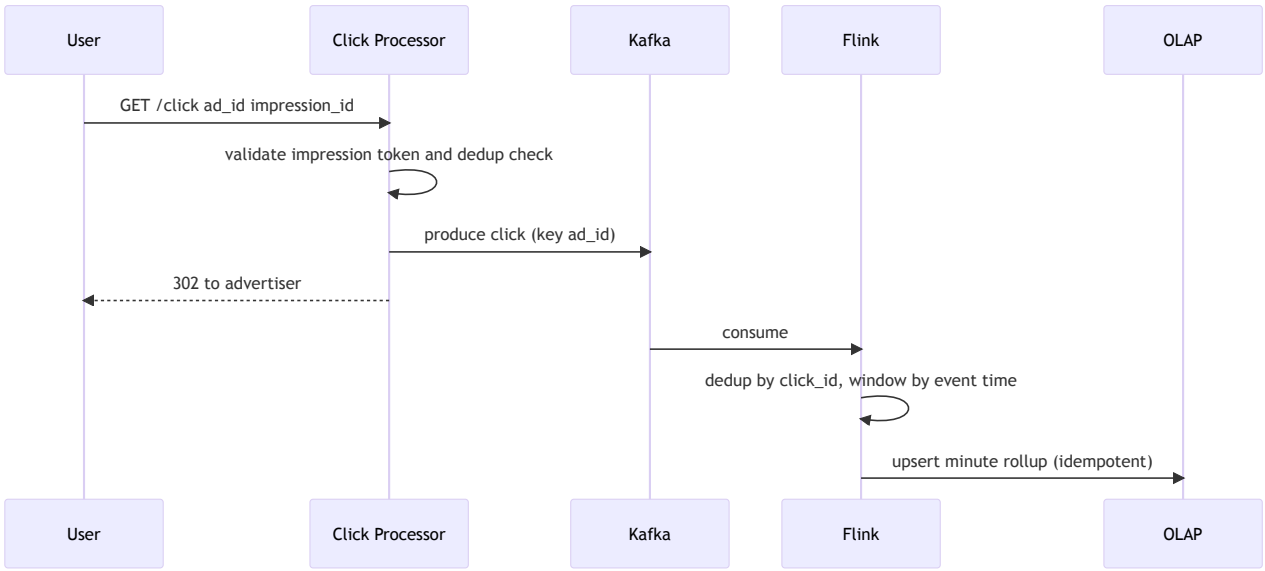
---

6. Architecture



---

7. Flow



---

8. Deep dive 1: ingest and aggregation

Approach	Verdict	Why
Write each click to SQL, aggregate with `GROUP BY`	Bad at scale	Write amplification; slow analytics
Batch only (Spark every N min)	Good	Accurate, but stale
Stream (Kafka + Flink) with windowed aggregation	Great	Near real-time; can combine with batch

Acknowledge the click once it's durably in **Kafka** (acks=all, replication factor 3), then redirect. Redirect shouldn't wait on aggregation. Flink checkpointing plus idempotent sink gives effectively-once.

9. Deep dive 2: hot ads

A viral ad hammers one partition keyed by `ad_id`. Mitigate: **key salting** (`ad_id#0..N`), aggregate partial counts per salt, then merge in a second stage; or pre-aggregate in the click processor in 1 s micro-batches before producing.

10. Deep dive 3: dedup and fraud

- **Impression id**: server issues a **signed, single-use impression token** embedded in the ad; the click must present it. Dedup `(impression_id)` via Redis `SET NX` with TTL (or in Flink state) so repeated clicks on one impression count once.
- Bot detection: rate limit per IP/user, anomaly detection offline; mark clicks invalid but **retain** raw record.
- Ads can be invalidated post hoc (billing correction) via batch recompute.

11. Deep dive 4: accuracy and reconciliation

Lambda-style: stream gives fast, approximate-fresh numbers; **batch from the raw S3 log** recomputes the truth periodically and overwrites rollups. Handles late events, bugs, and replay. Raw log is the **source of truth** (append-only, immutable).

12. Query layer

OLAP stores (columnar, pre-aggregated) answer time-range queries fast. Optional **pre-rollups** at minute, hour, day to avoid scanning; result cache for dashboards; sharded by ad_id/time.

---

13. Failure handling

Failure	Mitigation
Click processor crash	Stateless, LB reroutes; client retry safe via dedup
Kafka broker down	Replication ISR; producer retries with idempotence
Flink job failure	Restart from checkpoint, offsets replay
OLAP write duplicates	Upsert keyed `(ad_id, minute)`
Late/out-of-order	Event-time windows + watermarks + batch fix
Kafka lag	Alert; autoscale consumers

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Ingest	Kafka	Direct DB write	Extra infra; gains durability and buffering
Aggregation	Stream plus batch	Stream only	Dual pipelines to maintain
Dedup	Signed impression ids	IP heuristics	Needs impression tracking
Storage	Raw S3 + OLAP rollups	OLAP only	Cost for storing raw
Freshness	About 1 min	Sub-second	Window latency

---

15. Level expectations

- **Mid:** click log, redirect, DB aggregation.
- **Senior:** Kafka + Flink, partitioning, OLAP, idempotency, batch backup.
- **Staff+:** hot-key salting, watermarks/late data, fraud pipeline, reconciliation, exactly-once semantics.

16. Follow-ups

Conversion attribution, budget pacing (real-time spend limits), per-user frequency capping, multi-region ingest.

17. Summary checklist

- [ ] Durable Kafka before redirect, partition by ad_id (salt hot)
- [ ] Flink minute windows, idempotent upsert to OLAP
- [ ] Signed impression IDs for dedup
- [ ] Raw S3 log and batch reconciliation
- [ ] Cached query layer on pre-rolled aggregates

Designing Top-K (YouTube Top Videos / Trending)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**exact vs approximate counting, windowed aggregation, hot keys, merging shard top-Ks, serving**) → trade-offs. The hard part is **maintaining a top-K ranking over a firehose of events and sliding windows**.

---

0. Talk track

1. Clarify: top K videos by views for windows (hour, day, month, all-time), K up to 1000, low-latency reads.
2. Estimate: 70K view events/s; videos in the billions.
3. Naive (DB group by) to counters to stream aggregation.
4. Deep dives: pre-aggregation, sharded counters + heaps, Count-Min Sketch, time windows, hot partitions, read path.

---

1. Problem

Return the **K most viewed videos** for a given window. Views arrive as a stream; queries should return in milliseconds.

What makes it hard

- Enormous event rate and cardinality (billions of videos).
- **Time windows** slide; old events must expire.
- Skew: viral videos create hot keys.
- Accuracy vs cost trade-off.

---

2. Requirements

Functional

- Query top K (up to 1000) videos for **all-time, last hour, day, month**.

Out of scope: per-category tops, personalization, UI.

Non-functional

- Query latency **under 100 ms**.
- **Near real-time** (results within about 1 min; hourly window can lag a little).
- **Highly available**; eventual consistency acceptable.
- Scale: **70K views/s**, 1B videos, **approximate OK** for some windows (state this!).

---

3. Estimation

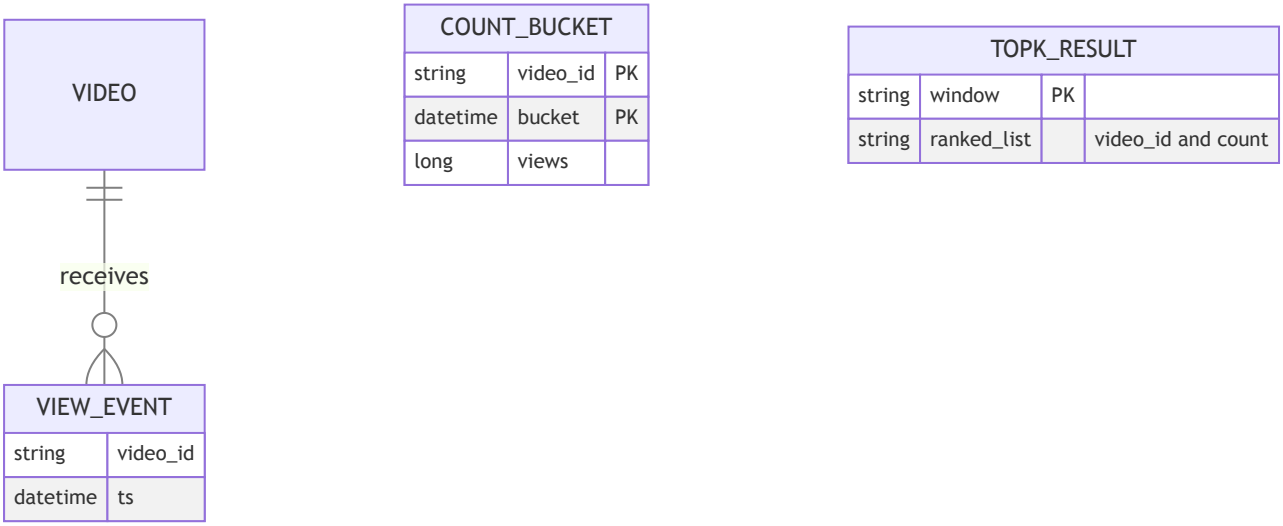
Quantity	Assumption	Result
Views	70K/s peak	**6B/day**
Event	about 50 B	300 GB/day raw
Distinct videos	1B, counter 16 B	**16 GB** counters: fits across a few nodes, even one big one
Top-K output	1000 x 16 B	**16 KB** per window: trivial to cache

Consequences

- The result is tiny and precomputable: **never compute top-K at query time**.
- Counters fit in memory when sharded; the pain is **write throughput** and **windowing**.

---

4. Entities



---

5. API

```
GET /views/top?window=hour&k=100   -> [ { "video_id":"v9", "views":1200000 }, ... ]
```

Windows: `hour` | `day` | `month` | `all`.

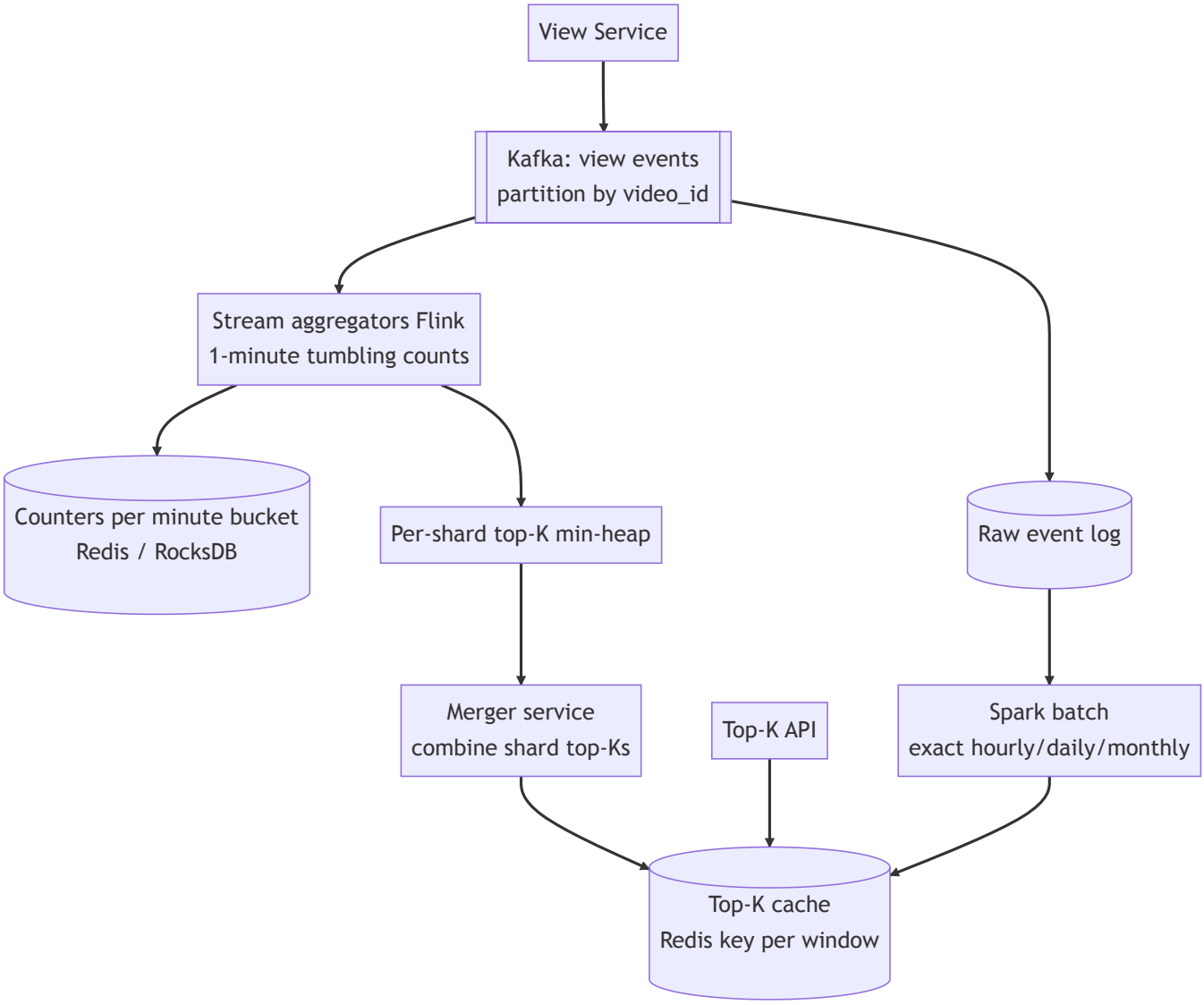
---

6. Evolution of the design

Step	Idea	Why it fails
1	`SELECT video_id, COUNT(*) ... GROUP BY ... ORDER BY LIMIT K`	Full scan per query
2	Per-video counters in a DB	Writes 70K/s on hot rows; top-K still needs a sort
3	**Stream pre-aggregation + per-shard top-K heaps + merge**	Good
4	Approximate counters (Count-Min Sketch) for huge key spaces	Memory bound, small error

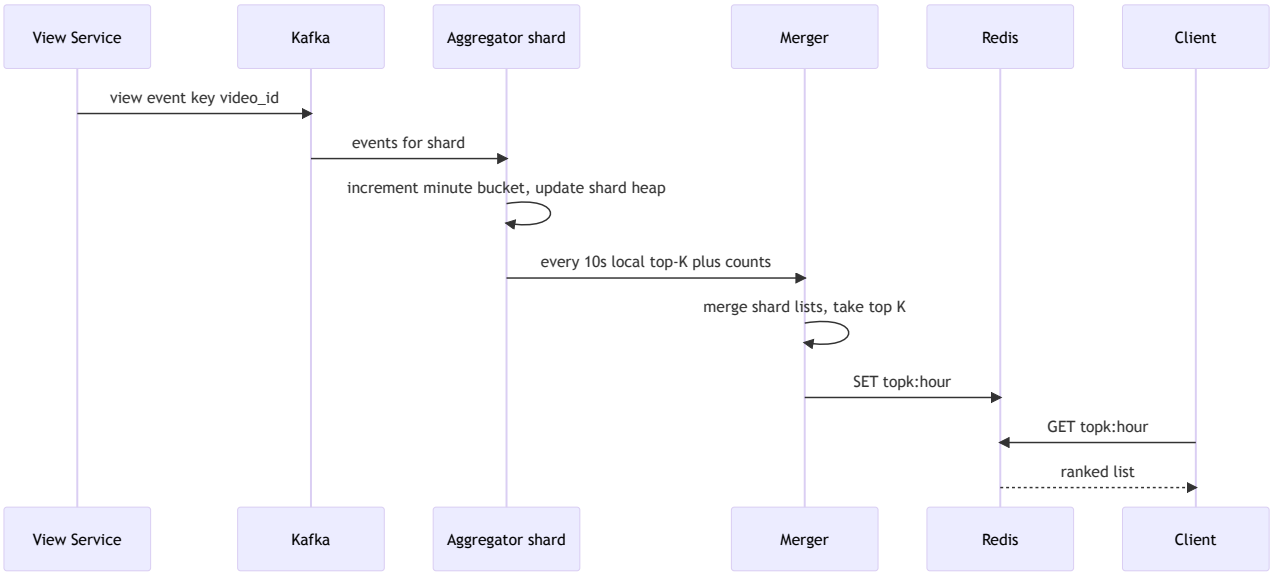
---

7. Architecture



---

8. Flow



---

9. Deep dive 1: partition by video so top-K merges are correct

Partition by `video_id` so **each video's count lives on exactly one shard**. Each shard keeps a **min-heap of size K**. The global top-K is then the top K of the union of shard top-Ks (**correct because any global top-K item must be in its own shard's top-K**). Merge cost: O(shards x K log K), tiny.

One-liner: *Shard by key, local top-K, merge K-lists.*

10. Deep dive 2: time windows

Approach	Verdict
Store raw events, recount per query	Bad
Bucketed counts (1-min buckets) and sum last N buckets for the window	Great for hour; for day/month use **hourly/daily rollup buckets**
Sliding sketch with decay	Good for approximate trending

Rollups: minute to hour to day. Expire old buckets (TTL). All-time window: running total counter. Hourly accuracy is the strictest; month can be recomputed in **batch** from the raw log (exact) and be hours stale.

11. Deep dive 3: counting at scale

- Exact counters in memory (16 GB total) are fine when sharded.
- If the key space were huge: **Count-Min Sketch** (fixed memory, overestimates only) plus a heap of candidates; or **Space-Saving / Misra-Gries** heavy hitters.
- **Hot keys**: a single viral video overloads its shard, so pre-aggregate in the View Service or Kafka consumer (batch 1 s, then add N), or salt the key and sum partials.

12. Deep dive 4: read path

Results are precomputed into Redis (`topk:{window}`), refreshed every 10-60 s; the API is a pure cache read, so latency is under 10 ms and the read side scales with replicas/CDN. Fallback to last good value on failure.

13. Failure handling

Failure	Mitigation
Aggregator crash	Checkpoint + Kafka offset replay; counters rebuilt from log
Duplicate events	Idempotent via event id dedup if needed, else tolerate (approximate)
Kafka lag	Alert; results stale but available
Redis loss	Recompute from counters or batch; API serves stale snapshot
Late events	Watermarks; batch layer fixes hourly/daily

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Accuracy	Exact where small, sketch where huge	Always exact	Memory vs small error
Freshness	10-60 s	Real time	Cost
Month window	Batch recompute	Streaming	Staleness (hours)
Merge	Shard top-K merge	Global heap	Small coordination step
Read	Precomputed cache	Compute on read	Storage trivial

---

15. Level expectations

- **Mid:** counters + sort; knows scans fail; adds cache.
- **Senior:** Kafka partitioned aggregation, heap per shard, windowing with buckets, batch for long windows.
- **Staff+:** Count-Min / heavy hitters, hot-key mitigation, correctness of merge, cost/accuracy trade-offs.

16. Follow-ups

Top-K per category/region, trending (velocity not total), decayed scores, real-time (sub-second) boards.

17. Summary checklist

- [ ] Precompute, never query-time top-K
- [ ] Shard by video_id, local min-heap size K, merge
- [ ] 1-minute buckets rolled up to hour/day; batch for month
- [ ] Salting or pre-aggregation for hot videos
- [ ] Redis result cache, stale-safe

Designing a Distributed Cache

> Interview-ready walkthrough: requirements → estimates → API → HLD → deep dives (**partitioning and consistent hashing, replication, eviction, hot keys, invalidation and write policies, client vs proxy routing, failure**) → trade-offs. The hard part is **keeping a huge in-memory key space available and balanced as nodes come and go**.

> Related: *In-memory KV store* doc in this folder covers a single node (data structures, persistence). This one is the **distributed** layer.

---

0. Talk track

1. Clarify: get/set/delete with TTL, LRU eviction, 1 TB, 100K QPS, under 10 ms.
2. Start with single node, then ask what breaks (memory, throughput, availability).
3. Deep dives: sharding (consistent hashing), replication, eviction/expiry, hot keys, client library, failure.

---

1. Problem

A shared in-memory cache fronting databases/services: **O(1) get/set**, TTL, eviction when full, scalable across nodes.

What makes it hard

- Data larger than one machine's RAM, QPS greater than one machine.
- **Adding/removing nodes** must not remap everything (cache avalanche).
- Hot keys, availability, and **consistency** with the source of truth.

---

2. Requirements

Functional

- `set(key, value, ttl)`, `get(key)`, `delete(key)`.
- Configurable **TTL**, **LRU** eviction.

Out of scope: complex types (Redis structures), transactions, durable persistence.

Non-functional

- **High availability** (cache loss must not take down the DB).
- **Low latency** (under 10 ms p99, ideally sub-ms), **1M+ QPS** cluster.
- **Scalable** to 1 TB+ and thousands of QPS growth.
- **Eventual consistency** acceptable.

---

3. Estimation

Quantity	Assumption	Result
Data	1 TB	**about 16 nodes at 64 GB** (about 3x with replicas)
QPS	1M	at about 100K ops/s per node = **10+ nodes** minimum
Item	1 KB avg	1B items
Hit ratio target	95%+	DB load = 5% of reads

Memory (not CPU) decides the node count; QPS and replication also push it up. **Plan around the larger of both.**

---

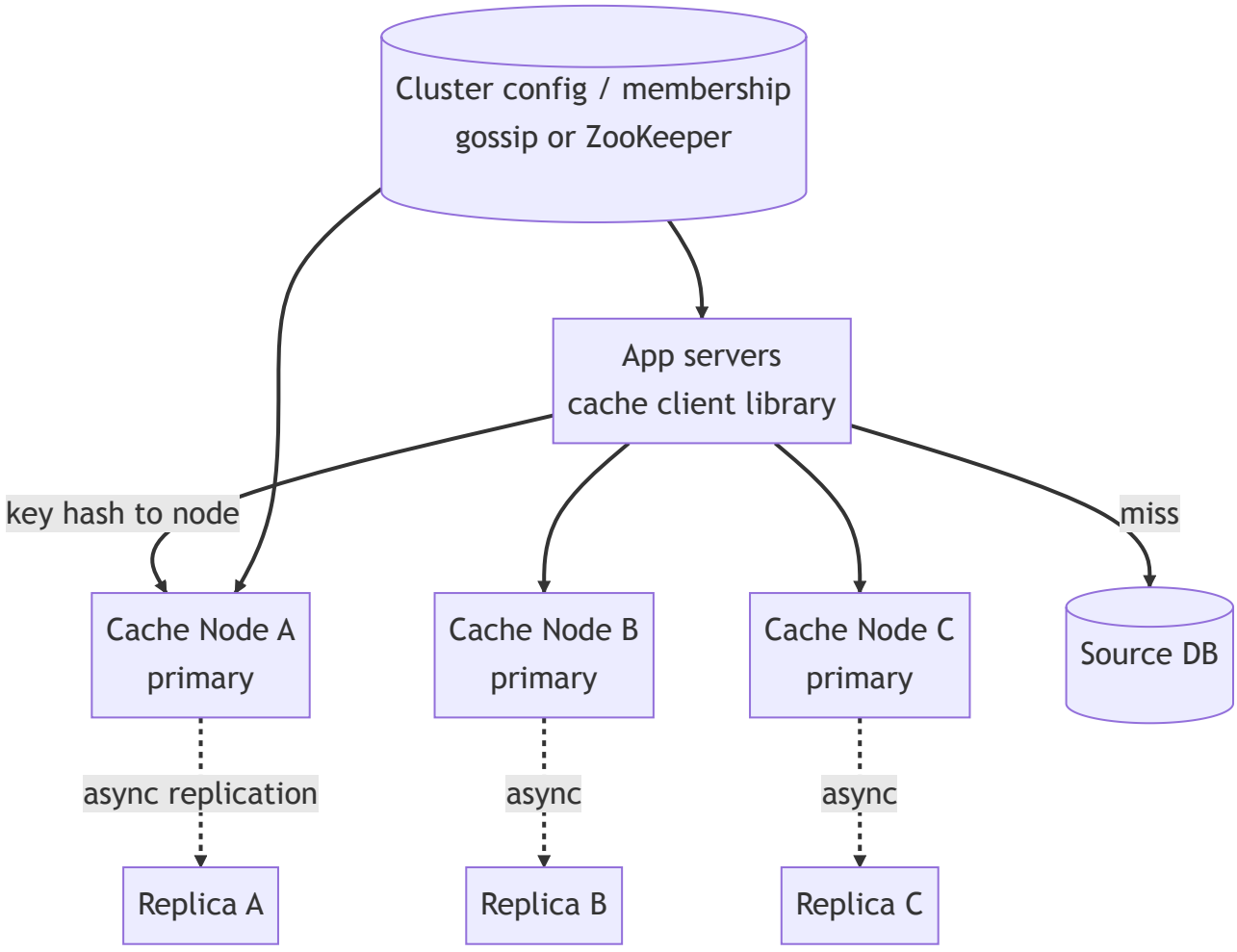
4. API

```
PUT  /cache/{key}  value, ttl     -> 200
GET  /cache/{key}                 -> 200 value | 404
DELETE /cache/{key}               -> 204
```

Binary protocol (RESP / memcached) in practice; a **smart client** hashes the key to a node.

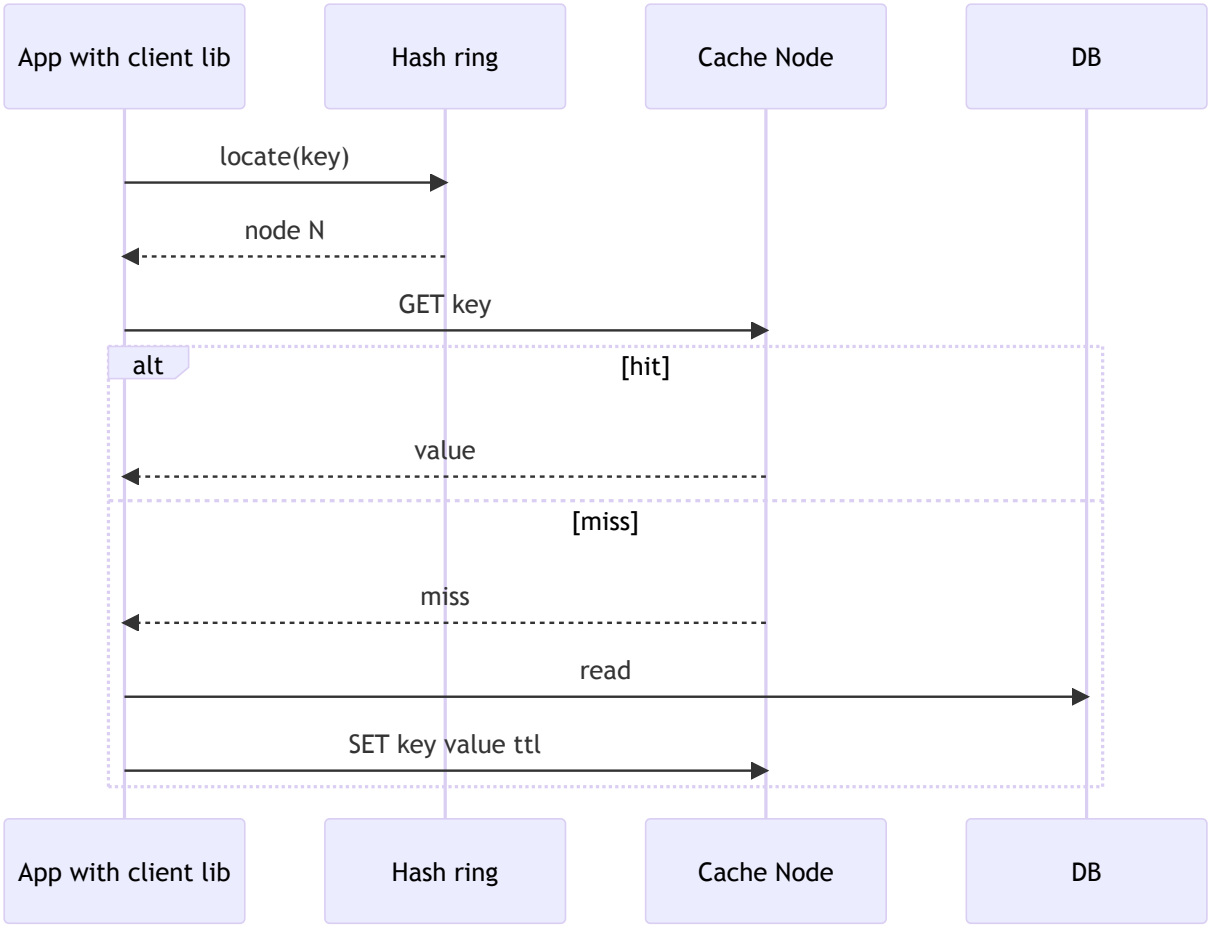
---

5. Architecture



---

6. Flow



---

7. Deep dive 1: partitioning

Approach	Verdict	Why
`hash(key) mod N`	Bad	Changing N remaps about all keys, causing a miss storm
Consistent hashing with virtual nodes	Great	Adding or removing a node moves about 1/N keys; vnodes even out load
Hash slots (Redis Cluster: 16384 slots)	Great	Explicit slot to node map; resharding moves slots; clients cache the map and follow `MOVED` redirects

One-liner: *Consistent hashing with vnodes so membership changes move about 1/N of keys.*

8. Deep dive 2: replication and availability

- Each primary has 1+ **replicas** (async for latency; promotion on failure by a failure detector/election).
- **Async replication can lose recent writes**; acceptable for a cache (miss, then refill).
- Reads from replicas boost read throughput at the cost of stale reads.
- Spread replicas across AZs. Alternative: no replicas, accept misses (memcached model) and rely on a warm DB capacity.

9. Deep dive 3: eviction and expiry

- **LRU** (hash map + doubly linked list, O(1)); approximated LRU / **LFU** (sampling) in Redis for lower overhead. LFU resists scans.
- **TTL**: lazy expiry on access plus periodic sampled sweep (or timer wheel) to reclaim memory.
- Use `maxmemory` + policy (`allkeys-lru`) to avoid OOM.

10. Deep dive 4: hot keys

A single key can overwhelm one node. Options: **local in-process L1 cache** with short TTL in clients, **replicate hot keys** to multiple nodes (key suffix `#0..R-1`, random read), detect via sampling counters, and request coalescing (singleflight).

11. Deep dive 5: write policies and consistency

Pattern	Behavior	Use
Cache-aside	App reads cache, then DB on miss; writes DB, then **invalidate**	Default
Write-through	Write cache and DB synchronously	Read-after-write, extra latency
Write-behind	Write cache, flush later	Fast writes; risk of loss
Read-through	Cache loads from DB itself	Simpler apps

Races (stale set after invalidate) are mitigated with short TTLs, versioned values, or delete-after-write with a delay. Stampedes: singleflight, locks, TTL jitter, probabilistic early refresh.

12. Deep dive 6: client vs proxy topology

	Smart client	Proxy (Twemproxy/Envoy)
Hops	1	2
Language support	Library per language	Any
Config updates	Client must refresh	Central

---

13. Failure handling

Failure	Mitigation
Node down	Replica promoted; consistent hashing re-routes; DB protected by rate limit and coalescing
Cache-wide cold start	Gradual warm-up, circuit breaker/load shedding, pre-warm top keys
Network partition	Cache may serve stale; prefer availability; clients fall back to DB with limits
Rebalance storm	Move slots gradually, throttle migration
Memory pressure	Eviction policy; alert on evictions and hit ratio

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Sharding	Consistent hash / slots	Mod N	Mapping complexity
Replication	Async	Sync	Possible lost writes
Write policy	Cache-aside	Write-through	Stale window
Topology	Smart client	Proxy	Client complexity
Eviction	Approx LRU/LFU	Exact LRU	Slight accuracy

---

15. Level expectations

- **Mid:** hash map + LRU, TTL, scaling by sharding.
- **Senior:** consistent hashing, replication/failover, hot keys, write policies.
- **Staff+:** rebalancing without storms, stampede protection, hit-ratio economics, multi-region, observability.

16. Follow-ups

Multi-region caches, cache warming on deploy, near-cache invalidation via pub/sub, memory-efficient encodings.

17. Summary checklist

- [ ] Memory-bound sizing, replicas across AZs
- [ ] Consistent hashing with vnodes or hash slots
- [ ] LRU/LFU plus TTL (lazy + sweep)
- [ ] Hot key replication and L1 cache, singleflight
- [ ] Cache-aside with jittered TTLs; protect the DB

Designing a Job Scheduler (Distributed Task Scheduler)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**scheduling at scale, finding due jobs, at-least-once execution, retries and idempotency, long-running jobs, exactly-once effects**) → trade-offs. The hard part is **reliably executing every job on time, once in effect**, despite worker crashes.

---

0. Talk track

1. Clarify: one-time and recurring (cron) jobs, immediate or future, status tracking, retries. 10K jobs/s.
2. Insight: separate **scheduling (when)** from **execution (run)**; durable store + queue.
3. Deep dives: efficient due-job lookup, queueing with visibility timeouts, at-least-once and idempotency, scale.

---

1. Problem

Users submit tasks to run **now, later, or on a schedule**; the system executes them reliably and reports status.

What makes it hard

- **Time accuracy** at scale (millions of pending jobs).
- Workers crash mid-job: **no lost jobs**, avoid **duplicates**.
- Retries, priority, long-running jobs, bursts at popular times (midnight).

---

2. Requirements

Functional

- Schedule jobs: **immediate, one-time at a future time, recurring (cron)**.
- Monitor job status.

Out of scope: UI, job dependencies/DAGs (mention), rich workflow.

Non-functional

- **High availability** (don't lose jobs).
- **Scale**: 10K jobs/s, up to 10M+ scheduled.
- **At-least-once** execution; **under 2 s** execution delay from due time.
- Jobs may run up to 2 hours.

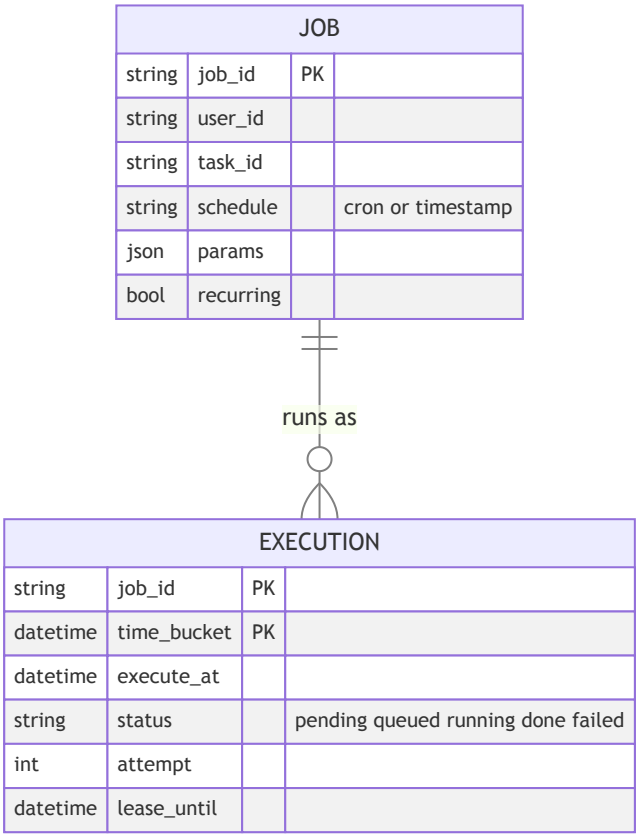
---

3. Estimation

Quantity	Assumption	Result
Throughput	10K executions/s	workers: avg 5 s job gives **50K concurrent**
Pending	10M jobs, 1 KB	**10 GB** metadata: fits a sharded DB, hot window in memory
Midnight burst	10x	queue buffering essential

---

4. Entities



`Job` is the definition; `Execution` is each occurrence (due time, status, attempts).

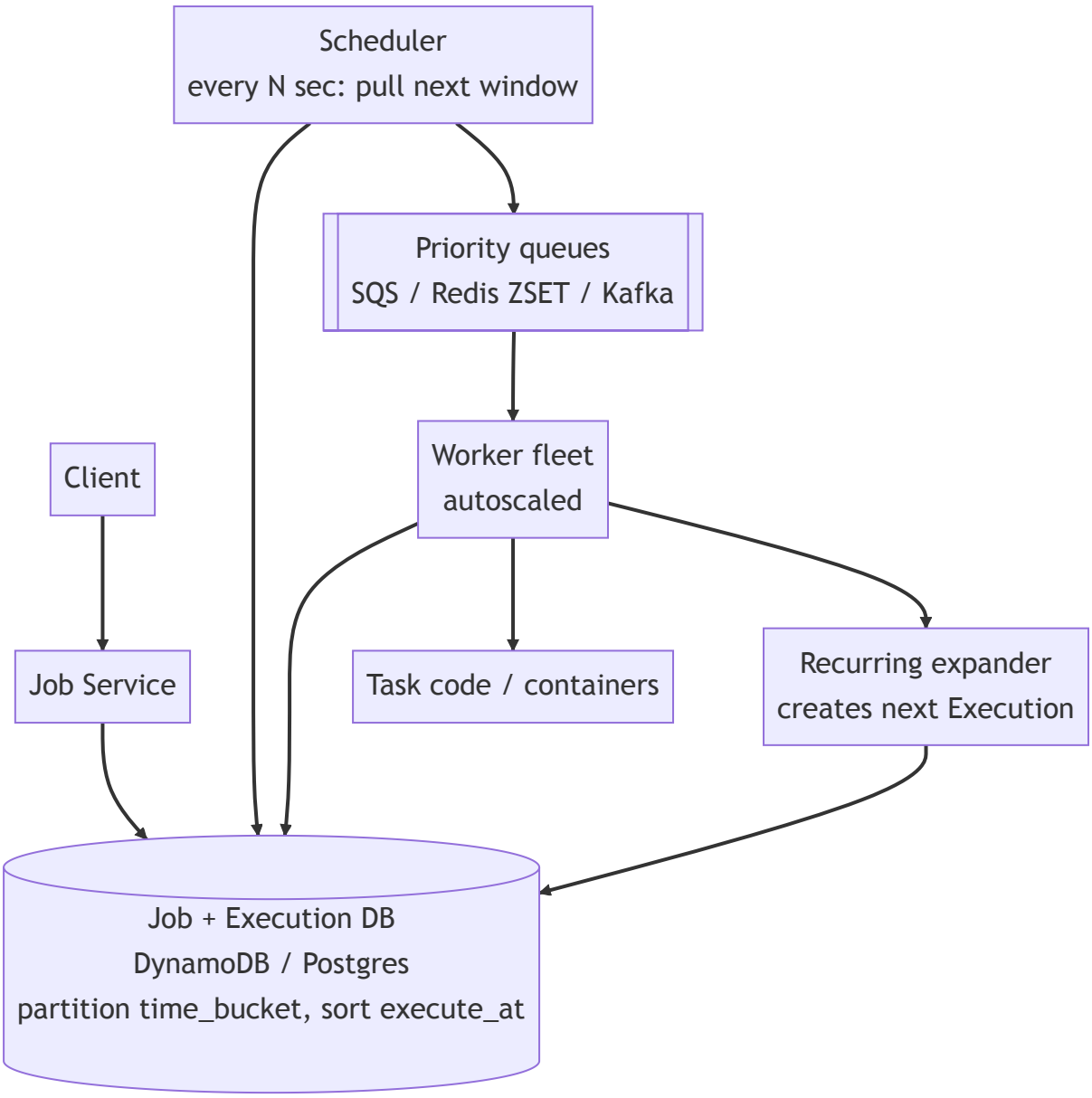
---

5. API

```
POST /jobs   { "task_id":"send_report", "schedule":"0 0 * * *", "parameters":{...} }  -> 201 { job_id
GET  /jobs?user_id=...&status=...&cursor=...
GET  /jobs/{id}                -> definition + execution history
DELETE /jobs/{id}              # cancel future runs
```

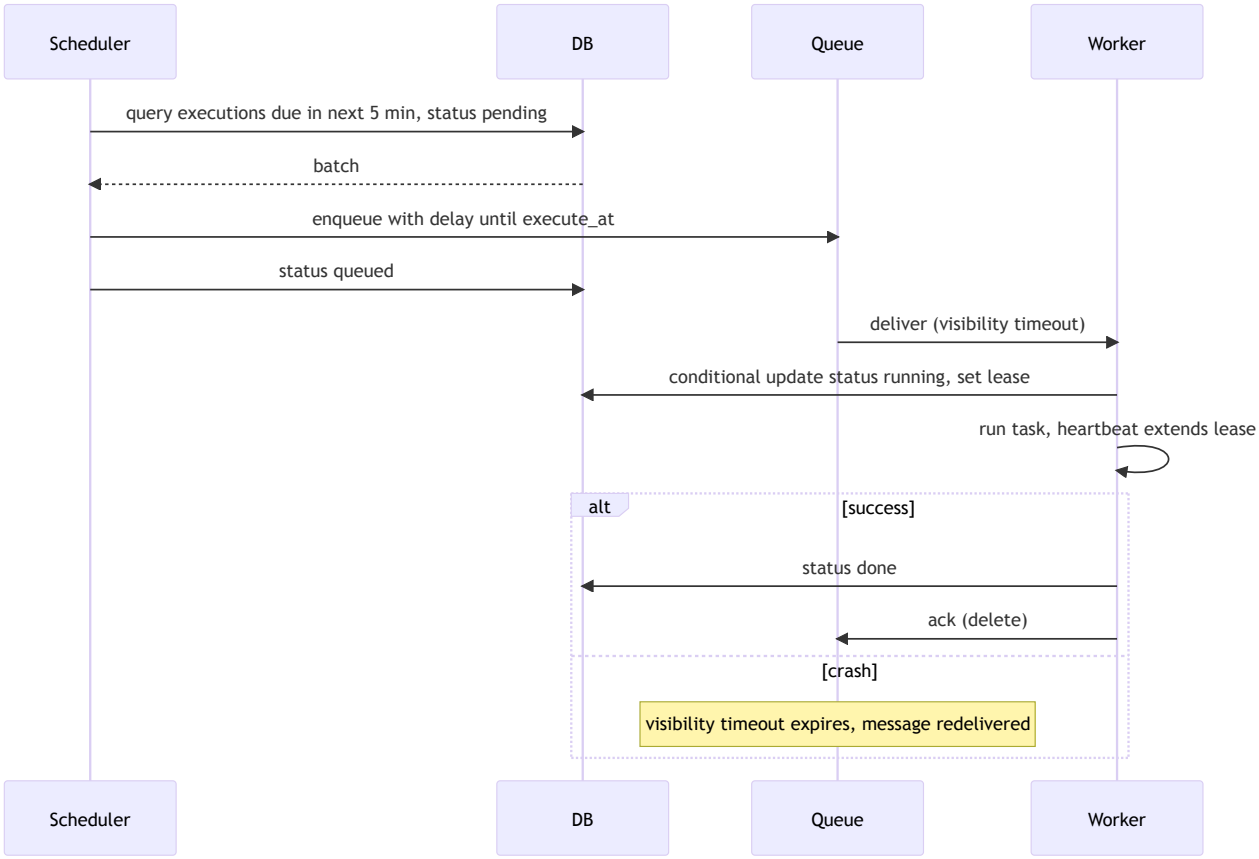
---

6. Architecture


Scheduler
every N sec: pull next window
Priority queues
SQS / Redis ZSET / Kafka
Client
Worker fleet
autoscaled
Job Service
Task code / containers
Recurring expander
creates next Execution
Job + Execution DB
DynamoDB / Postgres
partition time_bucket, sort execute_at

---

7. Flow



---

8. Deep dive 1: finding due jobs efficiently

Approach	Verdict	Why
Poll `WHERE execute_at <= now` over a full table	Bad	Full scans, hot index
Index on `execute_at` + cron every second	Good	Better; still contention and hot partition at "now"
Time-bucketed table: partition key = `execute_at` truncated to hour, sort key = `execute_at`	Great	Scheduler reads only upcoming bucket(s); writes spread across future buckets
Two-tier: DB poll every few minutes pulls near-future jobs into **delay queue / Redis ZSET**	Great	Exact timing without per-second DB polling

Scheduler instances each own bucket ranges (consistent hashing or leader election) to avoid duplicate enqueue; duplicates are harmless if workers claim with a conditional update.

One-liner: *DB holds the far future, a queue holds the near future.*

9. Deep dive 2: at-least-once with leases

- **Visibility timeout** (SQS) or **lease** field: a job is claimed by a worker; if the lease expires without completion, it is redelivered.
- Worker **heartbeats** extend the lease for long jobs (up to 2 h).
- Claim via **conditional write** (`status=queued AND lease_until<now`) to prevent two workers running one execution.
- Max attempts, **exponential backoff with jitter**, then DLQ and `failed` status.

10. Deep dive 3: idempotency (effectively exactly-once)

Execution can occur twice (lease expiry while slow, crash after side effect before ack). Therefore tasks must be **idempotent**: pass `execution_id` as an idempotency key to downstream calls; use transactional outbox/dedup tables. Be explicit: the scheduler guarantees **at-least-once**; the **task** provides idempotency.

11. Deep dive 4: recurring jobs and scale

- Recurring: on completion (or creation), compute the next fire time and insert the next `Execution`; avoids pre-materializing years.
- **Scale**: DB partitioned by time bucket and `job_id`; queues sharded by priority; worker autoscaling on queue depth; smoothing for midnight bursts by adding **jitter** to user cron times where acceptable.
- **Priority**: separate queues/weights so urgent jobs bypass backlog; isolate tenants (per-tenant concurrency limits) to prevent noisy neighbors.

---

12. Failure handling

Failure	Mitigation
Worker crash	Lease expires, redelivery
Scheduler crash	Another instance takes its buckets; re-enqueue safe (claims are conditional)
Queue outage	Jobs remain in DB `pending`; scheduler retries enqueue
DB failover	Multi-AZ; jobs not lost since durable first
Poison job	Attempt cap, DLQ, alerts
Clock skew	Use server-side time, NTP, tolerate small drift

---

13. Trade-offs

Decision	Chosen	Alternative	Given up
Delivery	At-least-once	Exactly-once	Task idempotency burden
Store + queue	DB + delay queue	DB only	Extra component
Granularity	Seconds	Millisecond-accurate	Cost/complexity
Cron expansion	Next occurrence only	Pre-expand	Slightly more write-on-complete logic
Claiming	Conditional write	Distributed lock	Needs DB support (cheap)

---

14. Level expectations

- **Mid:** jobs table, poll for due, workers, status.
- **Senior:** time-bucketed index, queue + visibility timeout, retries, idempotency, autoscale.
- **Staff+:** two-tier scheduling, tenant isolation, priority, long-job leases, burst smoothing, observability (lag, late-start SLO).

15. Follow-ups

Job DAG dependencies, rate-limited jobs, cancellation/pause, multi-region scheduling, timezone/DST handling for cron.

16. Summary checklist

- [ ] Definition vs execution tables, time-bucket partitioning
- [ ] Near-future jobs in a delay queue/ZSET
- [ ] Conditional-claim, lease, heartbeat, DLQ
- [ ] At-least-once plus idempotent tasks
- [ ] Autoscale by queue depth, tenant limits

Designing a Payment System (Stripe-style Payment Processing)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**idempotency, durability and no lost payments, webhooks, double-entry ledger, reconciliation, security/PCI**) → trade-offs. The hard part is **correctness**: money must never be lost, duplicated, or invented.

---

0. Talk track

1. Clarify: merchants initiate payments, customers pay by card, merchants check status and get webhooks. Not building the card network.
2. Priorities: **correctness and durability over latency**; 10K TPS.
3. Entities and API with **idempotency keys**.
4. HLD: API, payment DB, PSP/bank integration, webhooks.
5. Deep dives: idempotency, durable state machine, async bank calls and webhooks, security, reconciliation/ledger, scale.

---

1. Problem

A platform that lets **merchants charge customers** (cards, wallets), routes to banks/PSPs, tracks status, and notifies merchants.

What makes it hard

- **Exactly-once effect** of money movement over unreliable networks.
- External banks are slow, flaky, asynchronous.
- **Security/PCI**: card data is toxic.
- Audit and **reconciliation** with external truth.

---

2. Requirements

Functional

- Merchants initiate payment requests (a customer pays for an order).
- Users pay for products.
- Merchants query payment status.
- Merchants receive **webhook** updates.

Out of scope: payouts, fraud ML details, multi-currency FX, subscriptions.

Non-functional

- **Security** (PCI-DSS), **durability** (no payment lost), **strong consistency / transactional integrity**.
- **Idempotent** operations.
- **High availability** (but correctness before availability).
- Scale: **10K TPS**; latency of seconds acceptable (bank-bound).

---

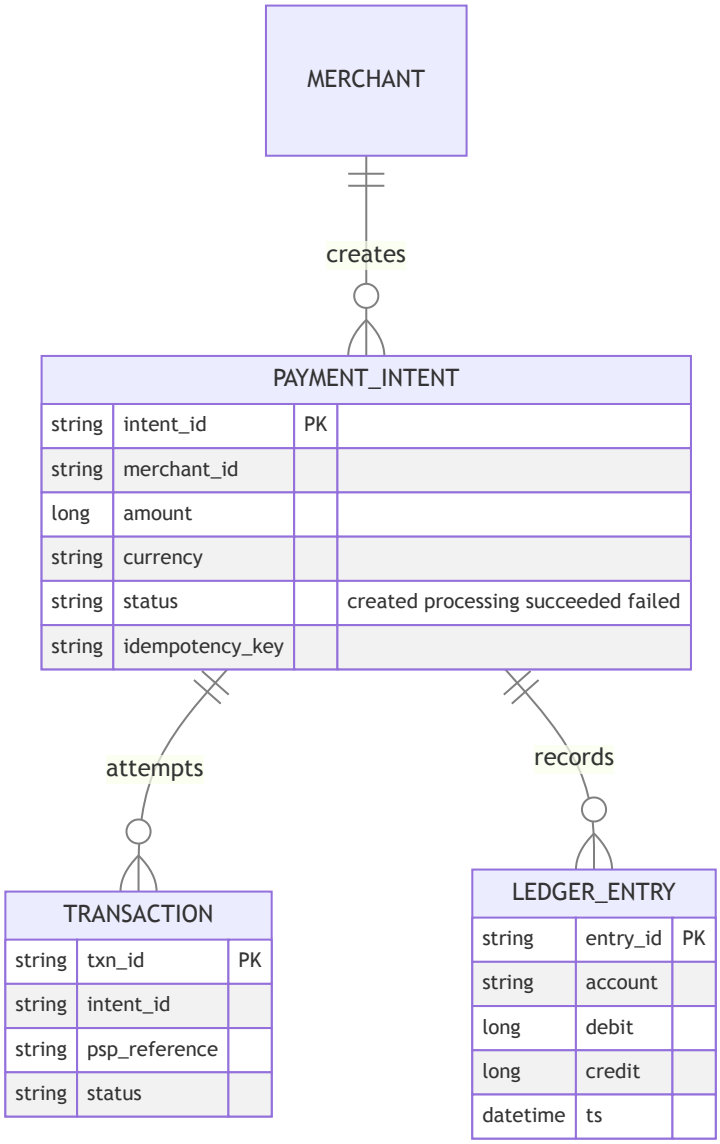
3. Estimation

Quantity	Assumption	Result
Peak TPS	10K	**about 860M/day** (upper bound)
Record	about 2 KB (events + metadata)	about 1.7 TB/day, so partition and archive
Webhooks	about 1 per transition x 3	about 30K/s outbound

Key point: volume is moderate; **consistency, auditing and failure handling** dominate, not raw throughput.

---

4. Entities



---

5. API

```
POST /payment-intents   Idempotency-Key: <uuid>
{ "amount": 4999, "currency":"USD", "order_id":"o1" }     -> 201 { intent_id, client_secret }

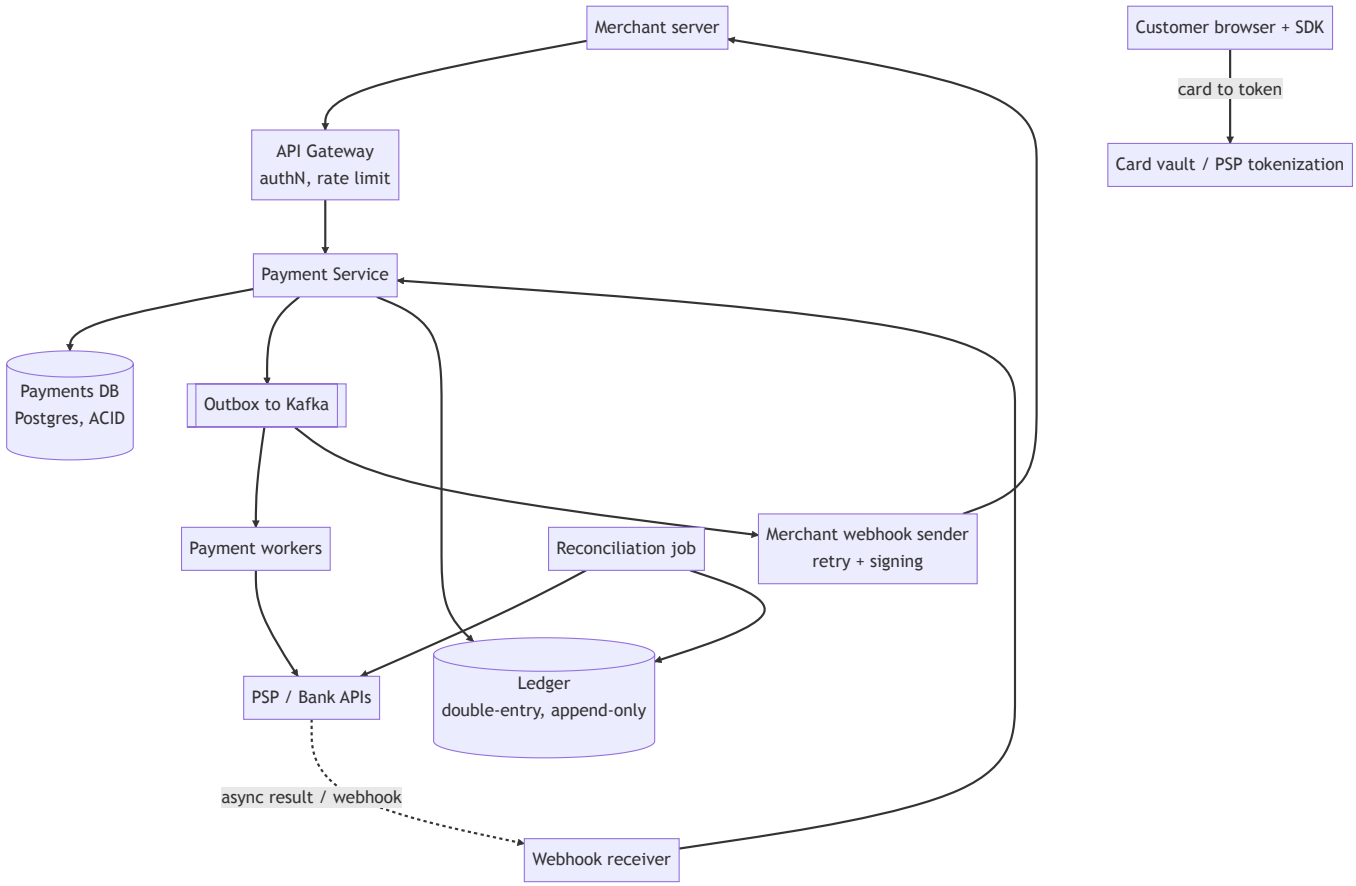
POST /payment-intents/{id}/confirm   Idempotency-Key
{ "payment_method_token":"tok_..." }                        -> 202 { status:"processing" }

GET  /payment-intents/{id}                                  -> { status }
POST (merchant endpoint) webhook: { event:"payment.succeeded", intent_id, signature }
```

Cards are **tokenized by the PSP/vault** in the browser SDK; our servers never see raw PANs (reduces PCI scope).

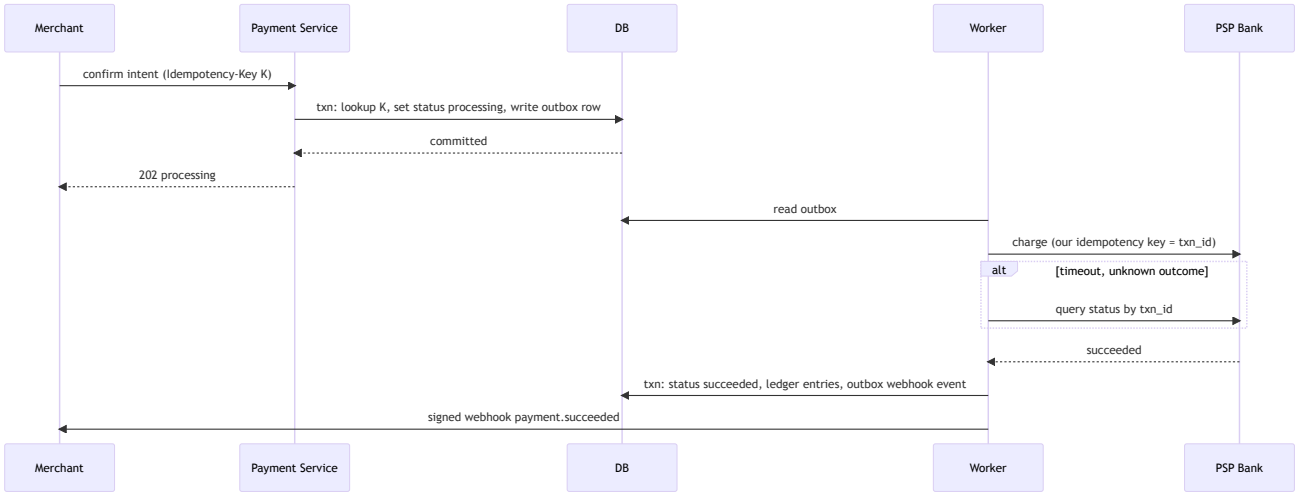
---

6. Architecture



---

7. Flow



---

8. Deep dive 1: idempotency

Approach	Verdict	Why
No keys, retry blindly	Bad	Double charges
Client-supplied `Idempotency-Key` stored with unique constraint, saved response replayed	Great	Retry-safe across network failures
Also pass an idempotency key (`txn_id`) to the **PSP**	Required	Protects the downstream call

Store `(merchant_id, key) -> request hash, response, status` with a **unique index**; the same key with a different body is a 422. Key TTL about 24 h+.

One-liner: *Idempotency key at every hop: merchant to us, us to bank.*

9. Deep dive 2: durability and the dual-write problem

Never "update DB then call Kafka" separately. Use the **transactional outbox**: write state and the outbox event in one DB transaction; a relay publishes events (CDC or poller). The payment is a **state machine** (`created to processing to succeeded|failed`), transitions via conditional updates, so a crash resumes from the persisted state. Use **sagas** with compensations (refund/void) across steps.

10. Deep dive 3: async bank calls and webhooks

- Bank may respond after minutes; support **both** synchronous result and **inbound webhook** from PSP (verify signature, dedupe by event id).
- Timeouts mean **unknown**, never "failed": reconcile by querying the PSP using the idempotency key.
- **Outbound webhooks** to merchants: at-least-once, HMAC-signed, retried with exponential backoff for days, DLQ, event ids for dedupe; merchants can also poll `GET`.

11. Deep dive 4: ledger and reconciliation

- **Double-entry, append-only ledger**: every movement has balanced debit/credit entries; balances derived, never updated in place. Immutability gives auditability.

- **Reconciliation**: nightly compare our ledger with PSP/bank settlement files; flag mismatches (missing, duplicate, amount) for investigation. This is the backstop for any bug.

12. Deep dive 5: security

- **PCI scope reduction**: tokenization/vault, TLS everywhere, encrypt at rest, HSM-managed keys.
- AuthN for merchants (API keys, OAuth), HMAC for webhooks, strict authorization (merchant sees own data).
- Fraud: velocity checks, risk scoring, 3-D Secure; rate limits.
- Least privilege, audit logs, secret rotation.

13. Deep dive 6: scale

Shard payments DB by `merchant_id` (or intent id hash), stateless services, Kafka partitioned by `intent_id` for ordering per payment, read replicas for status reads, archival of old data. Postgres ACID for the core; avoid eventual-consistent stores for money state.

---

14. Failure handling

Failure	Mitigation
Client retries	Idempotency key replay
Crash after bank charge	Resume via state + PSP query by key; outbox ensures events
Bank/PSP outage	Circuit breaker, failover to secondary PSP (routing), queue and retry
Webhook to merchant failing	Backoff retry, DLQ, polling fallback
DB failover	Synchronous replica for payments; RPO = 0
Duplicate PSP callback	Event-id dedupe

---

15. Trade-offs

Decision	Chosen	Alternative	Given up
Store	ACID SQL	NoSQL	Horizontal ease (shard by merchant)
Processing	Async with 202	Synchronous	Immediate answer
Consistency	Strong	Eventual	Availability in partitions
Events	Outbox	Dual write	Relay component
Card data	Tokenize at PSP	Store ourselves	Control; large PCI burden removed

---

16. Level expectations

- **Mid:** payment flow, DB, status API, retries.
- **Senior:** idempotency keys, async/webhooks, state machine, outbox, PCI tokenization.
- **Staff+:** ledger design, reconciliation, PSP failover/routing, saga compensation, audit, multi-region and cost.

17. Follow-ups

Refunds/chargebacks, multi-currency, payouts, subscriptions, fraud scoring, multi-PSP smart routing.

18. Summary checklist

- [ ] Idempotency keys end to end
- [ ] Outbox plus state machine, never dual-write
- [ ] Timeout means unknown, so query to resolve
- [ ] Append-only double-entry ledger and nightly reconciliation
- [ ] Tokenization, signed webhooks, retries

Designing Google Docs (Real-Time Collaborative Editing)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**concurrent edit conflicts: OT vs CRDT, server-ordered operations, WebSocket scaling, presence, storage and snapshots, offline**) → trade-offs. The hard part is **converging concurrent edits from many users without losing intent**.

---

0. Talk track

1. Clarify: create docs, concurrent edit by multiple users, see others' cursors, up to 100 editors per doc; offline out of scope.
2. Name the core problem: **concurrent edits conflict**.
3. Show naive fixes fail (last write wins, locking), arrive at **OT** (or CRDT).
4. HLD: WebSocket collaboration servers, per-document ordering, operation log + snapshots.

---

1. Problem

Many users edit one document at once; everyone sees changes in **real time** and all copies **converge** to the same text.

What makes it hard

- Two users edit simultaneously, so operation positions shift.
- Low latency (under 100 ms perceived) with intention preserved.
- Persist history; reconnect and catch up; scale to **millions of docs and 10M concurrent users**.

---

2. Requirements

Functional

- Create documents.
- **Edit concurrently** with real-time propagation.
- See **cursor and presence** of others.

Out of scope: rich formatting depth, permissions/sharing details, comments, offline editing, version history UI.

Non-functional

- **Eventual consistency**: all clients converge to the same state.
- **Low latency** (under 100 ms updates).
- **Scale**: 10M concurrent editors, 1B docs, up to 100 editors/doc.
- **Durable**; available.

---

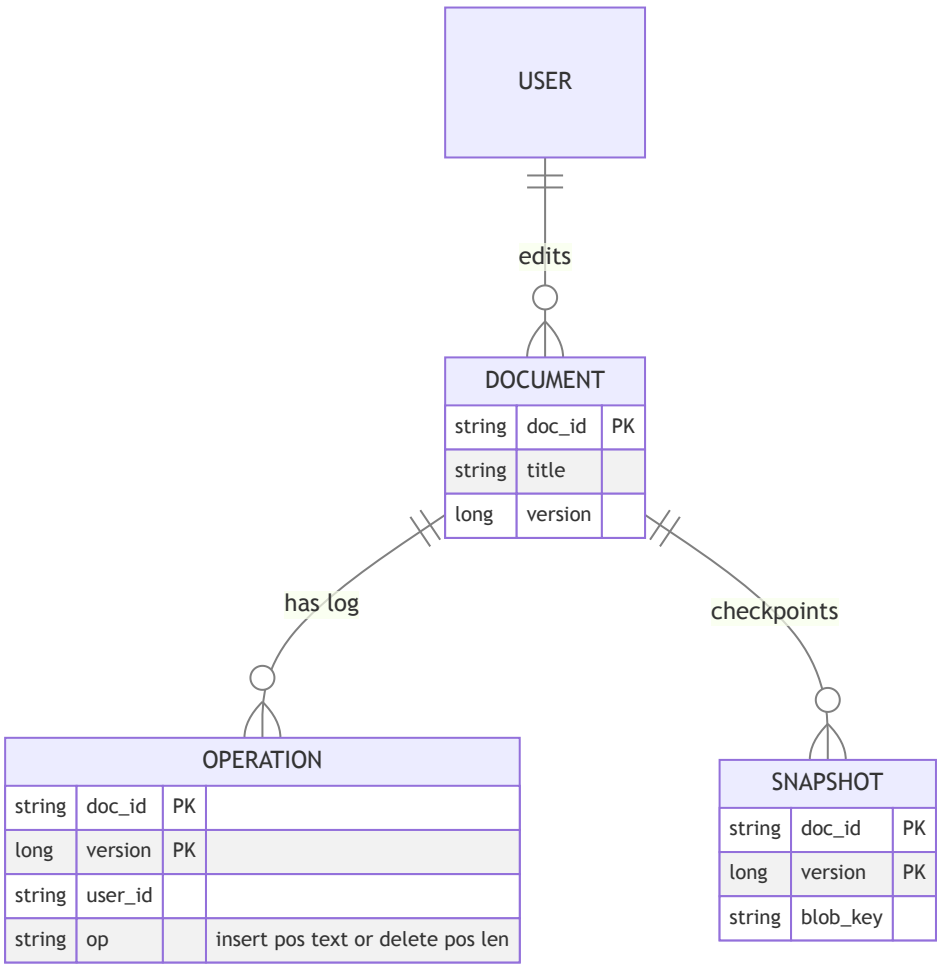
3. Estimation

Quantity	Assumption	Result
Concurrent users	10M	10M WebSockets, about 100K/server = **about 100+ servers**
Ops	1 op/s per active editor	**10M ops/s** worst case; avg typing less
Op size	about 100 B	about 1 GB/s ingress at the extreme
Docs	1B x 50 KB	**about 50 TB** snapshots

Consequence: a document is the **unit of ordering and sharding**. All editors of one doc connect to **the same collaboration server** (or shard).

---

4. Entities



---

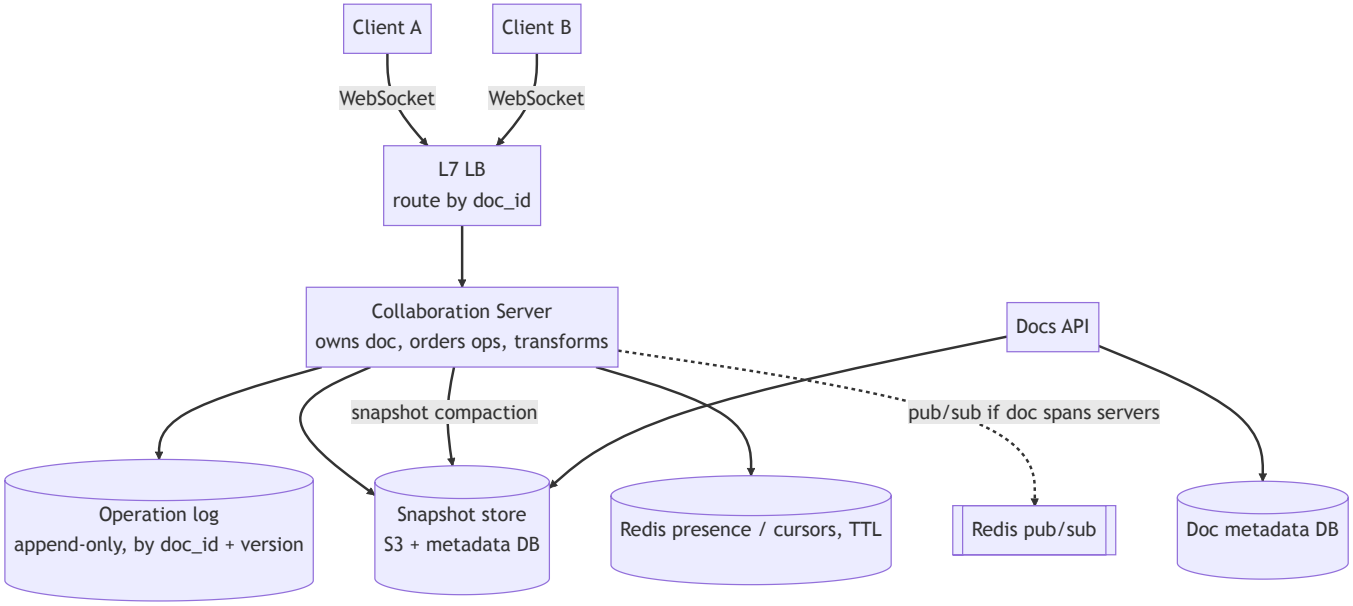
5. API

```
POST /docs                -> 201 { doc_id }
GET  /docs/{id}           -> latest snapshot + ops since snapshot (version)
WS   /docs/{id}/edit
  client -> { type:"op", baseVersion: 41, op: { insert: {pos: 5, text:"x"} } }
```

```
server -> { type:"ack", version: 42 }
server -> { type:"op", version: 43, op: {...}, user:"u2" }       // others' edits (transformed)
client/server -> { type:"cursor", user:"u2", pos: 17 }           // ephemeral presence
```

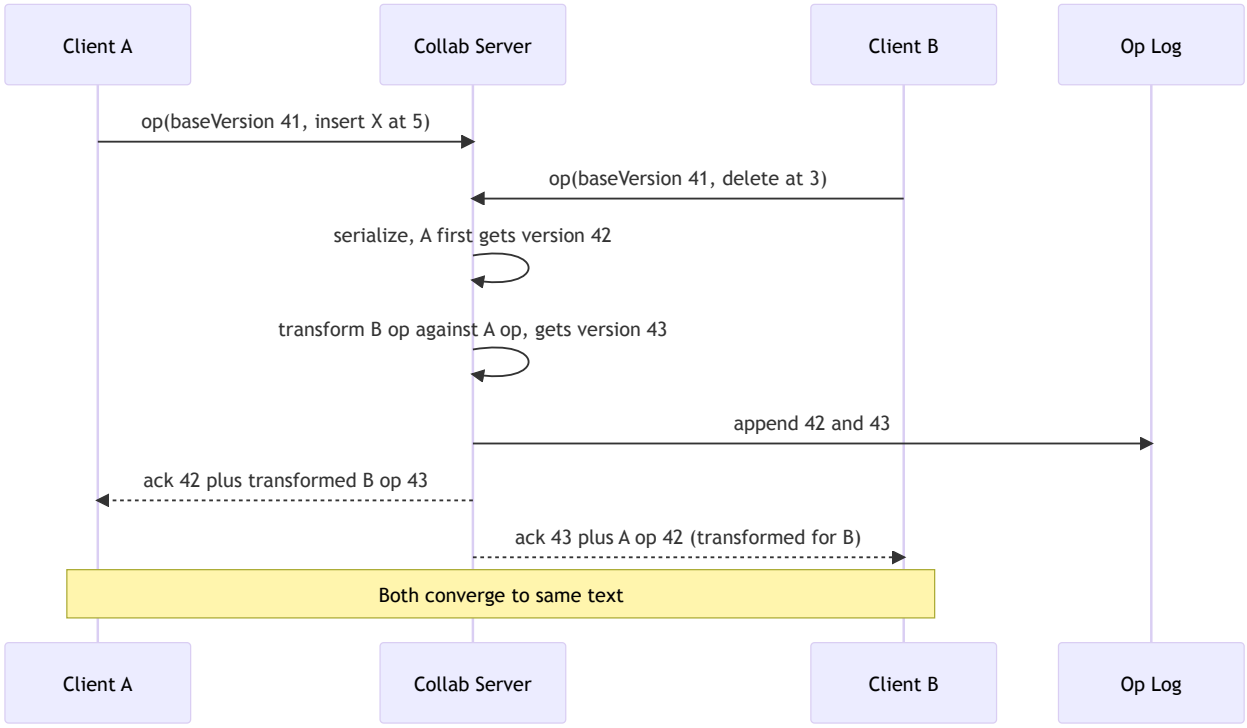
---

6. Architecture



---

7. Flow



---

8. Deep dive 1: concurrent edits

Approach	Verdict	Why
Send whole doc, last write wins	Bad	Loses edits
Locking sections/doc	Bad	Hurts collaboration
Operation-based with **position** only	Bad	Positions are stale after concurrent ops
Operational Transformation (OT) with a central server	Great	Server orders ops, transforms each against concurrent ones; clients converge. Google Docs approach; needs a central authority per doc
CRDT (Yjs, Automerge, RGA)	Great	Operations commute; no central ordering, supports peer-to-peer/offline; metadata overhead and tombstones grow

Example: doc `ABC`; A inserts `X` at 1, B deletes `C` at 2 concurrently. Without transform, B's delete after A's insert hits the wrong char; OT shifts B's index to 3.

One-liner: *Server assigns a total order per doc; transform concurrent ops (OT), or use CRDTs when you need offline/peer-to-peer.*

9. Deep dive 2: scaling WebSockets and routing

- **One doc = one owner server** (consistent hash by `doc_id`), so ordering is local and cheap. The LB routes by `doc_id`.
- Alternative for hot docs: dedicated shard; cap at 100 editors keeps load bounded.
- On server failure, clients reconnect (with last known version), another server **rebuilds state** from snapshot + log tail and resumes; version numbers make resync exact.
- Use a registry (Redis/ZooKeeper) of doc to server with leases to avoid two owners (split brain).

10. Deep dive 3: storage

- **Append-only op log** (durable write before ack), periodic **snapshots/compaction** (every N ops or minutes), so loading = latest snapshot + tail.
- Old ops retained for version history; cold docs archived to S3.
- Persist before ack, or batch with a small WAL; trade latency vs durability window.

11. Deep dive 4: presence and cursors

Ephemeral, not persisted: Redis with TTL or in-memory on the collab server, broadcast on change (throttled, e.g. 10/s). Loss is harmless.

12. Deep dive 5: client behavior

Optimistic local apply immediately; keep a queue of unacked ops; on ack of own op or incoming transformed remote ops, **rebase**. One in-flight op at a time (classic OT client) simplifies the state machine.

---

13. Failure handling

Failure	Mitigation
Collab server crash	Reconnect and reload snapshot + log tail; clients resend unacked ops (idempotent by op id)
Duplicate ops	Op ids/client sequence numbers
Network drop	Client queues, reconnects with last version
Log store failure	Replicated store, multi-AZ
Split brain on ownership	Lease/fencing token on op log writes

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Convergence	OT (central)	CRDT	Offline/P2P; but simpler, smaller state
Ownership	Per-doc server	Stateless any-server	Routing/ownership complexity
Storage	Log + snapshots	Full copies	Replay cost on load
Consistency	Eventual convergence	Strong locking	Transient divergence
Presence	Ephemeral	Persisted	Lost cursors on crash

---

15. Level expectations

- **Mid:** WebSocket updates, saving doc, notices conflict problem.
- **Senior:** OT or CRDT explained with an example, per-doc routing, log + snapshots.
- **Staff+:** server failover with fencing, hot-doc handling, op compaction, CRDT trade-offs, permissions on ops, rich-text structures.

16. Follow-ups

Offline editing (CRDT), undo/redo semantics, comments/suggestions, permissions, version history/restore.

17. Summary checklist

- [ ] Ops (not whole doc) over WebSocket, baseVersion
- [ ] Per-document owner orders and transforms (OT) or CRDT
- [ ] Op log plus snapshot compaction
- [ ] Redis ephemeral presence
- [ ] Reconnect with version, fencing on ownership

Designing LeetCode (Online Coding Judge)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**secure sandboxed code execution, async judging queue, scaling for contests, leaderboard, isolation and resource limits**) → trade-offs. The hard parts are **running untrusted code safely** and **bursty contest load**.

---

0. Talk track

1. Clarify: list problems, view a problem + editor, submit code, get a verdict, contest leaderboard.
2. Focus: execution safety and throughput; reads are easy.
3. HLD: API, problem store, submission queue, sandboxed workers, results.
4. Deep dives: isolation (containers, gVisor/Firecracker), async pipeline, contest scale, leaderboard.

---

1. Problem

Users solve coding problems online: write code, **run it against hidden tests**, and get **verdicts** (accepted, wrong answer, TLE, MLE, runtime error), with live **contest leaderboards**.

What makes it hard

- Executing **arbitrary untrusted code** (fork bombs, file/network access, infinite loops).
- Fast feedback (a few seconds) under bursty contest load (10K+ users submitting at once).
- Fair, consistent ranking.

---

2. Requirements

Functional

- View a problem list and individual problems.
- Write code in multiple languages and **submit**.
- Receive results (verdict, runtime, memory).
- View a **leaderboard** for live contests.

Out of scope: user accounts/auth details, discussion forums, plagiarism checks, custom judges.

Non-functional

- **Security and isolation** of user code.
- **Availability** for reads; correctness and fairness for contest scoring.
- **Execution latency**: under about 5 s typical.
- Scale: **100K concurrent contest users**, 10K submissions/min at peak.

---

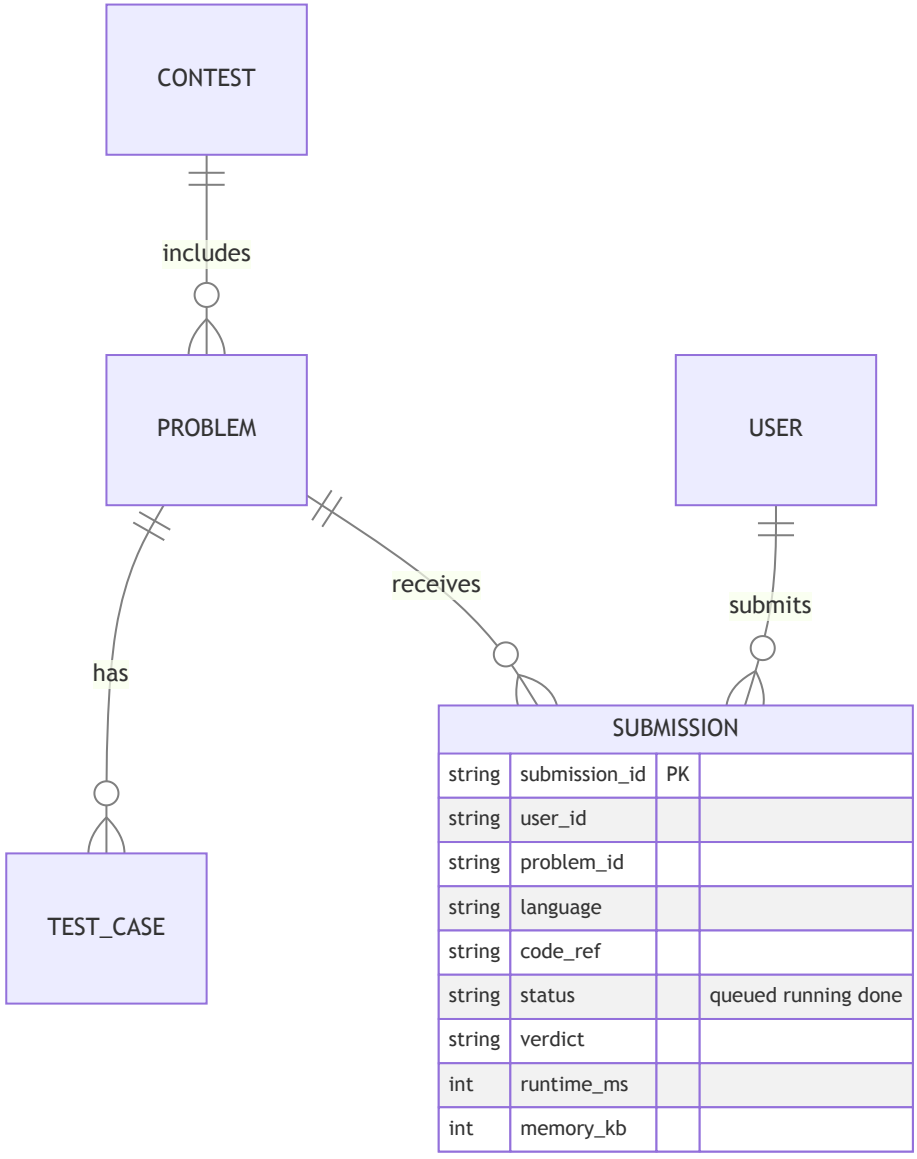
3. Estimation

Quantity	Assumption	Result
Contest users	100K	
Submissions	each about 5 in 2 h	**500K / 2 h**, bursts about 100/s to 1K/s
Run time	about 2-5 s CPU each x multiple tests	1K/s x 3 s = **3K concurrent sandboxes**
Problem data	tens of thousands	small, cache everything

Workload is **CPU-bound and bursty**, so use a **queue** plus **autoscaled worker pool**.

---

4. Entities



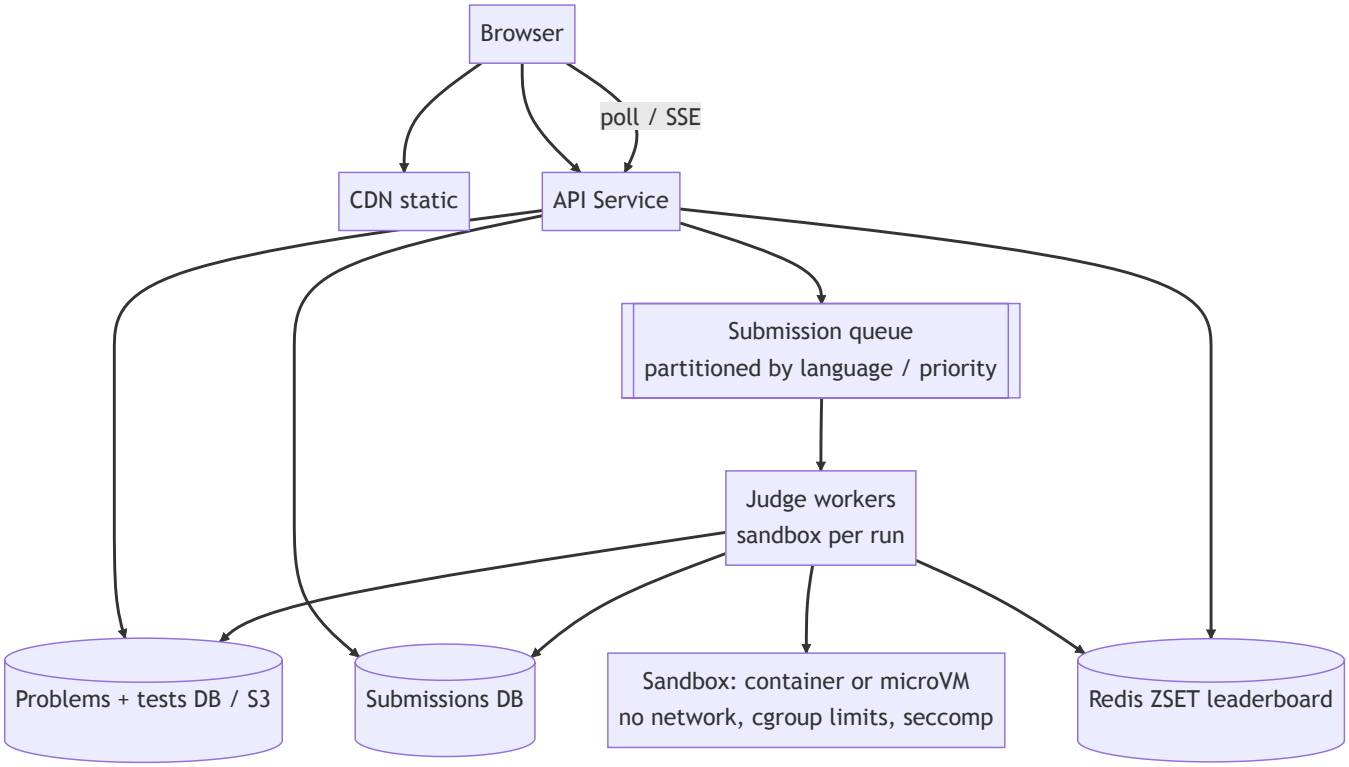
---

5. API

```
GET  /problems?page=&tag=              -> list
GET  /problems/{id}                    -> statement, starter code per language
POST /problems/{id}/submissions        { language, code }  -> 202 { submission_id }
GET  /submissions/{id}                 -> { status, verdict, runtime, memory }   # poll (or WebSocket/
GET  /contests/{id}/leaderboard?cursor= -> ranked users
```

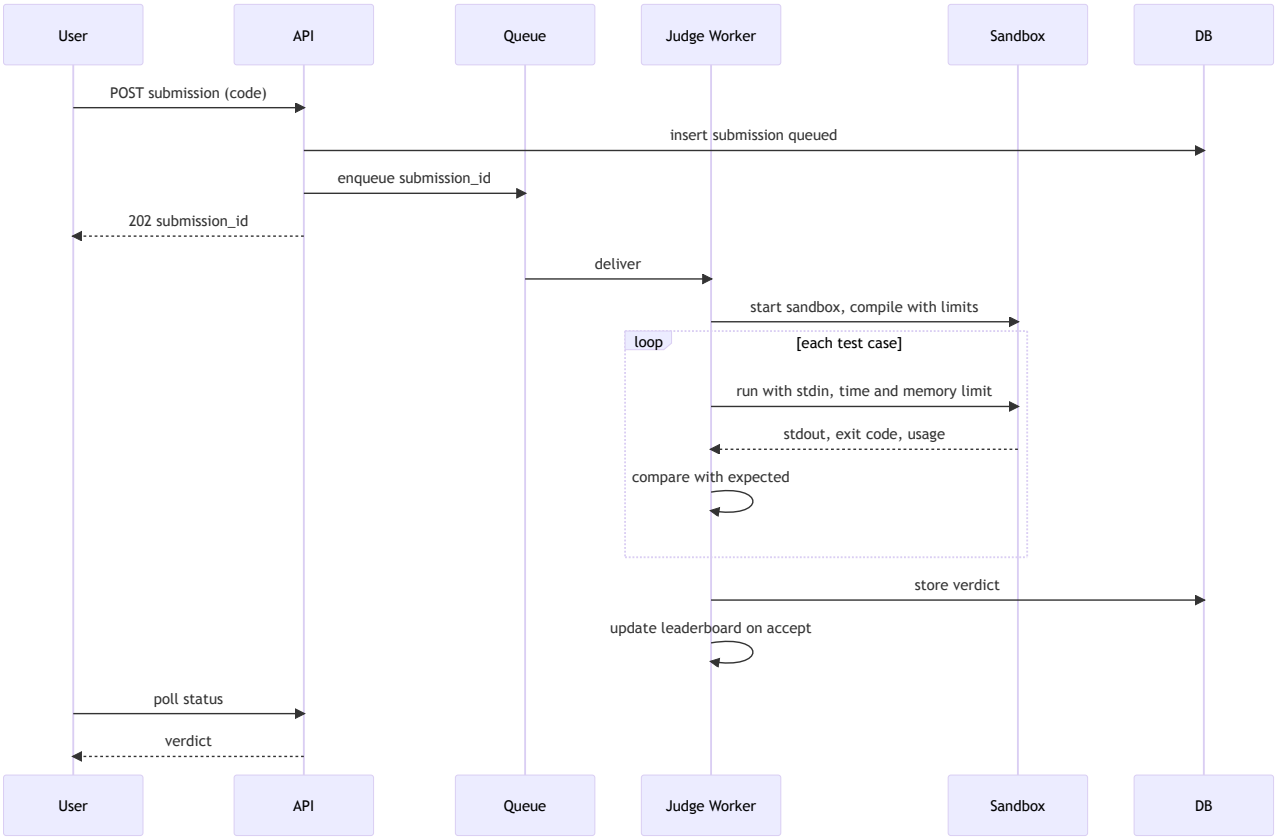
---

6. Architecture



---

7. Flow



---

8. Deep dive 1: safe execution

Approach	Verdict	Why
Run on the app server	Bad	Full compromise
Plain Docker containers	Good	Namespaces + cgroups, but shared kernel risk
Hardened containers (seccomp, no network, read-only FS, non-root, cgroup CPU/mem/pids limits, timeouts)	Great	Practical default
gVisor / Firecracker microVM	Great for hostile workloads	Stronger isolation, small overhead

Rules: no network, read-only root FS, small tmpfs, **memory cap**, **CPU time cap** plus wall-clock timeout (kill), **PID limit** (fork bombs), output size cap, drop capabilities. Fresh sandbox per submission (or reuse warm pool but reset), and **never trust the run result** inside the sandbox process.

One-liner: *Assume the code is hostile: no network, hard resource caps, fresh microVM/container, kill on timeout.*

9. Deep dive 2: async pipeline

Submission is **accepted immediately** (202), judged in the background. **Queue** (SQS/Kafka/Redis) gives buffering during bursts, retries on worker crash (visibility timeout), and priority lanes (contest over practice). Autoscale workers on queue depth; **warm pools** reduce cold start. Results via polling (simple), or SSE/WebSocket push. Idempotent by `submission_id`.

10. Deep dive 3: contest scale

- Pre-scale workers before the contest start; separate **contest worker pool** from practice.
- Cache problem statements/tests at workers; static content on CDN.
- Per-user submission **rate limits**; per-language queues; shed low-priority load.
- Hidden tests stored in S3, pulled and cached on workers.

11. Deep dive 4: leaderboard

Redis **sorted set** per contest: score = solved count and penalty time encoded into one number; `ZADD`/`ZREVRANGE` give rank in O(log n); `ZREVRANK` for a user's rank. Persist truth in the DB and rebuild on failure. Cache top pages; poll every few seconds (no strict real-time need).

12. Fairness and correctness

Deterministic judge hardware (same instance class), time limits relative to reference solution, rerun on suspect timing, per-test result storage, and idempotent score updates keyed by `(user, problem)` first accepted.

---

13. Failure handling

Failure	Mitigation
Worker crash	Message redelivered; submission re-judged
Sandbox escape attempt	Defense in depth (microVM), isolated worker network, no secrets on workers
Queue backlog	Autoscale, shed practice, show queue position
DB failure	Replicated; verdict writes retried (idempotent)
Redis loss	Rebuild leaderboard from submissions

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Isolation	Hardened containers / microVM	VMs per run	Startup cost vs security
Feedback	Async + poll	Sync request	Simplicity of immediate response
Workers	Warm pool	Cold start	Idle cost
Leaderboard	Redis ZSET	SQL ORDER BY	Rebuild logic
Queue	Priority lanes	Single FIFO	Config complexity

---

15. Level expectations

- **Mid:** submit, run in container, store result, simple queue.
- **Senior:** isolation details, queue plus autoscaling, leaderboard with Redis, rate limits.

- **Staff+:** microVMs, contest pre-scaling, fairness/determinism, abuse and cost, observability.

16. Follow-ups

Plagiarism detection, custom checkers/interactive problems, multi-language toolchains, code snapshots, run-against-samples fast path.

17. Summary checklist

- [ ] 202 + queue + autoscaled judge workers
- [ ] Sandbox: no network, cgroups, seccomp, timeouts, pid caps
- [ ] Idempotent verdict writes, retry on worker death
- [ ] Redis ZSET leaderboard
- [ ] Contest pool pre-scaled, rate limits

Designing Yelp (Business Search and Reviews)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**geospatial search, text + location query, rating aggregation, one review per user, read-heavy scaling**) → trade-offs. The hard parts are **location-based search** and **keeping average ratings fresh cheaply**.

---

0. Talk track

1. Clarify: search businesses (name/category/location), view business details, post reviews with rating, one review per user per business.
2. Insight: **read-heavy (100:1)**, search is the core; ratings consistency is relaxed.
3. HLD: business service, review service, search service with geo index.
4. Deep dives: geo search, rating aggregates, uniqueness constraint, search index sync.

---

1. Problem

Users find nearby businesses, read details and reviews, and leave **star ratings + text**.

What makes it hard

- Combine **text relevance + geo-radius + filters** at low latency.
- Average rating must update when reviews change, without scanning all reviews.
- Constraint: **one review per user per business**.

---

2. Requirements

Functional

- Search businesses by name, location, and category.
- View business details (info, reviews, photos).
- Leave reviews (1-5 stars + text).

Out of scope: adding businesses, photos pipeline, recommendations, check-ins, friends.

Non-functional

- **Low latency search** (under 500 ms), **high availability**, **eventual consistency** acceptable for ratings.
- **Read-heavy** (about 100:1).
- Scale: **100M DAU, 10M businesses**.

---

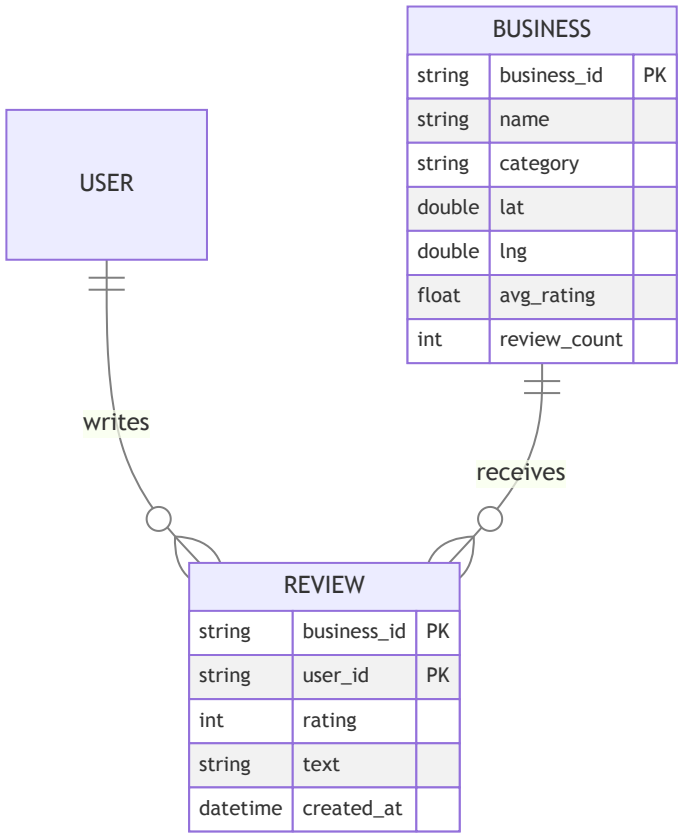
3. Estimation

Quantity	Assumption	Result
Searches	100M DAU x 5	500M/day, **about 6K/s avg, about 20K/s peak**
Reviews	1M/day	about 12/s
Business row	2 KB	10M x 2 KB = **20 GB**
Reviews	500M x 1 KB	**500 GB**
Search index	geo + text for 10M docs	tens of GB: fits a small ES cluster

Consequence: data is small; **query shape** (geo + text) and read QPS drive the design.

---

4. Entities



`(business_id, user_id)` as a **unique key** enforces one review per user.

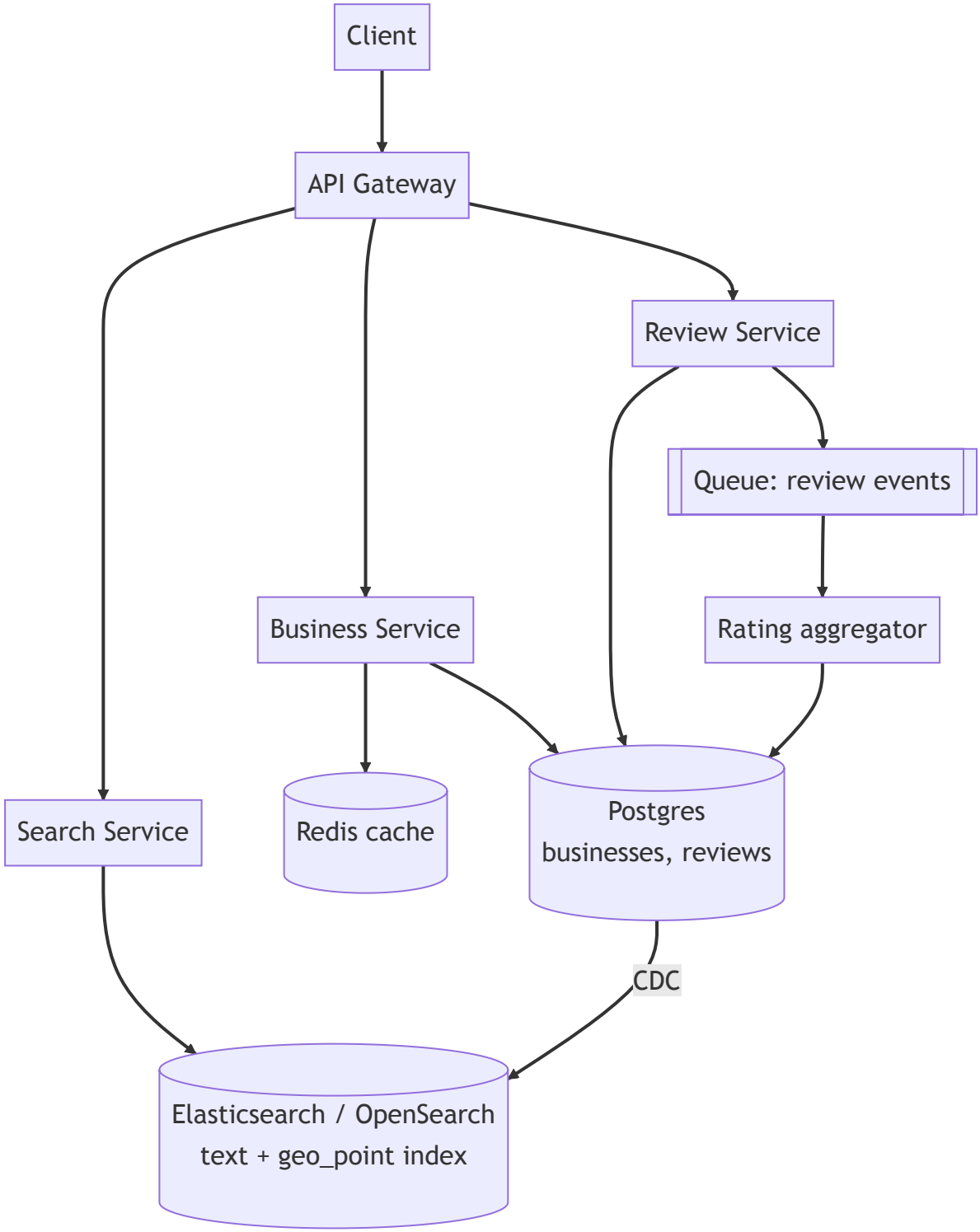
---

5. API

```
GET  /businesses?q=pizza&lat=37.77&lng=-122.41&radius=3km&category=&min_rating=&cursor=
GET  /businesses/{id}                    -> details + avg_rating
GET  /businesses/{id}/reviews?cursor=
POST /businesses/{id}/reviews  { rating: 5, text: "..." }    -> 201 | 409 already reviewed
```

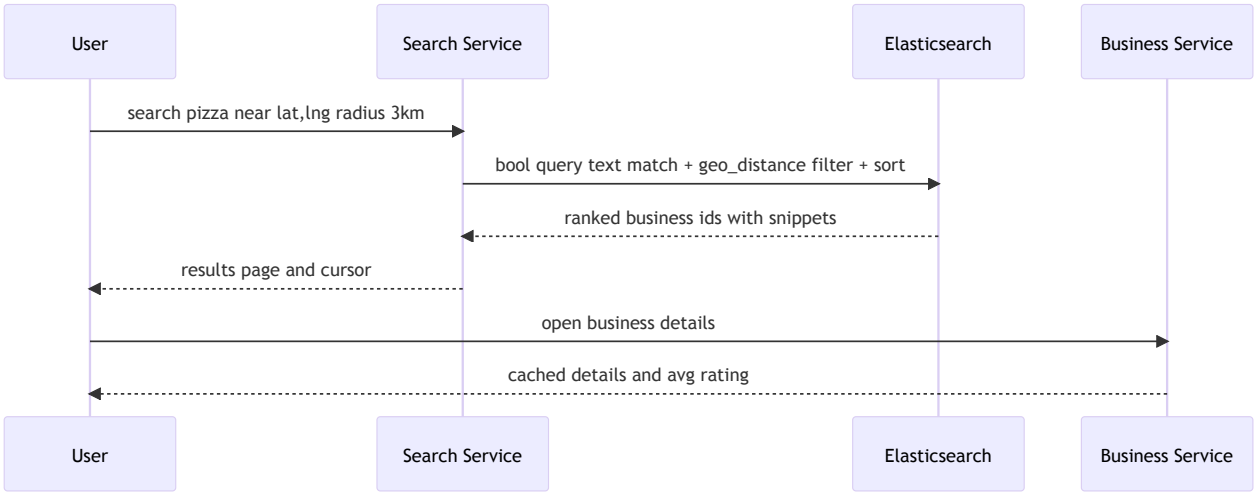
---

6. Architecture



---

7. Flow: search



---

8. Deep dive 1: geo + text search

Approach	Verdict	Why
SQL with `lat/lng BETWEEN` bounding box + `LIKE`	Bad at scale	Poor index use for 2D range plus text
Postgres PostGIS GIST + full-text	Good	Works at moderate scale, one system
Elasticsearch with `geo_point` and text analyzers	Great	Inverted index for text, BKD/geohash for geo; combined filter + relevance
Geohash/quadtree own index	Good	More control; more to build

Geo-index options: **geohash prefix** (lookup neighbors to avoid edge cases), **quadtree**, **S2/H3 cells**. Because businesses are **relatively static**, the index can be updated infrequently (batch/CDC). For dense areas (NYC) cells subdivide; for sparse, expand radius.

One-liner: *Static points + text: put them in a search engine with a geo index; update via CDC.*

9. Deep dive 2: ratings aggregation

Approach	Verdict
`AVG()` over reviews each view	Bad: scans
Incremental update: `new_avg = (old_avg * n + r) / (n + 1)` in the same transaction as the review insert	Good: exact, but hot row for viral businesses
Async aggregation (queue) every few seconds/batch	Great for hot businesses: eventually consistent, no write contention

Store `avg_rating` and `review_count` on the business row; handle **edits/deletes** by adjusting the delta. Occasionally reconcile from reviews. Ratings in ES may lag (ok) and are used for sorting.

10. Deep dive 3: one review per user

Unique constraint on `(business_id, user_id)`; 409 on duplicate. Alternative: allow editing (upsert) and update aggregate by delta.

11. Deep dive 4: scaling reads

- **Cache** business details and popular queries (short TTL); CDN for photos.
- Read replicas for Postgres; ES shards by geo region or hash, replicas for QPS.
- Partition reviews by `business_id`; page with cursor `(created_at, id)`.
- Hot businesses: cache reviews first page; coalesce requests.

---

12. Failure handling

Failure	Mitigation
ES down	Replicas; degrade to Postgres basic search or cached results
Index lag	Acceptable; reindex via CDC replay
Review write to DB fails	Client retry; unique key makes duplicates safe
Aggregator crash	Queue redelivery; idempotent by review_id; periodic recompute
Cache stampede	TTL jitter, singleflight

---

13. Trade-offs

Decision	Chosen	Alternative	Given up
Search	Elasticsearch	Postgres only	Operating two stores, eventual index
Rating update	Async aggregate	Synchronous	Slight staleness
Geo index	geo_point/geohash	Custom quadtree	Control
Store	Postgres	NoSQL	Constraint support is a win; scale fits
Consistency	Eventual for ratings	Strong	Fresh numbers

---

14. Level expectations

- **Mid:** tables, SQL search idea, review uniqueness.
- **Senior:** geo indexing choice, ES, caching, rating aggregation.
- **Staff+:** CDC sync, hot business handling, density-adaptive indexing, ranking relevance, fraud/spam reviews.

15. Follow-ups

Review spam/fraud detection, photos pipeline, personalization, business owner replies, map view clustering.

16. Summary checklist

- [ ] 100:1 reads; small data (500 GB), Postgres + replicas
- [ ] ES geo + text for search, CDC sync
- [ ] Unique (business_id, user_id)
- [ ] Incremental/async rating aggregates
- [ ] Caching and cursor pagination

Designing Strava (Fitness Activity Tracking)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**offline-first recording, GPS ingest, activity feed and fan-out, segments/leaderboards, storage of time-series routes**) → trade-offs. The hard parts are **offline tolerance on the client**, **GPS data volume**, and **social feed + leaderboards**.

---

0. Talk track

1. Clarify: record activities (run/ride) with GPS, view live stats, upload, share with friends, view friends' activities. Out: routes/segments (may add), coaching.
2. Client-side first: **recording happens on device, offline**; server stores and shares.
3. HLD: upload service, activity DB, feed, notifications.
4. Deep dives: GPS storage, offline sync, feed scaling, leaderboards.

---

1. Problem

Athletes track activities (GPS route, pace, distance, elevation), upload them, and **share/compete** with friends.

What makes it hard

- GPS tracks are time-series blobs (thousands of points).
- Recording must work **offline**; sync later and exactly once.
- Social feed with fan-out; **segment leaderboards** need geospatial matching.

---

2. Requirements

Functional

- Users **start, pause, stop** activities and see **live stats** (distance, time, pace, route).
- Users view **past activities** and details.
- Users **follow** friends and see their activities.

Out of scope: routes planning, premium features, clubs, kudos/comments (mention).

Non-functional

- **Availability over consistency**; feed eventual.
- **Offline-first** client.
- Scale: **10M DAU**, 1 activity/day/active user.
- Low-latency feed and history.

---

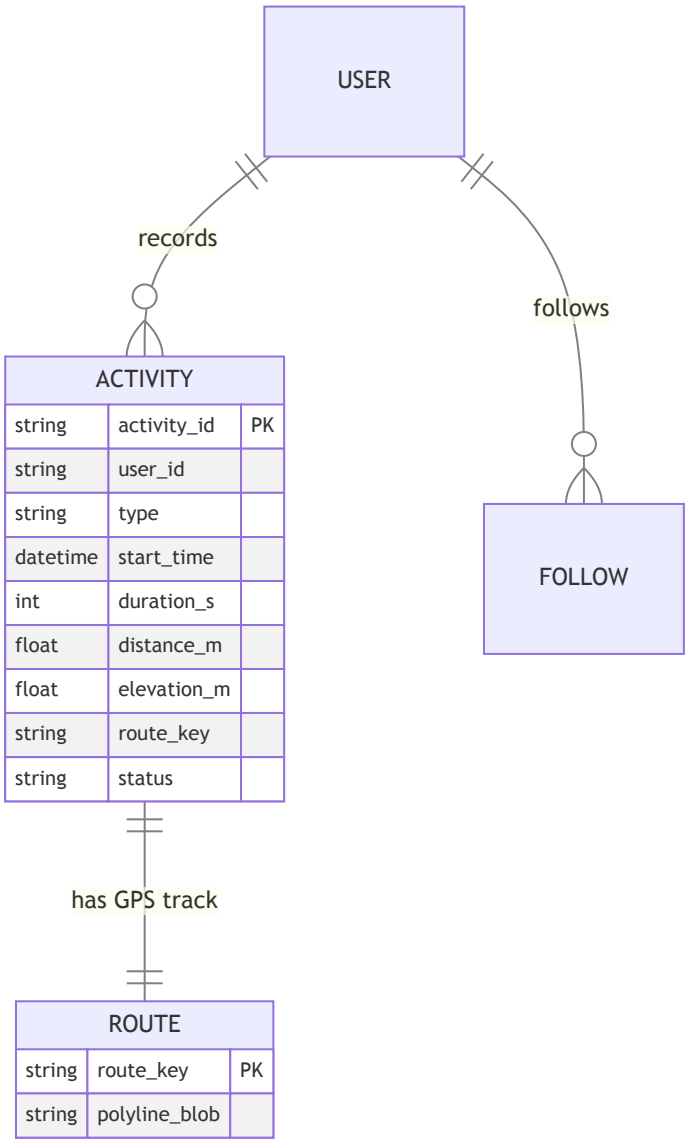
3. Estimation

Quantity	Assumption	Result
Activities	10M/day	about 115/s avg
GPS points	1 Hz, 1 hour: 3600 x 30 B	**about 100 KB per activity**
Daily GPS	10M x 100 KB	**1 TB/day**
Metadata	2 KB	20 GB/day
Yearly		about 365 TB of tracks: object storage

Consequence: **tracks in blob/object store** (or columnar time-series), metadata in a DB. Write rate is low; payloads are big.

---

4. Entities



---

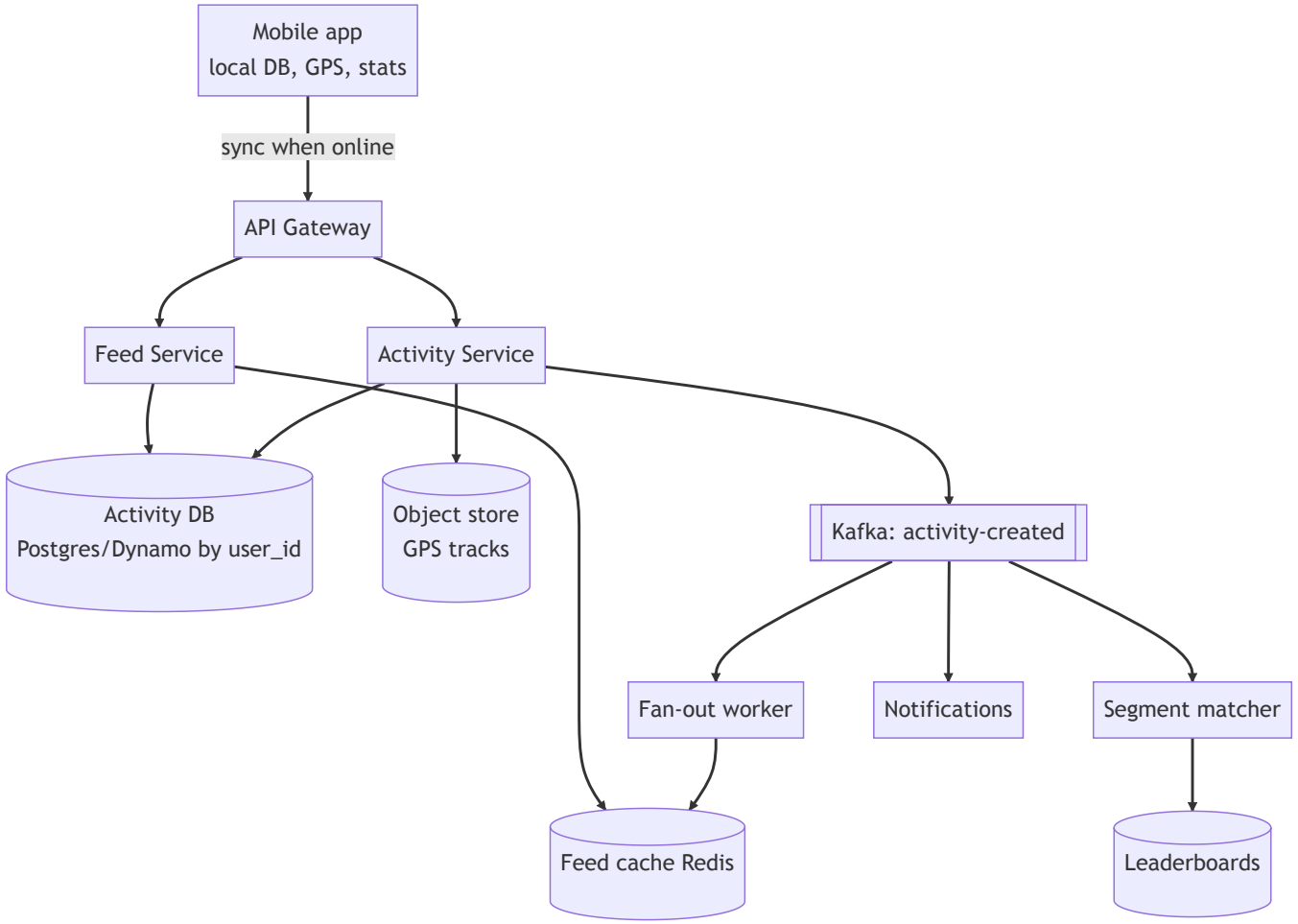
5. API

```
POST /activities                          { activity_id (client UUID), type, start_time }        -> 201
PATCH /activities/{id}   { state:"paused|resumed|stopped", points:[{ts,lat,lng,alt}], stats }  # perio
POST /activities/{id}/route               # upload full GPS track (multipart/chunked presigned) at the e
GET  /activities/{id}                     -> details + route url
GET  /feed?cursor=                        -> friends' activities
```

Client-generated **activity id** makes uploads **idempotent** (retry after reconnect).

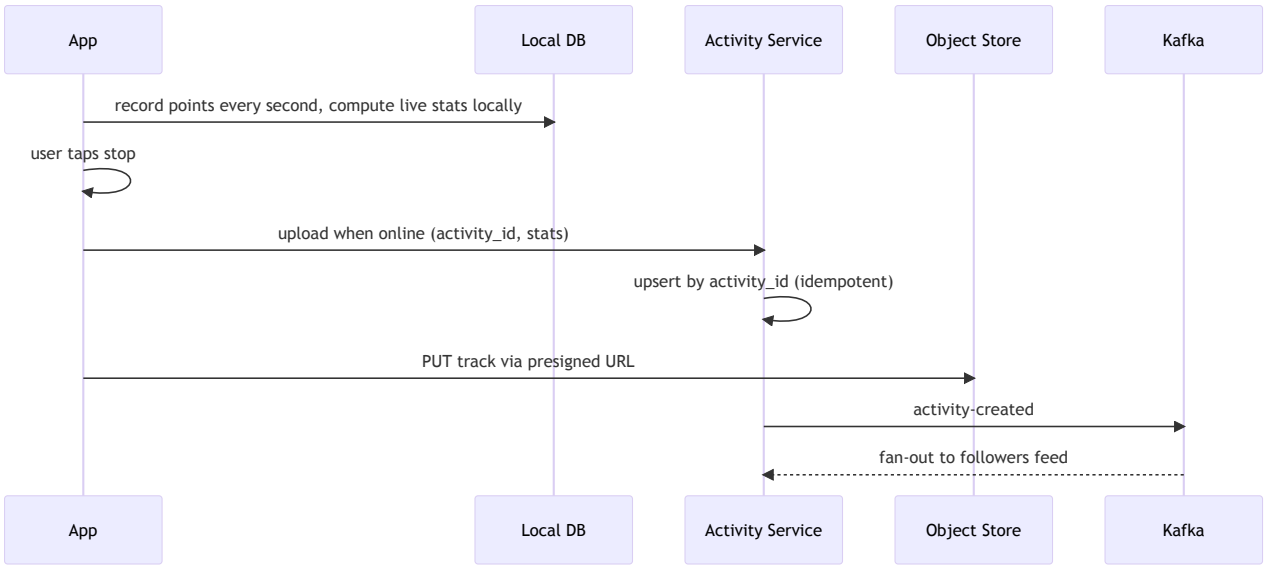
---

6. Architecture



---

7. Flow



---

8. Deep dive 1: offline-first recording

- All live metrics (distance, pace) are computed **on-device** (no network dependence; GPS + sensors continuously).
- Persist points to a **local database** every few seconds (crash-safe).
- Sync when connectivity returns; background upload with retry; **idempotent by client UUID**.
- For optional live sharing: periodically push batches over HTTP/WebSocket; server treats as best effort.
- Resolve conflicts: server is append-only for an activity; client wins for its own data.

One-liner: *The phone is the source of truth until sync; the server is idempotent.*

9. Deep dive 2: storing GPS data

Approach	Verdict
One DB row per point	Bad: 36B points/day
Blob per activity (compressed polyline / protobuf delta-encoded) in object store	Great: 1 write, cheap, fast fetch
Time-series/geo DB for analytics	Good add-on for segments, heatmaps

Downsample for map display (Douglas-Peucker) and keep raw for analysis; thumbnails/static map images pre-rendered.

10. Deep dive 3: feed

Same trade-off as news feed: friends typically few, so **fan-out on write** into per-user Redis feed lists; skip fan-out for accounts with huge followings (pull at read); keep **ids only** and hydrate. Pagination by cursor on `start_time`/id. Eventual consistency (seconds) is fine.

11. Deep dive 4: segments and leaderboards (extension)

Segment = predefined stretch of road. Match activities to segments by geohash/cell overlap, then verify by path distance; write efforts to a per-segment sorted set (`ZADD` by time). Do it **async** off the Kafka event.

12. Storage and scaling

Activity metadata sharded by `user_id`; tracks in S3 with lifecycle (infrequent access for old); CDN for static map images; read replicas for history.

---

13. Failure handling

Failure	Mitigation
Phone offline or crash	Local persistence; resume recording; sync later
Duplicate uploads	Idempotent by `activity_id`
Upload partially complete	Chunked/resumable via presigned multipart
Fan-out lag	Eventual feed; metrics on lag
Service outage	Client retries with backoff; recording unaffected

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Source of truth during activity	Device	Server live	Real-time sharing precision
Track storage	Blob	Row per point	Point-level queries (use secondary index)
Feed	Push fan-out	Pull	Write amplification
Consistency	Eventual	Strong	Fresh feed
Compression	Polyline/delta	Raw	CPU, fidelity (keep raw offline)

---

15. Level expectations

- **Mid:** upload activity, store in DB, friends feed.
- **Senior:** offline-first client, idempotent sync, blob storage for tracks, fan-out feed.
- **Staff+:** segment matching at scale, battery/sampling trade-offs, data lifecycle/cost, privacy zones.

16. Follow-ups

Live tracking (beacon), privacy zones, heatmaps, challenges, device integrations (Garmin), anti-cheat for segments.

17. Summary checklist

- [ ] On-device recording and local DB
- [ ] Client UUID for idempotent upload
- [ ] Object store for tracks, DB for metadata
- [ ] Kafka fan-out to feed cache; async leaderboards

- [ ] Lifecycle tiering of old tracks

Designing Robinhood (Stock Brokerage)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**live price fan-out to millions, order management and exactly-once, exchange integration, consistency and the ledger, spikes**) → trade-offs. The hard parts are **streaming prices to many clients** and **correct, idempotent order lifecycle with an external exchange**.

---

0. Talk track

1. Clarify: view live stock prices, place/cancel market and limit orders, view orders/portfolio. Exchange integration assumed (brokerage only).
2. Split: **read path** (prices, 20M DAU) vs **write path** (orders, strongly consistent).
3. HLD: order service + DB, exchange gateway, price feed fan-out.
4. Deep dives: price streaming (SSE/WebSocket + pub/sub), order state machine, exchange async updates, spike handling.

---

1. Problem

A brokerage app that lets users **watch live quotes**, **trade** through an external exchange, and track positions.

What makes it hard

- **Price updates** for thousands of symbols to millions of watching users.
- **Orders must not be duplicated or lost**; exchange is asynchronous and external.
- Market-open **spikes** (100x).

---

2. Requirements

Functional

- See **live prices** of stocks.
- Manage orders: **place, cancel** (market/limit), view status.

Out of scope: after-hours, ACH funding, options, advanced charts, ML.

Non-functional

- **High consistency for orders** (users' money).
- **Scale**: 20M DAU, about 5 trades/day each, about 10K symbols.
- **Low latency** price display (under 200 ms), order submit under about 200 ms.
- **Fault tolerant**; handle exchange partial outages.

---

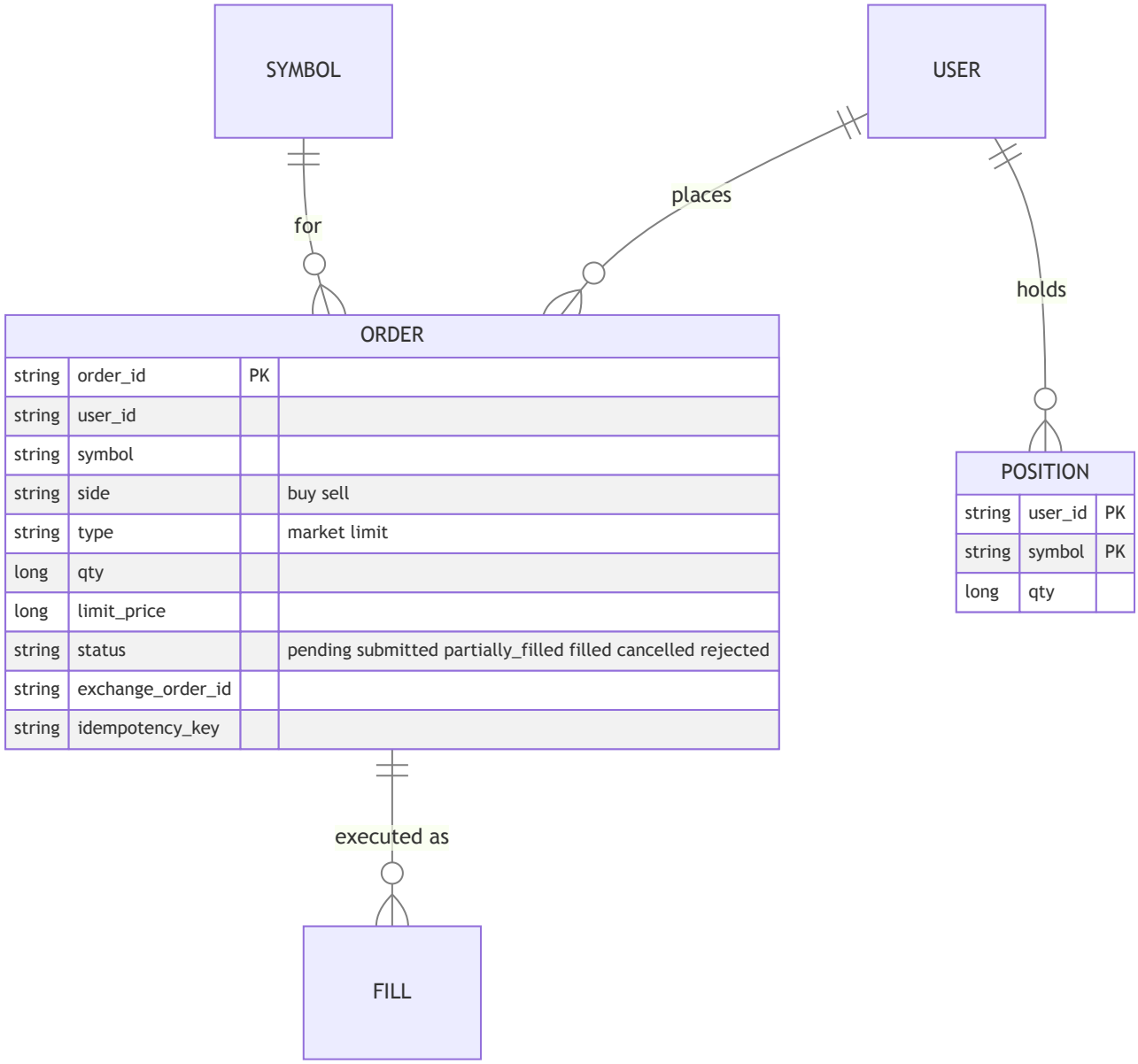
3. Estimation

Quantity	Assumption	Result
Orders	20M DAU x 5	**100M/day, about 1.2K/s avg, about 10K+/s at open**
Symbols	about 10K, updates up to 10/s	**100K exchange ticks/s** inbound
Price subscribers	20M DAU, each watching 5-20 symbols	**fan-out in the tens of millions of messages/s peak**
Order row	1 KB	100 GB/day

Consequence: ingest of prices is modest; **fan-out** is the heavy part. Order QPS is modest but needs **ACID**.

---

4. Entities



---

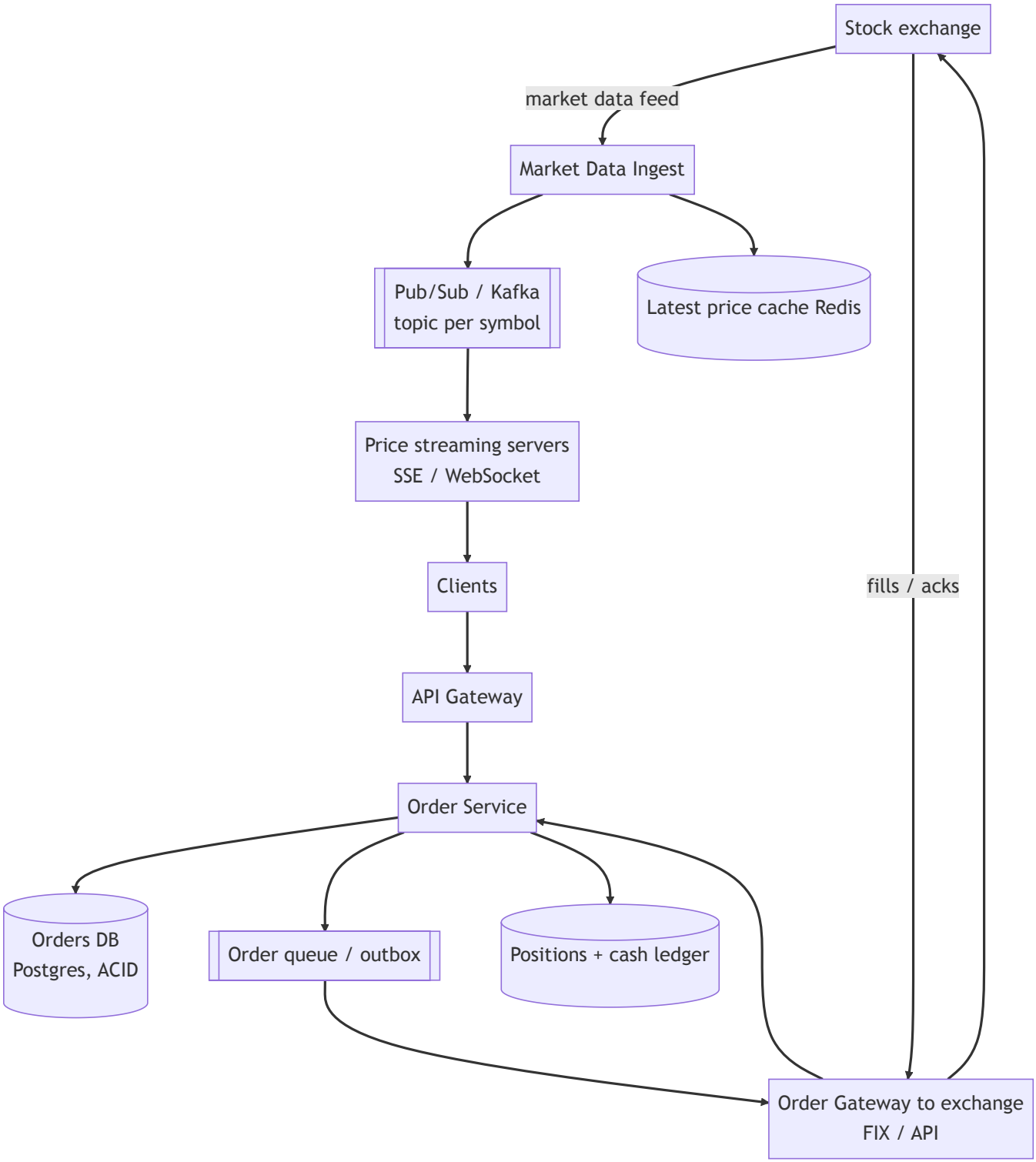
5. API

```
GET  /symbols/{sym}/price                        # snapshot (initial load)
SSE  /stream?symbols=AAPL,TSLA                   # live price updates (server push)
POST /orders  Idempotency-Key: <uuid>  { symbol, side, type, qty, limit_price }  -> 201 { order_id, st
DELETE /orders/{id}                               -> 202 cancel requested
GET  /orders?cursor=   GET /orders/{id}
```

User id from auth, never from the body.

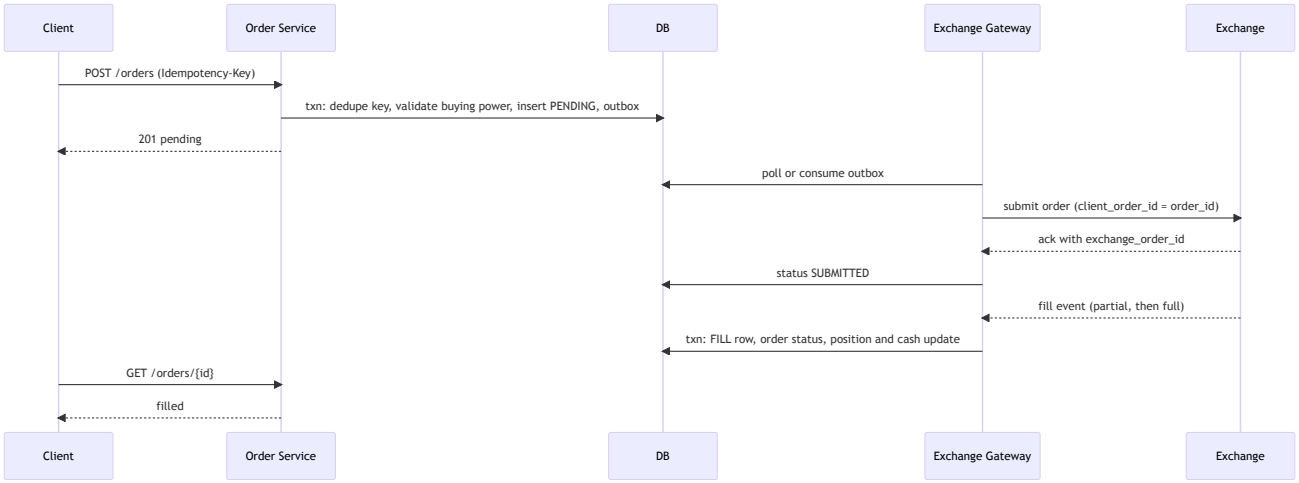
---

6. Architecture


Stock exchange
market data feed
Market Data Ingest
Pub/Sub / Kafka
topic per symbol
Latest price cache Redis
Price streaming servers
SSE / WebSocket
Clients
fills / acks
API Gateway
Order Service
Orders DB
Postgres, ACID
Order queue / outbox
Positions + cash ledger
Order Gateway to exchange
FIX / API

---

7. Order flow



---

8. Deep dive 1: live prices

Approach	Verdict	Why
Client polls every second	Bad	20M x 1/s = 20M req/s; stale
Server push (SSE/WebSocket) from price servers subscribed to a **pub/sub per symbol**	Great	Efficient; fan-out at the edge tier
Add **conflation** (send latest price at most every 200 ms) and coalesce per symbol	Great	Bounds outbound rate regardless of tick rate

Each streaming server subscribes (once) to only symbols its connected clients watch, keeping a `symbol to set of connections` map. Clients subscribe/unsubscribe on screen change. Use a latest-price cache for initial render and reconnects. SSE is simpler than WebSocket since traffic is **one-way**.

One-liner: *Pub/sub per symbol, edge servers fan out to sockets, conflate ticks.*

9. Deep dive 2: order consistency

- **ACID DB** (Postgres), partition by `user_id`. Buying-power check and order insert in one transaction.
- **Idempotency key** per order prevents duplicates from retries.
- **State machine** with conditional transitions; persist before calling the exchange (outbox).
- Exchange `client_order_id = order_id` makes submissions idempotent at the exchange.
- Cancel is a request; terminal state arrives asynchronously.

10. Deep dive 3: asynchronous exchange updates

Exchange replies arrive via callbacks/streams: the **gateway** consumes, maps to internal orders, and applies **idempotent** updates (dedupe by exec id). If the gateway restarts it reconciles by querying open orders. Also run end-of-day **reconciliation** between exchange reports and our ledger.

11. Deep dive 4: spikes at market open

Queue between order service and exchange gateway to smooth bursts; autoscale stateless services; rate limits per user; shed non-critical load (charts) first; pre-warm caches; isolate price streaming from order path so one cannot starve the other.

12. Failure handling

Failure	Mitigation
Streaming server dies	Client reconnects (SSE auto-reconnect), resubscribes, gets snapshot
Exchange link down	Orders stay PENDING in queue; user notified; no silent loss
Duplicate submission	Idempotency key and client_order_id
Order service crash mid-flow	Outbox + state machine resume
DB failover	Synchronous replica; strong consistency over availability
Position mismatch	Reconciliation and ledger audit

---

13. Trade-offs

Decision	Chosen	Alternative	Given up
Prices	Push + conflation	Poll	Connection management; exact tick-by-tick
Orders DB	Postgres ACID	NoSQL	Auto-scaling ease
Order submit	Async via outbox	Synchronous call	Immediate fill info
Transport	SSE	WebSocket	Bidirectional (not needed)
Consistency	Strong for orders, eventual for prices	All strong	Cost

---

14. Level expectations

- **Mid:** order table, price service, cache, polling or WebSocket.
- **Senior:** push fan-out via pub/sub, idempotent order lifecycle, exchange integration, spike handling.
- **Staff+:** conflation, reconciliation/ledger, exchange failure modes, regulatory/audit trail, multi-region.

15. Follow-ups

Margin/risk checks, order types (stop), options, fractional shares, tax lots, market-data licensing costs.

16. Summary checklist

- [ ] SSE/WebSocket price streaming from per-symbol pub/sub, conflated
- [ ] ACID order DB, idempotency key, state machine
- [ ] Outbox to exchange gateway, `client_order_id`
- [ ] Idempotent fill processing and reconciliation
- [ ] Queue smoothing for open spikes

Designing Tinder (Swipe-Based Matching)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**profile feed generation by location/preferences, swipe write path, mutual-match detection and consistency, avoiding repeats, scale**) → trade-offs. The hard parts are **efficient candidate feeds** and **detecting a mutual match reliably under concurrent swipes**.

---

0. Talk track

1. Clarify: profile with preferences, view a **feed of candidates**, swipe right/left, **match** when both right, notify. Chat out of scope.
2. Highlight: swipes are **high-volume writes**; matches need consistency; feed is a **geo + preference** query.
3. Deep dives: feed generation, no repeats (seen set), swipe storage and match check, notifications.

---

1. Problem

Show users nearby profiles that fit their filters; record swipes; when two users swipe right on each other, create a **match** and notify both.

What makes it hard

- **Feed**: location + age/gender filters + not-yet-swiped, at low latency for 20M DAU.
- **Swipe volume**: about 2B/day.
- **Match race**: A and B swipe right at nearly the same time; neither check must miss the other.
- Avoid re-showing seen profiles.

---

2. Requirements

Functional

- Create a profile with preferences (age range, distance, gender).
- View a **stack of profiles** matching preferences.
- **Swipe** like/pass on profiles.
- Get a **match** notification when mutual like.

Out of scope: chat, boosts/payments, photo verification, reporting.

Non-functional

- **Consistent matching** (a mutual like must always produce a match, exactly one).
- **Low latency feed** (under 300 ms), instant swipe response.
- **Scalable**: 20M DAU, about 100 swipes/day, so **2B swipes/day**.
- **Availability** over strict feed freshness.

---

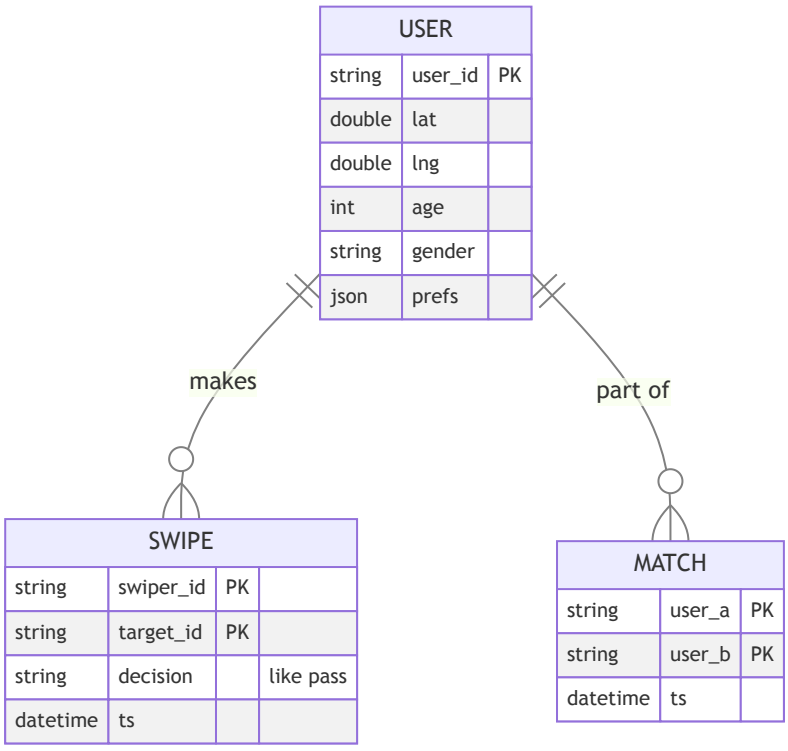
3. Estimation

Quantity	Assumption	Result
Swipes	20M x 100	2B/day, **about 23K/s avg, about 100K/s peak**
Swipe record	40 B	80 GB/day, about 30 TB/year
Profiles	100M x 5 KB	500 GB (photos in object store/CDN)
Feed requests	1 per 10 swipes	about 2.3K/s
Right-swipe rate	about 5-30%	matches are rare events

Consequence: swipes are **write-heavy and key-value shaped** (use a scalable store); feed reads are moderate but need a **geo index**.

---

4. Entities



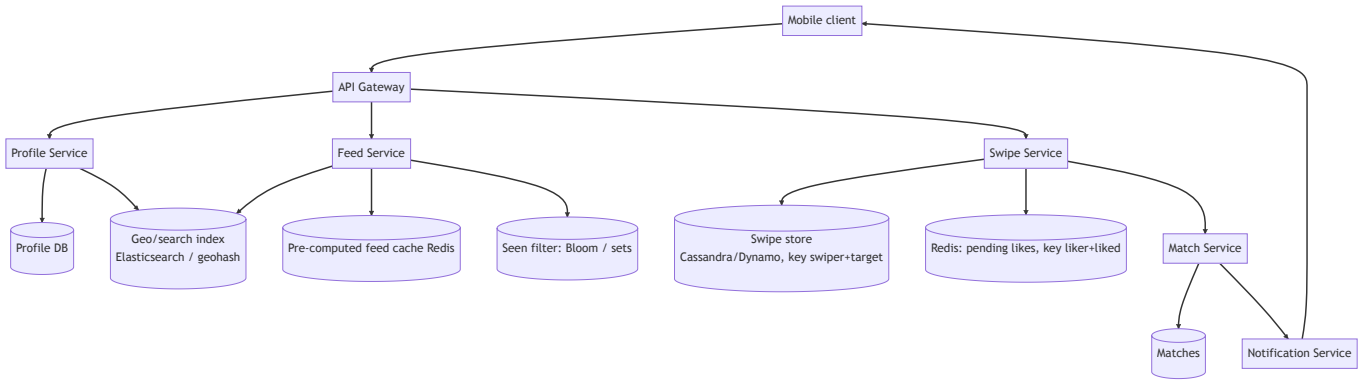
---

5. API

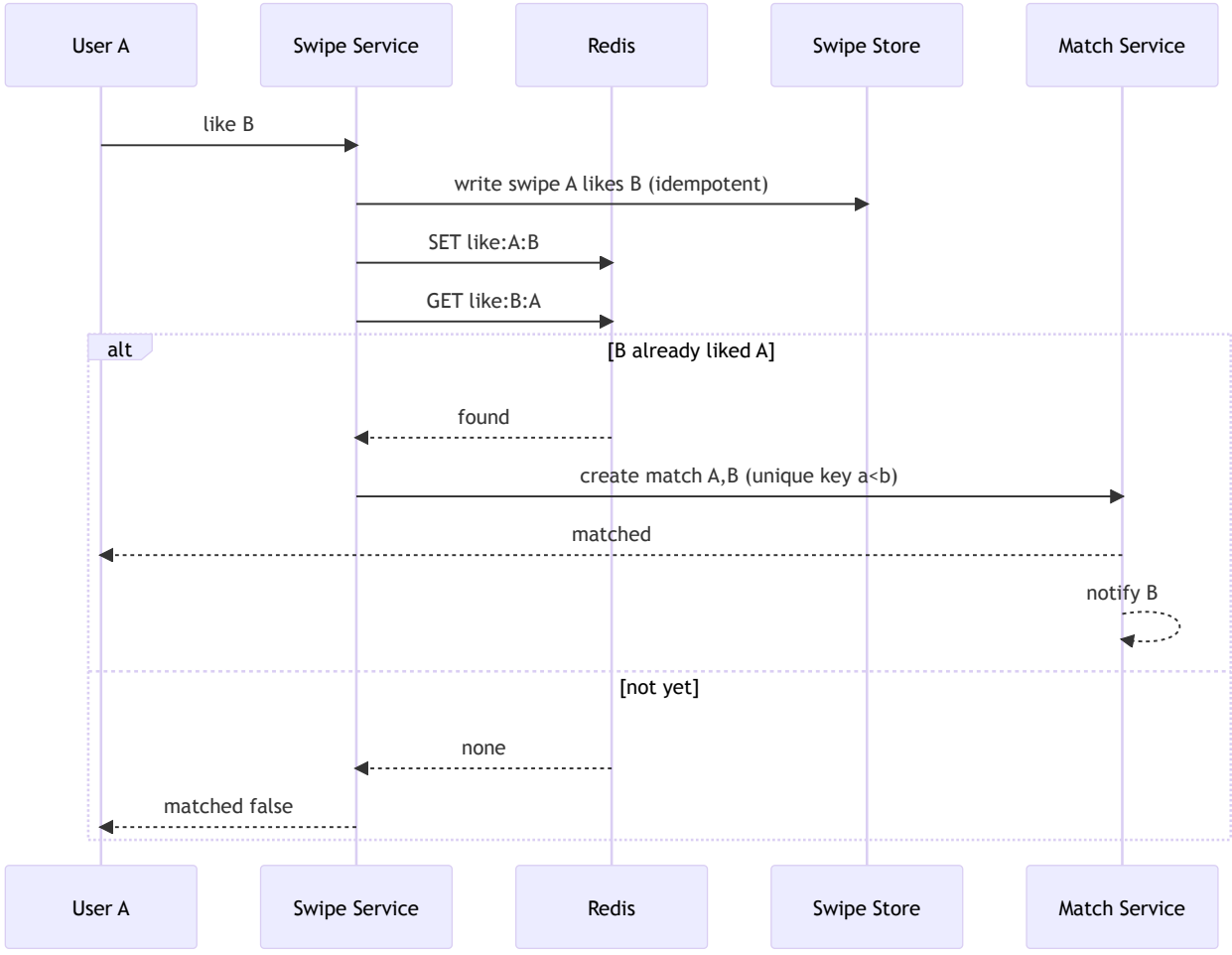
```
GET  /feed?lat=&lng=                    -> [ profile, ... ] (about 20, cursor-less, server tracks seen
POST /swipe   { target_id, decision:"like|pass" }    -> 200 { matched: true|false }
GET  /matches?cursor=
PUT  /profile  { prefs, ... }
```

Swipe is **idempotent** on `(swiper_id, target_id)`.

6. Architecture



7. Swipe and match flow



8. Deep dive 1: generating the feed

Approach	Verdict	Why
SQL with lat/lng bounding box + filters + `NOT IN swiped`	Bad	Slow at 100M users; huge NOT IN
Geo/search index (Elasticsearch geo_point + filters on age/gender)	Good	Efficient candidate retrieval
Pre-computed feed cache (batch generate candidate stacks per user, served from Redis; refresh when low)	Great	Fast; absorbs query cost; tolerate staleness (profiles change slowly)

Ranking: simple score (distance, activity recency, attractiveness/Elo-like, mutual fit). Geo cells (geohash/H3) shard the index by city. Users move slowly, so location updates are infrequent and batch-indexed.

9. Deep dive 2: don't show repeats

- Store swiped ids per user; filter candidates via a **seen set**. For heavy swipers (100K+ ids) use a **Bloom filter** per user (small false positive = skip a profile, acceptable) cached in Redis.
- Alternatively keep a per-user "swiped ids" in the swipe store and filter in the feed service after candidate retrieval (over-fetch, then filter).
- Reset or expire history after N months to allow re-surfacing.

10. Deep dive 3: swipe storage and mutual match consistency

- **Store**: Cassandra/DynamoDB keyed by `(swiper_id, target_id)`: simple, high write throughput, idempotent upsert.
- **Match detection**, three options:

Approach	Verdict	Why
Check DB on every like (read the reverse swipe)	Good	Simple; race: both write, both read before the other's write visible, then **missed match**
Atomic in Redis: store likes in a hash/set, `SADD`/Lua script that checks reverse and writes atomically	Great	Single-threaded atomicity per key; resolves the race
DB transaction with serializable isolation on the pair	Good	Correct, lower throughput

Pair partitioning: route both directions of a pair to the **same shard** (`key = min(A,B):max(A,B)`) so the check-and-set is local. Match row uses a **unique key on the ordered pair** so duplicates are impossible.

One-liner: *Put both directions of a pair on one shard and check-and-set atomically.*

11. Deep dive 4: notifications

Match event to Kafka, push through APNs/FCM; if the other user is in the app, WebSocket push. Notification is best effort (the match is persisted; shown at next open).

12. Deep dive 5: scale

Swipe path stateless and sharded by pair key; feed cache regenerated in background; profile photos on CDN; rate limit swipes (anti-bot, free-tier daily cap) via Redis counters.

---

13. Failure handling

Failure	Mitigation
Redis loss	Rebuild pending-like state from swipe store (recent window); reverse check falls back to DB
Duplicate swipe	Idempotent upsert
Missed match due to crash between steps	Write swipe first (durable), then check; reconcile job re-evaluates recent likes
Feed cache stale	Acceptable; refill on low watermark
Geo index lag	Acceptable

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Feed	Precomputed + geo index	On-demand query	Freshness, storage
Seen tracking	Bloom filter	Exact set	Rare skipped profile
Match check	Atomic Redis on pair shard	DB read-after-write	Extra store, rebuild logic
Swipe store	Wide-column	SQL	Ad hoc queries
Consistency	Strong for matches	Eventual	Latency/complexity

---

15. Level expectations

- **Mid:** profile + swipe tables, geo query, match on reverse swipe.
- **Senior:** feed precompute, seen filtering, swipe write scalability, match race.
- **Staff+:** pair-sharded atomic matching, Bloom filters, ranking pipeline, anti-abuse, multi-region.

16. Follow-ups

Super-like, rewind, boosts, ranking by ML, chat post-match, safety/reporting.

17. Summary checklist

- [ ] 2B swipes/day, KV swipe store keyed by pair
- [ ] Precomputed geo-filtered feed cache
- [ ] Seen filter (Bloom) to avoid repeats
- [ ] Pair-local atomic check for mutual like, unique match key
- [ ] Async push notification

Designing Google Calendar

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**recurring events and exceptions, time zones and DST, availability/conflict queries, sharing and invites, sync, notifications/reminders**) → trade-offs. The hard parts are **recurrence modeling** and **time-range queries across many calendars**.

---

0. Talk track

1. Clarify: create/edit/delete events, view day/week/month, invite attendees (RSVP), recurring events, reminders; multiple calendars.
2. Note: read-heavy, mostly per-user partitioned; correctness on recurrence and time zones.
3. Deep dives: recurrence storage (RRULE + exceptions), range queries, free/busy, invites and consistency, reminders, sync.

---

1. Problem

Users manage events (time, location, attendees), view calendars by time range, share/invite others, and get reminders, across devices.

What makes it hard

- **Recurring events** (infinite series, exceptions, "this and following").
- **Time zones/DST** semantics.
- Event shared by many attendees: **keep copies consistent**.
- Efficient **range queries** and **free/busy** across many people.

---

2. Requirements

Functional

- Create, read, update, delete events.
- View events by day/week/month.
- Invite others; RSVP (accept/decline).
- Recurring events and reminders (notifications).

Out of scope: room booking, video link integrations, smart scheduling AI.

Non-functional

- **Fast calendar view** (under 200 ms).
- **High availability**, eventual consistency across attendees (seconds).
- Scale: **500M users, 100M DAU**.
- Durable; no lost events.

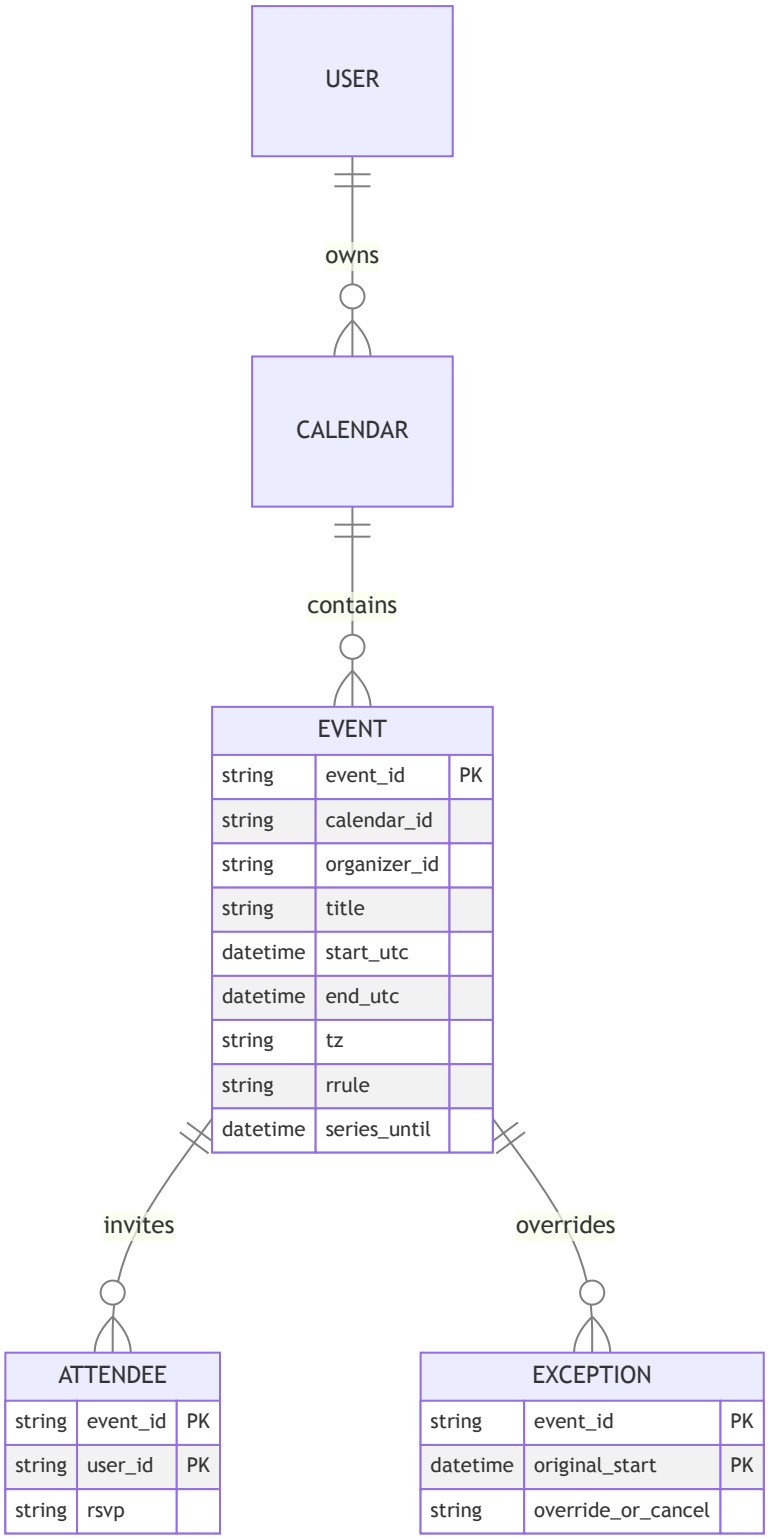
---

3. Estimation

Quantity	Assumption	Result
Events per user	about 200 active	100B event rows if fully materialized; so store **series**, not instances
View reads	100M DAU x 10	1B/day, **about 12K/s, about 50K/s peak**
Writes	20M/day	about 230/s
Read:write		about 50:1
Event row	1 KB	**about 100 TB**: sharded by user

---

4. Entities



---

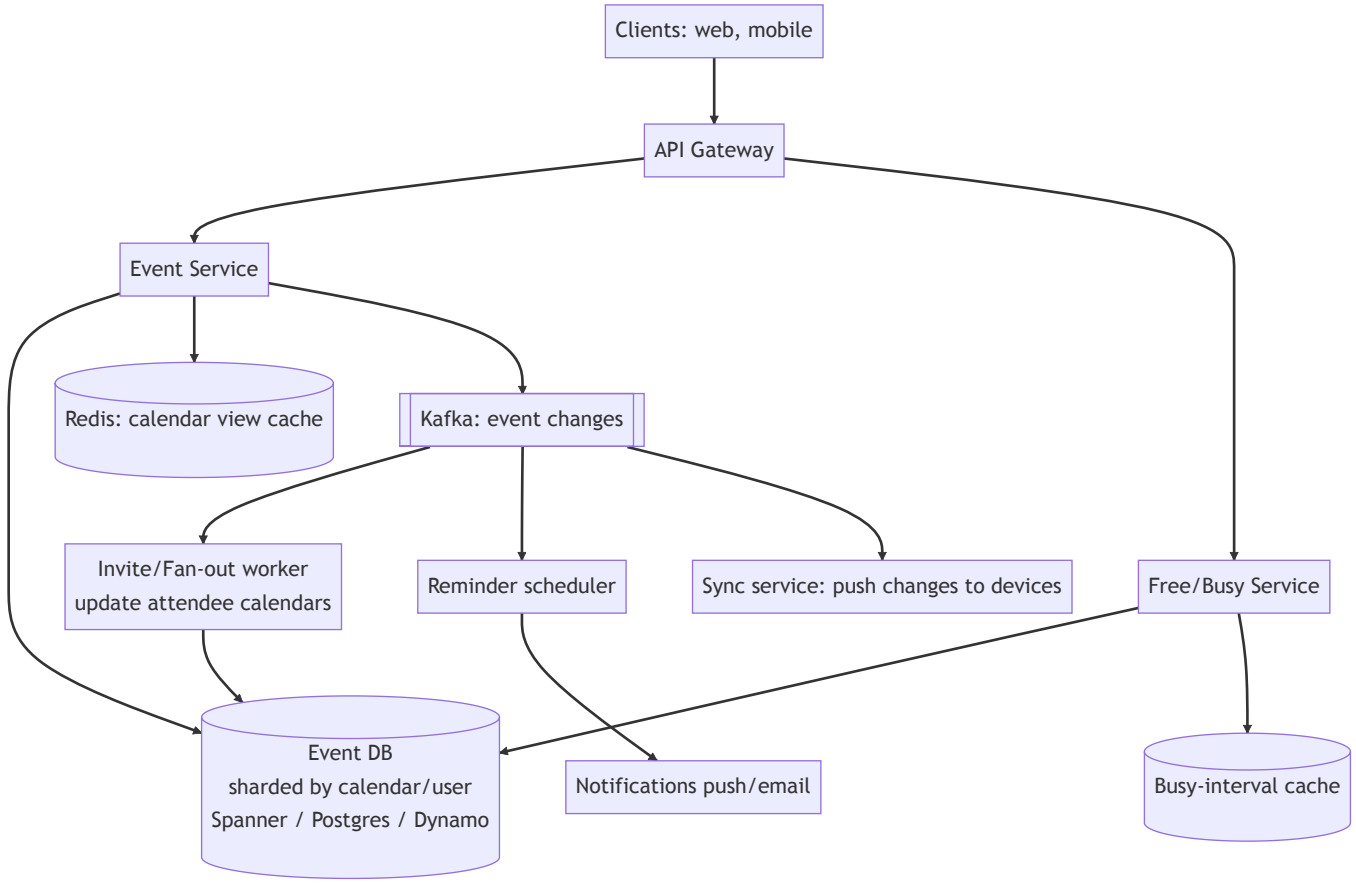
5. API

```
POST /events    { title, start, end, tz, rrule?, attendees:[...], reminders:[...] }   -> 201 { event_
GET  /events?from=2027-03-01&to=2027-03-08&calendar=...                                -> expanded insta
PATCH /events/{id}?scope=this|following|all   { ... }
DELETE /events/{id}?scope=...
```

```
POST /events/{id}/rsvp { response:"accepted" }
GET  /freebusy?users=u1,u2&from=&to=
```

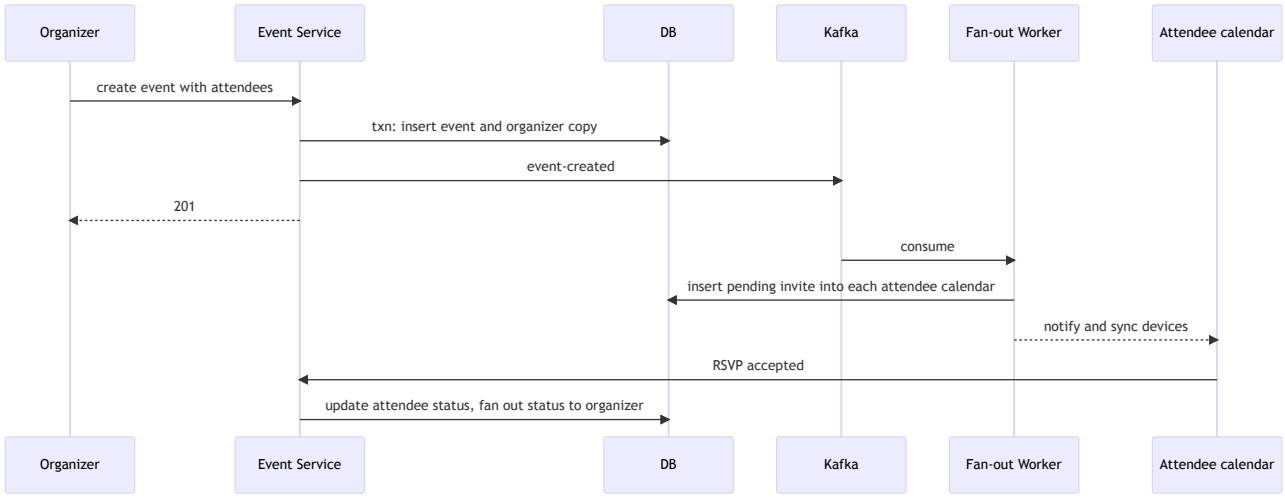
---

6. Architecture



---

7. Flow



---

8. Deep dive 1: recurring events

Approach	Verdict	Why
Materialize all instances	Bad	Infinite series; edits touch thousands of rows
Store the series once (RRULE) and **expand at query time** for the requested window	Great	Compact; small windows are cheap
Hybrid: store RRULE plus **exception rows** (modified/cancelled instances) overriding expansion; optionally materialize a rolling window (next 1-2 years) for indexed queries	Great	Efficient edits ("this", "this and following" splits the series: end old with `UNTIL`, create new)

Query: fetch events whose `[start, series_until]` overlaps the window, expand RRULEs, apply exceptions, sort. Cache expanded view per `(calendar, week)`; invalidate on change.

One-liner: *Store rules, expand on read, override with exceptions.*

9. Deep dive 2: time zones and DST

Store event instants in **UTC** plus the **IANA timezone** (e.g. America/New_York) and compute recurrences in **local wall time** (a 9 AM weekly meeting stays at 9 AM across DST, so UTC time shifts). All-day events are date-only, not instants. Convert at display using the viewer's tz.

10. Deep dive 3: range queries and indexing

Index `(calendar_id, start_utc)` to scan a window. For recurring events keep an upper-bound column (`series_until`, or max end) so overlap queries prune. Shard by `user_id/calendar_id`: a view hits one shard (plus shared calendars).

11. Deep dive 4: shared events and consistency

Model	Verdict
One canonical event referenced by attendees (shared row)	Single source of truth, fast updates; permission and per-user overrides need a separate table
Per-attendee copies (Google-style event in each calendar) with fan-out	Fast reads (single shard), per-user settings (reminders, colors); **updates must fan out**, eventual consistency

Choose copies with an **organizer as owner**: edits go to the organizer's event, then Kafka fan-out updates attendee copies. Conflicts: organizer wins; attendee only changes own RSVP/personal fields. Optimistic concurrency with `version`.

12. Deep dive 5: free/busy

`freebusy(users, window)` unions busy intervals (events that are `busy`, including expanded recurrences). Cache per-user **busy-interval summaries** for near-term windows, merging sorted intervals; respect privacy (return only busy/free, not details). Big attendee lists: parallel shard queries.

13. Deep dive 6: reminders

A **reminder scheduler** (like the Job Scheduler doc): time-bucketed table of `(fire_time, event, user)`; near-term items are pulled into a delay queue; at-least-once with dedupe; recurring events generate the next reminder after firing. Push/email via notification service.

14. Deep dive 7: sync across devices

Per-calendar **sync token / change log**; clients call `GET /changes?since=token`, with push (WebSocket/FCM) hints. Same cursor-based pattern as Dropbox.

---

15. Failure handling

Failure	Mitigation
Fan-out lag/failure	Retry via queue; idempotent per (event, attendee); eventual
Cache stale	Invalidate on change; short TTL
Shard outage	Replicas; other users unaffected
Reminder missed	Lease/retry; catch-up on recovery (send late flagged)
Concurrent edits	Version check, last-writer by organizer, conflict error

---

16. Trade-offs

Decision	Chosen	Alternative	Given up
Recurrence	RRULE + exceptions, expand on read	Materialize	Read-time compute
Shared events	Per-attendee copies	Single shared row	Fan-out complexity
Time storage	UTC + tz	Local only	Conversion care
Consistency	Eventual across attendees	Strong	Short divergence
Sharding	By user/calendar	By time	Cross-user queries need scatter

---

17. Level expectations

- **Mid:** event table, range query, invites.
- **Senior:** RRULE, time zones, fan-out for shared events, reminders.
- **Staff+:** exceptions and series splitting, free/busy at scale, sync tokens, DST correctness, multi-region.

18. Follow-ups

Meeting room resources, working hours/OOO, smart scheduling, external calendar interop (CalDAV/ICS), delegation.

19. Summary checklist

- [ ] Series stored as RRULE, exceptions as rows
- [ ] UTC plus IANA tz, recurrence in wall time
- [ ] Index (calendar_id, start); cache views
- [ ] Per-attendee copies, Kafka fan-out
- [ ] Reminder scheduler, sync tokens

Designing an Online Auction (eBay-style)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**consistent highest-bid under concurrency, real-time bid broadcast, auction end and fairness, hot auctions, payments handoff**) → trade-offs. The hard part is **never accepting a lower bid than the current highest and ending exactly once**.

---

0. Talk track

1. Clarify: sellers post items with start price and end time; buyers place bids; highest bid wins at end; users see live current bid.
2. Core: **strongly consistent bids** per item, real-time fan-out of the current price.
3. Deep dives: concurrency on bids (OCC / lock), real-time updates, end-of-auction, scale and hot items, failures.

---

1. Problem

Run time-boxed auctions: accept bids, always know the **current highest bid**, show it live, and **close** the auction at the deadline, picking one winner.

What makes it hard

- Concurrent bids on the same item; **no lost updates, no lower-bid overwrite**.
- Many viewers need **real-time** updates.
- Auction end must be **exactly once** and fair near the deadline (sniping).

---

2. Requirements

Functional

- Post an item for auction (start price, end time).
- Place a bid (must exceed the current highest).
- View an auction with the **current highest bid** (live).

Out of scope: search, payments/escrow details, shipping, reserve prices (mention), proxy/auto-bidding.

Non-functional

- **Strong consistency** for bids (highest bid correct).
- **Fault tolerant and durable** (no lost bids).
- **Real-time** price display (under a second).
- Scale: **10M concurrent auctions**, bids up to 10K/s overall; hot items 1000s of bidders.

---

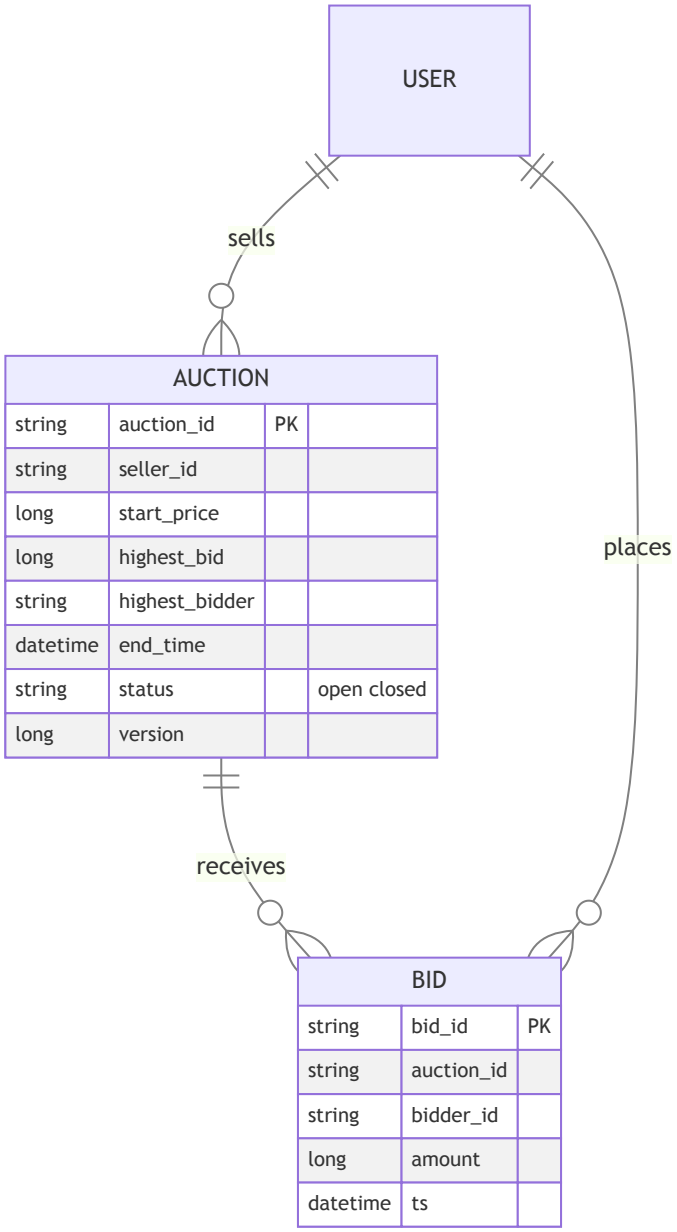
3. Estimation

Quantity	Assumption	Result
Auctions	10M active	metadata about 10M x 1 KB = 10 GB
Bids	100M/day	about 1.2K/s avg, **about 10K/s peak**
Viewers	1M watching hot items	fan-out in the millions of messages
Bid row	100 B	10 GB/day

Consequence: modest data; the difficulty is **contention per item** and **fan-out**.

---

4. Entities



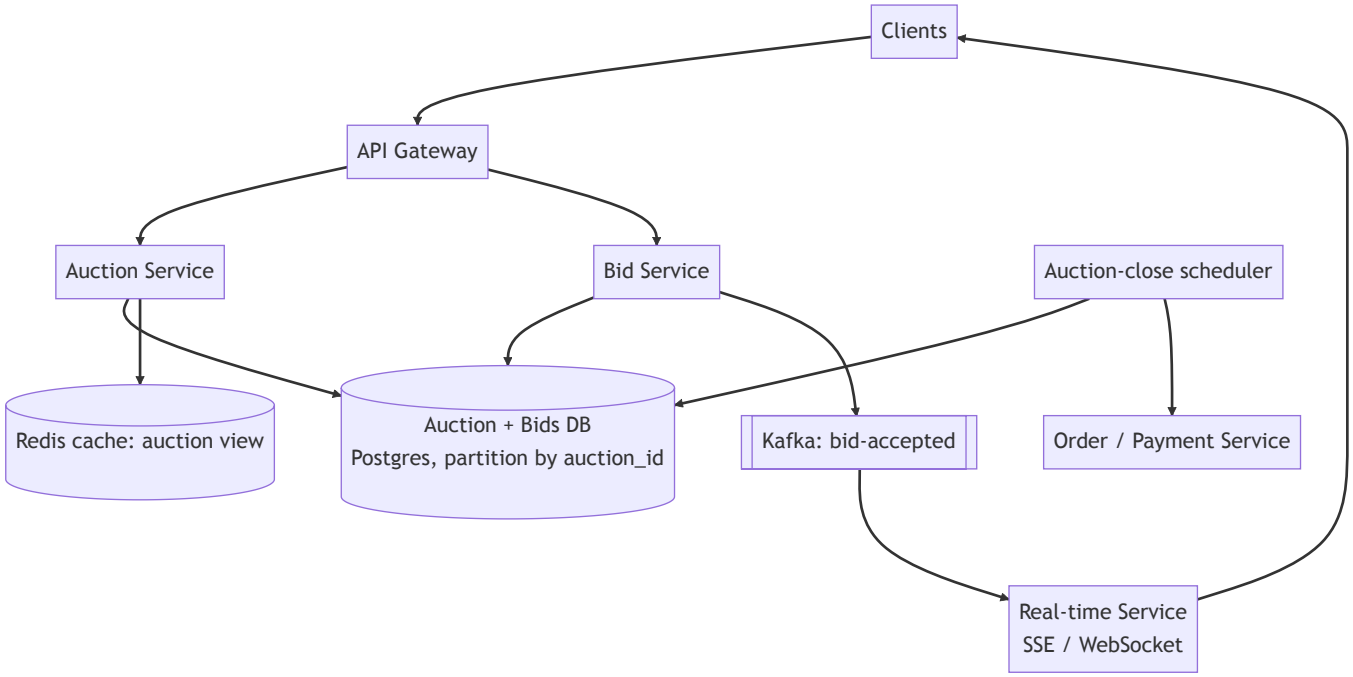
---

5. API

```
POST /auctions       { item, start_price, end_time }              -> 201 { auction_id }
GET  /auctions/{id}                                                -> { current_bid, end_time, ... }
POST /auctions/{id}/bids  Idempotency-Key  { amount }             -> 201 | 409 bid too low or auction
SSE  /auctions/{id}/stream                                         # live bid updates
```

---

6. Architecture



---

7. Bid flow



---

8. Deep dive 1: consistent highest bid

Approach	Verdict	Why
Read highest, compare in app, write	Bad	Race: two bids both pass the check
Pessimistic lock (`SELECT ... FOR UPDATE` on the auction row)	Good	Correct; serializes per auction; contention on hot items but fine
Optimistic concurrency / conditional update (`WHERE highest_bid < :amount` or `version = :v`)	Great	No long locks; the DB atomically guarantees monotonic max; loser gets 409
Cache-based (Redis) check + later DB write	Risky	Source of truth ambiguity; needs Lua + durability
Serialize via queue per auction (partition by auction_id; single consumer)	Great at scale	Natural ordering; adds latency, async result

Keep **all writes for an auction on one shard/leader** (partition by `auction_id`). Idempotency key on bids so retries don't duplicate. Reject bids after `end_time` using the **DB clock**.

One-liner: *Conditional update on the auction row makes "highest bid" a DB invariant.*

9. Deep dive 2: real-time updates

Approach	Verdict
Polling every second	OK for small scale; heavy at 1M viewers
SSE/WebSocket with **pub/sub topic per auction**; servers subscribe only to auctions their clients watch	Great
Conflate to at most 1-2 updates/s for hot auctions	Great

Cache current state in Redis for first paint; broadcast deltas from Kafka after commit (never before).

10. Deep dive 3: ending the auction

Single closer: **scheduler** picks auctions with `end_time <= now` and does a conditional `UPDATE status='closed' WHERE status='open'` (exactly-once by CAS), reads the winner, and emits a `won` event to payments/orders and notifications. Bids accepted only when `now < end_time` in the same conditional statement, so there is a **crisp cutoff**. Anti-sniping extension (extend by 2 min if a bid lands in the last minute) is a product option, implemented by updating `end_time` and rescheduling.

11. Deep dive 4: hot auctions and scale

- Contention on one row is bounded by DB single-row update rate (about thousands/s), plenty for any auction; **admission** control: per-user rate limit, drop obviously low bids at the edge using the cached current price (fast reject, final decision still in DB).
- Partition by `auction_id`; read replicas/cache for views; CDN for item media.
- Fan-out tier scales horizontally; hot auctions' topics replicated across more servers.

12. Payments handoff

On close, create an order and invoke payment (idempotent, see Payment doc); if the winner fails to pay, offer to the next highest bid (bids are retained, ordered).

---

13. Failure handling

Failure	Mitigation
Bid service retry	Idempotency key
DB primary down	Failover (sync replica); bids retried; no accepted bid lost
Broadcast failure	Clients re-fetch current state; SSE resume
Scheduler crash	Another instance; CAS close is idempotent
Clock skew	Use DB time for cutoff
Cache stale	Never used for decisions

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Concurrency	Conditional update	Distributed lock	Retry on 409
Store	SQL ACID	NoSQL	Auto-scale (partition by auction)
Updates	Push with conflation	Poll	Connection mgmt
Close	Scheduler + CAS	Timer per auction in memory	Slight delay (seconds)
Read path	Cached view	Direct DB	Slight staleness for display only

---

15. Level expectations

- **Mid:** tables, check highest then write, polling.
- **Senior:** race analysis, OCC/locking, SSE/WebSocket, scheduler-based close.
- **Staff+:** sharding/ordering per auction, anti-sniping, fraud (shill bidding), reconciliation with payments, cost.

16. Follow-ups

Proxy bidding (max bid auto-increments), reserve prices, Dutch auctions, bid retraction rules, fraud detection.

17. Summary checklist

- [ ] Conditional UPDATE enforces monotonic highest bid
- [ ] Partition by auction_id; idempotent bids
- [ ] Post-commit Kafka to per-auction pub/sub to SSE
- [ ] CAS close by scheduler; DB-time cutoff
- [ ] Payments handoff with fallback to runner-up

Designing Facebook Live Comments

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**real-time delivery to millions of viewers, SSE vs WebSocket, pub/sub fan-out, hot streams, ordering/pagination, rate limiting**) → trade-offs. The hard part is **broadcasting a firehose of comments to millions of concurrent viewers of one stream**.

---

0. Talk track

1. Clarify: post comments on a live video, **viewers see new comments in real time**, scroll back history; scale 1M+ viewers on a stream.
2. Note: one-to-many broadcast; **availability over consistency**, eventual ordering fine.
3. Deep dives: transport (polling vs SSE vs WebSocket), pub/sub fan-out tiers, hot stream scaling, storage/pagination, moderation/rate limits.

---

1. Problem

Viewers watching a live video post comments and see **other viewers' comments appear within a second**.

What makes it hard

- A single popular stream has **millions of watchers** and thousands of comments/s.
- Each comment must be broadcast to every watcher of that stream.
- Late joiners need **recent history**; scrolling needs pagination.

---

2. Requirements

Functional

- Viewers **post comments** on a live video.
- Viewers see **new comments in real time**.
- Viewers can **view older comments** (scroll back).

Out of scope: reactions, replies/threading, moderation tools, video delivery.

Non-functional

- **Scale**: millions of concurrent viewers across thousands of live streams; hot streams with 1M+.
- **Low latency** (under 200 ms-1 s).
- **Availability over consistency** (a viewer may miss a comment briefly; eventual order OK).
- Cost-efficient connections.

---

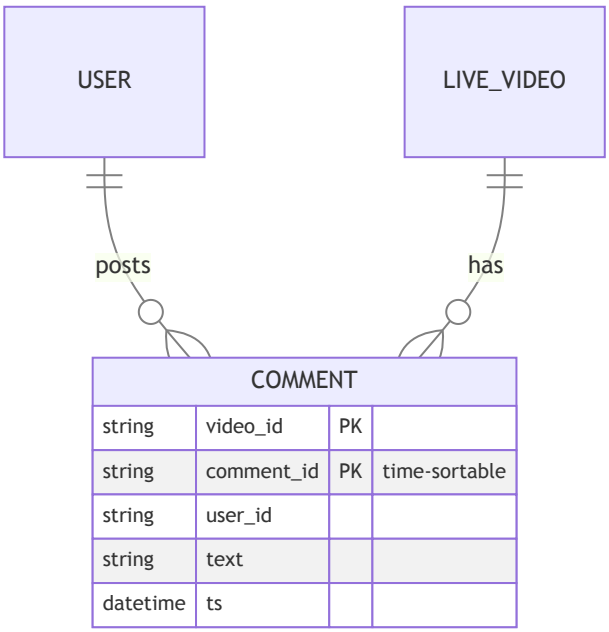
3. Estimation

Quantity	Assumption	Result
Concurrent viewers	10M across platform	10M connections
Hot stream	1M viewers, 1K comments/s	naive fan-out = 1M x 1K = **1B deliveries/s**, infeasible, so **sample and rate-limit** what each viewer receives
Displayed rate	cap about 5-20 comments/s per viewer	1M x 20 = **20M msgs/s** for the hottest stream
Comment	200 B	storage modest: 1K/s x 200 B = 200 KB/s
Connections/server	100K	**100 servers** for 10M

Consequence: human readability bounds useful rate (nobody reads 1K/s): **sampling/conflation is a feature**. The cost center is **outbound fan-out**.

---

4. Entities



---

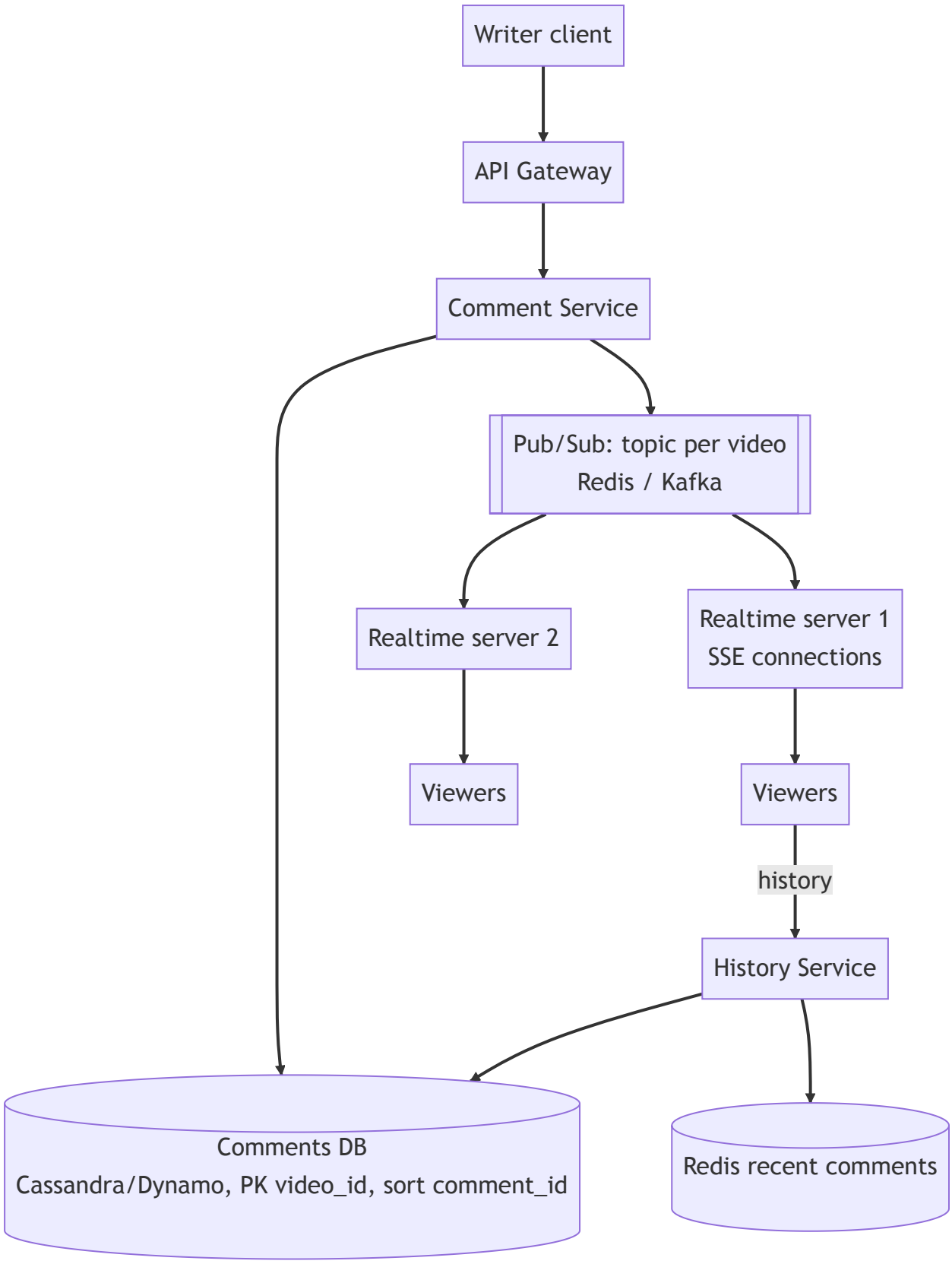
5. API

```
POST /videos/{id}/comments   { "text":"hi" }                          -> 201 { comment_id }
GET  /videos/{id}/comments?before=<comment_id>&limit=50               -> history (cursor pagination)
SSE  /videos/{id}/comments/stream?since=<comment_id>                  -> live new comments (server p
```

Cursor on a **time-sortable id** (Snowflake/ULID) enables history and `since` catch-up on reconnect.

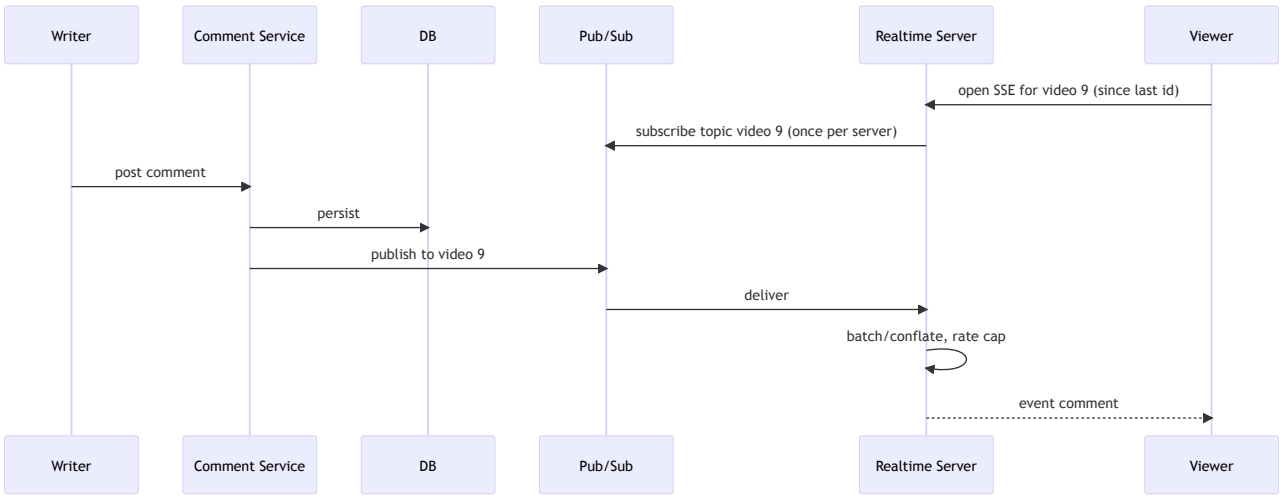
---

6. Architecture


Writer client
API Gateway
Comment Service
Pub/Sub: topic per video
Redis / Kafka
Realtime server 2
Realtime server 1
SSE connections
Viewers
Viewers
history
History Service
Comments DB
Cassandra/Dynamo, PK video_id, sort comment_id
Redis recent comments

---

7. Flow



---

8. Deep dive 1: transport

Approach	Verdict	Why
Polling every N s	Good for small	Simple; load = viewers/N req/s; 1M viewers means 200K+ rps and delay
Long polling	OK	Fewer empties, still connection churn
SSE (server push over HTTP)	Great	One-way fits comments; auto-reconnect with `Last-Event-ID`; works through proxies/HTTP2
WebSocket	Great	Bidirectional; needed if viewers send over the same socket; heavier infra

Posting uses normal HTTP POST; viewing uses SSE. WebSocket is not required since reads dominate and are one-way.

One-liner: *SSE for server-to-viewer broadcast, plain POST for writes.*

9. Deep dive 2: fan-out architecture

Naive: comment service pushes to every viewer: too many connections for one box. Use **tiers**:

1. Comment Service publishes **once** to the video's topic.
2. Each **realtime server** subscribes to topics for videos its viewers watch (one subscription per server per video, not per viewer).
3. Realtime server fans out locally to its connected viewers (in-memory map `video_id to connections`).

Cost: publish = O(servers hosting watchers), local delivery = O(viewers/server). **Viewers of one video are spread across servers** by load balancing; or **co-locate by video_id** (consistent hashing) to reduce subscriptions but risk hot servers. For hottest videos, use hierarchical pub/sub (regional relays) so a publish crosses the network few times.

10. Deep dive 3: scaling a hot stream

- **Sample/conflate**: each realtime server sends at most N comments/s per viewer (e.g. newest or random sample); drop the rest (humans can't read more).
- **Batch** sends every 100-250 ms (fewer syscalls, packets).
- **Shard the topic** (video#0..K-1) when the pub/sub itself is hot; servers subscribe to their shard.

- **Admission/rate limits** on posters (per-user, per-video) and spam filters before publish.
- Autoscale realtime tier by connection count; **graceful reconnect** with jitter.

11. Deep dive 4: history and catch-up

Store comments in a wide-column store keyed `(video_id, comment_id)` so range scans return newest-first quickly. Keep the **latest N** per live video in Redis for fast initial load. On SSE reconnect, `since=<last id>` queries the missing tail (bounded) then resumes live. Ordering: time-sortable ids; minor cross-server reorder is acceptable (eventual).

12. Deep dive 5: moderation and abuse

Rate limit per user/video, profanity/spam filters (sync lightweight, async heavier ML), shadow-ban, delete events published to topic so clients remove the comment.

---

13. Failure handling

Failure	Mitigation
Realtime server dies	Clients reconnect (SSE auto), `Last-Event-ID` catch-up
Pub/sub node down	Replicated/sharded; clients degrade to polling history
DB slow	Publish does not wait for DB for display (or minimal durable buffer), eventual persistence
Reconnect storm	Jitter, connection rate limits, staged rollouts
Spam flood	Rate limits, circuit breaker on stream

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Transport	SSE	WebSocket	Bidirectional on same channel
Delivery	Sampled/conflated	Every comment	Completeness (users can't read it anyway)
Fan-out	Topic per video + server-local	Per-viewer push from service	Pub/sub dependency
Consistency	Eventual	Strong order	Minor reordering
Storage	Wide-column	SQL	Ad hoc queries

---

15. Level expectations

- **Mid:** post and poll, DB table, pagination.
- **Senior:** SSE/WebSocket choice, pub/sub, server-local fan-out, history and catch-up.
- **Staff+:** hot-stream math and conflation, hierarchical fan-out, shard hot topics, reconnect storms, moderation pipeline.

16. Follow-ups

Reactions (counters with sharding), pinned comments, replies, language translation, geo-restricted comments.

17. Summary checklist

- [ ] POST write, SSE read
- [ ] Per-video pub/sub, per-server subscription, local fan-out
- [ ] Rate cap and sampling for hot streams
- [ ] Time-sortable ids, since-cursor catch-up, Redis recent cache
- [ ] Spam/rate limits, jittered reconnects

Designing Facebook Post Search

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**inverted index, sharding the index, near-real-time indexing, ranking and recency/likes, caching, scaling reads and handling hot terms**) → trade-offs. The hard parts are **index size and sharding**, **fast freshness**, and **ranking by relevance + engagement**.

---

0. Talk track

1. Clarify: search posts by keyword, results sorted by recency or likes, posts created and liked continuously. Out: personalization, typeahead, privacy ACL (mention).
2. Estimate: 1B posts/yr? 100K searches/s reads; likes are write-heavy.
3. Evolve: SQL `LIKE` to inverted index to sharded index to separate ingest path, ranking, caching.

---

1. Problem

Users search all public posts by keywords and get results sorted by **recency** or **like count**, with fresh posts discoverable quickly.

What makes it hard

- Large corpus (trillions of tokens) so need a **distributed inverted index**.
- **Freshness**: new posts searchable in seconds.
- **Likes change constantly** so ranking by likes means high write rate into the index.
- Search QPS is huge; skewed terms.

---

2. Requirements

Functional

- Create posts.
- **Like** posts.
- **Search** posts by keyword.
- Sort by **recency** or **likes**.

Out of scope: personalization, typeahead, media search, privacy/ACL filtering, spelling correction.

Non-functional

- **Low latency** search (under 500 ms).
- **High throughput**: about 10K searches/s peak (scale up as required), 1B+ users.
- **Near-real-time** indexing of new posts (seconds to a minute).
- **High availability**, **eventual consistency** acceptable.
- Efficient, sharded storage.

---

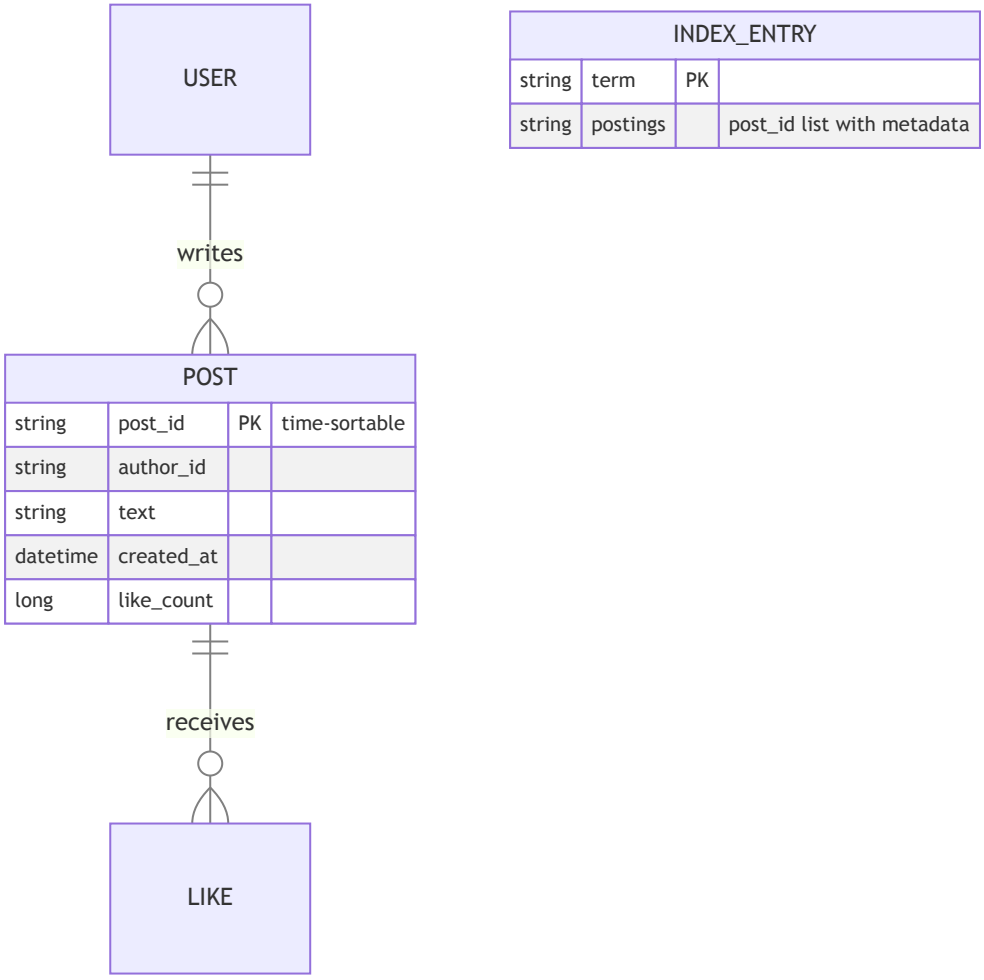
3. Estimation

Quantity	Assumption	Result
Posts	10B total, 200 B text avg	**2 TB** raw text
Index	postings (doc ids per term) about 1-2x text	**about 2-4 TB**: sharded over dozens of nodes, RAM/SSD
New posts	100M/day	about 1.2K/s
Likes	1B/day	**about 12K/s avg** (write-heavy)
Searches	100M/day	about 1.2K/s avg, about 10K/s peak

Consequence: index > one machine; **shard**. Likes dominate writes: do **not** reindex on every like (batch/aggregate).

---

4. Entities



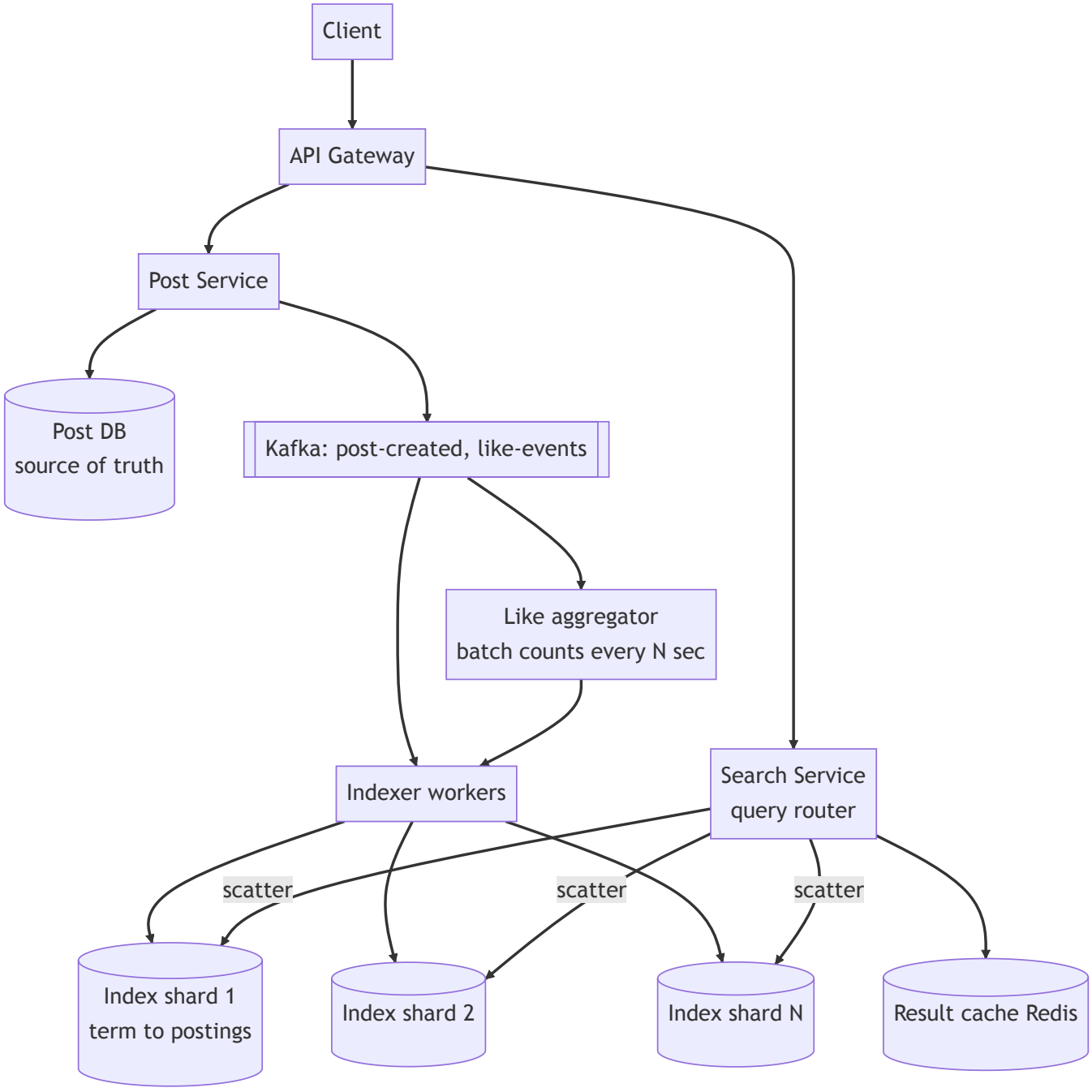
---

5. API

```
POST /posts       { text }                                  -> 201
POST /posts/{id}/like                                       -> 204
GET  /search?q=taylor+swift&sort=recent|likes&cursor=  -> { posts:[...], next_cursor }
```

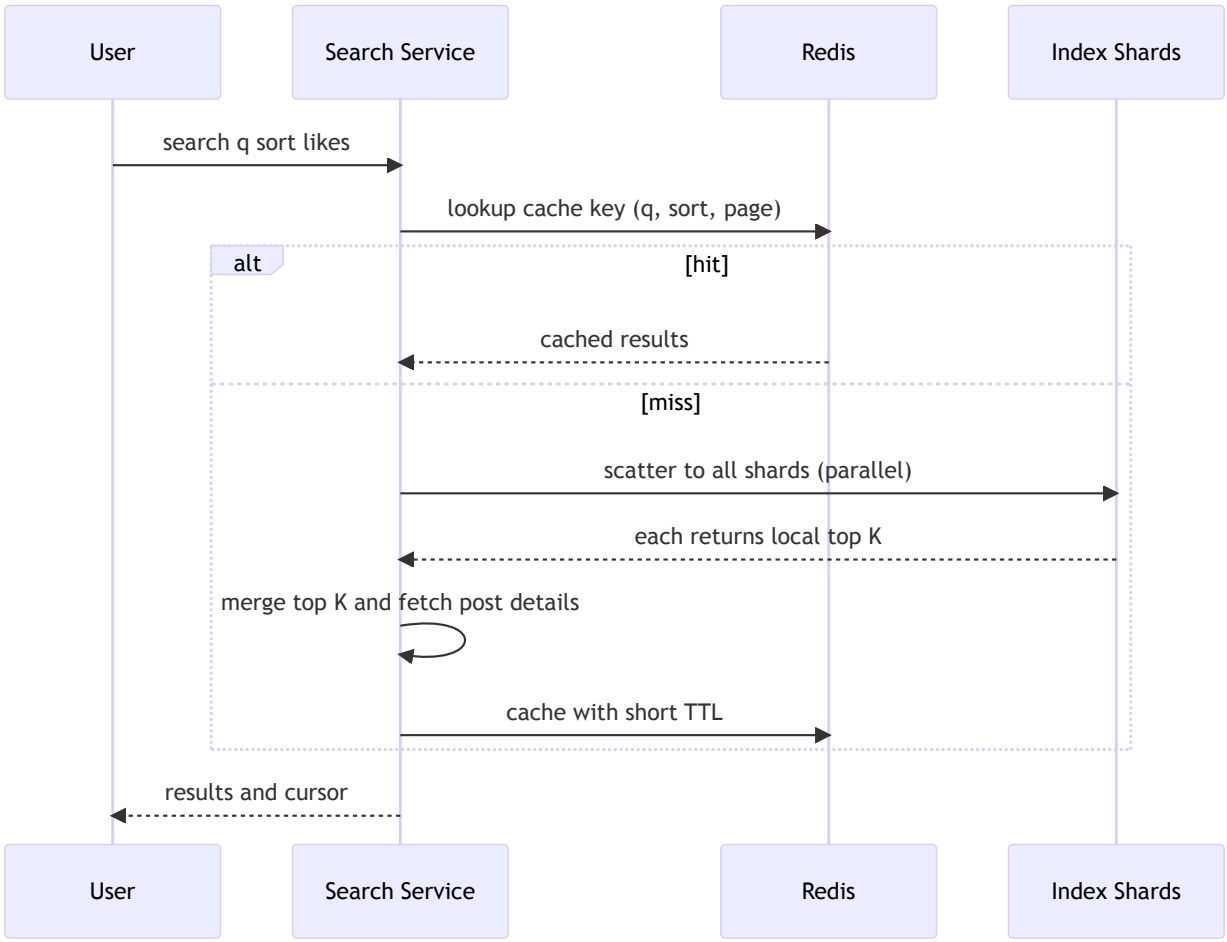
---

6. Architecture



---

7. Search flow



---

8. Deep dive 1: the index

Approach	Verdict	Why
`SELECT ... WHERE text LIKE '%term%'`	Bad	Full scan
DB full-text index	Good	Works to moderate scale; hard to shard
Inverted index: `term to sorted list of post ids` (Lucene segments in Elasticsearch/OpenSearch or custom)	Great	Multi-term queries by posting-list intersection

Tokenize, lowercase, stem, drop stopwords at index time. Posting lists sorted by the **sort key** (e.g. by recency/doc id descending) so top-K can **early terminate**. Store compressed (delta-encoding, bitmaps). For "likes" sort, keep per-term lists ordered by like bucket, or keep like count as a doc value and score at query time.

9. Deep dive 2: sharding

Scheme	Pros	Cons
By document (post_id hash/time range): each shard indexes a subset of docs fully	Even load, simple indexing, writes touch one shard	Query is **scatter-gather** to all shards (latency = slowest shard)
By term: each shard owns some terms	Query hits few shards	Hot terms, multi-term joins cross-shard, writes touch many shards

Choose **document-sharded** (what ES/Lucene do); mitigate tail latency with replicas (hedged requests), per-shard timeouts, partial results. **Time-based partitioning** (recent index hot on SSD/RAM, older archived) helps recency queries: search recent shards first and stop when K found.

One-liner: *Shard by document, scatter-gather top-K, merge.*

10. Deep dive 3: freshness

New posts go through **Kafka to indexer** to a small in-memory **real-time segment** (searchable in about 1 s, refresh interval), periodically merged into larger segments. Post DB remains truth; the index is rebuildable (replay Kafka / backfill from DB).

11. Deep dive 4: likes and ranking

Likes arrive at 12K/s and hot posts get floods: don't reindex on each. **Aggregate** counts in Redis/stream per post, flush every few seconds to the index as a partial doc update (or store as a doc value updated in batch). Ranking for "likes" can be exact top-K by `like_count` with slight staleness; bucketed counts (log scale) reduce update churn. Combine relevance (BM25) with engagement and recency if ranking mixed.

12. Deep dive 5: caching and hot terms

Cache results for popular queries (short TTL, key = normalized query + sort + page); trending terms are skewed, so cache hit ratio is high. Request coalescing for stampedes. Replicate shards holding hot data; CDN not applicable (dynamic).

13. Deep dive 6: pagination

Cursor by `(sort_key, post_id)` (search-after) instead of offset to avoid deep-page cost across shards.

---

14. Failure handling

Failure	Mitigation
Shard down	Replicas serve; partial results with flag as fallback
Slow shard	Hedged requests, timeouts
Indexer lag	Queue lag alert; searches stale not broken
Index corruption	Rebuild from DB/Kafka (reindex)
Like storm on one post	Aggregation buffer; skip per-like indexing

---

15. Trade-offs

Decision	Chosen	Alternative	Given up
Sharding	By document	By term	Scatter-gather fan-out cost
Freshness	Near real time	Batch	Indexing complexity
Likes in index	Batched	Realtime exact	Slightly stale ranking
Engine	Elasticsearch/Lucene	Custom	Control; saves time
Cache	Short-TTL results	None	Freshness (seconds)

---

16. Level expectations

- **Mid:** inverted index idea, DB for posts.
- **Senior:** sharded inverted index, scatter-gather, Kafka indexing, like batching, caching.
- **Staff+:** tiered/time-partitioned indexes, tail-latency control, ranking blends, reindex/backfill strategy, capacity math.

17. Follow-ups

Typeahead/autocomplete, personalized ranking, privacy filtering at query time, spell correction, multi-language analyzers.

18. Summary checklist

- [ ] Inverted index, doc-sharded, replicated
- [ ] Scatter-gather top-K merge, hedged requests
- [ ] Kafka real-time indexing, DB as truth
- [ ] Batch like counts into index
- [ ] Cursor pagination and result cache

Designing a Price Tracking Service (CamelCamelCamel)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**getting price data: crawling vs user-submitted, price-history time series, detecting changes and alerting at scale, politeness and anti-bot, notification fan-out**) → trade-offs. The hard part is **continuously and cheaply monitoring millions of products and alerting the right users**.

---

0. Talk track

1. Clarify: track Amazon product prices, show history charts, users set price-drop alerts. 500M products, 1M users watching.
2. Key problem: **no official API**, so data acquisition is scraping + crowd sourcing.
3. HLD: tracker (crawler), price store (time series), alert evaluator, notifier.
4. Deep dives: data acquisition, history storage, alert matching, scale.

---

1. Problem

Show historical prices for products and **notify users** when a price crosses their threshold.

What makes it hard

- **Data acquisition** without an API at **500M products**.
- Prices change rarely (so most scrapes are wasted) but must be caught timely for popular products.
- Matching a price change to **many user alerts** efficiently.

---

2. Requirements

Functional

- View **price history** for a product (chart).
- **Subscribe** to price-drop alerts (threshold or any drop).
- Receive **notifications** (email/push).

Out of scope: search UX, user auth, affiliate links, multi-retailer details.

Non-functional

- **Scale**: 500M products, 1M active users.
- **Reasonable freshness**: popular products within minutes/hours, tail daily.
- **High availability**, **eventual consistency** fine.
- Alert delivery reliable; no spam (dedupe).

---

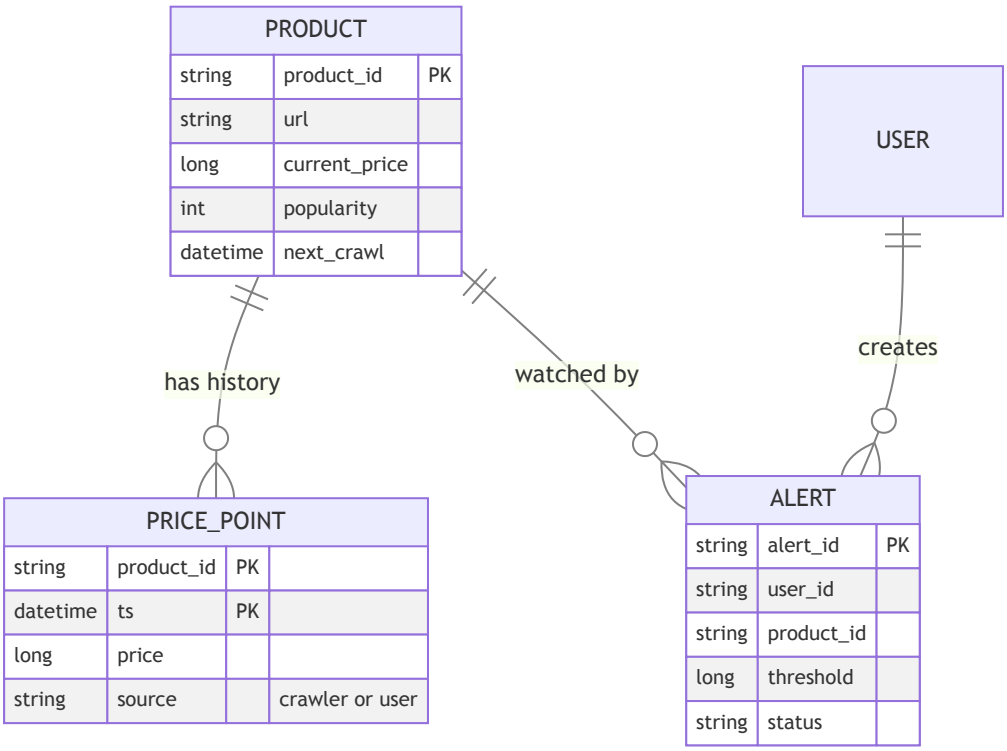
3. Estimation

Quantity	Assumption	Result
Products	500M	metadata 500M x 1 KB = **500 GB**
Price points	change events only (avg 10/yr/product)	5B/yr x 50 B = **250 GB/yr** (tiny)
Scrapes	tiered: top 1M hourly, 50M daily, rest weekly	1M x 24 + 50M + 450M/7 = **about 140M scrapes/day, about 1.6K/s**
Alerts	1M users x 5 alerts	**5M alert rules**
Price changes/day	about 1-5% of scrapes	about 3M events/day, trivial

Consequence: storage is small; **crawl scheduling** (who to refresh when) is the lever.

---

4. Entities



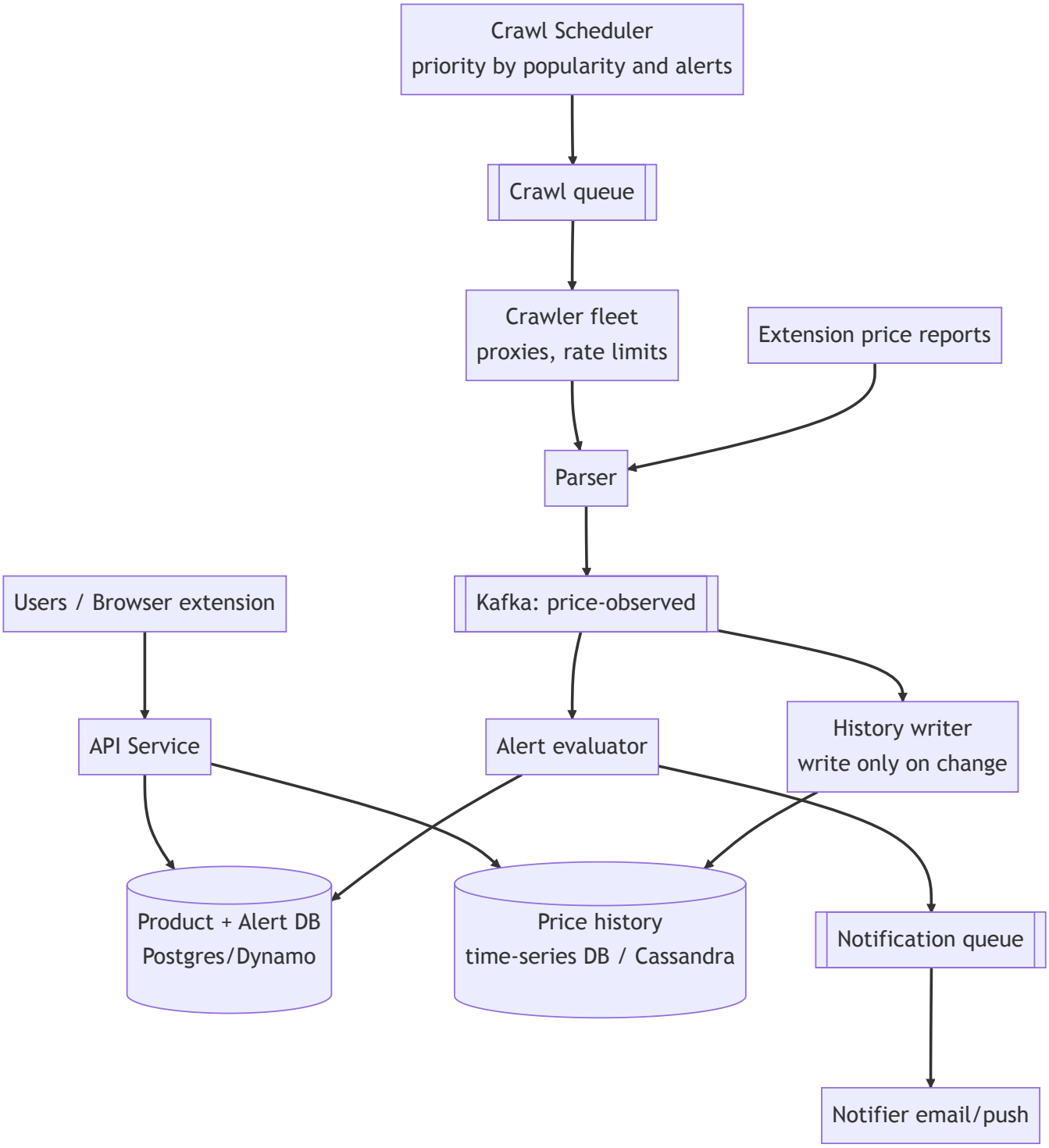
---

5. API

```
GET  /products/{id}/prices?from=&to=&granularity=day    -> [{ts, price}]
POST /alerts        { product_id, threshold }            -> 201
DELETE /alerts/{id}
POST /products     { url }                               # user adds a product to track
```

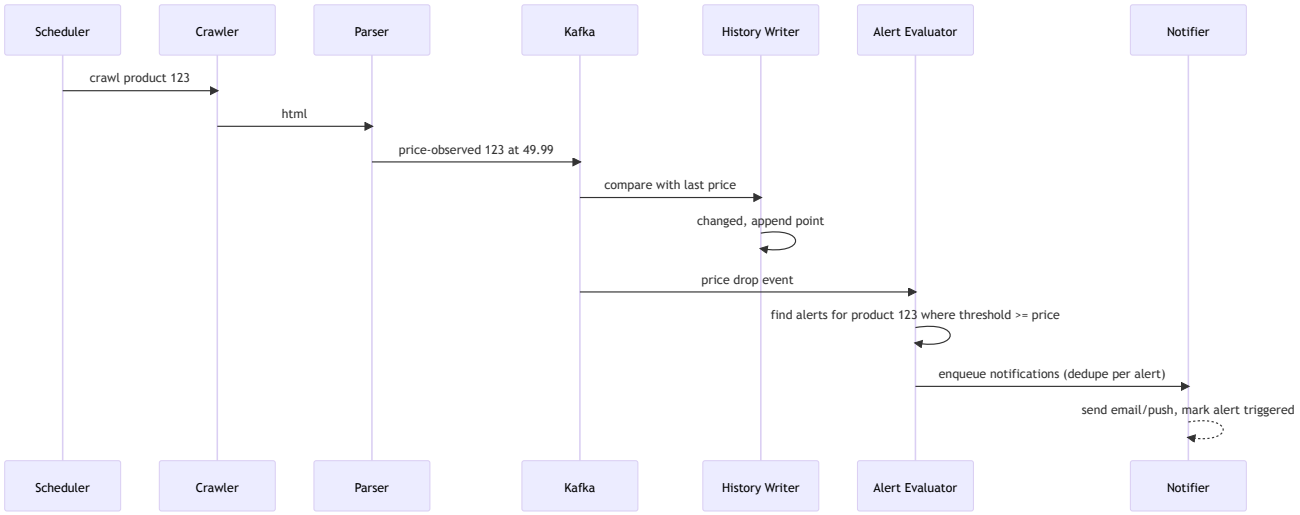
---

6. Architecture



---

7. Flow



---

8. Deep dive 1: getting the data

Approach	Verdict	Why
Official retailer API	Ideal but often unavailable/limited	Quotas, ToS
Web crawling (see Web Crawler doc)	Good	Needs proxy rotation, anti-bot handling, parsing breakage; legal/ToS risk
Crowdsourcing: browser extension reports prices users see	Great complement	Scales free with users, covers popular items; trust/validation required (outlier detection, multiple reports)

Hybrid: user reports for popular products (high frequency), **targeted crawls** for products with active alerts, slow background crawl for the tail. Respect robots/ToS; throttle per domain; rotate IP pools.

9. Deep dive 2: crawl scheduling

Priority = f(popularity, number of active alerts, volatility, time since last crawl). A scheduler picks due products (priority queue / time-bucketed index) and feeds the crawl queue; adaptive intervals: back off for stable prices, speed up when changes are frequent. Products with **zero** watchers and low popularity are refreshed rarely.

One-liner: *Spend crawl budget where prices matter: popular or watched products.*

10. Deep dive 3: price history storage

Append only **on change** (or at most every N hours) to keep series small. Store `(product_id, ts) to price` in a **time-series store** or Cassandra/Dynamo (PK product_id, sort ts). Charts: query range, downsample (daily min/max) via precomputed rollups; cache popular product charts. Keep source and confidence flags.

11. Deep dive 4: alert matching

Approach	Verdict
Scan all alerts on every price change	Bad
Index alerts by `product_id` sorted by threshold; on event query `threshold >= new_price AND status=active`	Great
Evaluate in Kafka consumer partitioned by product_id	Great (ordering per product)

Notify **once** per alert crossing (mark triggered, optionally re-arm after the price rises above threshold). Idempotency key per `(alert, price_event)` so retries don't duplicate. Batch/digest for users with many alerts; throttle per user.

12. Deep dive 5: notifications

Queue to workers sending email/push via providers (SES/FCM/APNs) with retries and backoff; bounce handling; preference management. Same pattern as the Notification Service doc.

---

13. Failure handling

Failure	Mitigation
Crawler blocked / parser breaks	Detect via drop in success rate, fall back to user reports, alert engineers
Bad price (outlier, error page)	Validate (range vs history, multiple sources) before accepting
Kafka lag	Alerts delayed, not lost; replay
Duplicate event processing	Idempotent history write (same ts/price), alert dedupe
Notification provider down	Retry queue, secondary provider

---

14. Trade-offs

Decision	Chosen	Alternative	Given up
Data source	Crawl + crowdsource	API only	Legal/ops risk, data quality work
Crawl policy	Adaptive priority	Uniform	Scheduler complexity
History	On-change points	Every scrape	Resolution of unchanged periods (implicit)
Alerts	Indexed by product	Full scan	Extra index
Consistency	Eventual	Strong	Late alerts by minutes

---

15. Level expectations

- **Mid:** crawler, DB, alert scan, email.
- **Senior:** tiered crawl scheduling, time series store, Kafka pipeline, alert index, dedupe.
- **Staff+:** crowdsourcing validation, anti-bot economics, adaptive polling, legal/ToS, cost per scrape, multi-retailer scale.

16. Follow-ups

Multiple retailers, price prediction ("wait to buy"), coupon tracking, regional prices, variant/SKU handling.

17. Summary checklist

- [ ] Hybrid data: user reports plus targeted crawl
- [ ] Priority-based crawl scheduler
- [ ] Append history on change only, rollups for charts
- [ ] Alerts indexed by product and threshold
- [ ] Idempotent alert and notification pipeline

Designing Gopuff (Local Delivery Service)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**nearby DC discovery, inventory aggregation across fulfillment centers, no overselling at order time, scaling reads vs consistent writes**) → trade-offs. The hard parts are **availability queries across nearby inventory** and **atomic order placement that doesn't oversell**.

---

0. Talk track

1. Clarify: customers see **available items** (within delivery range), place orders; inventory lives in many **distribution centers (DCs)**; 10-minute delivery promise.
2. Split: **availability reads** (10K/s, eventual) vs **orders** (strong consistency).
3. Deep dives: nearby-DC lookup, inventory aggregate query, atomic order with inventory decrement, scale and partitioning.

---

1. Problem

Show what products are **actually available near the user** and let them order, decrementing stock at the right DC.

What makes it hard

- Availability = union of inventory across **all DCs within range** of the user.
- **Overselling**: two users buy the last item.
- Read-heavy browse (10K req/s) with strongly consistent order writes.

---

2. Requirements

Functional

- Query **item availability** by delivery location (aggregated over nearby DCs).
- **Place orders** for multiple items.

Out of scope: payments, driver/courier logistics, search ranking, returns.

Non-functional

- **Availability queries**: under 100 ms, 10K req/s, **eventual consistency OK**.
- **Orders**: **strong consistency** (no oversell); about 10M orders/day.
- Scale: 10K items, 1K DCs.

---

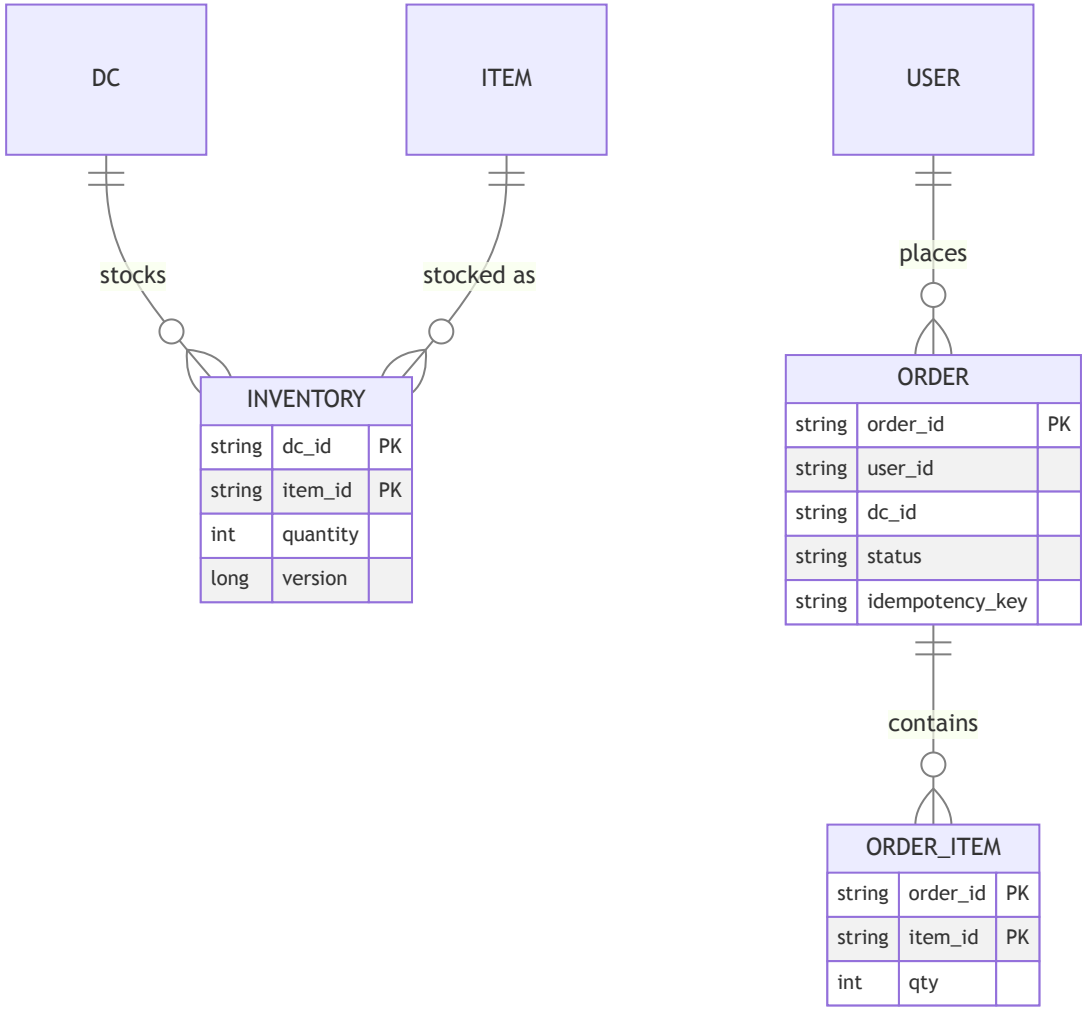
3. Estimation

Quantity	Assumption	Result
Orders	10M/day	about 115/s avg, **about 500/s peak**
Availability reads		**10K/s**
Inventory rows	10K items x 1K DCs	**10M rows x 100 B = 1 GB** fits in memory
Order size	5 items	50M inventory updates/day

Consequence: data is tiny; one strongly consistent DB handles orders, and **a cache plus read replicas** serve availability.

---

4. Entities



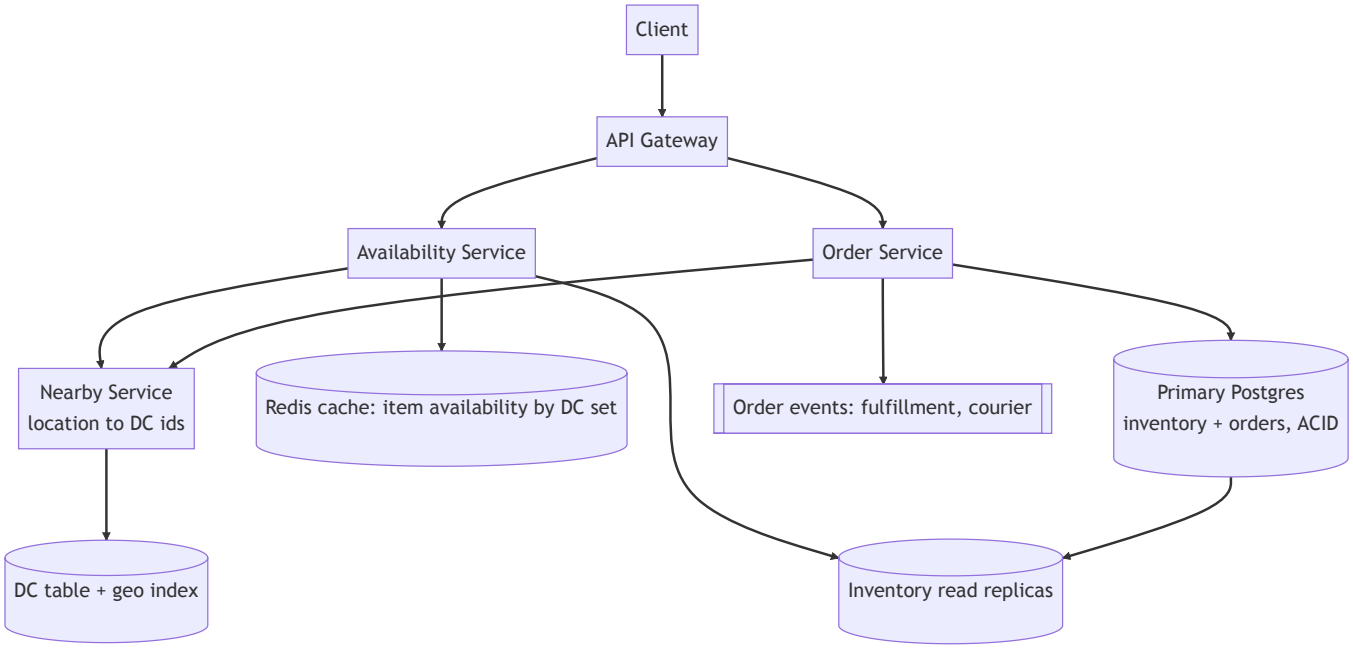
---

5. API

```
GET  /availability?lat=&lng=&item_ids=...&page=      -> [{ item_id, total_available }]
POST /orders  Idempotency-Key  { items:[{item_id, qty}], lat, lng }   -> 201 { order_id } | 409 insuf
```

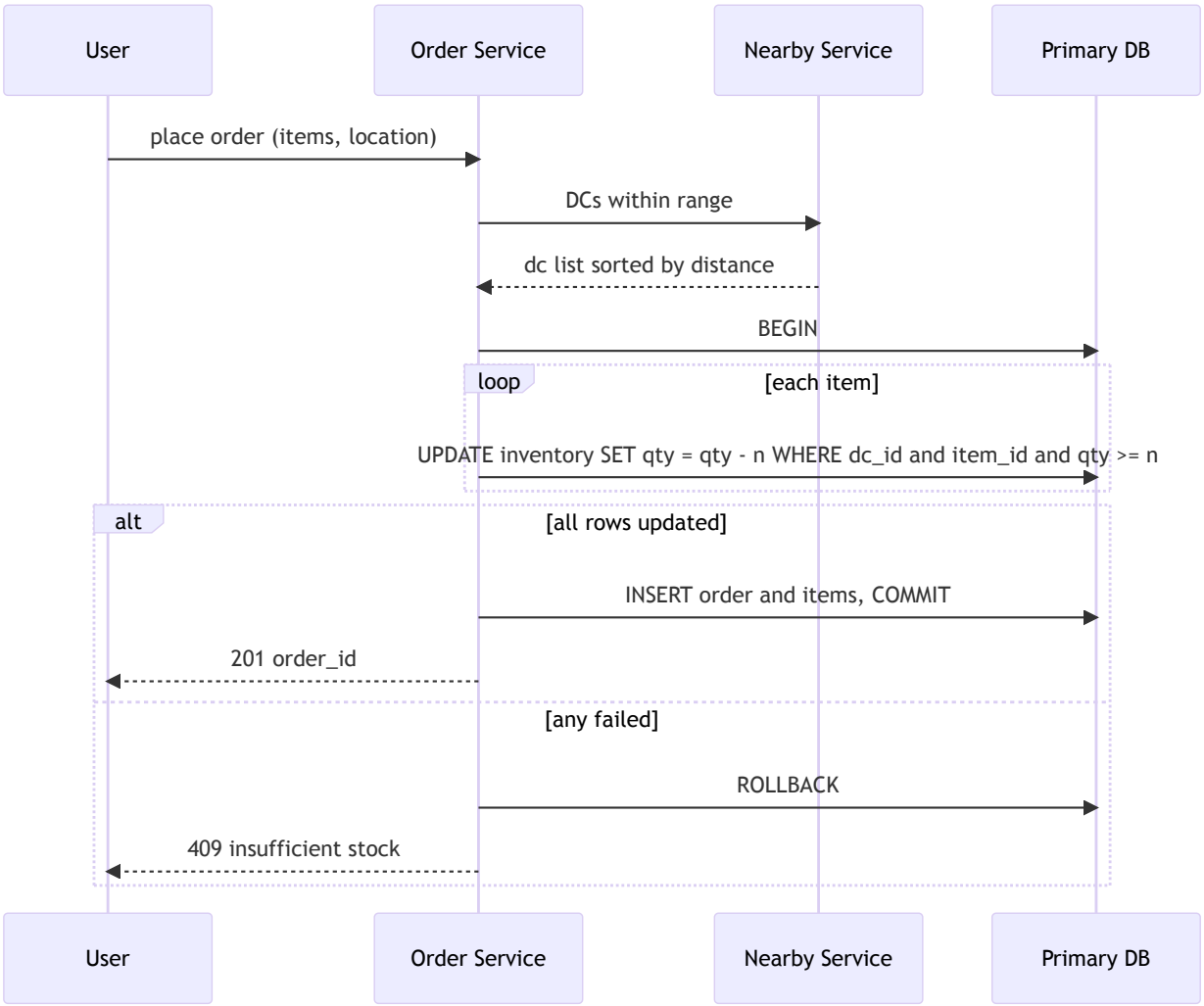
---

6. Architecture



---

7. Order flow



---

8. Deep dive 1: finding nearby DCs

Approach	Verdict	Why
Call Google Maps for travel time to every DC per request	Bad	Slow, costly
Geo filter (lat/lng bounding box or geohash cells) then travel-time/ETA service for few candidates	Great	1K DCs is small
Precompute **service-area polygons / cell to DC list**, cached	Great	DC set changes rarely; O(1) lookup

DCs are few and static: keep them in memory in the Nearby Service, **precomputed by grid cell**; refresh on DC changes.

9. Deep dive 2: availability queries

Availability for item X at location = **sum of quantity across nearby DCs**.

Approach	Verdict
Query primary DB with join per request	Bad at 10K/s
Read replicas + indexed `(item_id, dc_id)` with `WHERE dc_id IN (...)`	Good
Cache per `(cell, item)` availability with short TTL (seconds)	Great: stale by seconds is acceptable (order path re-checks)

Alternatively cache per-DC inventory and sum in the service. Stale reads cause occasional "sorry, out of stock" at order time: acceptable trade-off for latency.

One-liner: *Approximate on read, exact on write.*

10. Deep dive 3: no overselling

Approach	Verdict	Why
Read qty, check, then update	Bad	Race: both see 1
Single transaction with conditional decrement (`qty >= n`)	Great	Atomic per row; relies on a **single DB** (or one shard per DC)
`SELECT FOR UPDATE` then update	Good	Correct but longer locks
Distributed transaction (2PC) across DBs	Avoid	Complex; partition inventory by DC so each order touches one shard

Which DC? Pick the nearest DC that can fulfil **all** items (orders fulfilled from one DC, simplifying atomicity). If none can, reject or split orders (product decision). Use **idempotency key** to prevent double orders. Short **reservation** (hold) before payment if payment is in scope (see Ticketmaster).

11. Deep dive 4: scaling

- Inventory table **partitioned by `dc_id`** (or region) so orders in a region only touch one shard; availability reads fan out to a few shards.
- Read replicas and cache absorb 10K/s; primary handles about 500 order tx/s.
- Updates from warehouse systems (restock) apply as increments; cache invalidated/TTL'd.
- Hot items (viral products) use conditional update contention: still fine at this volume; add queueing per item/DC if needed.

---

12. Failure handling

Failure	Mitigation
Primary DB failover	Sync replica; order retries safe (idempotency)
Replica lag	Order path hits primary, so correctness preserved
Cache down	Fall back to replicas with rate limit
Nearby service down	Cached cell to DC map; static fallback
Duplicate order request	Idempotency key

---

13. Trade-offs

Decision	Chosen	Alternative	Given up
Availability freshness	Cached, seconds stale	Exact	Occasional failed order
Order atomicity	Single DC, single tx	Multi-DC split	Fulfilment flexibility
DB	Postgres ACID	NoSQL	Horizontal ease (partition by DC solves)
DC lookup	Precomputed cells	Live routing API	Slight ETA inaccuracy
Hold inventory	At order time	At cart add	Cart-abandonment locking

---

14. Level expectations

- **Mid:** inventory table, nearby DC query, order transaction.
- **Senior:** read/write split, caching availability, race prevention, partition by DC.
- **Staff+:** reservation strategy, multi-DC order splitting, restock pipelines, demand-driven placement, consistency boundaries.

15. Follow-ups

Cart reservations, split shipments, substitution suggestions, predictive restocking, courier assignment integration.

16. Summary checklist

- [ ] Tiny data, so ACID Postgres, partition by DC
- [ ] Precomputed cell to DC map
- [ ] Cached/replica availability sums (stale OK)
- [ ] Conditional decrement in one transaction, idempotent orders
- [ ] Order path always hits primary

Designing a Metrics Monitoring System (Datadog / Prometheus-style)

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**ingestion at scale, push vs pull, time-series storage and compression, downsampling/rollups, query performance, alerting, cardinality**) → trade-offs. The hard parts are **write-heavy ingestion**, **TSDB storage layout**, and **high-cardinality blowups**.

---

0. Talk track

1. Clarify: ingest metrics from services (CPU, latency, counters), query dashboards over time ranges, **alerting**.
2. Insight: **write-heavy, append-only, time-ordered**, reads are recent-heavy aggregations.
3. HLD: agents, ingest gateway, Kafka, TSDB, query service, alert evaluator.
4. Deep dives: ingestion and pipeline, TSDB design, rollups/retention, query path, alerting, cardinality.

---

1. Problem

Collect time-series metrics from thousands of hosts/services, store them cheaply, let users **query and graph** them, and **alert** on conditions.

What makes it hard

- Millions of data points/s, append-only.
- Fast range/aggregate queries over huge history.
- **Cardinality explosion** (label combos).
- Alerts must be **timely and reliable even when the system is under stress**.

---

2. Requirements

Functional

- **Ingest** metrics (name, labels/tags, timestamp, value).
- **Query** time ranges with aggregation (avg, p99, sum by tag).
- **Dashboards** and **alerts** with notifications.

Out of scope: logs, traces, anomaly detection ML.

Non-functional

- **Scale**: 10K hosts x 1K metrics each = **10M series**, 10 s scrape: **1M points/s**.
- **High write throughput**; ingestion must not lose data.
- Query latency: seconds for dashboards (under 1 s for recent).
- **Availability** high (monitoring must outlive the thing monitored); retention (raw 15 days, rollups 1 year+).

---

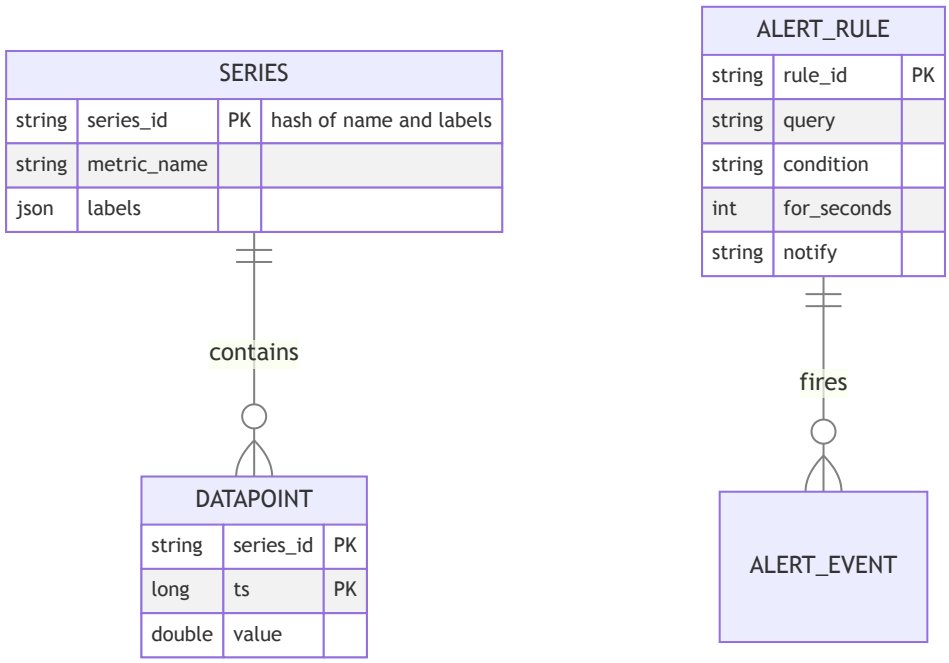
3. Estimation

Quantity	Assumption	Result
Series	10M	
Rate	1 point / 10 s per series	**1M points/s**
Point	16 B raw (8 ts + 8 val); **compressed about 1.5 B** (Gorilla delta-of-delta)	raw 16 MB/s = **1.4 TB/day**; compressed **about 130 GB/day**
Retention 15 d raw		**about 2 TB** compressed
Alert rules	100K evaluated every 30 s	about 3.3K evals/s

Consequence: compress aggressively; **recent data in memory/SSD**, old data downsampled to cheap storage.

---

4. Entities



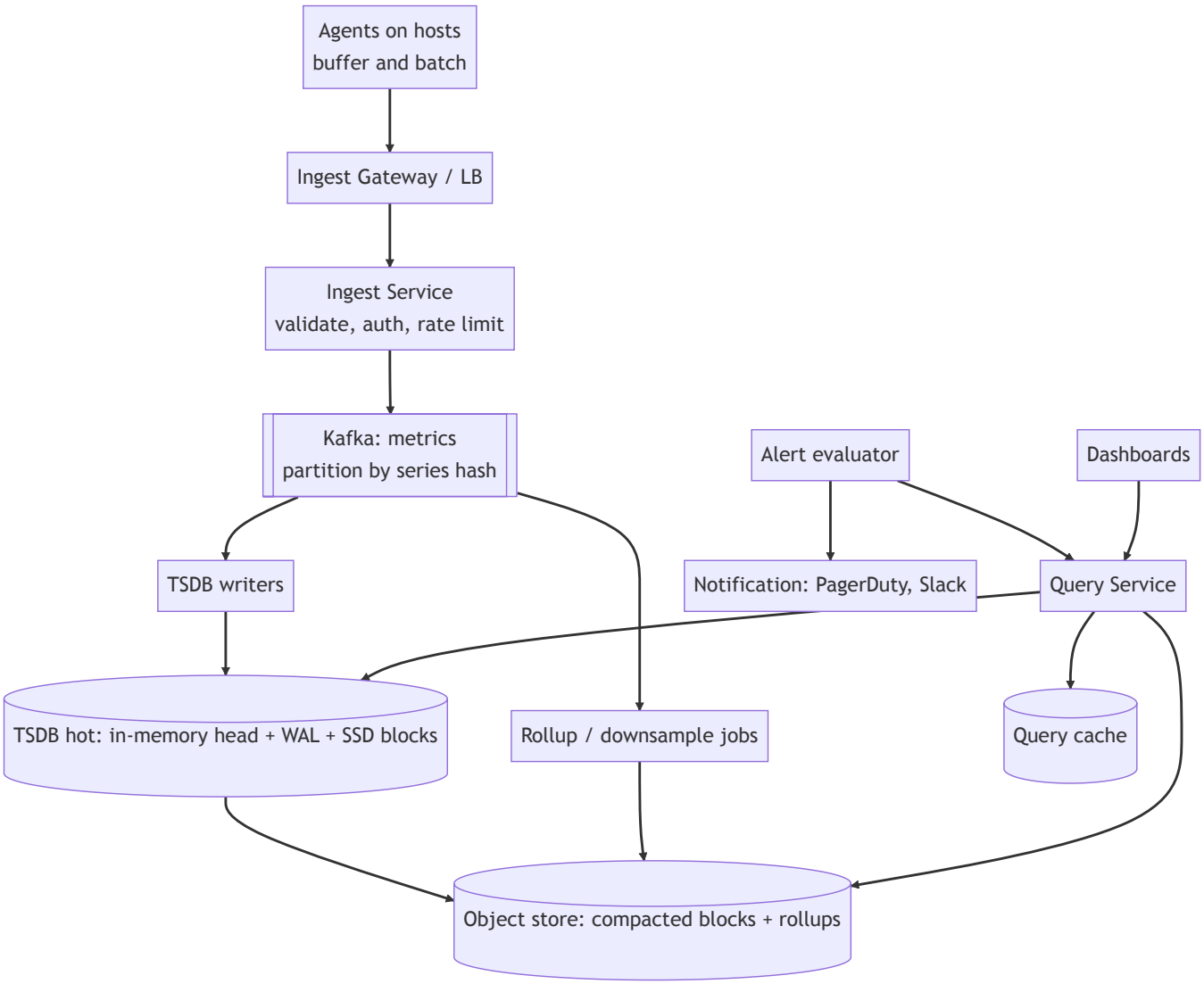
---

5. API

```
POST /v1/write   [{ "name":"cpu", "labels":{"host":"a1","dc":"us"}, "ts":..., "value":0.73 }, ...]
GET  /v1/query_range?query=avg(cpu{dc="us"})&start=&end=&step=60s     -> series of [ts,value]
POST /v1/alerts  { query, condition:"> 0.9", for:"5m", notify:"pagerduty" }
```

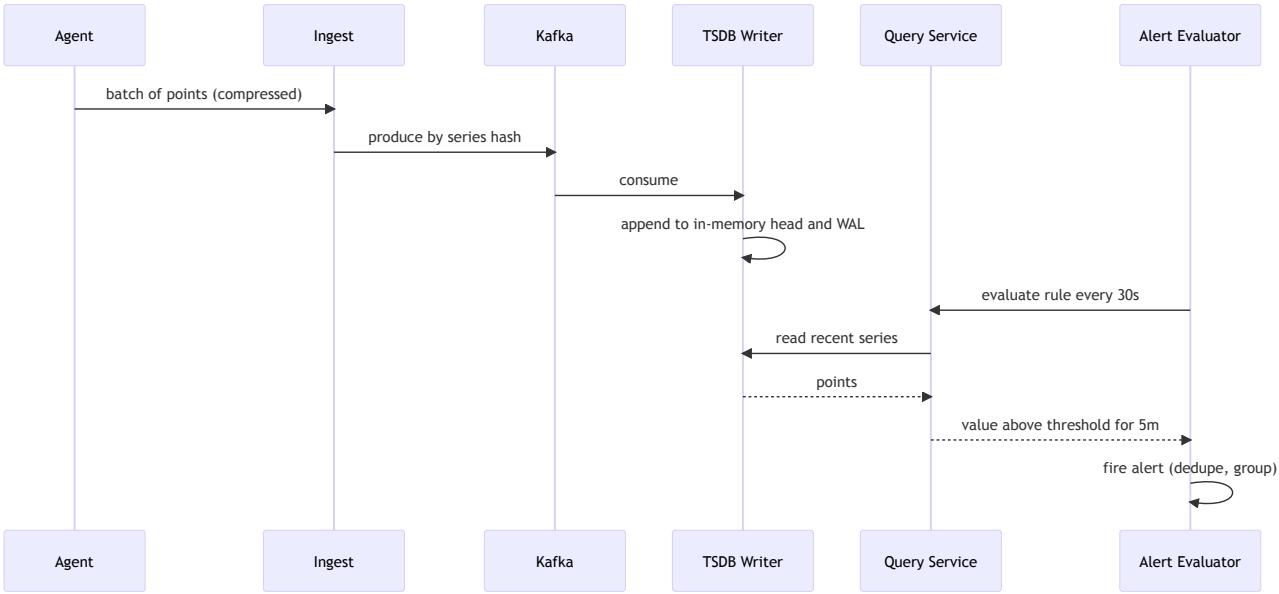
---

6. Architecture


Agents on hosts
buffer and batch
Ingest Gateway / LB
Ingest Service
validate, auth, rate limit
Kafka: metrics
partition by series hash
Alert evaluator
Dashboards
TSDB writers
Notification: PagerDuty, Slack
Query Service
TSDB hot: in-memory head + WAL + SSD blocks
Rollup / downsample jobs
Query cache
Object store: compacted blocks + rollups

---

7. Flow


Agent
Ingest
Kafka
TSDB Writer
Query Service
Alert Evaluator
batch of points (compressed)
produce by series hash
consume
append to in-memory head and WAL
evaluate rule every 30s
read recent series
points
value above threshold for 5m
fire alert (dedupe, group)
Agent
Ingest
Kafka
TSDB Writer
Query Service
Alert Evaluator

---

8. Deep dive 1: ingestion

Approach	Verdict	Why
Services write directly to the DB	Bad	Overload, coupling
Pull (Prometheus scrape)	Good	Central control of targets, simple health detection (`up`); needs service discovery; harder through NAT
Push via agent to gateway	Good	Works for short-lived jobs and across networks; need auth and backpressure
Agent batching + Kafka buffer	Great	Absorbs bursts, decouples ingest from storage, enables replay

Agents **batch and compress**, retry with backoff, buffer locally (bounded) during outages. Backpressure with rate limits per tenant. Partition by **series hash** so a series' points land on one writer (ordering, good compression).

9. Deep dive 2: time-series storage

Approach	Verdict	Why
General SQL row per point	Bad	Too much overhead, slow aggregates
Wide-column (Cassandra)	OK	Scales; poor compression and query ergonomics
Purpose-built TSDB: in-memory head block + WAL, flush to immutable **time-partitioned blocks**, columnar, **Gorilla compression** (delta-of-delta timestamps, XOR floats)	Great	About 1-2 bytes/point; fast scans

An **inverted index** from labels to series ids supports queries like `host=a1,dc=us`. Writes are append-only (sequential), so throughput is high. Shard by series hash across nodes; replicate (2-3x) for durability or rely on Kafka/WAL replay.

One-liner: *Append-only head with WAL, flush to compressed time blocks, label index for lookup.*

10. Deep dive 3: retention and rollups

Raw (10 s) for 15 days; **downsample** to 1 min (7 days to 90 days) and 1 h (1 year+) storing min/max/sum/count (so averages and quantile approximations remain computable). Query service picks the resolution from the requested range (`step`). Old blocks move to object storage (cheap), with **TTL deletion** by block, so deleting is dropping a file.

11. Deep dive 4: query performance

- Range queries touch few time blocks; **block time bounds + label index** prune.
- **Query cache** for dashboard refreshes (cache by query + aligned time window; only recompute the newest slice).
- Parallelize across shards and merge; limit series per query; timeouts.
- Percentiles: store **histograms/sketches (t-digest)** rather than raw p99 values, since percentiles can't be averaged.

12. Deep dive 5: alerting

An **evaluator** runs rules periodically (every 30 s), with `for: 5m` conditions to suppress flapping, groups/deduplicates alerts, and uses **state** (pending, firing, resolved) persisted so restarts do not re-page. Make the alerting path **independent** from heavy dashboard load (separate query pool, priority). HA via redundant evaluators with dedupe at the notifier; add a **dead man's switch** (heartbeat alert) to detect monitoring failure.

13. Deep dive 6: cardinality

Each unique label combination creates a series; unbounded labels (user_id, request_id) explode memory and index. Enforce **limits per tenant/metric**, reject or drop offending labels, provide cardinality dashboards, and guide users to put high-cardinality data in logs/traces.

---

14. Failure handling

Failure	Mitigation
Ingest node down	LB reroutes; agents retry and buffer
Writer crash	Replay Kafka from offset; WAL recovery
Storage node loss	Replicas / re-read from object storage and Kafka
Query overload	Query limits, caching, shed low priority
Alert evaluator dies	Standby takes over; state persisted
Cardinality bomb	Tenant limits and circuit breaker

---

15. Trade-offs

Decision	Chosen	Alternative	Given up
Ingestion	Agent push via Kafka	Pull scrape	Target discovery simplicity
Storage	Custom TSDB blocks	Generic DB	Build/ops cost
Resolution	Downsampled over time	Keep raw forever	Fine detail on old data
Percentiles	Sketches	Raw points	Slight approximation
Replication	Kafka + WAL + 2 replicas	Single copy	Storage cost

---

16. Level expectations

- **Mid:** agents, DB, dashboards, alert polling.
- **Senior:** Kafka buffering, TSDB reasoning, rollups/retention, query cache.
- **Staff+:** compression math, cardinality governance, multi-tenancy, alert reliability, monitoring-the-monitor.

17. Follow-ups

Distributed tracing correlation, anomaly detection, SLO burn-rate alerts, multi-region federation, cost allocation per tenant.

18. Summary checklist

- [ ] Agent batching, Kafka buffer by series hash
- [ ] TSDB: head + WAL + compressed time blocks, label index
- [ ] Rollups and retention tiers, object storage for cold
- [ ] Query cache, sketches for percentiles
- [ ] Alerts with state, `for`, dedupe, dead-man switch; cardinality limits

Designing a Notification Service

> Interview-ready walkthrough: requirements → estimates → entities → API → HLD → deep dives (**multi-channel delivery, queues and priority, retries and at-least-once with dedupe, user preferences and rate limiting, templating, scheduling, third-party provider failures**) → trade-offs. The hard parts are **reliable fan-out through flaky third parties** and **not spamming users**.

---

0. Talk track

1. Clarify: send **push, SMS, email, in-app** notifications triggered by services or schedules; user preferences; scale 10B/day.
2. Core: asynchronous pipeline with **queues per channel**, workers calling providers (APNs/FCM/Twilio/SES).
3. Deep dives: reliability, priority, preferences/dedupe/throttle, provider failover, bulk campaigns, observability.

---

1. Problem

A central service other teams call to notify users across channels, **reliably, once, respecting preferences**, with bursts (marketing blast to 100M users).

What makes it hard

- Third-party providers are rate-limited and flaky.
- **Exactly-once feel** from at-least-once infrastructure.
- Priorities: OTP/security alerts must not queue behind a marketing blast.
- User fatigue: dedupe, caps, quiet hours.

---

2. Requirements

Functional

- Send notifications via **push (iOS/Android), SMS, email, in-app**.
- **Templates** and personalization.
- **User preferences** (channel opt-in/out, quiet hours).
- **Scheduled** and bulk (campaign) sends; delivery status tracking.

Out of scope: campaign targeting/segmentation UI, analytics dashboards, in-depth content creation.

Non-functional

- **Scale**: 10B notifications/day, bursts to 1M+/s for campaigns.
- **Latency**: transactional (OTP) under seconds; bulk is best effort.
- **At-least-once delivery** with dedupe; high availability; durable.

---

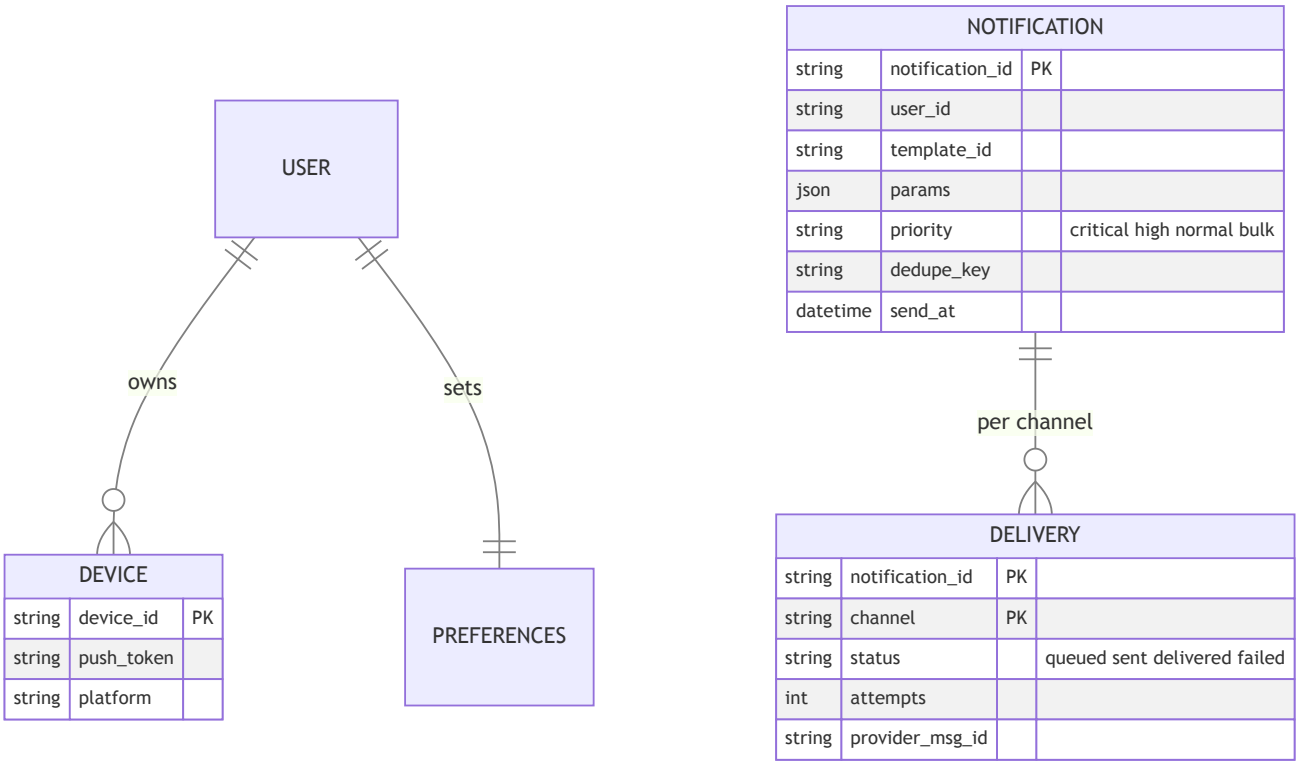
3. Estimation

Quantity	Assumption	Result
Volume	10B/day	**about 115K/s avg, about 1M/s peak**
Payload	1 KB	10 TB/day in queues
Channels	70% push, 20% email, 10% SMS	SMS is the cost/limit driver (carrier rate limits)
Status events	3 per notification	30B/day logs: **sampled/aggregated** storage

Consequence: everything async; **Kafka-class queues** with per-channel partitions; autoscaled workers; careful provider rate limiting.

---

4. Entities



---

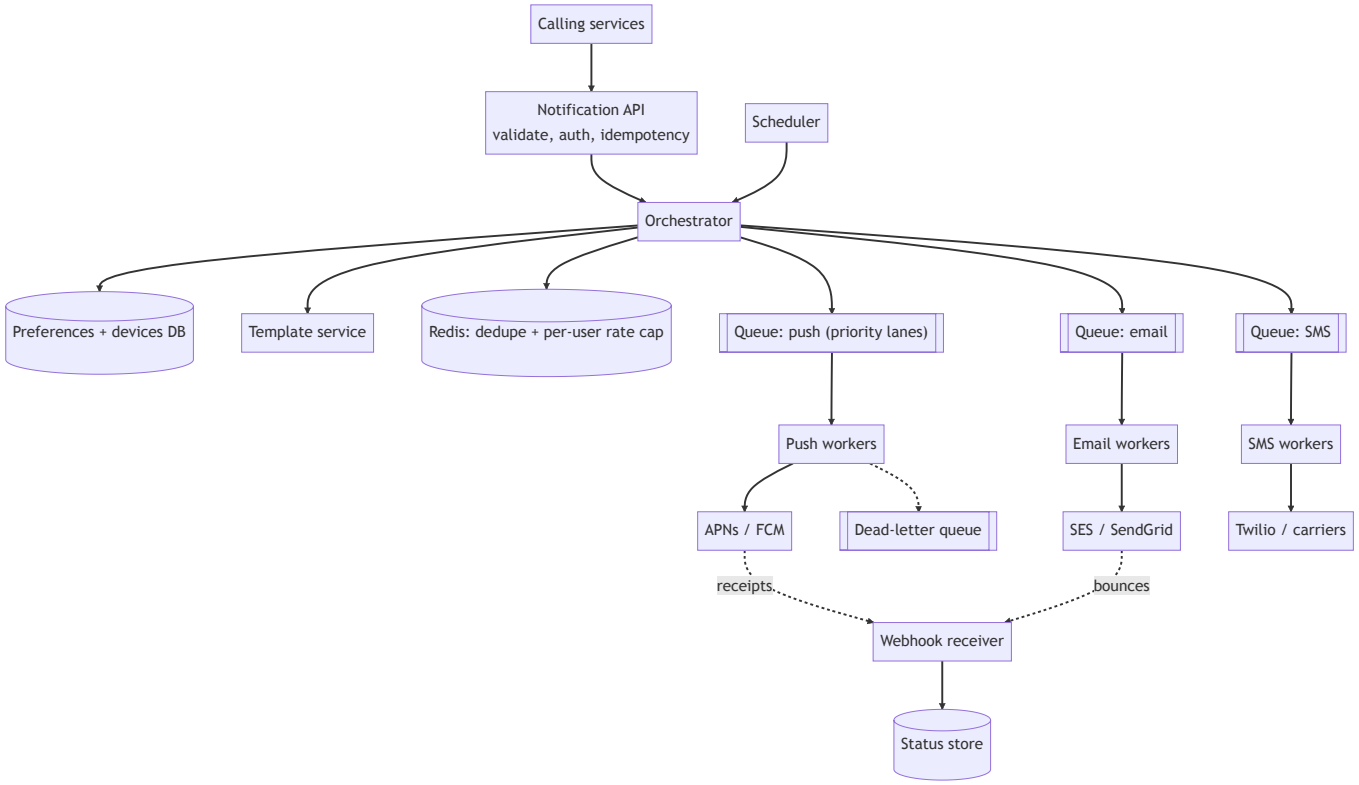
5. API

```
POST /v1/notifications   Idempotency-Key
{ "user_ids":["u1"], "template":"order_shipped", "params":{...}, "priority":"high", "channels":["push"
-> 202 { notification_id }
GET  /v1/notifications/{id}      -> delivery status per channel
PUT  /v1/users/{id}/preferences  { email:true, sms:false, quiet_hours:"22:00-07:00" }
```

Providers' delivery receipts/bounces arrive via webhooks to us.

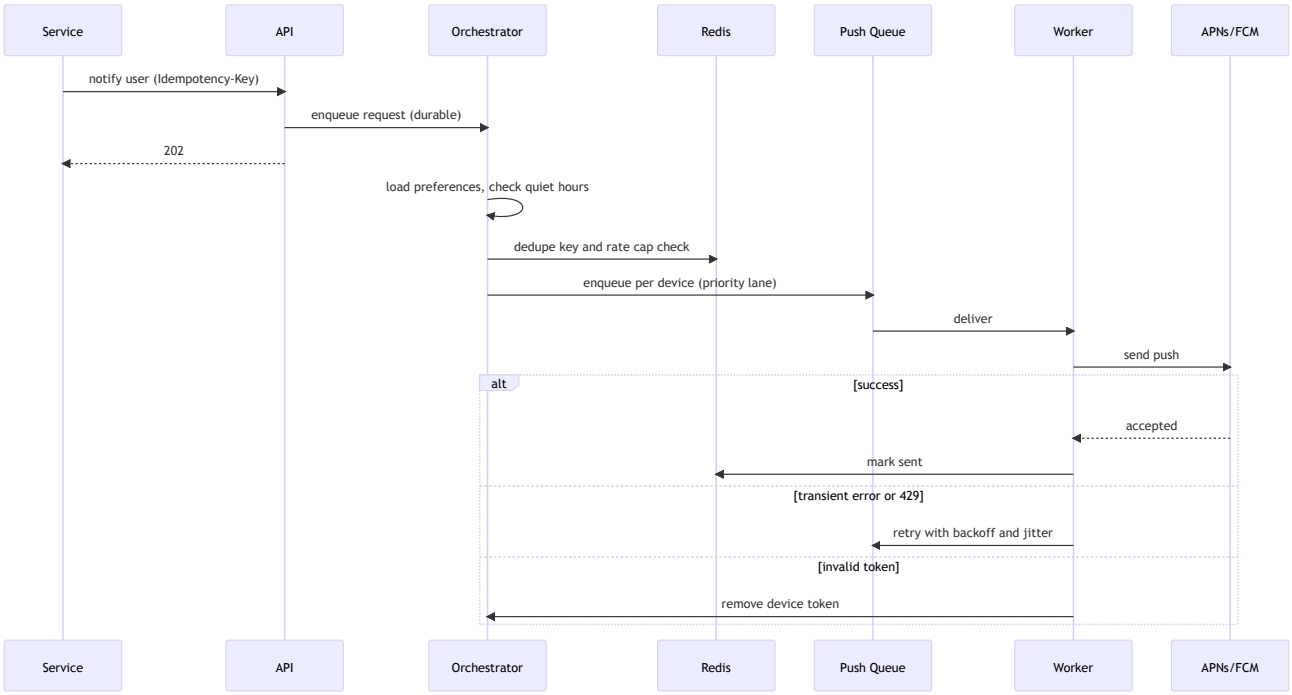
---

6. Architecture



---

7. Flow



---

8. Deep dive 1: reliability

- **Durable ingestion**: persist the request (Kafka/DB) **before** returning 202.

- Workers use **at-least-once** consumption (ack after provider accepted); duplicates controlled via **dedupe key** `(user, notification_id, channel)` in Redis/DB checked just before sending (provider message ids help as well).
- **Retries**: exponential backoff + jitter for transient errors/429, capped attempts, then DLQ and alert; permanent errors (invalid token, bounced email) are not retried and update device/contact state.
- **Circuit breaker** per provider; **failover** to a secondary provider (e.g. second SMS vendor) when error rate spikes.
- Outbox pattern between request DB and queue to avoid dual-write loss.

One-liner: *Persist first, retry transient, drop permanent, dedupe at send, DLQ the rest.*

9. Deep dive 2: priority and isolation

Separate **queues/lanes per channel and priority** (critical OTP, high transactional, normal, bulk). Workers weight critical lanes first; bulk campaigns are **rate-paced** (token bucket per provider and per tenant) so they cannot starve transactional. Dedicated worker pools and quotas per team prevent noisy neighbors. Autoscale on queue depth and age.

10. Deep dive 3: preferences, dedupe, throttling

- Preferences cached (Redis), consulted by the orchestrator; respect opt-out and **quiet hours** (delay to window), **regulatory** (unsubscribe, consent).
- **Per-user frequency caps** (e.g. 5 marketing/day) with Redis counters; **collapse keys** (replace stale notifications of same topic); digest batching.
- Timezone-aware scheduling.

11. Deep dive 4: bulk and scheduled

A campaign to 100M users: split recipients into **shards/chunks** processed by parallel orchestrator tasks that expand to per-user messages gradually (pace to provider limits). Scheduler (time-bucketed, see Job Scheduler doc) triggers sends at `send_at`. Progress tracked as aggregates, not per-message rows only.

12. Deep dive 5: templating and content

Template service with versioned templates, localization, and variable substitution; render in orchestrator or worker (render late to keep queues small). Validate params; sandbox rendering; link tracking/unsubscribe links.

13. Deep dive 6: status and observability

Status store updated from provider webhooks/receipts (eventual); keep summaries per notification and sampled detail given 30B events/day. Track delivery rate, latency percentiles, queue age, provider error rates, DLQ size; alert on lag rather than just errors.

---

14. Failure handling

Failure	Mitigation
Provider outage	Circuit breaker; failover provider; queue and delay
Worker crash	Message redelivered; dedupe prevents double-send
Queue backlog	Autoscale; shed/expire bulk (TTL on messages)
Redis loss (dedupe)	Fall back to DB check with rate limit; rare duplicate acceptable for non-critical
Poison message	Attempt cap, DLQ, inspection tooling
Token invalidation storm	Batch cleanup, ignore known-bad tokens

---

15. Trade-offs

Decision	Chosen	Alternative	Given up
Delivery	At-least-once + dedupe	Exactly-once	Rare duplicate on dedupe-store loss
Processing	Async queues	Sync call to providers	Immediate status
Priority	Separate lanes	Single queue	More infra, but protects OTP
Bulk	Paced fan-out	Blast	Slower campaign completion
Status	Eventual via webhooks	Sync	Fresh delivery info

---

16. Level expectations

- **Mid:** API, queue, workers per channel, provider integration.
- **Senior:** retries/DLQ, dedupe, priority lanes, preferences, rate limiting, scheduling.
- **Staff+:** provider failover economics, bulk pacing, tenant isolation, compliance, observability at 30B events/day.

17. Follow-ups

A/B testing content, delivery-time optimization, in-app inbox with read state, rich push, cross-channel fallbacks (push then SMS).

18. Summary checklist

- [ ] Durable ingest, 202, outbox
- [ ] Queue per channel and priority, autoscaled workers
- [ ] Dedupe key, retries with backoff + jitter, DLQ
- [ ] Preferences, quiet hours, frequency caps
- [ ] Provider circuit breakers/failover, receipts via webhooks